



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

Un zkBridge para CARDANO utilizando pruebas de conocimiento cero

Tesis de Licenciatura en Ciencias de la Computación

Lorenzo Ruiz Díaz

Director: Matías López y Rosenfeld

Codirector: Bruno Weisz

Buenos Aires, 2026

DESARROLLO DE UN zkBridge EN CARDANO

Esta tesis estudia el diseño e implementación de un zkBridge para CARDANO. El objetivo es autenticar un evento ocurrido en una cadena de origen y habilitar una acción correspondiente en una cadena destino, sin depender de un operador centralizado y respetando los límites reales de ejecución on-chain. Para ello se propone una arquitectura que separa una capa de actualización, encargada de mantener estado remoto autenticado, y una capa de aplicación, encargada de consumir esa evidencia para autorizar el flujo $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$.

La construcción evita verificar directamente el consenso completo de CARDANO dentro de un circuito. En su lugar, utiliza MITHRIL como mecanismo de certificación de estado, una verificación STM particionada en dos fases sobre Aiken y argumentos Groth16 auxiliares para pertenencia de transacciones y actualización de un SMT anti-replay. El prototipo muestra que esta composición puede expresarse en el modelo eUTxO mediante UTxOs canónicos, NFTs de estado, *reference scripts* y `proof_receipt`, manteniéndose dentro de los presupuestos de unidades de ejecución disponibles.

El resultado principal es una prueba de viabilidad para una instancia isomorfa de CARDANO: no un bridge productivo heterogéneo completo, sino una arquitectura verificable que delimita con precisión sus supuestos de seguridad, sus invariantes on-chain y los componentes que quedan como trabajo futuro.

Palabras clave: zkBridge, CARDANO, MITHRIL, eUTxO, pruebas de conocimiento cero.

DEVELOPMENT OF A zkBridge IN CARDANO

This thesis studies the design and implementation of a zkBridge for CARDANO. The goal is to authenticate an event that occurred on a source chain and enable a corresponding action on a destination chain, without relying on a centralized operator and while respecting the actual limits of on-chain execution. To this end, the work proposes an architecture that separates an update layer, responsible for maintaining authenticated remote state, from an application layer, responsible for consuming that evidence to authorize the $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$ flow.

The construction avoids verifying the full CARDANO consensus protocol directly inside a circuit. Instead, it uses MITHRIL as a state-certification mechanism, a two-phase STM verification implemented in Aiken, and auxiliary Groth16 arguments for transaction membership and anti-replay SMT updates. The prototype shows that this composition can be expressed within the eUTxO model through canonical UTxOs, state NFTs, *reference scripts* and `proof_receipt`, while remaining within the available execution-unit budgets.

The main result is a viability proof for an isomorphic CARDANO instance: not a complete production-ready heterogeneous bridge, but a verifiable architecture that precisely states its security assumptions, its on-chain invariants and the components left for future work.

Keywords: zkBridge, CARDANO, MITHRIL, eUTxO, zero-knowledge proofs.

AGRADECIMIENTOS

Comencé la carrera formalmente en el año 2022, pero en verdad fue mucho antes. Todo empezó cuando en 2017 comencé a participar en la Olimpiada Matemática Argentina, y especialmente en 2019 cuando llegué al Nacional y empecé forjar mis más grandes grupos de amigos. Ese camino me condujo a esta facultad, a esta carrera, y a este trabajo final. Me gustaría dar máxima gratitud a Patricia, Flora, Matías, Silvia, Gabriel y Verónica.

Quiero agradecer a toda mi familia: especialmente a mis padres Gustavo y Guillermina, por su apoyo; y a mis abuelos Carlos, Ana, Lorenzo y Marta. Sé que los árboles genealógicos Ruiz Díaz y Herrera van a estar orgullosos de tener un nuevo universitario.

Todas estas personas, y seguro muchas más, son las que me trajeron hasta acá: nunca estuve solo. Pido sinceras disculpas si me olvido de alguien. Quiero mencionar especialmente a Federico Brisnicoff, Tomás Molezzi, Santiago Manzanares, Carolina Castellucci, Franco Lotti, Sebastián Naddeo, Antonella Berzotti, Martina Ranieri, Facundo Álvarez Motta, Martina Álvarez Motta, Joaquín Fernández, Pilar Palmarino, Lucas Sandleris, Nicolás Schaievitch, Lisandro Acuña, Olivia López Taiana, Francisco Cirelli, Matteo Ginhson, Nicolás Pérez Stefan, Tomás Castro, Sofía Kutianski, Lucas Blake, Dante Culaciat, Nicolás Yañez, el Chino Criboli, Franco Bongiovanni, Franco de Rosa, Facundo Decroix, Oriana Martínez y Franco Bottazzi.

También quiero mencionar expresamente a Matías y Bruno, mi director y codirector respectivamente, ya que sin ellos, esta tesis no hubiera sido posible o su calidad hubiera sido menor.

Por último, quiero agradecer a la comunidad de la cooperativa Eryx por estos diez meses maravillosos que pasé junto a ellos, previo a mi emigración a México, y en los cuales pude desarrollar el proyecto que hoy presento como Tesis de Licenciatura.

A la Olimpiada Matemática Argentina.

Índice general

Notación	XV
1.. Introducción	1
2.. Bridges y zkBridge	3
2.1. Bridges en la Blockchain	3
2.1.1. ¿Qué es un Bridge?	3
2.1.2. Ejemplo de uso	3
2.1.3. Modelos de seguridad de Bridges	4
2.2. Problemas de seguridad en Bridges	5
2.3. Resumen del paper zkBridge	6
2.3.1. Esquema general de un zkBridge	6
2.3.2. Un zkBridge en la práctica	8
3.. Cardano y Modelo eUTxO	9
3.1. El modelo eUTxO	9
3.1.1. De UTxO a eUTxO: validadores, datums, redeemers, contexto	9
3.1.2. Máquinas de estado en eUTxO	10
3.1.3. Características de eUTxO relevantes para aplicaciones complejas	11
3.2. Smart Contracts en Cardano: Plutus y Aiken	11
3.2.1. Plutus, Plutus Core y PlutusTx	11
3.2.2. Aiken	11
3.3. Límites del protocolo y su importancia en un zkBridge	12
3.3.1. Contabilización de recursos: unidades de ejecución, determinismo y validación en dos fases	12
3.3.2. Límites de tamaño y cómputo	12
3.3.3. Cómo los límites estrictos moldean el espacio de diseño	13
3.4. Praos y el modelo operativo de proof-of-stake	13
3.4.1. Proof of stake, stake pools y SPOs	13
3.4.2. Estructura temporal: slots y épocas	14
3.4.3. Elección de líderes mediante VRF	14
3.4.4. KES y certificados operativos	14
3.4.5. <i>Light clients</i> en Cardano	15
3.5. Direcciones futuras	15
3.5.1. Peras	15
3.5.2. Leios	15
3.6. Conclusiones	16
4.. Mithril	17
4.1. Introducción y motivación	17
4.2. Protocolo STM y esquemas de firma	18
4.2.1. Intuición de la lotería y del umbral	19
4.2.2. Eras y rutas criptográficas en Mithril	20

4.2.3.	Fases del protocolo	21
4.2.4.	Registro cerrado y clave de verificación agregada SNARK	22
4.2.5.	Esquema de firmas en Lagrange	22
4.2.6.	Firmas individuales y agregación	23
4.3.	Cadena de certificados	25
4.3.1.	Estructura matemática de un certificado	25
4.3.2.	Diseño matemático de la cadena	26
4.3.3.	Verificación de la CertChain	27
4.3.4.	Relación con el flujo de StakeDist	28
4.4.	Entidades, certificados y versiones	28
4.4.1.	Pythagoras y el camino <i>legacy</i>	29
4.4.2.	Lagrange como transición	31
4.5.	API del <i>aggregator</i> e integración	32
4.5.1.	Endpoints relevantes	33
4.5.2.	Flujo del zk-bridge-operator	33
4.6.	Relación NP del circuito STM	34
4.7.	Halo2 y complejidad	35
4.7.1.	Qué aporta Halo2 en este capítulo	35
4.7.2.	Del protocolo SNARK al circuito STM	35
4.7.3.	Representación del circuito en Halo2	36
4.7.4.	De la relación NP a una aritmetización <i>PLONKish</i>	36
4.7.5.	Restricciones específicas de la relación STM	36
4.7.6.	Compilación algebraica y verificación global	37
4.7.7.	Argumentos de permutación, <i>lookups</i> y columnas auxiliares	37
4.7.8.	Uso de crates de MIDNIGHT	37
4.7.9.	Grado algebraico del circuito	38
4.7.10.	Costo estructural del <i>prover</i>	38
4.8.	Puente a la implementación final	39
5..	Decisiones de Diseño	41
5.1.	Restricciones de Cardano que gobiernan el diseño	42
5.2.	Arquitectura	42
5.2.1.	Capas: actualización y aplicación	42
5.2.2.	Actores, responsabilidades y flujo del sistema	43
5.3.	Estado en eUTxO : atomicidad y concurrencia	43
5.4.	Elección de estrategia ZK	44
5.4.1.	Alternativas consideradas	44
5.4.2.	Esquema de argumentos ZK	45
5.4.3.	Implicancias de diseño	45
5.5.	Transacciones del protocolo	45
6..	Implementación del zkBridge	47
6.1.	Objetivo y alcance del bloque inicial	47
6.2.	Componentes principales del prototipo	48
6.3.	Representación del estado del bridge en eUTxO	48
6.4.	Flujo transaccional del bloque inicial	49
6.4.1.	Actualización de StakeDist mediante tx_{sd}	49
6.4.2.	Transacción de <i>locking</i> en la cadena de origen	50

6.4.3.	Transacción de minting en la cadena destino	51
6.5.	Decisiones de ingeniería y preparación para los circuitos reales	52
6.6.	Formalización de los validadores principales	53
6.6.1.	Validador de bootstrap de StakeDist	53
6.6.2.	Validador de actualización de StakeDist	54
6.6.3.	Validador de <i>minting</i> del zkBridge	54
6.6.4.	Validador del conjunto de transacciones procesadas	54
6.7.	Del circuito Halo2 de Mithril a Cardano	55
6.7.1.	Necesidad y alcance de la adaptación de <code>plutus-halo2-verifier-gen</code>	55
6.7.2.	Optimizaciones del validador Aiken	56
6.7.3.	Partición de la verificación en dos transacciones	57
6.7.4.	Validez del transcript y <i>binding</i> entre fases	58
6.7.5.	Conexión de UTxOs, tx_{sd} y tx_{mint}	59
6.8.	El circuito <code>tx_set_update</code>	60
6.8.1.	Modelo matemático del SMT	60
6.8.2.	<i>Public inputs</i> y <i>witness</i> del circuito	61
6.8.3.	La relación NP de <code>tx_set_update</code>	61
6.8.4.	Familias de restricciones del circuito	62
6.8.5.	Vínculo con <code>txs_updater_minting</code>	63
6.9.	El circuito de pertenencia a una TxSnapshot	63
6.9.1.	Modelo matemático del TxSnapshot Pythagoras de Mithril	63
6.9.2.	<i>Public inputs</i> y <i>witness</i> del circuito	64
6.9.3.	La relación NP de <code>snapshot_membership</code>	64
6.9.4.	Familias de restricciones del circuito	65
6.9.5.	Vínculo exacto con <code>minting_validator</code>	66
6.9.6.	Transición experimental a Lagrange	66
6.10.	Flujo final del zkBridge	67
6.10.1.	El ecosistema Tx3/trix/cshell/dolos	67
6.10.2.	Estrategia de tests con <code>aiken check</code>	68
6.10.3.	El flujo principal del zkBridge	68
7..	Resultados	71
8..	Conclusiones y trabajo futuro	73
8.1.	Conclusiones	73
8.1.1.	Modelo de seguridad y alcance del prototipo	73
8.2.	Trabajo futuro	74
8.2.1.	Sobre CIP-133	74
8.2.2.	Sobre ‘zk-bridge-operator’	74
8.2.3.	Sobre ‘preview testnet’ de Cardano	75
8.2.4.	Sobre σ_{gen} en la era Lagrange	75
	Apéndice	83
A..	Primitivas Criptográficas	85
A.1.	Cuerpos Finitos	85
A.1.1.	Cuerpos	85

A.1.2.	Cuerpos Finitos	85
A.1.3.	Estructura multiplicativa del grupo no nulo de $\mathbb{F}_q$	86
A.2.	Problema del logaritmo discreto (DLP)	86
A.3.	Polinomios	88
A.3.1.	Anillo de polinomios	88
A.3.2.	Lema de Schwartz-Zippel	89
A.3.3.	Interpolación de polinomios	90
A.4.	Curvas Elípticas	91
A.4.1.	Representación y compresión de puntos	95
A.4.2.	Curvas elípticas útiles para este trabajo	95
A.5.	Pairings bilineales	96
A.6.	Hashes	97
A.6.1.	Funciones de hash criptográficas	97
A.6.2.	Domain separation	98
A.6.3.	Hash hacia curvas elípticas	98
A.6.4.	Firmas Schnorr sobre curvas elípticas	99
A.7.	Merkle Trees	99
A.7.1.	Definición e intuición	99
A.7.2.	Merkle Proofs	100
A.8.	Sparse Merkle Trees (SMT)	102
A.8.1.	Definición e Intuición	102
A.8.2.	Sparse Merkle Proofs	103
A.8.3.	Actualización de un Sparse Merkle Tree	105
B..	Aritmetización y Construcción de un SNARK	107
B.1.	Introducción a pruebas ZK	107
B.2.	SNARKs	111
B.3.	Circuitos Aritméticos	114
B.3.1.	Introducción a los circuitos aritméticos	114
B.3.2.	Construcción de un ejemplo	115
B.3.3.	Lo que tenemos hasta ahora	117
B.3.4.	Limitaciones de los circuitos aritméticos	118
B.4.	Sistemas de Restricciones R1CS (Rank-1 Constraint Systems)	118
B.4.1.	Introducción a R1CS	118
B.4.2.	Construcción de un ejemplo	120
B.4.3.	Lo que tenemos hasta ahora	121
B.5.	QAP: Quadratic Arithmetic Programs	122
B.5.1.	Equivalencia entre R1CS y QAP	122
B.5.2.	Construcción de un ejemplo	124
B.5.3.	Lo que tenemos hasta ahora	124
B.6.	Esquemas de Compromiso (Commitment Schemes)	125
B.6.1.	Motivación y definición formal	125
B.6.2.	Compromisos Vectoriales	126
B.6.3.	Compromisos Polinomiales	127
B.6.4.	De compromisos polinomiales a identidades de QAP	128
B.7.	KZG	129
B.7.1.	Definición formal e intuición	129

B.7.2.	Trusted Setup	131
B.7.3.	Aperturas múltiples y GWC19	131
B.7.4.	Aperturas por conjuntos de puntos en Halo2 Multi-Open	133
B.7.5.	Lo que tenemos hasta ahora	134
B.8.	Protocolo Fiat–Shamir y No-Interactividad	135
B.8.1.	De protocolos interactivos a no interactivos	135
B.8.2.	Transformación Fiat–Shamir en KZG	136
B.8.3.	Limitaciones	137
B.9.	Groth16	138
B.9.1.	El protocolo Groth16	138
B.9.2.	Groth16 como extensión natural de KZG	140
B.9.3.	Complejidad de Groth16	141
B.9.4.	Aplicación de Groth16 al proyecto	142

NOTACIÓN

Notación matemática y algebraica

Símbolo	Significado
$\mathbb{F}, \mathbb{K}$	Cuerpo (finito, base, escalar), según el contexto.
$\mathbb{F}_p, \mathbb{F}_q, \mathbb{F}_r$	Cuerpos finitos de orden p , q y r .
$\text{char}(\mathbb{K})$	Característica del cuerpo $\mathbb{K}$.
$\mathbb{G}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$	Grupos algebraicos usados en esquemas con pairings.
$e(P, Q)$	Pairing bilineal evaluado en $P \in \mathbb{G}_1$ y $Q \in \mathbb{G}_2$.
$\mathcal{E}, \mathcal{O}$	Curva elíptica y su punto al infinito.
$\mathcal{H}$	Función de hash genérica.
$\mathbb{P}$	Operador de probabilidad.
λ, negl	Parámetro de seguridad y función negligible.
$\mathcal{O}, \Omega, \Theta$	Notación asintótica estándar.
$\text{P}, \text{NP}, \text{IP}, \text{PSPACE}$	Clases de complejidad computacional.

Pruebas sucintas y aritmetización

Símbolo	Significado
$\mathcal{P}, \mathcal{V}$	<i>Prover</i> y <i>verifier</i> de un protocolo.
$\mathcal{R}, \mathcal{L}$	Relación y lenguaje asociados a un problema de verificación.
$\mathcal{X}, \mathcal{W}, \Pi$	Espacios de <i>public inputs</i> , <i>witness</i> y pruebas/argumentos.
Setup, Prove, Verify	Algoritmos de protocolos de pruebas/argumentos.
crs, SRS	<i>Common Reference String</i> y <i>Structured Reference String</i> .
NIZK, SNARK, zk-SNARK, STARK, zk-STARK	Familias de pruebas no interactivas y sucintas.
R1CS, QAP, PCS, VCS	Modelos y primitivas de aritmetización y compromiso.
KZG, Fiat–Shamir, Groth16, PLONK, Halo2	Protocolos usados recurrentemente en la tesis.
$\mathcal{C}, \mathcal{C}$	Circuito aritmético y restricción genérica de un circuito.
$\mathcal{H}, Z_{\mathcal{H}}$	Dominio de evaluación y polinomio anulador asociado.
$Q, Z_{\text{perm}}, Z_{\text{lookup}}$	Polinomio cociente y polinomios auxiliares de permutación y <i>lookup</i> .

Merkle Trees

Símbolo	Significado
$\mathcal{T}$	Árbol de Merkle genérico.
$r_{\mathcal{T}}$	Raíz del árbol $\mathcal{T}$.

Símbolo	Significado
π	Prueba de inclusión de Merkle.
Path	Camino autenticado dentro del árbol.
ℓ, h, i	Hoja, nodo hermano e índice de una prueba de inclusión.
SMT	<i>Sparse Merkle Tree</i> .
$\perp$	Hoja o nodo vacío en un SMT.
$r_{\text{in}}, r_{\text{out}}$	Raíces inicial y final del anti-replay.
sib	Vector de nodos hermanos en una prueba de Merkle.

Cardano y validadores

Símbolo	Significado
Bridge, zkBridge	Bridge genérico y bridge sucinto propuesto en la tesis.
$\mathcal{A}, \mathcal{B}$	Cadenas de origen y destino del bridge.
s, tx, e	Estado, transacción y evento abstractos del protocolo.
$\text{tx}_{\text{lock}}, \text{tx}_{\text{mint}}, \text{tx}_{\text{burn}}, \text{tx}_{\text{unlock}}, \text{tx}_{\text{updater}}$	Transacciones canónicas del flujo del bridge.
UTxO, UTxOs, eUTxO, eUTxOs	Modelo UTxO clásico y su extensión en CARDANO.
PLUTUS, UPLC	Lenguaje y <i>assembly</i> de scripts en CARDANO.
OUROBOROS	Protocolo de consenso de CARDANO.
epoch, slot	Época y slot del protocolo OUROBOROS.
$\delta, \rho, \nu, \text{tx}$	Componentes semánticos de un validador en CARDANO.

Mithril, certificados y snapshots

Símbolo	Significado
MITHRIL, STM	Protocolo MITHRIL y esquema de <i>stake-based threshold multisignatures</i> .
<i>aggregator</i> , zk-bridge-operator	<i>Aggregator</i> de MITHRIL y operador del zkBridge que consume sus artefactos.
Pythagoras, Lagrange	Eras relevantes de MITHRIL para la tesis: ruta <i>legacy</i> y ruta <i>SNARK-friendly</i> .
legacy, rigid	Esquemas de <i>hashing</i> del mensaje de protocolo en las rutas Pythagoras y Lagrange.
σ , StakeDist	Participación individual y distribución de stake certificada.
MithrilStakeDistribution, CardanoTransactions	Tipos principales de entidades firmadas por MITHRIL usados en el zkBridge.
$\text{Reg}^{\text{closed}}, \text{Reg}^*$	Registro cerrado por participante y registro autenticado del comité de una época.
$\text{vk}^{\text{sch}}, \text{sk}^{\text{sch}}$	Claves pública y secreta Schnorr de la ruta Lagrange.
τ, J, I	Umbral de lotería por participante, conjunto de índices evaluados y subconjunto de índices ganados.
AVK, AVK, AVK, $r_{\mathcal{R}}$	Claves de verificación agregadas y compromiso Merkle del comité autenticado.

Símbolo	Significado
L_i, w	Hoja autenticada del participante i y <i>witness</i> asociado a la prueba STM.
$\text{msg}, \mathcal{H}, \mu$	Mensaje de protocolo certificado, operador de <i>hash</i> rígido y <i>digest</i> firmado en la ruta Lagrange .
PP	Parámetros del protocolo STM, típicamente (k, m, ϕ_f) y sus variantes por época.
UniqueSchnorrSignature, StandardSchnorrSignature	Variantes de firma Schnorr usadas por MITHRIL en la era Lagrange .
$\text{cert}, \mathcal{C}$	Certificado individual y familia de certificados de MITHRIL.
$\text{cert}_{\text{gen}}, \text{cert}_{\text{ts}}^{\text{first}}$	Certificado génesis y primer certificado válido de una época.
$\text{cert}_{\text{sd}}, \text{cert}_{\text{ts}}$	Certificados de distribución de stake y de <i>snapshot</i> de transacciones.
$h, h^-, \text{meta}, \text{sm}$	Hash del certificado, hash del predecesor, metadatos y mensaje efectivamente firmado.
CertChain	Cadena de certificados enlazados que autentica la evolución de MITHRIL entre épocas.
$\text{pred}, \text{ChainVerify}, \text{CertVerify}$	Predecesor de un certificado, verificación de la cadena completa y verificación individual.
$\text{TxSnapshot}, \mathcal{S}$	Snapshot de transacciones individual y familia de snapshots expuestos por MITHRIL.
$\text{hash}_{\text{snap}}, b^*, r_{\text{tx}}$	Hash del snapshot, bloque certificado y raíz autenticada del conjunto de transacciones.
$\mathcal{T}_u, r_u, \rho_u$	Conjunto, raíz local y hoja maestra del rango u en CardanoTransactions legacy .
$\pi_{\text{master}}, \pi_u, J^*$	Pruebas de Merkle del nivel maestro y del nivel por rango, y conjunto final de índices retenidos.
$\mathbf{x}, \mathbf{w}$	Entradas públicas y privadas de los circuitos ligados a STM y a snapshots.
$\mathcal{R}_{\text{STM-H2}}, \mathcal{L}_{\text{STM-H2}}$	Relación y lenguaje del circuito STM formalizado en la tesis.
$\mathcal{R}_{\text{tx-set-update}}, \mathcal{R}_{\text{tx-snapshot-membership}}$	Relaciones de los circuitos de actualización del conjunto y de pertenencia a snapshots.

1. INTRODUCCIÓN

Esta tesis estudia el problema de construir un zkBridge para CARDANO. En términos generales, un Bridge—noción introducida en la Sección 2.1.1— busca transferir valor o información entre dos cadenas sin exigir a sus usuarios confianza plena en un intermediario centralizado. En nuestro caso, el objetivo es más específico: diseñar una arquitectura que permita tomar un evento ocurrido en una cadena de origen, autenticarlo criptográficamente y habilitar una acción correspondiente en una cadena destino, todo ello respetando las restricciones reales del entorno de ejecución de CARDANO.

El desafío no es solamente criptográfico. Un zkBridge práctico debe combinar, al mismo tiempo, un mecanismo de autenticación del estado remoto, una representación compacta de la evidencia relevante, una estrategia de verificación on-chain compatible con límites estrictos de cómputo y tamaño, y un modelo de estado que preserve invariantes como la atomicidad y la prevención de *replay*. En CARDANO, estas exigencias quedan además condicionadas por el modelo eUTxO, desarrollado en la Sección 3.1, y por los límites de ejecución reunidos en la Sección 3.3.

La propuesta desarrollada en este trabajo parte de una decisión central: no intentar verificar directamente dentro de un circuito ZK todo el consenso nativo de CARDANO, sino apoyarse en MITHRIL como mecanismo de certificación compacta del estado relevante, estudiado en el Capítulo 4. Sus certificados se autentican mediante un esquema de *stake-based threshold multisignatures* (STM), explicado en la Sección 4.2, cuya verificación Halo2/MIDNIGHT se divide en dos fases según la Sección 6.7.3. Para los circuitos auxiliares de pertenencia a *snapshots* y actualización del SMT anti-replay se utiliza Groth16, presentado en la Sección B.9. Sobre esa base, la tesis organiza el zkBridge en una capa de actualización, que mantiene en la cadena destino un estado autenticado y reutilizable, y una capa de aplicación, que consume esa autenticación para autorizar operaciones concretas del protocolo.

Para seguir el cuerpo principal de la tesis conviene tener familiaridad previa con nociones básicas de blockchains, transacciones, contratos inteligentes y criptografía de clave pública. También se utilizan de manera recurrente herramientas algebraicas y criptográficas como cuerpos finitos, curvas elípticas, funciones de hash, Merkle Trees, argumentos de conocimiento cero, circuitos aritméticos, R1CS, QAP, KZG, Fiat–Shamir y Groth16. Esos temas no se desarrollan linealmente dentro del cuerpo principal, para no interrumpir la discusión sobre el diseño del zkBridge, sino que se recopilan como material de apoyo al final del manuscrito: el Apéndice A cubre las primitivas criptográficas y estructuras autenticadas de base, mientras que el Apéndice B desarrolla la aritmetización y la construcción de un SNARK. En cambio, los conceptos específicos de CARDANO, eUTxO, MITHRIL y del flujo implementado se introducen progresivamente en los capítulos centrales.

Con ese marco, el recorrido del manuscrito avanza desde los antecedentes sobre Bridges y zkBridge del Capítulo 2 hacia las restricciones de CARDANO del Capítulo 3, la certificación con MITHRIL del Capítulo 4 y las decisiones arquitectónicas del Capítulo 5. El Capítulo 6 concreta luego esas decisiones en una implementación por etapas del camino principal $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$, cuyos resultados se evalúan en el Capítulo 7. Los fundamentos necesarios para sostener esa discusión pueden consultarse en los Apéndices A y B.

2. BRIDGES Y zkBridge

2.1. Bridges en la Blockchain

2.1.1. ¿Qué es un Bridge?

Las blockchains modernas se diseñan, en general, como sistemas autocontenidos: cada red mantiene su propio consenso, estado global y reglas de validación. Como consecuencia, una blockchain no puede verificar directamente el estado de otra sin algún mecanismo adicional. Esta limitación dificulta la *interoperabilidad*, es decir, la capacidad de transferir información o activos entre redes distintas.

Los Bridges (puentes) constituyen mecanismos diseñados para resolver este problema [1]. En términos generales, un Bridge permite que eventos ocurridos en una blockchain $\mathcal{A}$ puedan ser reconocidos y utilizados en otra blockchain $\mathcal{B}$. Esto habilita funcionalidades muy útiles para los usuarios, como:

- Realizar transferencias de activos entre blockchains.
- Ejecutar aplicaciones descentralizadas que utilizan múltiples blockchains.
- Sincronizar estados o mensajes entre sistemas distribuidos independientes.

Formalmente, es posible modelar un Bridge como un protocolo que conecta dos blockchains $\mathcal{A}$ y $\mathcal{B}$ mediante dos funciones principales:

- Detectar que un evento e ocurrió en el estado de $\mathcal{A}$.
- Demostrar o argumentar en la blockchain $\mathcal{B}$ que el evento e es válido según las reglas de $\mathcal{A}$.

Observamos que un Bridge es esencialmente *unidireccional*: es decir, se tiene una blockchain origen $\mathcal{A}$ donde ocurren eventos, hacia una blockchain destino $\mathcal{B}$ donde se actúa de forma acorde a esos eventos. Un Bridge no es agnóstico a los eventos; por el contrario, generalmente está diseñado para capturar una o más clases particulares de eventos.

Si se provee un Bridge unidireccional desde $\mathcal{A}$ hacia $\mathcal{B}$ y otro desde $\mathcal{B}$ hacia $\mathcal{A}$, podríamos decir que es un *Bridge bidireccional*.

2.1.2. Ejemplo de uso

El caso más común de interoperabilidad entre blockchains es la transferencia de activos. Supongamos que un usuario desea transferir un token desde una blockchain $\mathcal{A}$ hacia una blockchain $\mathcal{B}$. Una estrategia ampliamente utilizada es el modelo *lock-mint*:

1. El usuario bloquea un activo x en la blockchain $\mathcal{A}$.
2. El Bridge desde $\mathcal{A}$ hacia $\mathcal{B}$ detecta este evento.
3. En la blockchain $\mathcal{B}$ se emite (minte) un nuevo activo x' , que representa el activo original x bloqueado en $\mathcal{A}$.

Si posteriormente el usuario desea regresar el activo a $\mathcal{A}$, el proceso se invierte mediante el esquema *burn-unlock* (esto sería un Bridge de $\mathcal{B}$ hacia $\mathcal{A}$):

1. El usuario destruye (burn) el activo representado x' en $\mathcal{B}$.

2. El Bridge desde $\mathcal{B}$ hacia $\mathcal{A}$ detecta este evento.
3. El activo original x es desbloqueado en $\mathcal{A}$.

En este modelo, la seguridad del sistema depende de que el Bridge pueda verificar correctamente eventos entre cadenas. Si un atacante logra falsificar una prueba de bloqueo o destrucción, podría crear activos sin respaldo, comprometiendo completamente el sistema y destruyendo todo su valor.

2.1.3. Modelos de seguridad de Bridges

Existen múltiples formas de implementar el mecanismo de verificación entre blockchains. Estas implementaciones difieren principalmente en su modelo de confianza y en los supuestos criptográficos que utilizan.

Bridges basados en multisignature

Una de las implementaciones más simples consiste en delegar la verificación a un conjunto de operadores externos. En este modelo, un conjunto de entidades observa los eventos de una blockchain y firma colectivamente mensajes que certifican dichos eventos.

Una herramienta fundamental en este contexto es el esquema de firma múltiple *multi-signature*. En un esquema (t, n) -multisignature, un mensaje es válido si al menos t de los n participantes producen una firma válida. Si $\sigma_i = \text{Sign}_{sk_i}(m)$ denota la firma del i -ésimo participante sobre un mensaje m , entonces cualquier conjunto de la forma $\{\sigma_{i_1}, \dots, \sigma_{i_t}\}$ permite verificar la autenticidad de m .

En un Bridge multisignature, los operadores firman un mensaje que afirma que un evento ocurrió en la cadena de origen. La cadena de destino verifica que el mensaje posee al menos t firmas válidas. Este modelo es conceptualmente simple y eficiente, pero introduce un supuesto de confianza en los operadores. Si un subconjunto suficientemente grande es comprometido, el sistema puede ser atacado.

Bridges basados en *light clients*

Un enfoque más descentralizado consiste en que una blockchain verifique directamente el consenso de otra mediante un *light client*, idea que se remonta a la fundación de BITCOIN [2]. En este modelo, $\mathcal{B}$ mantiene una representación compacta del estado de $\mathcal{A}$. Típicamente se verifican encabezados de bloques, reglas del protocolo de consenso, e inclusión de transacciones.

Supongamos que las transacciones de un bloque se representan en forma compacta usando una raíz de Merkle r . Si se quiere probar que una transacción tx está incluida en un bloque de $\mathcal{A}$ con encabezado h y raíz r , se debería verificar que h efectivamente pertenece a la cadena consensuada de $\mathcal{A}$, y verificar una prueba de Merkle π de la inclusión de tx ; estas estructuras se definen formalmente en las Definiciones A.22 y A.23. La seguridad de este modelo depende entonces de la seguridad del consenso de la blockchain de origen.

Bridges basados en pruebas criptográficas

Otro enfoque es usar pruebas criptográficas sucintas, como zk-SNARK, definidos en la Definición B.4, para demostrar que un evento ocurrió en $\mathcal{A}$ [3]. En este modelo, un prover $\mathcal{P}$ genera una prueba o argumento que certifica que un bloque de $\mathcal{A}$ es válido según sus

reglas de consenso. Luego, se puede hacer un programa en $\mathcal{B}$ que actúe como el verificador $\mathcal{V}$ de la prueba generada.

Se conoce popularmente a un Bridge de esta clase como un zkBridge.

Composability

En la práctica, los sistemas de interoperabilidad suelen combinar varios de los modelos anteriores. Por ejemplo, es posible construir un Bridge donde un *light client* mantiene el estado de la cadena remota, mientras que un conjunto de operadores confiables actúa como relayers que transmiten información entre redes. Del mismo modo, sistemas basados en pruebas criptográficas pueden requerir operadores que generen las pruebas o mantengan la infraestructura necesaria para producirlas. Por esta razón, los modelos de seguridad de Bridges suelen ser *componibles*.

2.2. Problemas de seguridad en Bridges

A pesar de su importancia para la interoperabilidad entre blockchains, los Bridges han demostrado ser uno de los componentes más vulnerables del ecosistema. En los últimos años se han producido numerosos ataques que resultaron en pérdidas económicas de gran magnitud. Se estima que, desde 2020, los ataques a Bridges han ocasionado pérdidas por miles de millones de dólares, incluyendo incidentes individuales de cientos de millones [4].

Existen dos razones principales por las cuales estos sistemas constituyen objetivos particularmente atractivos para atacantes. En primer lugar, muchos diseños concentran grandes cantidades de activos en contratos custodios, lo que implica que una vulnerabilidad puede comprometer fondos significativos. En segundo lugar, los Bridges presentan una superficie de ataque amplia, ya que involucran múltiples componentes: contratos inteligentes desplegados en dos blockchains distintas, así como infraestructura off-chain encargada de transmitir eventos entre ellas.

Diversas clases de vulnerabilidades han sido explotadas en sistemas de interoperabilidad. Una de ellas corresponde a errores en la verificación de pruebas criptográficas utilizadas para demostrar eventos entre cadenas. Un ejemplo representativo ocurrió en el ataque al *Binance Bridge* en 2022, donde una falla en la verificación de Merkle Proofs permitió construir una prueba fraudulenta correspondiente a una transacción inexistente, y como consecuencia, el contrato en la cadena destino liberó activos que nunca habían sido bloqueados en la cadena de origen.

Otra categoría relevante corresponde a errores lógicos en los contratos inteligentes que implementan los componentes del Bridge. El ataque al *Nomad Bridge* en 2022 explotó una inicialización incorrecta del contrato que permitía aceptar mensajes inválidos como válidos. Una vez descubierto el exploit, múltiples usuarios replicaron la transacción maliciosa, lo que produjo pérdidas masivas en pocas horas.

También se han observado vulnerabilidades asociadas a actualizaciones del sistema o código heredado. En el caso del protocolo *Qubit Finance*, inconsistencias entre funciones antiguas y nuevas del contrato permitieron emitir eventos de depósito sin que los activos correspondientes hubieran sido transferidos realmente, lo que llevó a la creación de tokens sin respaldo.

Otra fuente frecuente de incidentes es el compromiso de claves privadas utilizadas por operadores del Bridge. Muchos sistemas dependen de validadores que firman mensajes para autorizar transferencias entre cadenas. Si un atacante obtiene suficientes claves privadas,

puede generar mensajes fraudulentos que el sistema aceptará como válidos. Este tipo de ataque ocurrió, por ejemplo, en el *Ronin Bridge*, donde los atacantes comprometieron múltiples validadores y autorizaron transferencias fraudulentas por un valor cercano a los 600 millones de dólares.

La recurrencia de estos ataques ha motivado el desarrollo de arquitecturas de interoperabilidad más robustas, ya que las vulnerabilidades explotadas no corresponden a debilidades criptográficas fundamentales, sino a errores de implementación o supuestos de confianza incorrectos. En particular, un zkBridge busca reducir la dependencia en operadores privilegiados y disminuir la superficie de ataque del sistema mediante pruebas criptográficas verificables. Esta idea prepara el análisis de la Sección 2.3, donde presentamos la arquitectura propuesta por el paper zkBridge.

2.3. Resumen del paper zkBridge

2.3.1. Esquema general de un zkBridge

El trabajo *zkBridge: Trustless Cross-chain Bridges Made Practical* [3] propone una arquitectura general de bridge *trustless* cuyo objetivo es permitir que una blockchain $\mathcal{B}$ verifique, de manera eficiente, hechos ocurridos en otra blockchain $\mathcal{A}$, sin depender de un comité privilegiado de validadores. La idea central es reemplazar la confianza en un comunicador por argumentos sucintos que certifiquen que la evolución del consenso de $\mathcal{A}$ fue correctamente verificada off-chain, y que su resultado puede ser aceptado on-chain en $\mathcal{B}$. El paper enfatiza que el problema principal de los *light clients* tradicionales no es conceptual sino práctico: verificar *headers* y mantener su estado on-chain puede ser demasiado costoso, especialmente cuando la cadena de origen usa consenso PoS o esquemas de firma poco amigables para aritmetización, como ocurre en varios sistemas modernos.

La base de la construcción es un *light client protocol*, siguiendo el modelo de seguridad introducido en la Sección 2.1.3. En el modelo del paper, un *light client* no almacena ni reejecuta toda la blockchain de origen, sino solamente la información necesaria para verificar una fracción de sus propiedades. El paper toma esta idea como punto de partida y observa que un bridge entre $\mathcal{A}$ y $\mathcal{B}$ puede reducirse a lo siguiente: mantener en $\mathcal{B}$ un estado compacto que represente el progreso del *light client* de $\mathcal{A}$, y luego usar ese estado para verificar eventos concretos (por ejemplo, que una transacción fue incluida en un bloque, o que cierto valor aparece en el estado de un contrato).

El obstáculo es que ejecutar directamente ese *light client* dentro de $\mathcal{B}$ suele ser demasiado caro. zkBridge traslada entonces esa verificación a uno o más provers off-chain, que generan argumentos sucintos de que la transición del *light client* fue correcta. En el diseño concreto del paper, esta tarea se organiza como una red de provers para paralelizar la generación de pruebas y sostener la propiedad de *liveness*; no es una condición conceptual de todo bridge basado en argumentos ZK, sino una decisión arquitectónica de esa construcción. A alto nivel, la arquitectura descrita por el paper tiene tres componentes:

1. Una red de provers, en el sentido del modelo del paper. Cualquier nodo de esta red puede consultar nodos completos de $\mathcal{A}$, obtener el siguiente *block header* h y construir un argumento ZK π_h .
2. Un contrato on-chain en $\mathcal{B}$, llamado *updater contract*, y que notaremos $\mathcal{SC}_{\text{updater}}$, el cual recibe π_h y, si es válida, actualiza el estado del *light client* en $\mathcal{B}$ para registrar el nuevo *header* h .

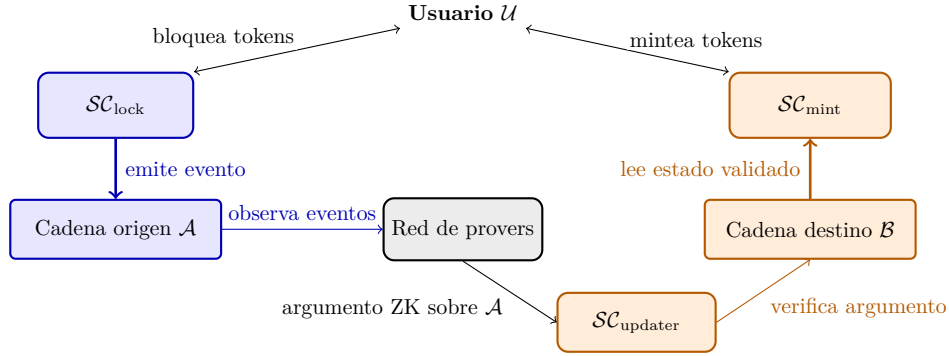


Fig. 2.1: Esquema operativo de un zkBridge entre una cadena origen $\mathcal{A}$ y una cadena destino $\mathcal{B}$, siguiendo el modelo del paper. El usuario bloquea activos en $\mathcal{SC}_{\text{lock}}$, una red de provers observa el evento y genera un argumento ZK sobre el estado de $\mathcal{A}$, el contrato $\mathcal{SC}_{\text{updater}}$ verifica ese argumento en $\mathcal{B}$, y el contrato $\mathcal{SC}_{\text{mint}}$ usa el estado autenticado para emitir los activos envueltos.

3. Contratos de aplicación desplegados en $\mathcal{B}$ que usan la información ya aceptada por $\mathcal{SC}_{\text{updater}}$ para realizar verificaciones específicas del bridge, como transferencias de activos. Esta separación modular es importante: el *updater* no sabe nada de la lógica particular de una aplicación, sino que solamente mantiene una vista autenticada, compacta y consistente de $\mathcal{A}$.

Conceptualmente, zkBridge desacopla la verificación del consenso de la cadena remota del uso de sus eventos. Como muestra la Figura 2.1, en el modelo del paper una red de provers genera argumentos criptográficos que certifican el estado de la cadena $\mathcal{A}$, los cuales son verificados por el contrato $\mathcal{SC}_{\text{updater}}$ en $\mathcal{B}$. Los contratos de aplicación pueden entonces consultar este estado autenticado para ejecutar operaciones cross-chain.

La intuición es la siguiente. Supongamos que queremos convencer a un contrato en $\mathcal{B}$ de que un cierto hecho ocurrió en $\mathcal{A}$. Primero, $\mathcal{SC}_{\text{updater}}$ debe haber aceptado el *header* del bloque correspondiente de $\mathcal{A}$ donde se registró el hecho. Luego, el contrato de aplicación obtiene ese *header* desde el *updater* y verifica el argumento ZK correspondiente. El problema queda así separado en dos capas:

1. Por un lado, demostrar que cierto *header* h pertenece a la historia válida de $\mathcal{A}$.
2. Por otro lado, demostrar que dentro de ese bloque o de ese estado autenticado ocurre el evento que nos interesa.

En una blockchain genérica, *consistencia* significa que los participantes honestos no terminan aceptando historias incompatibles como definitivas, mientras que *liveness* significa que el sistema no queda detenido: las transacciones o bloques válidos eventualmente pueden ser incorporados si la red sigue operando bajo sus supuestos. El modelo de seguridad del paper supone que ambas blockchains tienen estas propiedades, que la cadena de origen posee un *light client protocol* correcto, que el protocolo de argumentos ZK usado tiene la propiedad de solidez, y que existe al menos un nodo honesto en la red de provers. Bajo estas hipótesis, zkBridge obtiene heredadas las propiedades de consistencia y *liveness*:

- **Consistencia:** $\mathcal{B}$ no debe aceptar dos historias incompatibles de $\mathcal{A}$, ni aceptar un *header* que no pueda pertenecer a una ejecución válida del consenso de $\mathcal{A}$. En tér-

minos de bridge, esto evita que un contrato en $\mathcal{B}$ actúe sobre eventos pertenecientes a una historia falsa o conflictiva.

- **Liveness:** si $\mathcal{A}$ sigue produciendo bloques y existe al menos un prover honesto capaz de observarlos y generar argumentos válidos, entonces el estado autenticado mantenido en $\mathcal{B}$ eventualmente puede avanzar. En términos de bridge, esto evita que el sistema quede permanentemente congelado pese a que la cadena origen progresa.

2.3.2. Un zkBridge en la práctica

Hasta aquí, la propuesta sería conceptualmente atractiva pero todavía podría ser impracticable. El punto técnico central del paper es justamente mostrar cómo hacer que la generación de estos argumentos sea suficientemente rápida. El cuello de botella aparece porque verificar el consenso de algunas blockchains dentro de un circuito aritmético puede requerir verificar cientos de firmas digitales y otras operaciones costosas. El paper observa, sin embargo, que estos circuitos suelen tener una estructura paralela: contienen muchas copias idénticas de un mismo subcircuito pequeño. A partir de esta observación, los autores diseñan *deVirgo*, un protocolo de argumentos distribuido que explota esa estructura repetitiva para paralelizar la generación de argumentos casi linealmente en la cantidad de máquinas disponibles.

Aunque *deVirgo* acelera fuertemente al prover, el tamaño de los argumentos sigue siendo demasiado grande para que puedan ser verificados cómodamente on-chain. Para resolver esto, los autores aplican una capa recursiva adicional: primero generan un argumento grande con *deVirgo*, y luego demuestran con *Groth16* que el argumento generado con *deVirgo* es válido. Así, *Groth16*—desarrollado en la Sección B.9— se usa para comprimir un argumento ya generado por un sistema más escalable para el prover, lo que se traduce en una verificación on-chain barata (argumentos de tamaño constante, y tiempo de verificación bajo). Esta idea se conoce como *proof recursion*. En esta tesis, sin embargo, esa ruta queda como antecedente conceptual del paper original: el prototipo final del Capítulo 6 no implementa recursión de pruebas, sino que adapta la idea general de verificación sucinta a CARDANO mediante una verificación STM Halo2/MIDNIGHT particionada en dos transacciones y circuitos *Groth16* dedicados para pertenencia a *snapshots* y actualización del SMT anti-replay.

Finalmente, el paper valida empíricamente esta arquitectura implementando un prototipo: un bridge desde Cosmos hacia ETHEREUM. Los resultados muestran que la compresión recursiva reduce drásticamente tanto el tamaño del argumento como el costo de verificación on-chain, lo que hace viable el uso de este esquema en blockchains programables como ETHEREUM.

3. CARDANO Y MODELO eUTxO

Este capítulo introduce las capas de CARDANO que condicionarán y darán forma al modelo general de zkBridge presentado en la Sección 2.3.1: el modelo eUTxO, la capa de ejecución de smart contracts (PLUTUS/UPLC y herramientas contemporáneas como Aiken), los límites de recursos a nivel de protocolo (tamaño de transacción/bloque y presupuestos de unidades de ejecución), y el protocolo de consenso (OUROBOROS PRAOS). El objetivo no es proponer aún un diseño de puente, sino establecer el espacio de diseño técnico y los límites estrictos. Para nuestro proyecto, CARDANO es una blockchain de interés porque:

1. Combina un libro contable de estilo UTxO con scripts de validación expresivos (el modelo eUTxO), lo que modifica los patrones típicos de diseño.
2. Fue construido en torno a la validación determinista de transacciones y a una contabilidad explícita de recursos, lo que hace que los costos en el peor caso y los modos de fallo sean más predecibles, pero también introduce límites estrictos.

CARDANO es una blockchain donde los bloques son producidos por *stake pools* en nombre de los delegadores, bajo la familia de protocolos de consenso OUROBOROS. Los *stake pools* proporcionan alta disponibilidad y conectividad; los delegadores aportan stake y comparten recompensas, mientras que un *stake pool operator* ejecuta la infraestructura que participa en la producción de bloques.

A nivel de libro contable, CARDANO sigue un paradigma UTxO (*Unspent Transaction Output*, como BITCOIN): las transacciones consumen *outputs* previamente no gastados y crean nuevos *outputs* que podrán gastarse en el futuro. Cada UTxO puede consumirse *una sola vez y en su totalidad*. CARDANO extiende este modelo mediante eUTxO, permitiendo que los *outputs* lleven datos y que las direcciones estén respaldadas por *scripts*, lo que habilita contratos inteligentes expresivos sin adoptar un modelo global de estado completamente basado en cuentas (como el de ETHEREUM).

En CARDANO, la validación de transacciones es determinista. Intuitivamente: si una transacción está bien formada y hace referencia a un conjunto válido de entradas, su resultado de validación (éxito/fracaso) y sus comisiones pueden saberse de forma exacta antes de enviarla on-chain. Esto es gracias a tener scripts puros que devuelven *True/False*, argumentos explícitos fijados dentro de la transacción, y presupuestos explícitos de recursos que limitan la ejecución de scripts.

Para un zkBridge, el determinismo es valioso porque los Bridge operators pueden precomputar costos y determinar el éxito off-chain. El determinismo también conlleva parámetros concretos del protocolo que limitan el cómputo y los datos permitidos por transacción/bloque, analizados en la Sección 3.3.

3.1. El modelo eUTxO

3.1.1. De UTxO a eUTxO: validadores, datums, redeemers, contexto

En el modelo UTxO tradicional, un *output* es principalmente un par (*dirección, valor*). Gastar un *output* requiere proporcionar el witness necesario para desbloquear esa dirección (usualmente una firma). CARDANO generaliza esto en dos direcciones clave: las

direcciones pueden incorporar lógica arbitraria (scripts), y los *outputs* pueden transportar datos específicos interpretables por los scripts.

Esto es formalizado [5] como una extensión donde cada *output* contiene un *script* de validación (o su hash) y un *datum*, y para poder gastarlo se proporciona un *redeemer*. Los validadores reciben información adicional sobre la transacción en la que se gasta el *output* (*contexto de transacción*, por ejemplo, otros *outputs* gastados en esa misma transacción). Se puede pensar a los *scripts* de validación como contratos. Hay dos características necesarias para codificar máquinas de estados como *scripts* de validación de CARDANO:

- **Transporte de estado:** los *outputs* incluyen un datum δ para almacenar datos específicos del contrato (estado).
- **Continuidad del contrato:** los validadores pueden inspeccionar una transacción para asegurar que el script continúa con la lógica de validación prevista.

Una forma compacta de expresar la idea central de validación en eUTxO para *spending validators* (como se presenta en el artículo) es: $\nu(\text{UtxoRef}, \delta, \rho, \text{tx}) = \text{True}$ donde ν es el validador, δ es el datum, ρ es el redeemer, y tx es la transacción que gasta el *output* (o un contexto derivado de ella).

En el caso de *minting validators*, la interfaz local cambia porque el script no valida el gasto de un UTxO concreto sino una política de acuñación. En ese caso, usaremos la notación $\nu_{\text{mint}}(\text{PolicyId}, \rho, \text{tx}) = \text{True}$ donde el parámetro *policyId* identifica la política que autoriza la operación de *minting*.

Esto es relevante para el diseño de zkBridge porque típicamente necesita estado persistente en cadena (por ejemplo, el estado de un *light client* o un commitment del mismo), e invariantes fuertes a través de actualizaciones (por ejemplo, protección contra repetición de transacciones, y correcta secuenciación de transiciones de estado). El modelo eUTxO proporciona un mecanismo explícito y analizable para ambos, pero el estado debe representarse y actualizarse utilizando UTxOs que se gastan y generan. Esto implica que no existe un estado global mutable compartido, sino una secuencia explícita de transiciones de estado.

3.1.2. Máquinas de estado en eUTxO

En [5] se argumenta que el modelo soporta patrones de contrato expresivos, incluyendo máquinas de estado generales, mientras preserva las propiedades del modelo UTxO y evita el estado global mutable compartido. Lo demuestra mediante una construcción formal (*Constraint Emitting Machines*) y enfatiza en que los autores de contratos pueden imponer invariantes sobre cadenas completas de transacciones.

Una conclusión práctica es que muchas aplicaciones en CARDANO modelan el estado de la aplicación como un pequeño conjunto de UTxOs que modelan estado. Cada actualización consume un UTxO de estado y produce uno nuevo que codifica el siguiente estado. Esto es directamente relevante para un zkBridge, donde un modelado natural es que un UTxO contiene el estado canónico del Bridge (por ejemplo, el último commitment aceptado), mientras que cada actualización lo consume y crea un UTxO de estado sucesor. La ventaja es la claridad: el propio ledger impone restricciones de uso único y de orden. Este patrón resulta particularmente adecuado para aplicaciones que requieren fuertes garantías de consistencia, como es el caso de los Bridges.

3.1.3. Características de eUTxO relevantes para aplicaciones complejas

Más allá del modelo conceptual, CARDANO ha evolucionado con características pragmáticas del *ledger* diseñadas para reducir el tamaño de las transacciones y mejorar el rendimiento de aplicaciones intensivas en *scripts*. El manual de eUTxO [6] y la documentación de CARDANO destacan (entre otras) las siguientes características existentes desde la era Vasil:

- **Reference Inputs (CIP-31):** permiten inspeccionar UTxOs sin gastarlos.
- **Inline Datums (CIP-32):** permiten adjuntar datums directamente (no solo por hash).
- **Reference Scripts (CIP-33):** permiten reutilizar scripts sin tener que volver a proporcionarlos en cada transacción.

3.2. Smart Contracts en Cardano: Plutus y Aiken

3.2.1. Plutus, Plutus Core y PlutusTx

La validación on-chain de CARDANO ejecuta en última instancia *Untyped Plutus Core* (UPLC), la representación de bajo nivel evaluada por el motor de *smart contracts* del nodo. La creación de contratos a alto nivel no está limitada a un solo lenguaje, siempre que la cadena de herramientas compile a UPLC.

Históricamente, PLUTUS en CARDANO se ha utilizado de forma ambigua: puede referirse a Plutus Core/UPLC como formato de ejecución, a PlutusTx/Plinth como una vía de compilación basada en Haskell, o a la plataforma/herramientas más amplia de PLUTUS alrededor de estos componentes.

El manual de eUTxO [6] describe de manera similar a PLUTUS como el lenguaje de *scripting* de CARDANO y señala que los autores normalmente no escriben PLUTUS directamente; en su lugar, un *plugin* del compilador de Haskell genera el código on-chain. Los *scripts* de PLUTUS son validadores puros, ejecutados durante la validación de transacciones, que devuelven **True/False**. Las transacciones llevan *redeemers* y UTxOs con estado en sus *datum*, donde se permite una evaluación determinista y la predicción de comisiones para la futura ejecución on-chain.

3.2.2. Aiken

Aiken [7] es un lenguaje moderno, diseñado específicamente para escribir scripts validadores en CARDANO. Es un lenguaje funcional puro, pequeño, pensado para la robustez y la facilidad de uso, que enfatiza un tipado estático fuerte con inferencia de tipos, módulos y una experiencia moderna de herramientas. Es importante destacar que Aiken compila directamente a UPLC (el formato on-chain), sin requerir Haskell. Se destacan varias ventajas frente a PLUTUS:

- **Menor barrera de entrada:** está diseñado como un lenguaje y toolchain inspirado en Rust (pese a ser un lenguaje funcional); que compila directamente a UPLC.
- **Herramientas y ciclos de retroalimentación:** Aiken integra compilación, tests, reportes de traza y de unidades de ejecución de los nodos de cardano, lo que proporciona herramientas útiles al desarrollador. Además, tiene soporte LSP.

- **Simplicidad deliberada:** Aiken busca mantenerse pequeño y evita características avanzadas del sistema de tipos (por ejemplo, *typeclasses*). Al mismo tiempo, permite extender el sistema de tipos al desarrollador, y ofrece inferencia de tipos.

Para una base de código de **zkBridge**, estas diferencias pueden impactar directamente en la rapidez con la que el equipo puede iterar sobre la lógica de verificación on-chain, qué tan confiablemente se puede medir y optimizar el uso de recursos (unidades de ejecución) antes del despliegue, y qué tan mantenible y auditable permanece el código del validador a lo largo del tiempo. Las restricciones on-chain no desaparecen, pero mejores herramientas pueden reducir sustancialmente el costo de trabajar dentro de esas limitaciones.

3.3. Límites del protocolo y su importancia en un zkBridge

3.3.1. Contabilización de recursos: unidades de ejecución, determinismo y validación en dos fases

CARDANO utiliza una contabilización explícita de recursos para scripts no nativos (fase 2), expresada en *unidades de ejecución* (**ExUnits**) que registran métricas como el uso de memoria y pasos abstractos de ejecución. Las transacciones especifican un presupuesto de unidades de ejecución, y el nodo que las ejecuta verifica que estos respeten los límites del protocolo. Este diseño contribuye al determinismo y *garantiza* la terminación de los scripts.

Para limitar vectores de DoS y trabajo no compensado en los nodos, CARDANO implementa un esquema de validación en dos fases y los *collateral inputs*: la fase 1 verifica que la transacción esté bien formada y pueda pagar comisiones, mientras que la fase 2 evalúa los scripts. Si la fase 2 falla, *collateral inputs* (UTxOs especiales referenciados por la transacción) se utilizan para compensar a la red por el trabajo realizado.

Para un **zkBridge**, esto tiene una consecuencia operativa: cualquier transacción que incluya una verificación de prueba pesada debe construirse con presupuestos correctos y *collateral input* suficiente, y debe mantenerse válida respecto al estado on-chain.

3.3.2. Límites de tamaño y cómputo

Una transacción de actualización de un **zkBridge** típicamente necesita incluir (directa o indirectamente) pruebas de conocimiento cero, entradas públicas y referencias al estado actual del Bridge. Por lo tanto, tanto los límites de *tamaño en bytes* como los de **ExUnits** son restricciones de diseño de los validadores.

La Tabla 3.1 resume parámetros del protocolo comúnmente citados que acotan el rendimiento y la complejidad. Los valores numéricos exactos pueden cambiar mediante actualizaciones del protocolo; el punto clave para el diseño es que estos son *límites duros* impuestos por las reglas de consenso. Los valores pueden ser modificados mediante el mecanismo de gobernanza del protocolo. En los límites de **ExUnits**, Cardano distingue dos componentes: unidades de memoria y pasos de CPU.

En particular, estos límites imponen restricciones directas sobre el tamaño máximo de pruebas verificables on-chain y sobre la complejidad de los validadores.

- El tamaño de la transacción incluye al witness, los redeemers, los datums (especialmente inline datums), metadatos y reference scripts. Incluso si los argumentos producidos por **Groth16** son sucintos, la carga completa y actualización puede acercarse

Parámetro	Significado	Valor
<code>maxTxSize</code>	Límite de una tx serializada (bytes)	16,384
<code>maxBlockBodySize</code>	Límite de un block body serializado (bytes)	90,112
<code>maxBlockHeaderSize</code>	Límite de un block header serializado (bytes)	1,100
<code>maxTxExUnits</code>	Límite de <code>ExUnits</code> por tx	memoria: 1.4e7 CPU: 1e10
<code>maxBlockExUnits</code>	Límite de <code>ExUnits</code> por block	memoria: 5e7 CPU: 4e10
<code>collateral Percentage</code>	Colateral necesario como % de comisiones	150
<code>maxCollateral Inputs</code>	Límite de collateral inputs por tx	3

Tab. 3.1: Parámetros de protocolo que imponen límites de recursos relevantes a aplicaciones con validation scripts pesados.

a `maxTxSize`. Los reference scripts y reference inputs pueden reducir la sobrecarga de bytes repetida entre transacciones.

- Cualquier verificación de prueba on-chain debe encajar dentro de `maxTxExUnits`. Además, el límite a nivel de bloque `maxBlockExUnits` hace que sea más lento para los nodos que elijan la transacción pesada una vez que esta llegue a la mempool.

3.3.3. Cómo los límites estrictos moldean el espacio de diseño

Aunque posponemos la discusión detallada de las decisiones de diseño para el Capítulo 5, es útil explicar las consecuencias típicas que estos límites imponen sobre un `zkBridge`. El modelo `eUTxO` representa naturalmente el estado como un `UTxO`. Actualizar ese estado requiere consumirlo, lo que impone una forma de secuencialidad sobre las actualizaciones del estado canónico del `Bridge`. Por lo tanto, el throughput está acotado tanto por la contención del `UTxO` como por los límites de recursos por bloque.

Además, la verificación ZK puede ser computacionalmente costosa dependiendo del sistema de prueba y de las primitivas criptográficas disponibles en la máquina virtual on-chain. Si una verificación directa no entra en `maxTxExUnits`, un `Bridge` debe elegir al menos una de las siguientes alternativas:

1. Un sistema de pruebas más amigable para verificadores auxiliares acotados. En esta tesis, `Groth16` se usa en las relaciones de pertenencia a `snapshots` y actualización del SMT anti-replay, desarrolladas respectivamente en las Secciones 6.9 y 6.8, debido a su verificación eficiente y tamaño de prueba constante.
2. Reducir el trabajo on-chain usando proof recursion o proof aggregation.
3. Dividir la verificación en múltiples transacciones cuando el objeto criptográfico completo no entra en presupuesto. Este es el caso de los certificados MITHRIL, cuya verificación STM sigue el camino Halo2/MIDNIGHT y requiere la partición en dos fases de la Sección 6.7.3.

3.4. Praos y el modelo operativo de proof-of-stake

3.4.1. Proof of stake, stake pools y SPOs

En CARDANO, la participación en el consenso se realiza a través de *stake pools*. Un stake pool es un nodo servidor que mantiene el ledger y produce bloques, además de agregar el

stake de los delegadores. El operador del pool ejecuta la infraestructura del nodo, mientras que los delegadores adoptan un rol pasivo delegando su stake. Las recompensas provienen de comisiones de transacción y expansión monetaria, y se distribuyen en cada época con una división explícita entre costos/margen del operador y los delegadores.

El proceso de recompensas es estocástico porque la producción de bloques es probabilística: los pools son seleccionados para producir bloques con una probabilidad monótona creciente (de una forma definida por el protocolo) a su stake.

3.4.2. Estructura temporal: slots y épocas

OUROBOROS PRAOS [8] es un protocolo *proof of stake* que divide el tiempo en *slots* agrupados en *épocas*. En cada *slot*, puede haber uno o más líderes elegibles, y muchos *slot* pueden estar vacíos (sin líder). Esto difiere de los esquemas deterministas de selección de líderes y es clave para las suposiciones de seguridad y red de OUROBOROS PRAOS.

En el paper de OUROBOROS PRAOS, la elección de líderes se describe como una *prueba local privada* utilizando una VRF (*Verifiable Random Function*, o función aleatoria verificable) sobre el timestamp actual y un η específico de la época. Solo la parte elegida sabe que es líder hasta que produce un bloque que contiene la prueba.

El coeficiente de slot activo de CARDANO (denotado f) controla con qué frecuencia se espera que los slots contengan un bloque. En los parámetros públicos usuales, se apunta a producir un bloque cada aproximadamente veinte segundos.

3.4.3. Elección de líderes mediante VRF

El paper de OUROBOROS PRAOS formaliza la probabilidad de elección de líder en términos de la fracción de stake α y el coeficiente de slot activo f . Específicamente, la función de elección de líder es: $\phi_f(\alpha) = 1 - (1 - f)^\alpha$. Esta formulación garantiza (entre otras propiedades) *agregación independiente*: dividir el stake entre muchas identidades no incrementa trivialmente la probabilidad de liderazgo más allá del stake combinado. Este mecanismo evita la necesidad de coordinación global para la elección de líderes.

El mecanismo VRF cumple dos roles clave:

- **Impredecibilidad:** los líderes no se conocen de antemano, reduciendo ataques dirigidos a futuros líderes de slots.
- **Verificabilidad pública** una vez producido un bloque, otros nodos pueden verificar la prueba VRF y aceptar la elegibilidad del líder.

3.4.4. KES y certificados operativos

OUROBOROS PRAOS está diseñado para resistir adversarios adaptativos de forma más robusta que variantes anteriores mediante el uso de *Key Evolving Signatures (KES)* y propiedades mejoradas de aleatoriedad/VRF. El paper de OUROBOROS PRAOS desarrolla explícitamente las firmas evolutivas dentro de un marco componible y enfatiza que la seguridad hacia adelante ayuda a prevenir ataques de “retrodatación” en los que un adversario intenta reescribir la historia usando claves comprometidas.

La práctica operativa de CARDANO refleja estas ideas mediante distintos tipos de claves y certificados:

- **Claves frías (offline) del operador** utilizadas para emitir certificados.

- **Claves KES (hot)** utilizadas en la producción de bloques y rotadas periódicamente; la documentación señala la necesidad operativa de actualizarlas regularmente (por ejemplo, “cada 90 días”).
- **Claves VRF** almacenadas en certificados operativos y utilizadas para demostrar la elegibilidad de liderazgo de slots en OUROBOROS PRAOS.

3.4.5. *Light clients* en Cardano

Una dificultad fundamental en el diseño de **Bridges** sobre CARDANO es la obtención de una representación verificable del estado de la blockchain sin volver a ejecutar completamente el protocolo de consenso. Es decir, necesitamos usar *light clients*, conforme al modelo presentado en la Sección 2.1.3.

Dentro del ecosistema de CARDANO, el protocolo MITHRIL [9] propone una solución en esta dirección mediante el uso de firmas agregadas ponderadas por stake para certificar snapshots del estado de la blockchain. En lugar de requerir el procesamiento completo de la cadena o la verificación explícita de la lógica de consenso, MITHRIL permite trabajar con certificados compactos que argumentan la validez de un estado acordado por una fracción significativa del stake de la red. Esto permite diseñar mecanismos de verificación más eficientes, tanto en términos de tamaño de datos como de costo computacional on-chain.

Esto permite diseñar mecanismos de verificación más eficientes, tanto en términos de tamaño de datos como de costo computacional on-chain, lo cual resulta especialmente relevante en el contexto de las restricciones del modelo eUTxO. El protocolo MITHRIL se analiza en profundidad en el Capítulo 4.

3.5. Direcciones futuras

3.5.1. Peras

OUROBOROS PERAS se describe como una extensión de OUROBOROS PRAOS que reduce el horizonte de finalización introduciendo votaciones periódicas por parte de un comité de nodos productores de bloques para certificar bloques. Los bloques certificados reducen más rápidamente la probabilidad de *rollback*, haciendo que la cadena sea efectivamente “irreversible” antes bajo los supuestos del protocolo. Esto se logra a partir de rondas de votación del orden de uno a dos minutos y destacan explícitamente a los **Bridges** como uno de los principales casos de uso beneficiados.

3.5.2. Leios

OUROBOROS LEIOS se plantea como un rediseño sustancial orientado a incrementar el *throughput* manteniendo fuertes propiedades de seguridad. Los materiales públicos sobre OUROBOROS LEIOS describen multiplicadores significativos de *throughput* y discuten *trade-offs* como un aumento en la latencia y en el uso de recursos; el resumen del paper de IOG enfatiza que las variantes actuales de OUROBOROS están limitadas no por los recursos de hardware en bruto, sino por dependencias de datos y comunicación del protocolo, lo que motiva un rediseño del algoritmo.

3.6. Conclusiones

En conjunto, el modelo $eUTxO$, los límites estrictos de recursos y el protocolo de consenso de CARDANO definen un espacio de diseño altamente restringido pero bien estructurado. Estas características condicionan fuertemente las decisiones del $zkBridge$, desarrolladas en el Capítulo 5 después de introducir la capa de certificación de MITHRIL.

4. MITHRIL

4.1. Introducción y motivación

Un bridge entre cadenas necesita decidir cuándo puede tratar como auténtico un estado que se originó en otra red. En este trabajo, el `zkBridge` debe utilizar información derivada de CARDANO sin reejecutar dentro de un circuito ZK todas las reglas de su consenso. Este capítulo estudia la capa de certificación que hace posible esa decisión y prepara las interfaces que luego utilizarán el diseño del Capítulo 5 y la implementación del Capítulo 6.

La decisión arquitectónica es utilizar MITHRIL como mecanismo intermedio de autenticación de estado [9, 10]. MITHRIL no sustituye al consenso de CARDANO: no decide qué cadena es válida ni produce bloques. Cumple, en cambio, un papel similar al de un *light client*, porque certifica ciertos datos derivados del estado de CARDANO para que un sistema externo pueda tratarlos como objetos autenticados.

En el `zkBridge`, esta evidencia se consume en dos capas. La capa de actualización mantiene en $\mathcal{B}$ un UTxO canónico de `StakeDist`, inicializado con un certificado génesis y renovado mediante certificados estándar de MITHRIL. La capa de aplicación utiliza certificados de *snapshots* de transacciones para justificar reclamos puntuales, como la pertenencia de una tx_{lock} a un conjunto certificado. Esta separación permite conservar el estado de certificación que debe persistir y transportar sólo la evidencia necesaria para cada operación.

En la terminología de MITHRIL, una *entidad firmada* no es un actor del protocolo, sino el tipo de dato que un certificado autentica. `MithrilStakeDistribution` designa la distribución de participación propia del protocolo: un artefacto serializable, consultable mediante la API, que enumera los *signers* registrados junto con su *stake* y sus claves de verificación. El certificado asociado autentica ese comité y los parámetros de la época, y se utiliza para mantener el estado persistente de la capa de actualización. `CardanoTransactions`, por su parte, designa el compromiso global de un conjunto de transacciones, representado por una raíz de Merkle. Su certificado se materializa en un cert_{ts} y se utiliza para justificar pruebas puntuales de pertenencia, como la inclusión de una tx_{lock} . La primera evidencia debe persistir en el bridge; la segunda se transporta sólo durante la operación que la consume [10, 11].

Para fijar el mapa conceptual, distinguimos cuatro actores. Los *signers* son nodos operados por participantes que aportan *stake* y producen firmas individuales. El *aggregator* es una instancia del software de MITHRIL ejecutada como nodo o servicio de red: coordina a los *signers*, recibe sus registros y firmas, construye certificados y los expone mediante una API. Su rol no exige aportar *stake* ni integrar el comité firmante, y sus resultados no se aceptan por confianza en el nodo, sino después de verificar criptográficamente los certificados. El *verifier* o cliente realiza esa validación, incluida la de la `CertChain`; finalmente, el operador del `zkBridge` consume los artefactos verificados para preparar argumentos ZK y actualizar el estado del puente [12]. Una *era* identifica una versión o carril criptográfico del protocolo, como `Pythagoras` o `Lagrange`. El *dispatch* por era es la selección de la ruta de hash, firma y verificación que corresponde a la era indicada por el certificado.

El término *registro* se utiliza en dos niveles relacionados. Cada *signer* aporta una entrada individual con su clave de verificación y la información de *stake* necesaria para participar. Una vez reunidas las entradas correspondientes a una época, el conjunto se

cierra y forma un registro autenticado colectivo, que denotamos por Reg^* ; de él se derivan el comité y la clave de verificación agregada AVK. Por lo tanto, el registro cerrado no pertenece a un firmante particular: reúne y autentica las inscripciones individuales del comité de esa época [13].

El punto de partida de la cadena es un certificado génesis, que actúa como raíz de confianza y se verifica con la clave génesis vk_{gen} ; en la notación de este capítulo lo denotamos por cert_{gen} . Las actualizaciones estándar quedan ligadas a un recibo persistente que certifica que una prueba STM ya fue verificada; en la implementación, este objeto se representa mediante un `proof_receipt`. Cada recibo está asociado a un dominio, que identifica el tipo de certificado y el flujo que se está verificando. En este trabajo usamos, entre otros, el dominio `stake_distribution_standard` para las actualizaciones del estado de participación y el dominio `cardano_transactions` para los certificados de *snapshots* de transacciones.

La motivación criptográfica es evitar que el bridge tenga que incorporar detalles de alto costo y fuerte inestabilidad semántica de OUROBOROS PRAOS, como VRF, firmas KES, nonces de época, selección de líderes y reglas de cadena. En su lugar, MITHRIL expone el registro autenticado colectivo, claves agregadas de verificación, cadenas de certificados y mensajes de protocolo con estructura fija. Esta interfaz es suficientemente fuerte para autenticar el estado relevante y suficientemente compacta para ser reutilizada por la capa de aplicación del zkBridge.

La notación que usamos en este capítulo es una abstracción propia de la tesis. Su propósito es conservar la semántica criptográfica que necesita el zkBridge: participantes ponderados por *stake*, registros autenticados, mensajes certificados, firmas agregadas, testigos, entradas públicas y relaciones verificables por circuitos. No vamos a reproducir uno a uno los tipos, módulos o nombres internos de la implementación de MITHRIL; los detalles de serialización, *dispatch* por era, compatibilidad de hashes y estructuras auxiliares del *aggregator* sólo aparecen cuando afectan esa semántica.

La Sección 4.2 describe el protocolo STM y sus esquemas de firma; la Sección 4.3 formaliza la cadena de certificados; y las Secciones 4.4 y 4.5 identifican los certificados relevantes y presentan la interfaz del *aggregator*. Sobre esa base, la Sección 4.6 formula la relación NP del circuito STM y la Sección 4.7 estudia su implementación en Halo2 con KZG. Finalmente, el Capítulo 5 decide qué estado y qué evidencia debe usar el zkBridge, y el Capítulo 6 muestra cómo esas decisiones se concretan con `proof_receipt` de verificación STM y argumentos Groth16 auxiliares.

El flujo abstracto entre estos actores se resume en la Figura 4.1. La figura no pretende describir los formatos internos: sólo muestra que el *aggregator* recibe evidencia de los *signers*, que un cliente o el operador del bridge verifica los certificados, y que el *dispatch* determina qué ruta criptográfica debe aplicarse.

4.2. Protocolo STM y esquemas de firma

El núcleo criptográfico de MITHRIL es un esquema de *stake-based threshold multisignatures* (STM), cuya meta es convencer a un *verifier* de que una fracción suficiente del *stake* del sistema firmó un mensaje dado [9]. No queremos pedir firmas a todos los participantes activos de la red, sino obtener una evidencia agregada a partir de una lotería ponderada por *stake*. En esta sección primero describimos esa lotería en términos conceptuales; luego distinguimos las eras y rutas criptográficas relevantes; después fijamos la notación básica

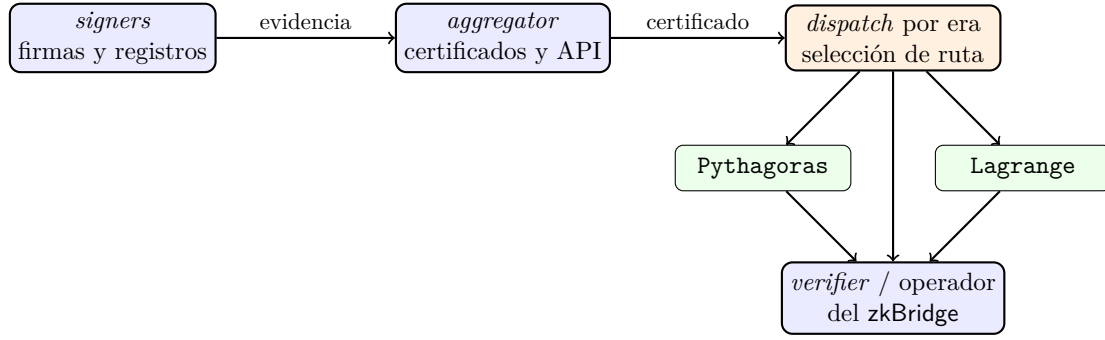


Fig. 4.1: Flujo conceptual de certificación de MITHRIL. Los *signers* aportan registros y firmas individuales al *aggregator*, que reúne esa evidencia, construye certificados y los expone mediante su API. Un *dispatch* por era selecciona la ruta criptográfica correspondiente, Pythagoras o Lagrange, y el *verifier* o el operador del zkBridge valida el certificado y utiliza el resultado para preparar argumentos ZK o actualizar el estado del puente.

del protocolo; y recién al final pasamos al detalle de la ruta más útil para circuitos, *public inputs* estructurados y pruebas ZK.

4.2.1. Intuición de la lotería y del umbral

Supongamos que, para una época determinada, los *signers* ya aportaron sus entradas individuales y que ese conjunto fue cerrado como un registro autenticado Reg^* . Este registro colectivo determina el comité y la clave agregada de verificación AVK que se utilizarán durante la época. Cuando aparece un mensaje M , por ejemplo la raíz de un *snapshot* de CARDANO, cada participante del comité evalúa m posiciones de una lotería pública. Para cada posición, la función de elegibilidad combina el *stake* relativo del participante con el parámetro ϕ_f . Un participante puede no ganar ninguna posición, ganar una o ganar varias. Si gana, firma M y envía al *aggregator* la firma junto con los índices ganadores y las pruebas que permiten verificar su pertenencia al comité y su elegibilidad. El *aggregator* descarta evidencias inválidas, resuelve duplicados y conserva como máximo una justificación por índice. La agregación sólo es válida cuando reúne al menos k índices distintos [13, 9].

En una primera lectura, STM puede pensarse como una lotería pública ponderada por *stake*. El comité y la clave de verificación agregada no se vuelven a sortear para cada índice: quedan fijados por el registro autenticado de la instancia o época correspondiente. Lo que cambia para cada mensaje es la evaluación de las posiciones de la lotería, porque la elegibilidad depende del mensaje firmado y del índice que se está justificando.

La unidad que se cuenta es la *cantidad de índices de lotería ganadores distintos*. En particular, **no** es simplemente la cantidad de participantes que enviaron firmas. El protocolo tiene un parámetro m , que fija el conjunto de índices posibles $J = \{0, \dots, m-1\}$, y un parámetro k , que indica cuántos de esos índices deben quedar justificados para aceptar la agregación. Para un participante i con peso relativo $w_i = s_i/S$, la probabilidad idealizada de ganar un índice es

$$\Pr[\text{ganar un índice} \mid i] = 1 - (1 - \phi_f)^{w_i}.$$

Por lo tanto, cuanto mayor es ϕ_f , mayor es la probabilidad de elegibilidad; el umbral de

aceptación, sin embargo, sigue siendo el número k de índices distintos y no un simple conteo de firmas.

Así, para un mensaje M , un participante i puede ganar cero, uno o varios índices. Si no gana ninguno, no aporta evidencia útil para esa operación. Si gana al menos uno, envía una firma individual junto con la información necesaria para justificar esos índices. El *aggregator* reúne firmas, descarta las inválidas, resuelve colisiones cuando varios participantes justifican el mismo índice y sólo puede producir una evidencia agregada válida cuando alcanza al menos k índices distintos. En la ruta **Concatenation**, la prueba de cada firma incluye además la información de registro y los caminos de Merkle necesarios para comprobar que el participante pertenece al comité autenticado [14].

Esta distinción es importante para el zkBridge: no alcanza con decir que se acumularon muchas firmas si todas explican las mismas posiciones de lotería. Lo que el certificado STM debe expresar es que un subconjunto autenticado del comité firmó el mensaje y que, bajo la lotería ponderada por *stake*, quedaron cubiertos al menos k índices válidos y distintos.

4.2.2. Eras y rutas criptográficas en Mithril

Antes de pasar al detalle matemático conviene separar tres niveles de vocabulario. En la implementación de MITHRIL, una *era* identifica una versión del protocolo soportada por el software. En la copia local revisada del repositorio, las eras explícitas son **Pythagoras** y **Lagrange**. Una *ruta criptográfica*, en cambio, describe cómo se representa y verifica la evidencia agregada dentro de una era. Finalmente, un esquema de firma como Schnorr o BLS12-381 no es una era por sí mismo, sino una primitiva usada por alguna ruta.

En el flujo de transacciones de MITHRIL, **CardanoTransactions** identifica la entidad firmada que certifica un conjunto de transacciones de CARDANO mediante una raíz de Merkle, asociada a una época y a un punto de corte de la cadena [11]. No representa una transacción individual.

En esta tesis, **Pythagoras** designa el camino productivo pre-**Lagrange** que consume el prototipo integrado. A efectos de la exposición, cuando hablemos del camino *legacy* de MITHRIL para **CardanoTransactions**, lo identificamos con **Pythagoras**: es el carril disponible hoy para el flujo real del *aggregator* y usa el tipo de agregación **Concatenation**, que combina las firmas individuales BLS12-381 de los participantes elegibles en una firma agregada verificable bajo la AVK [14]. El término *legacy* queda reservado para el detalle de compatibilidad del *msg*, es decir, el esquema de hash pre-**Lagrange** que aparece en la implementación.

Por contraste, **Lagrange** nombra la transición *SNARK-friendly*. En esa ruta, la agregación se modela como una evidencia de tipo SNARK: el registro conserva claves y umbrales compatibles con circuitos, el mensaje firmado se organiza mediante una preimagen rígida, y las firmas individuales relevantes son firmas Schnorr sobre curvas aptas para la aritmetización. Por lo tanto, Schnorr debe leerse como el esquema de firma usado por la ruta **Lagrange**/SNARK, no como una tercera era de MITHRIL.

Con esta convención, el resto del capítulo usa **Pythagoras** para referirse al camino productivo *legacy*/pre-**Lagrange**, **Lagrange** para la ruta *SNARK-friendly*, **Concatenation** y **Snark** para los tipos de agregación, y Schnorr o BLS12-381 para las primitivas de firma correspondientes.

4.2.3. Fases del protocolo

La documentación oficial organiza MITHRIL en tres fases: establecimiento, inicialización y operación [13]. Consideremos un conjunto fijo de participantes $P = \{1, \dots, n\}$, donde cada participante $i \in P$ tiene un *stake* $s_i \in \mathbb{N}$. El stake total del sistema es

$$S = \sum_{i=1}^n s_i,$$

y el peso relativo del participante $1 \leq i \leq n$ viene dado por $w_i = s_i/S$. El protocolo queda parametrizado por una tripla (k, m, ϕ_f) , donde m es el número de loterías evaluadas por participante, k es el mínimo de índices ganadores distintos requeridos para la firma agregada y $\phi_f \in (0, 1]$ controla la probabilidad de elegibilidad [9, 13].

Fase de establecimiento. Se fijan el grupo criptográfico subyacente, los algoritmos de firma, las funciones de hash relevantes y los parámetros (k, m, ϕ_f) . El resultado de esta fase es una instancia del protocolo

$$\Pi_{\text{Mithril}} = (\mathcal{G}, \text{Sign}, \text{Verify}, k, m, \phi_f),$$

donde $\mathcal{G}$ está compuesto por la familia de grupos y curvas elegidas. Esta descripción todavía es independiente de si la evidencia agregada se materializa mediante la ruta **Concatenation** o mediante la ruta SNARK.

Fase de inicialización. Cada participante i genera material criptográfico propio y registra una entrada asociada a su *stake*. Una vez reunidas todas las entradas, el registro se cierra y pasa a un estado autenticado

$$\text{Reg}^* = \{\text{Reg}_1^{\text{closed}}, \dots, \text{Reg}_n^{\text{closed}}\},$$

del que se deriva una clave de verificación agregada **AVK**. En términos conceptuales, esa **AVK** actúa como un *commitment* al comité autenticado y al *stake* con el que se evaluará la lotería. Los detalles de qué campos exactos entran en cada hoja dependen de la ruta criptográfica elegida; para la ruta **Lagrange**/SNARK se desarrollan en la Subsección 4.2.4.

La clave de verificación agregada no es global ni independiente del tiempo: **AVK** _{n} autentica el registro cerrado y el comité cuya evidencia se utilizará durante la época n .

Fase de operación. Dado un mensaje M , cada *signer* i computa una firma individual y evalúa su elegibilidad sobre los índices J . El *output* local del *signer* puede modelarse como un par (σ_i, I_i) , donde $I_i \subseteq J$ es el conjunto de índices ganadores para el par (i, M) .

La condición de elegibilidad para cada índice $j \in J$ puede escribirse como un predicado $\text{Win}(i, j, M) \in \{0, 1\}$, y entonces $I_i = \{j \in J : \text{Win}(i, j, M) = 1\}$. El *aggregator* recibe los pares (σ_i, I_i) , descarta aquellos que no verifican, deduplica índices repetidos y únicamente puede producir una firma agregada válida si el conjunto total de índices ganadores distintos satisface:

$$\left| \bigcup_{i=1}^n I_i \right| \geq k$$

Observamos entonces que no alcanza con acumular muchas firmas si en realidad están justificando los mismos índices. El protocolo STM produce una firma agregada que certifica que un subconjunto autenticado del comité, firmó el mensaje M con al menos k índices válidos y distintos.

4.2.4. Registro cerrado y clave de verificación agregada SNARK

Una vez fijado el stake total S , cada participante queda representado por una entrada cerrada que incorpora, además del *stake*, una clave Schnorr y el umbral de lotería correspondiente. Si denotamos esa entrada por $\text{Reg}_i^{\text{closed}}$, la parte relevante para la AVK de la ruta **Lagrange/Snark** es $L_i = (\text{vk}_i^{\text{sch}}, \tau_i)$, donde vk_i^{sch} es la clave de verificación Schnorr del participante y τ_i es el *lottery_target_value* derivado de (ϕ_f, s_i, S) . El código calcula τ_i durante el cierre del registro como una aproximación (en el cuerpo base de Jubjub $\mathbb{F}_q$), de

$$p \cdot (1 - (1 - \phi_f)^{w_i}),$$

donde $w_i = \frac{s_i}{S}$ y p es la escala entera ya mencionada. Para pasar esta hoja al Merkle Tree, usamos enc para denotar una serialización canónica a bytes; en particular, enc_{64} y enc_{32} denotan serializaciones canónicas de 64 y 32 bytes, respectivamente. La serialización de la hoja es

$$\text{enc}(L_i) = \text{enc}_{64}(\text{vk}_i^{\text{sch}}) \parallel \text{enc}_{32}(\tau_i),$$

de donde se obtiene la hoja $\ell_i = \mathcal{H}(\text{enc}(L_i))$. Entonces, la AVK está dada por la raíz del Merkle Tree del conjunto de hojas así como el *stake* total:

$$\text{AVK} = (r_{\mathcal{R}}, S), \quad \mathcal{L} = \{\ell_i\}_{i=1}^n,$$

donde las hojas se ordenan canónicamente antes de armar el árbol, primero por τ_i y luego por vk_i^{sch} [14].

4.2.5. Esquema de firmas en Lagrange

En esta era de MITHRIL, se produce para cada participante una clave de firmado sk_i^{sch} y su correspondiente clave de verificación vk_i^{sch} . Este es el camino más importante para la verificación STM de certificados del zkBridge, ya que para obtener argumentos sucintos verificables on-chain en $\mathcal{B}$ no alcanza con transportar la BLS12-381 *multisignature* del camino **Pythagoras**. Esto no significa que toda la evidencia del bridge haya migrado a **Lagrange**: en la versión del repositorio de MITHRIL evaluada en esta tesis —commit bd24b9d48, de mayo de 2026—, la pertenencia de **CardanoTransactions** en el prototipo integrado se verifica mediante el camino **Pythagoras** y la agregación **Concatenation**. La ruta **Lagrange** se presenta aquí como una transición *SNARK-friendly*, todavía no utilizada por ese flujo integrado.

Dentro de este camino, aparecen dos variantes de firma Schnorr. La primera es la **StandardSchnorrSignature**, formada solo por un *challenge* c y su correspondiente respuesta r . Según la propia implementación, esta variante se usa en el SNARK cert_{gen} . Su desafío se recomputa como un hash **Poseidon** del *domain separation tag*, la clave de verificación, el punto aleatorio y el mensaje. La segunda es la **UniqueSchnorrSignature**, que agrega un *commitment_point* determinista dependiente solo del mensaje y de la clave Schnorr sk^{sch} . Ese punto permite evitar *grinding attacks* sobre la lotería de *aggregator*, y por lo tanto es la forma que el circuito Halo2 reutiliza después [14].

En lo que sigue, las etiquetas DST_ * denotan etiquetas fijas de separación de dominio; el subíndice identifica el contexto criptográfico. En particular, DST_{seed} se concatena con el *digest* del mensaje para derivar la semilla pública *seed*.

Algoritmo 1 StandardSchnorrSignature en la era Lagrange

Requiere: Mensaje $\mathbf{M} = (m_1, \dots, m_t)$, clave secreta Schnorr sk^{sch}

Asegura: $\sigma = (r, c)$

- 1: $\text{vk} \leftarrow \text{sk}_{\text{sch}} \cdot G$
 - 2: $\rho \leftarrow \text{RandomNonZeroScalar}()$
 - 3: $R \leftarrow \rho \cdot G$
 - 4: $c \leftarrow \text{Poseidon}(\text{DST}_{\text{std}} \parallel \text{vk} \parallel R \parallel \mathbf{m})$
 - 5: $r \leftarrow \rho - c \cdot \text{sk}_{\text{sch}}$
 - 6: **return** (r, c)
-

Algoritmo 2 UniqueSchnorrSignature en la era Lagrange

Requiere: Mensaje $\mathbf{M} = (m_1, \dots, m_t)$, clave Schnorr sk^{sch}

Asegura: $\sigma = (C, r, c)$

- 1: $\text{vk} \leftarrow \text{sk}_{\text{sch}} \cdot G$
 - 2: $H \leftarrow \text{HashToCurve}(\mathbf{m})$
 - 3: $C \leftarrow \text{sk}_{\text{sch}} \cdot H$ ▷ commitment_point
 - 4: $\rho \leftarrow \text{RandomNonZeroScalar}()$
 - 5: $R_1 \leftarrow \rho \cdot H$
 - 6: $R_2 \leftarrow \rho \cdot G$
 - 7: $c \leftarrow \text{Poseidon}(\text{DST}_{\text{uniq}} \parallel H \parallel \text{vk} \parallel C \parallel R_1 \parallel R_2)$
 - 8: $r \leftarrow \rho - c \cdot \text{sk}^{\text{sch}}$
 - 9: **return** (C, r, c)
-

El Algoritmo 3 está formulado para reflejar la semántica observable en el código de MITHRIL. En particular, el participante no firma una Merkle Root aislada ni devuelve una firma “a medio terminar” para que el *aggregator* elija luego los índices. La secuencia correcta es: derivar primero el *digest* rígido μ del mensaje, producir después una **UniqueSchnorrSignature** sobre ese valor y, finalmente, calcular en forma local el conjunto I_i de índices de loterías ganadas. La notación $\text{LotteryEval}(\sigma_i, j)$ se refiere a la derivación determinista previamente expuesta.

4.2.6. Firmas individuales y agregación

Durante la fase de operación, cada participante evalúa para un mensaje msg y para cada índice $j \in J = \{0, \dots, m-1\}$ si gana la lotería correspondiente a ese índice. Si resulta elegible para uno o más índices, genera y envía firmas individuales asociadas a esos índices. La agregación posterior no presupone honestidad del *aggregator*: cualquier tercero con acceso a las firmas individuales puede verificar que son correctas y filtrar aquellas que no satisfacen los chequeos de elegibilidad, así como hacer la agregación [13]. Esta propiedad es útil para el zkBridge, porque evita introducir un supuesto adicional de confianza sobre el *aggregator*.

Algoritmo 3 Firma de un participante con chequeo de lotería en Lagrange

Requiere: Mensaje msg , clave secreta Schnorr sk_i^{sch} , $\text{lottery_target_value } \tau_i$, parámetro m

Asegura: Par (σ_i, I_i) con $\sigma_i = (C_i, r_i, c_i)$ y $I_i \subseteq J = \{0, \dots, m-1\}$, o LotteryLost

```

1:  $\mu \leftarrow \mathcal{H}(\text{msg})$   $\triangleright \text{ProtocolMessage}::\text{compute\_hash}()$ 
2:  $\sigma_i = (C_i, r_i, c_i) \leftarrow \text{UniqueSchnorrSignature}(\mu, \text{sk}_i^{\text{sch}})$ 
3:  $I_i \leftarrow \{j \in J : \text{LotteryEval}(\sigma_i, j) \leq \tau_i\}$ 
4: if  $I_i = \emptyset$  then
5:   return LotteryLost
6: else
7:   return  $(\sigma_i, I_i)$ 
8: end if

```

Agregación en Lagrange. En la ruta SNARK, el *aggregator* recibe un conjunto de firmas

$$\Sigma = \{(\sigma_i^{\text{sch}}, I_i, i)\}_{i \in P'},$$

del conjunto de participantes $P' \subseteq P$ que le enviaron firmas sobre el mensaje hashado $\mu = \mathcal{H}(\text{msg})$.

El primer paso consiste en descartar las firmas que no verifiquen respecto de la vk^{sch} registrada para cada participante, o cuyos índices reclamados no coincidan con los que se recomputan a partir de τ_i . Este cómputo usa la información de la hoja $L_i = (\text{vk}_i^{\text{sch}}, \tau_i)$, que luego será enlazada al Merkle tree comprometido por AVK. El resultado es un conjunto de candidatos válidos $\Sigma_{\text{val}} \subseteq \Sigma$.

Recordemos que en STM el umbral se mide en índices de lotería distintos, no en cantidad de firmas, para poder cubrir al menos k valores distintos de J . Si definimos

$$J_{\text{cand}} = \bigcup_{(\sigma_i^{\text{sch}}, I_i, i) \in \Sigma_{\text{val}}} I_i,$$

la agregación solo puede avanzar cuando $|J_{\text{cand}}| \geq k$. Una vez superado ese umbral, el código debe elegir qué índices utilizar, pero sin beneficiar o perjudicar a ningún participante específico. Primero, deriva una *seed* pública $\text{seed} = \text{SHA-256}(\text{DST}_{\text{seed}} \parallel \mu)$. Para seleccionar qué índices sobreviven, calcula para cada índice candidato j un puntaje

$$h_{\text{idx}}(j) = \text{SHA-256}(\text{DST}_{\text{idx}} \parallel \text{seed} \parallel j),$$

y conserva exactamente los k índices con menor valor. Denotamos por J^* a ese conjunto, con $|J^*| = k$ y $J^* \subseteq J_{\text{cand}}$.

Como para cada $j \in J^*$ puede haber varios participantes que lo hayan ganado, el *aggregator* hace un desempate, que depende de una segunda familia de hashes:

$$h_{\text{dedup}}(j, i) = \text{SHA-256}(\text{DST}_{\text{dedup}} \parallel \text{seed} \parallel j \parallel i),$$

de modo que el participante retenido para el índice j es

$$i^*(j) = \arg \min \{h_{\text{dedup}}(j, i) : j \in I_i\}.$$

Esta selección deja un mapa ordenado $\hat{\Sigma} : J^* \rightarrow P'$, con exactamente un participante por índice y exactamente k índices retenidos.

El paso final produce un *witness* $w \in \mathcal{W}$ para el circuito de Halo2 de MITHRIL. Para cada $j \in J^*$, el *aggregator* construye

$$w_j = \left(L_{i^*(j)}, \text{Path}_{i^*(j)}, \sigma_{i^*(j)}^{\text{sch}}, j \right),$$

donde $\text{Path}_{i^*(j)}$ es el Merkle Path de $L_{i^*(j)}$ hasta $r_{\mathcal{R}}$. Entonces,

$$w = \left\{ \left(L_{i^*(j)}, \text{Path}_{i^*(j)}, \sigma_{i^*(j)}^{\text{sch}}, j \right) \right\}_{j \in J^*} \in \mathcal{W}.$$

Posteriormente, la Sección 4.6 formula la relación que demuestra que todas esas entradas son coherentes respecto de la AVK, del mismo hash μ y de la condición de quórum $|J^*| = k$; la Sección 4.7 estudia su compilación a Halo2. Es por esto que esta subsección se enfoca en mostrar cómo todo puede ser expresado de forma determinística, para llevarlo luego a un circuito con menos dificultades.

4.3. Cadena de certificados

Si el protocolo STM descrito en la Sección 4.2 solo produjera una evidencia agregada sobre un mensaje aislado *msg*, todavía tendríamos un problema fundamental: cómo confiar en la StakeDist con la que fue construida la AVK que autentica al registro cerrado Reg^* y con la que se verifica esa evidencia. La respuesta de MITHRIL es la cadena de certificados (*CertChain*), cuyo objetivo es certificar precisamente la evolución auténtica de Reg^* de una época a la siguiente y evitar ataques en los que un *aggregator* deshonesto genere certificados formalmente bien contruidos pero anclados en una visión falsa del sistema [15, 16]. Para nuestro zkBridge, la *CertChain* otorga autenticidad al estado publicado on-chain. De otro modo, la evidencia agregada de un certificado cert_{ts} no basta.

En la ruta *Lagrange*, el circuito Halo2 tiene un contrato acotado: recibe como entradas públicas el mensaje (o su *digest*) y la clave de verificación agregada, y demuestra que existen k entradas del comité autenticado con caminos de Merkle válidos, firmas Schnorr correctas e índices de lotería ganadores. Este circuito verifica la evidencia STM del certificado, pero no verifica por sí mismo la *CertChain*. La cadena se verifica por separado: se comprueban los hashes de los certificados, sus enlaces con los predecesores, la consistencia de las épocas, la transición de la AVK y los parámetros, y la firma del certificado génesis.

Advertencia de alcance. Con la terminología de la Subsección 4.2.2, la estructura de la *CertChain* descrita aquí es común a *Pythagoras* y *Lagrange*: en ambos casos, cada certificado enlaza con uno previo, fija la época actual y compromete la clave y los parámetros de la época siguiente. Lo que cambia entre eras no es la lógica de encadenamiento, sino la ruta criptográfica de la agregación. En *Pythagoras*, que en esta tesis coincide con el camino *legacy*/pre-*Lagrange* del flujo productivo, esa evidencia es una BLS12-381 *multi-signature* de tipo *Concatenation*. En *Lagrange*, la evidencia relevante para circuitos se lee como una ruta *Snark*: un argumento sucinto construido en Halo2 sobre firmas Schnorr individuales, su pertenencia a J^* con $|J^*| = k$ respecto de la AVK y del *msg* asociados al certificado.

4.3.1. Estructura matemática de un certificado

Usamos cert_{gen} para denotar el certificado génesis (que se verifica con vk_{gen}) y $\text{cert}_{p,n}$ para denotar un certificado no génesis emitido en la época n de CARDANO bajo un *trigger* p .

Para distinguir el primer certificado de una época, lo notamos $\text{cert}_n^{\text{first}}$. En la documentación de MITHRIL, el *trigger* p identifica el instante o condición que determina la creación de un certificado. Al combinarse con la época n , forma el *beacon* que identifica esa instancia de certificación. El significado concreto de p depende del artefacto certificado; por ejemplo, para los snapshots de la base de datos de CARDANO puede ser el número de un archivo inmutable. En nuestra notación, $\text{cert}_{p,n}$ representa el certificado asociado a ese *beacon* [15].

Podemos idealizar un certificado de MITHRIL como una tupla

$$\text{cert}_{p,n} = (h_{p,n}, h_{p,n}^-, n, \text{meta}_{p,n}, \text{msg}_{p,n}, \text{sm}_{p,n}, \text{AVK}_n, \sigma_{p,n}),$$

donde $h_{p,n}$ es el hash del certificado (sin contar al mismo $h_{p,n}$), $h_{p,n}^-$ es el hash del certificado predecesor en la *CertChain*, $\text{meta}_{p,n}$ resume metadatos de protocolo, $\text{msg}_{p,n}$ es el mensaje de protocolo autenticado por $\text{cert}_{p,n}$, $\text{sm}_{p,n}$ es el mensaje firmado derivado de $\text{msg}_{p,n}$, AVK_n es la clave de verificación agregada para la época n (excepto si $\text{cert}_{p,n} = \text{cert}_{\text{gen}}$), y $\sigma_{p,n}$ es la firma de $\text{cert}_{p,n}$, que puede ser génesis o no según el caso.

El código computa el hash del certificado como una concatenación hashada de todos sus campos estructurales, de modo que cualquier alteración en el certificado modifica $h_{p,n}$, y por lo tanto rompe la *CertChain*. Esquemáticamente,

$$h_{p,n} = \mathcal{H}(h_{p,n}^- \parallel n \parallel H(\text{meta}_{p,n}) \parallel H(\text{msg}_{p,n}) \parallel \text{sm}_{p,n} \parallel \text{AVK}_n \parallel \sigma_{p,n}).$$

Los metadatos $\text{meta}_{p,n}$ incluyen la versión del protocolo, los parámetros $\text{PP}_n = (k_n, m_n, \phi_{f,n})_n$ de la época n , los tiempos de inicio y sellado, y la lista de participantes que contribuyeron a la agregación [15]. Aunque PP_n forma parte de $\text{meta}_{p,n}$, la condición de encadenamiento exige además que los parámetros que habilitan la época n hayan sido comprometidos en el msg del certificado predecesor. El mensaje $\text{msg}_{p,n}$ es un diccionario tipado de partes cuyo contenido depende de qué es lo que se está certificando. Entre esas partes aparecen de manera recurrente el número de época actual, la siguiente clave de verificación AVK_{n+1} , los parámetros de protocolo siguientes PP_{n+1} y el resumen del artefacto propiamente dicho, por ejemplo Merkle Root de transacciones o de *StakeDist*.

El certificado génesis cert_{gen} ocupa un lugar especial. No depende de una agregación previa, sino de una firma bajo la clave pública vk_{gen} . En la práctica, cert_{gen} es la raíz de autenticidad de toda la *CertChain*, porque todo certificado posterior termina trazando su validez desde él [15].

4.3.2. Diseño matemático de la cadena

En la semántica implementada por Mithril, la *StakeDist* asociada a una época n sólo queda estabilizada al cierre de esa época, y los certificados emitidos incorporan AVK_{n+1} y los parámetros que habilitan su uso en la época $n+1$ [15]. Por eso la cadena de certificados debe garantizar que AVK_{n+1} y PP_{n+1} con los que se verificará la época $n+1$ ya hayan sido firmados en un certificado anterior. En particular, $\text{cert}_{n+1}^{\text{first}}$ debe apoyarse en una certificación válida de la transición desde la época n .

En esta tesis usamos la convención de que AVK_n es la clave agregada que autentica el comité y los parámetros utilizados para verificar la época n . La primera certificación de esa época debe obtenerla del certificado predecesor (que es el génesis en el caso de la primera época) y los certificados posteriores de la misma época conservan esa misma clave.

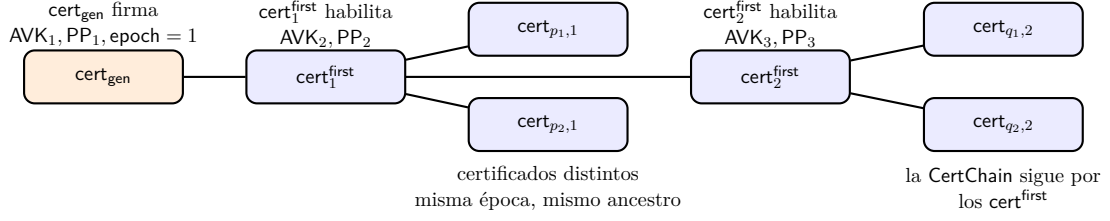


Fig. 4.2: Estructura conceptual de una CertChain de MITHRIL por épocas. El cert_{gen} actúa como raíz de autenticidad y firma la información que habilita la época 1; cada $\text{cert}_n^{\text{first}}$ habilita la AVK y los parámetros de la época siguiente; otros certificados de la misma época pueden compartir ancestro sin ser parte del camino mínimo que conecta épocas consecutivas.

Entonces, la CertChain debe garantizar que la información necesaria para usar AVK_n y PP_n ya haya sido firmada en un certificado anterior. Por lo tanto

$$\{\text{AVK}_n, \text{PP}_n, n\} \subseteq \text{msg} \left(\text{pred} \left(\text{cert}_n^{\text{first}} \right) \right),$$

En cambio, para un certificado posterior $\text{cert}_{p,n}$ dentro de la misma época, no es necesario reintroducir una nueva AVK. Basta con que se mantenga la consistencia interna:

$$\text{AVK}(\text{cert}_{p,n}) = \text{AVK}(\text{pred}(\text{cert}_{p,n})), \quad \text{PP}(\text{cert}_{p,n}) = \text{PP}(\text{pred}(\text{cert}_{p,n})).$$

Por eso la documentación afirma que, aunque puedan existir muchos certificados en una misma época, no hace falta encadenarlos linealmente: alcanza con preservar una senda válida con al menos un certificado por época y con la transición correcta entre épocas consecutivas [15].

4.3.3. Verificación de la CertChain

La validez de la CertChain hasta un certificado $\text{cert}_{p,n}$ se apoya en la validez de sí mismo así como todos sus predecesores.

$$\text{ChainVerify}(\text{cert}_{p,n}) = \text{CertVerify}(\text{cert}_{p,n}) \wedge \bigwedge_{i=1}^n \text{cert}_i^{\text{first}} \wedge \text{CertVerify}(\text{cert}_{\text{gen}}),$$

donde en particular observamos que existe al menos un certificado válido por época desde n hasta el génesis [15]. Por recursividad, tenemos que

$$\text{ChainVerify}(\text{cert}_{p,n}) = \text{CertVerify}(\text{cert}_{p,n}) \wedge \text{ChainVerify}(\text{pred}(\text{cert}_{p,n}))$$

La verificación de un certificado individual se descompone en capas. Para el caso génesis, $\text{CertVerify}(\text{cert}_{\text{gen}})$ requiere

$$\text{Verify}_{\text{gen}}(\text{vk}_{\text{gen}}, \text{cert}_{\text{gen}}.\text{sm}, \sigma_{\text{gen}}) = 1,$$

así como los chequeos estándar de hash del certificado y de su msg . Además, el número de época cargado por el certificado debe coincidir con la época declarada dentro de su msg .

Para un certificado no génesis $\text{cert}_{p,n}$, la condición se vuelve

$$\text{CertVerify}(\text{cert}_{p,n}) = \text{HashOk} \wedge \text{SigOk} \wedge \text{EpochOk} \wedge \text{PrevHashOk} \wedge \text{AVKChainOk} \wedge \text{PPChainOk}.$$

En esta descomposición:

- HashOk significa que el ‘hash’ del certificado coincide con la recomputación de sus campos.
- SigOk significa que la agregación del certificado es válida bajo la AVK y los PP con los que ese certificado declara haber sido firmado. En *Pythagoras*, esto coincide con verificar la ruta *Concatenation*; en *Lagrange*, debe leerse respecto de la ruta *Snark* y, cuando corresponda, de la AVK SNARK adicional enlazada por la cadena.
- EpochOk significa que la época declarada en el certificado coincide con la época dentro del mensaje firmado.
- PrevHashOk significa que el h^- del certificado coincide con el hash de su predecesor.
- AVKChainOk y PPChainOk se diferencian según el tipo de certificado: si $\text{cert}_{p,n}$ es el primer certificado de su época, entonces su predecesor debe haber firmado la AVK y los parámetros de la nueva época:

$$\text{AVK}_n \in \text{msg}(\text{pred}(\text{cert}_n^{\text{first}})), \quad \text{PP}_n \in \text{msg}(\text{pred}(\text{cert}_n^{\text{first}})),$$

mientras que en caso contrario se debe preservar la misma AVK y los mismos PP de su predecesor.

4.3.4. Relación con el flujo de StakeDist

La formulación anterior fija la base criptográfica del flujo de *StakeDist* que usará nuestro zkBridge. En la fase de *bootstrap*, *stake_distribution_genesis_tx* crea el UTxO canónico de *StakeDist* con un cert_{gen} autenticado por vk_{gen} , como se formaliza en la Subsección 6.6.1. En las actualizaciones recurrentes, *stake_distribution_standard_tx* consume el UTxO que contiene el certificado padre junto con un *proof_receipt* del dominio *stake_distribution_standard*, y produce otro UTxO con el mismo NFT y un certificado nuevo, imponiendo los invariantes detallados en la Subsección 6.6.2.

4.4. Entidades, certificados y versiones

No todos los artefactos certificados por MITHRIL cumplen el mismo rol dentro de esta tesis. Para el zkBridge, la distinción relevante no pasa por volver a describir la mecánica STM de la Sección 4.2 ni la CertChain de la Sección 4.3, sino por identificar qué certificados alimentan estado persistente y cuáles sólo acompañan verificaciones puntuales. En el prototipo desarrollado, esta separación aparece de forma explícita en los tipos de entidad firmada *MithrilStakeDistribution* y *CardanoTransactions*. Como se aclaró en la Sección 4.1, estos nombres clasifican los datos autenticados por un certificado; no designan participantes del protocolo.

- Un artefacto *MithrilStakeDistribution* materializa la lista de participantes registrados, su *stake* y sus claves de verificación. Su certificado autentica el comité de MITHRIL P'_n y los parámetros PP_n de una época n . Colabora en sostener la CertChain y por lo tanto el estado persistente de la capa de actualización del zkBridge.
- Una entidad firmada *CardanoTransactions* certifica una Merkle Root obtenida de un conjunto de transacciones de CARDANO. Colabora en respaldar argumentos de pertenencia de tx_{lock} en $\mathcal{A}$ para autorizar tx_{mint} en $\mathcal{B}$.

En esta sección, *CardanoTransactions* denota una entidad firmada de MITHRIL que certifica un conjunto de transacciones de CARDANO, representado mediante una raíz de

Merkle y asociado a un intervalo de bloques. No se refiere a una transacción individual. Por otra parte, **Concatenation** denota el tipo de agregación de firmas utilizado por la ruta productiva **Pythagoras**: la evidencia combina las firmas individuales BLS12-381 de los participantes elegibles en una firma agregada verificable bajo la AVK correspondiente [11, 14].

Desde el punto de vista del **zkBridge**, la diferencia central entre **MithrilStake Distribution** y **CardanoTransactions** es operacional. El primero fija el estado autenticado que la capa de actualización mantiene en $\mathcal{B}$; mientras que el segundo respalda una prueba efímera de pertenencia sobre un compromiso global de transacciones. No conviene persistir esta segunda evidencia en el datum cuando lo que realmente debe sobrevivir entre transacciones es el certificado que autentica el comité y los parámetros con los que luego se valida un cert_{ts} .

La documentación oficial refuerza la asimetría del lado transaccional. Un certificado de **CardanoTransactions** está diseñado para un conjunto masivo de transacciones de CARDANO y por eso firma un Merkle Root en lugar de transacciones aisladas [11].

4.4.1. Pythagoras y el camino *legacy*

En la ruta **Pythagoras**, el conjunto certificado de transacciones no se representa mediante un único Merkle Tree plano. Se utiliza una estructura de dos niveles, que puede verse como un Merkle tree de Merkle trees.

Primero, cada rango contiguo de 15 bloques tiene un Merkle Tree local cuyas hojas son las transacciones del rango. La raíz local se combina con la identificación del rango para formar una hoja maestra. Luego, las hojas maestras se organizan en un segundo Merkle Tree; la raíz de este árbol maestro es la r_{tx} firmada por MITHRIL.

Por eso, demostrar que una transacción pertenece al conjunto certificado requiere dos pruebas complementarias: una π_u , que demuestra su pertenencia al árbol local de su rango, y una π_{master} , que demuestra que la raíz local pertenece al árbol maestro cuya raíz final es r_{tx} . La Figura 4.3 resume esta composición antes de introducir la notación formal.

En la versión del repositorio de MITHRIL evaluada en esta tesis (commit **bd24b9d48**, de mayo de 2026) el **zkBridge** consume el camino **Pythagoras** con agregación de tipo **Concatenation**, que aparece integrado de punta a punta en el flujo real del *aggregator* [12]. El operador del **zkBridge** trabaja contra esa interfaz al ejecutar **tx prove** sobre respuestas reales de la API del *aggregator*.

Para esta sección, $\mathcal{B}_\ell$ denota el conjunto de cadenas de bytes de longitud ℓ . En particular, $\mathcal{B}_{32}$ representa un digest binario de 32 bytes, mientras que la codificación hexadecimal ASCII correspondiente pertenece a $\mathcal{B}_{64}$.

En esta sección, entonces, **Pythagoras** nombra la era/carril productivo, mientras que *legacy* nombra la forma concreta de hash y prueba de pertenencia asociada a ese carril. En esta ruta *legacy*, el mensaje efectivamente autenticado por el certificado es una Merkle Root $r_{\text{tx}} \in \mathcal{B}_{32}$, la cual es la del **msg** definido en la Subsección 4.3.1 [14]. El **TxSnapshot** expuesto por la API del *aggregator* agrega además un número de bloque certificado b^* y un identificador derivado. En la fórmula siguiente, $\text{be64}(b)$ denota la codificación de b en 8 bytes, con el byte más significativo primero:

$$\text{TxSnapshot} = (\text{hash}_{\text{snap}}, r_{\text{tx}}, b^*), \quad \text{hash}_{\text{snap}} = \text{SHA-256}(\text{hex}(r_{\text{tx}}) \parallel \text{be64}(b^*)),$$

pero el objeto firmado por MITHRIL sigue siendo r_{tx} .

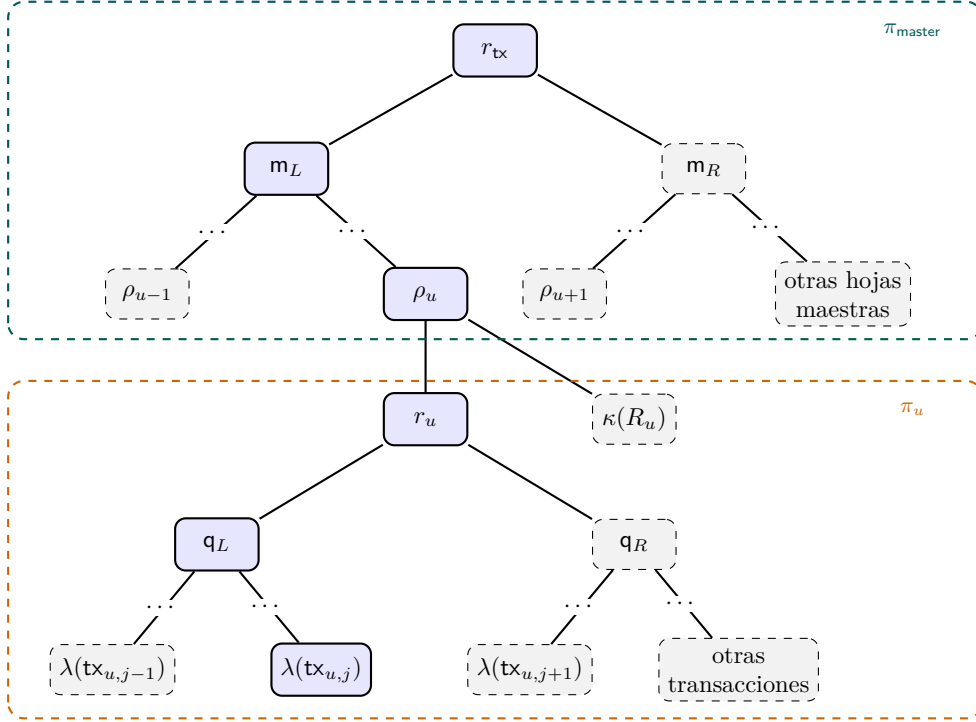


Fig. 4.3: Estructura de dos niveles de **CardanoTransactions legacy**: un Merkle Tree local agrupa las transacciones de un rango de bloques R_u y produce r_u ; esta raíz, combinada con $\kappa(R_u)$, forma la hoja maestra ρ_u de un segundo Merkle Tree cuya raíz firmada es r_{tx} . La prueba completa de pertenencia combina una π_u local y una π_{master} del árbol maestro.

Dado $u \in \mathbb{N}$, sea $R_u = [15u, 15(u + 1)) \subset \mathbb{N}$ el *block range* canónico determinado por la fórmula

$$\text{start}(b) = \left\lfloor \frac{b}{15} \right\rfloor \cdot 15.$$

Para un número de bloque b^* , definimos el conjunto de rangos completos

$$I(b^*) = \{u \in \mathbb{N} : 15(u + 1) - 1 \leq b^*\}$$

y, para cada $u \in I(b^*)$, la secuencia ordenada de transacciones $\mathcal{T}_u = (\text{tx}_{u,1}, \dots, \text{tx}_{u,m_u})$, donde el orden está dado por número de bloque y luego por hash de la transacción.

Si $\lambda(\text{tx}) \in \mathcal{B}_{64}$ denota la codificación ASCII hexadecimal del hash de tx , entonces la raíz local del rango R_u es r_u del Merkle Tree $\mathcal{T}_{\mathcal{L}, \text{BLAKE2S-256}}$ con $L = \{\lambda(\text{tx}_{u,i})\}_{1 \leq i \leq m_u}$.

El segundo nivel vuelve a comprometer estas raíces por rango. Si

$$\kappa(R_u) = \text{ascii}(15u - 15(u + 1))$$

denota la clave serializada del rango, entonces la *hoja maestra* asociada a R_u es $\rho_u = \text{BLAKE2S-256}(\kappa(R_u) \parallel r_u)$.

Ordenando los rangos por su cota inferior, la Merkle Root firmada por el certificado es r_{tx} del árbol maestro $\mathcal{T}_{\mathcal{L}, \text{BLAKE2S-256}}$ para $\mathcal{L} = \{\rho_{u_i}\}_{1 \leq i \leq s}$, con $u_1 < \dots < u_s$, $u_i \in I(b^*)$ donde solamente intervienen los rangos no vacíos. Esta es la formalización precisa de la estructura de dos niveles mostrada en la Figura 4.3: cada Merkle Tree local comprime

las transacciones de 15 bloques contiguos y el Merkle Tree maestro comprime las hojas formadas a partir de las raíces de esos subárboles [11, 14].

La prueba que devuelve el endpoint de transacciones refleja exactamente esta estructura. Para un subconjunto $Q \subseteq \bigcup_{u \in I(b^*)} \mathcal{T}_u$, tenemos

$$\pi_Q = \left(\pi_{\text{master}}, \{(u, \pi_u)\}_{u \in U_Q} \right), \quad U_Q = \{u : Q \cap \mathcal{T}_u \neq \emptyset\},$$

donde cada π_u es una Merkle Proof en el árbol de R_u , mientras que π_{master} es una Merkle Proof de las hojas ρ_u en el Merkle Tree maestro con raíz r_{tx} . En la implementación, este objeto aparece como `CardanoTransactionsSetProof`, cuyo campo `transactions_proof` es un `MKMapProof<BlockRange>` que primero verifica las Merkle Proofs por rango, luego la Merkle Proof maestra y finalmente la pertenencia de cada hash al conjunto certificado [14]. En consecuencia,

$$\text{Verify}_{\text{tx-set}}(Q, \pi_Q, r_{\text{tx}}) = 1$$

si y sólo si todas las ramas locales y la rama maestra llegan a la misma raíz r_{tx} . Ésta es la identidad que debe ser verificada por el `zkBridge`, en este caso de forma off-chain con un circuito aritmético, previo a verificar el argumento sucinto de forma on-chain.

4.4.2. Lagrange como transición

Lagrange es la era que introduce el carril *SNARK-friendly*; no cambia el rol lógico del certificado dentro del `zkBridge`. Lo que sí cambia es la representación matemática del mensaje firmado y de la evidencia que luego debe verificar el `zkBridge`. El propio código de MITHRIL deja visibles esas modificaciones aunque el *aggregator* productivo todavía opere sobre el carril `Pythagoras`. Por eso, el diseño del bridge queda preparado para **Lagrange**, pero el flujo integrado actual conserva `Pythagoras` para la pertenencia de `CardanoTransactions`. El Capítulo 6 refleja esta separación: el camino `Pythagoras` se modela en la Subsección 6.9.1, mientras que la geometría **Lagrange** queda presentada como transición experimental en la Subsección 6.9.6.

El mensaje firmado deja de ser sólo un hash legacy. En `Pythagoras`, el certificado termina firmando el valor $\mu_{\text{legacy}} = \text{msg.compute_hash}()$ con esquema `legacy`, es decir, un hash calculado sobre el diccionario dinámico de partes del mensaje. Esta representación es flexible para el software del *aggregator*, porque permite agregar o quitar partes del mensaje sin cambiar una preimagen rígida global, pero es menos conveniente para un circuito: el verificador aritmético debería reproducir una lógica de serialización dinámica o aceptar como dato externo una frontera difícil de auditar.

En la ruta **Lagrange**, ‘rigid’ es el esquema de hash de los mensajes de protocolo compatible con circuitos **SNARK**. A diferencia de `legacy`, no hashea directamente un diccionario dinámico: separa las partes variables del mensaje de ciertos campos estructurales y los reinyecta en segmentos de tamaño fijo, precedidos por etiquetas conocidas. En nuestra notación, `rigid` designa este esquema y μ el digest obtenido a partir de esa preimagen rígida.

Esta elección responde a una necesidad concreta del circuito: el verificador debe recibir una preimagen cuya longitud, segmentación y campos relevantes estén determinados de antemano. De ese modo, puede imponer restricciones sobre posiciones conocidas sin reproducir la lógica de un diccionario dinámico ni aceptar una serialización ambigua.

En la ruta **Lagrange**, en cambio, **ProtocolMessage** pasa al esquema **rigid** y el repositorio ya fija una preimagen rígida, de longitud y segmentación controladas, de la forma:

$$\mu = \text{SHA-256}(\text{digest} \parallel d \parallel \text{next_aggregate_verification_key} \\ \parallel a \parallel \text{next_protocol_parameters} \parallel p \\ \parallel \text{current_epoch} \parallel e),$$

donde $d = \text{SHA-256}(\text{parts}_{\text{dyn}})$ y $\text{parts}_{\text{dyn}}$ es la proyección del mensaje que conserva las partes dinámicas (entre ellas, para **CardanoTransactions**, la raíz del **snapshot**) pero excluye los segmentos rígidos reinyectados por separado. Las etiquetas **digest**, **next_aggregate_verification_key**, **next_protocol_parameters** y **current_epoch** actúan como separadores de dominio dentro de una preimagen con ranuras estables. Por lo tanto, el cambio central no es sólo cambiar una función de hash por otra: es transformar el mensaje firmado en una palabra estructurada cuya forma puede fijarse como interfaz pública de un **SNARK** [14].

El conjunto de transacciones deja de representarse como un árbol de dos niveles. El segundo cambio afecta la evidencia de pertenencia de transacciones. En **Pythagoras**, la prueba *legacy* de **CardanoTransactions** reproduce una estructura de dos niveles: primero certifica la pertenencia de una transacción dentro del árbol de un rango de bloques y luego certifica que la raíz de ese rango aparece como hoja dentro del árbol maestro firmado por **MITHRIL**. Esa geometría coincide con la interfaz productiva actual, pero obliga al circuito a modelar claves de rango, hojas maestras y dos Merkle Proofs enlazadas por la misma raíz final. El circuito experimental **mithril_lagrange_tx_membership_experimental.circom** anticipa una interfaz más directa. Todas las transacciones pasan a organizarse en un Merkle Tree plano, sin partición por rango de bloques, sin $\kappa(R_u)$, sin hojas maestras y sin recursión. Si enumeramos las transacciones certificadas como $\mathcal{T} = (\text{tx}_0, \text{tx}_1, \dots, \text{tx}_{N-1})$, la representación sugerida por ese circuito puede verse como la raíz $r_{\text{tx}}^{\text{Lag}}$ del Merkle Tree $\mathcal{T}_{\mathcal{L}, \text{Poseidon}^*}$, donde $L = \{\text{Poseidon}^*(\lambda(\text{tx}_i))\}_{0 \leq i < N}$ y **Poseidon*** representa la variante *Midnight Poseidon* planeada por **MITHRIL** para el camino **future_snark**. En el experimento actual esa función todavía está *mockeada* con **Poseidon255**, por lo que la igualdad anterior debe leerse como el objetivo matemático de la transición, no como el resultado final implementado en el **zkBridge**.

El **README** del circuito aclara, sin embargo, que este camino todavía no está conectado al operador ni es compatible con **tx prove**, precisamente porque el formato del *witness* todavía no coincide con la forma de prueba que entrega el *aggregator* actual. Por eso, en el Capítulo 6 no aparecen dos alternativas productivas equivalentes: aparece un circuito plenamente alineado con el camino **Pythagoras** hoy soportado por el servicio, y otro de carácter experimental que adelanta cómo debería verse la ruta **Lagrange** cuando esa interfaz deje de ser sólo inferible desde el código y pase a estar efectivamente desplegada.

4.5. API del *aggregator* e integración

A partir del rol del *aggregator* definido en la Sección 4.1, esta sección describe su interfaz concreta. **MITHRIL** fue presentado como protocolo y como estructura de certificados. Sin embargo, el **zkBridge** no interactúa con una abstracción matemática, sino con una instancia concreta del software *aggregator*, ejecutada como nodo o servicio de red. Este componente,

entre otras cosas, expone artefactos y pruebas a través de una API REST [12]. Para este desarrollo, es la interfaz práctica entre el estado certificado de CARDANO y el *relayer* off-chain que prepara argumentos para el zkBridge. La documentación oficial de MITHRIL sobre CardanoTransactions ayuda a interpretar mejor este flujo [11].

4.5.1. Endpoints relevantes

La implementación local del operador del zkBridge (**zk-bridge-operator**) consume un subconjunto pequeño pero muy expresivo de la API del *aggregator*. El **zk-bridge-operator** necesita descubrir entidades ya certificadas, verificar su coherencia y transformar la respuesta del *aggregator* en entradas canónicas para sus circuitos. Los endpoints relevantes son los siguientes:

Ruta (GET)	Respuesta principal	Uso en zk-bridge-operator
/	capacidades del <i>aggregator</i>	detectar entidades firmadas soportadas y versionado
/status	estado actual	registrar contexto operativo del <i>aggregator</i>
/certificate/genesis	cert _{gen}	ancla inicial para sincronizar la CertChain
/certificates	certificados recientes	obtener certificados recientes
/certificate/{hash}	certificado individual	reconstruir la CertChain y obtener mensajes concretos
/artifact/cardano-transaction/{hash}	detalle de un snapshot de transacciones	obtener el certificado de un snapshot
/proof/cardano-transaction	transacciones certificadas	obtener una Merkle Proof para un conjunto de transacciones concreto

Tab. 4.1: Endpoints del *aggregator* utilizados por el **zk-bridge-operator**

4.5.2. Flujo del zk-bridge-operator

Organizamos el **zk-bridge-operator** en dos comandos. El primero, **relayer sync-certificates**, descarga el cert_{gen}, consulta certificados recientes, filtra aquellos relacionados con **MithrilStakeDistribution** y reconstruye hacia atrás la cadena necesaria para persistirla localmente (hasta el último estado sincronizado, potencialmente puede llegar a ser toda la cadena). Como *output*, escribe metadatos y una colección local de certificados serializados por su hash, lo que es práctico para usar como caché verificable de los certificados con los que trabajan los circuitos del zkBridge.

El segundo comando, **tx prove <transaction-hash>**, es el que conecta la API de MITHRIL con el zkBridge. Este flujo consulta una Merkle Proof de pertenencia para la transacción indicada, descarga el certificado de **CardanoTransactions** asociado, busca el *snapshot* correspondiente y luego ejecuta tres chequeos:

1. Verifica la Merkle Proof de la transacción con el cliente de MITHRIL.

2. Verifica la **CertChain** del certificado referenciado hasta el cert_{gen} (potencialmente, sino el certificado más reciente ya validado), es decir que hay una sucesión válida $\text{cert}_{\text{gen}} \rightsquigarrow \text{cert}_1^{\text{first}} \rightsquigarrow \dots \rightsquigarrow \text{cert}_{p,n}$.
3. Reconstruye el mensaje esperado msg a partir de las transacciones verificadas y comprueba que coincida con el de $\text{cert}_{p,n}$. Es decir, que corresponda a una r_{tx} autenticada por alguna **AVK** válida y encadenada en la **CertChain** correcta.

Finalmente, si los chequeos fueron exitosos, el **zk-bridge-operator** exporta *inputs* válidos para los circuitos **snapshot_membership** y **tx_set_update**, desarrollados respectivamente en las Secciones 6.9 y 6.8.

4.6. Relación NP del circuito STM

Como se explicó en la introducción, el *aggregator* reúne y valida las evidencias individuales producidas por los participantes. En la ruta **Lagrange**, las normaliza como un *witness* estructurado $w = (w_1, \dots, w_k) \in \mathcal{W}$, que luego recibe y verifica el circuito **Halo2**. El circuito argumenta que existen k participantes legítimos del comité autenticado por **AVK**, cada uno con una firma Schnorr válida sobre μ , un Merkle correcto e índice de lotería ganador. Los objetos

$$\text{Reg}_{\text{snark}}^* = \{L_1, \dots, L_n\}, \quad \text{AVK} = (r_{\mathcal{R}}, S), \quad \mu = \mathcal{H}(\text{msg}),$$

ya quedaron fijados en la Subsección 4.2.4.

El circuito recibe como *public inputs* los componentes de msg codificados como elementos del cuerpo $\mathbb{F}_q$ en el que se aritmetiza el circuito, es decir, el cuerpo base de **Jubjub** y el cuerpo escalar de **BLS12-381**. Entonces, $\mathbf{x} = (x_1, \dots, x_t) \in \mathbb{F}_q^t = \mathcal{X}$, y se determinan unívocamente tanto μ y $r_{\mathcal{R}}$. Cuando la implementación incluye puntos de curva, éstos aparecen codificados por sus coordenadas en $\mathcal{E}(\mathbb{F}_q)$, mientras que los escalares de respuesta y *challenge* que intervienen en las verificaciones de firma Schnorr viven en $\mathbb{F}_r$, el cuerpo escalar de **Jubjub**.

Por otra parte, el *witness* contiene exactamente una entrada por cada índice de $J^* = \{j_1, \dots, j_k\}$ con $j_1 < \dots < j_k$. En particular, la Subsección 4.2.6 establece que cada entrada tiene la forma $w_t = (L_{i_t}, \text{Path}_{i_t}, \sigma_{i_t}^{\text{sch}}, j_t)$ con $1 \leq t \leq k$, y donde $\sigma_{i_t}^{\text{sch}} = (C_{i_t}, r_{i_t}, c_{i_t})$ es una **UniqueSchnorrSignature**, Path_{i_t} es el Merkle Path hasta $r_{\mathcal{R}}$, y $j_t \in \{0, \dots, m-1\}$ es el índice reclamado por el participante. Finalmente, $\mathbf{w} = (w_1, \dots, w_k) \in \mathcal{W}$.

Introducimos ahora la relación NP $\mathcal{R}_{\text{STM-H2}} \subseteq \mathcal{X} \times \mathcal{W}$. Decimos que $\mathcal{R}_{\text{STM-H2}}(\mathbf{x}, \mathbf{w}) = 1$ si y sólo si se satisfacen simultáneamente las cuatro condiciones desarrolladas abajo. El lenguaje asociado a la relación es $\mathcal{L}_{\text{STM-H2}} = \{\mathbf{x} \in \mathcal{X} \mid \exists \mathbf{w} \in \mathcal{W} : \mathcal{R}_{\text{STM-H2}}(\mathbf{x}, \mathbf{w}) = 1\}$. Semánticamente, esto significa:

Existen exactamente k participantes del comité autenticado por **AVK**, cada uno con Merkle Path correcto, **UniqueSchnorrSignature** válida sobre μ e índice de lotería de **MITHRIL** ganador, tales que los k índices son estrictamente crecientes.

Esta formulación hace explícito que el circuito no prueba “todo **MITHRIL**”, sino sólo la consistencia de un conjunto de k testigos seleccionados respecto del comité autenticado, del mensaje público y del mecanismo de lotería.

1. *Correctitud del Merkle Tree.* Para cada $1 \leq t \leq k$, la hoja L_{i_t} debe pertenecer al Merkle Tree de raíz $r_{\mathcal{R}}$. Esta condición es la verificación de una Merkle Proof sobre el hash ℓ_t de L_{i_t} . El circuito verifica $\text{Verify}(\ell_t, \text{Path}_{i_t}, r_{\mathcal{R}}) = 1$.

2. *Correctitud de las UniqueSchnorrSignature.* Para cada entrada $w_t \in \mathbf{w}$, el circuito debe verificar que $\sigma_{i_t}^{\text{sch}} = (C_{i_t}, r_{i_t}, c_{i_t})$ es una firma válida sobre μ bajo $\text{vk}_{i_t}^{\text{sch}}$. Como μ es el mismo para cada w_t , el punto $H_t = \text{HashToCurve}(\mu) \in \mathcal{E}(\mathbb{F}_q)$ es fijo. Se recomputan $R_{1,t}, R_{2,t} \in \mathcal{E}(\mathbb{F}_q)$ como:

$$R_{1,t} = r_{i_t} \cdot H_t + c_{i_t} \cdot C_{i_t}, \quad R_{2,t} = r_{i_t} \cdot G + c_{i_t} \cdot \text{vk}_{i_t}^{\text{sch}},$$

y se deriva el *challenge* $c'_{i_t} = \mathcal{H}(\tau_{\text{sig}}, H_t, \text{vk}_{i_t}^{\text{sch}}, C_{i_t}, R_{1,t}, R_{2,t}) \in \mathbb{F}_r$. La condición de firma correcta es entonces $c_{i_t} = c'_{i_t}$.

3. *Correctitud de la lotería de MITHRIL.* La elegibilidad del índice j_t en J^* se reduce a verificar que ese índice gana la lotería para su UniqueSchnorrSignature y su umbral τ_{i_t} . Sea $\lambda = \mathcal{H}(\tau_{\text{ot}}, \mu)$ el prefijo público de lotería. Luego el circuito recomputa el valor unívoco (gracias a UniqueSchnorrSignature) $e_t = \mathcal{H}(\lambda, x(C_{i_t}), y(C_{i_t}), j_t) \in \mathbb{F}_r$. La condición de elegibilidad es $e_t \leq \tau_{i_t}$, que es exactamente $\text{Win}(i_t, j_t, \text{msg}) = 1$. El circuito no necesita recomputar el *stake* original s_{i_t} del participante.

4. *Unicidad y orden de índices.* Como $\mathbf{w}$ ya llega al circuito con exactamente k índices seleccionados, se verifica su ordenamiento $j_1 < j_2 < \dots < j_k$. Esto también garantiza $|J^*| = k$ porque son índices distintos.

4.7. Halo2 y complejidad

4.7.1. Qué aporta Halo2 en este capítulo

En este capítulo, Halo2 se utiliza como un sistema de aritmetización *PLONKish*: transforma la relación abstracta $\mathcal{R}_{\text{STM-H2}}$ en un circuito organizado en filas, columnas y restricciones locales [17]. Las columnas contienen valores asignados por el *prover*, constantes del circuito o entradas públicas; las restricciones locales verifican operaciones del circuito, mientras que los argumentos de permutación y los *lookups* permiten expresar igualdades entre celdas y pertenencia a tablas. La prueba se construye mediante compromisos polinomiales, cuya instancia concreta utiliza KZG.

La definición abstracta de SNARK y la relación entre circuitos y restricciones algebraicas se presentan en la Definición B.4 y en las secciones iniciales del Apéndice B; el funcionamiento de KZG se desarrolla en la Sección B.7. Esta sección se concentra únicamente en cómo la aritmetización concreta representa los cuatro chequeos de $\mathcal{R}_{\text{STM-H2}}$ y en las consecuencias estructurales de esa compilación para el circuito STM.

4.7.2. Del protocolo SNARK al circuito STM

Como se formaliza en la Definición B.4, un SNARK recibe una instancia pública x , un *witness* w y produce una prueba π que el verificador puede comprobar sin reconstruir el cómputo completo. En este capítulo, la instancia pública está formada por los parámetros

de la época, el mensaje y los compromisos publicados; el *witness* contiene las evidencias de los firmantes, sus caminos de Merkle y los valores auxiliares necesarios para verificarlas.

El objetivo de esta sección no es volver a definir el protocolo **SNARK**, sino describir cómo la relación $\mathcal{R}_{\text{STM-H2}}$ se traduce a las restricciones del circuito que luego procesa Halo2.

4.7.3. Representación del circuito en Halo2

En Halo2, un circuito se organiza como un *layout*: una disposición tabular de filas y columnas que permite reutilizar la misma estructura de restricciones a lo largo del dominio de evaluación. Cada fila representa una instancia local del cálculo, mientras que las columnas almacenan los valores que participan de esas restricciones.

Las columnas *advice* contienen valores asignados por el *prover*; las columnas *fixed* contienen constantes determinadas por la configuración del circuito; y las columnas *instance* contienen las entradas públicas que recibe el verificador. Esta distinción es relevante para el circuito STM, porque las evidencias de los firmantes forman parte del *witness*, mientras que el mensaje, los parámetros de época y los compromisos forman parte de la instancia pública.

4.7.4. De la relación NP a una aritmetización *PLONKish*

Sea $\mathcal{H} = \{\omega^0, \omega^1, \dots, \omega^{n-1}\} \subseteq \mathbb{F}_q$ un subgrupo multiplicativo de orden n , y sea $Z_{\mathcal{H}}(X) = \prod_{h \in \mathcal{H}} (X - h) = X^n - 1 \in \mathbb{F}_q[x]$ su *vanishing polynomial*. La aritmetización de Halo2 consiste en asociar a cada columna del *layout* un polinomio de grado menor que n interpolado sobre $\mathcal{H}$. En lo que sigue usamos estos nombres sólo para describir la aritmetización concreta, mientras que la relación abstracta continúa expresándose mediante $\mathbf{x}$ y $\mathbf{w}$. Notamos:

- $A_1(X), \dots, A_a(X)$ a los polinomios de columnas *advice*,
- $F_1(X), \dots, F_f(X)$ a los polinomios de columnas *fixed*,
- $I_1(X), \dots, I_u(X)$ a los polinomios de columnas *instance*.

Entonces, cada fila $0 \leq r < n$ impone identidades locales de la forma

$$G_j(A_1(\omega^r), \dots, A_a(\omega^r), F_1(\omega^r), \dots, F_f(\omega^r), I_1(\omega^r), \dots, I_u(\omega^r)) = 0,$$

donde cada G_j es un polinomio multivariado de bajo grado.

4.7.5. Restricciones específicas de la relación STM

Aplicado al circuito STM, esto significa que los cuatro chequeos ya introducidos en $\mathcal{R}_{\text{STM-H2}}(\mathbf{x}, \mathbf{w}) = 1$ deben reescribirse como igualdades locales sobre filas del *layout*. En particular:

- los chequeos de Merkle Path se traducen a iteraciones de $\mathcal{H}$ entre filas consecutivas;
- los chequeos de firma se traducen a restricciones sobre puntos de $\mathcal{E}(\mathbb{F}_q)$, escalares en $\mathbb{F}_r$ y desafíos recomputados;
- la lógica de lotería en $\mathbb{F}_r$ se representa con elementos de $\mathbb{F}_q$;
- la unicidad de índices se traduce a una cadena de desigualdades estrictas entre filas adyacentes.

4.7.6. Compilación algebraica y verificación global

Todas estas familias de restricciones se agregan luego en una única identidad global. Más precisamente, si notamos N a la combinación aleatorizada de *gates*, condiciones de borde, consistencia con *public inputs*, permutaciones y *lookups*, entonces el *prover* debe exhibir un polinomio $Q \in \mathbb{F}_q[x]$ tal que $N(X) = Q(X) \cdot Z_{\mathcal{H}}(X)$.

Se debe verificar que $N(x) = 0$ para todo $x \in \mathcal{H}$. Se utiliza KZG como PCS para los polinomios involucrados, de modo que esa identidad se pueda verificar mediante Open en pocos puntos aleatorios derivados con Fiat–Shamir.

4.7.7. Argumentos de permutación, *lookups* y columnas auxiliares

Halo2 utiliza argumentos de permutación y de *lookup* para expresar igualdades entre celdas y pertenencia a tablas, respectivamente. El tratamiento general de estos mecanismos pertenece a la descripción del protocolo y se remite al Apéndice B. En el circuito STM sólo nos interesa su consecuencia concreta: las permutaciones garantizan la consistencia entre valores reutilizados en distintas filas del *layout*, mientras que los *lookups* permiten implementar los *range checks* de la lotería.

En el circuito STM, los *lookups* se usan para *range checks*. La comparación de lotería $e_t \leq \tau_{i_t}$ se interpreta sobre una representación en $\mathbb{F}_q$, y no sobre $\mathbb{F}_r$. Como $\mathbb{F}_q$ no tiene un orden total compatible con $\mathbb{F}_r$, el circuito compara representantes canónicamente elegidos en $[0, q)$. Cada elemento $x \in \mathbb{F}_q$ se descompone como $x = x_{\text{lo}} + 2^{127}x_{\text{hi}}$, con $0 \leq x_{\text{lo}} < 2^{127}$, y $0 \leq x_{\text{hi}} < 2^{128}$.

Estas cotas se fuerzan mediante *lookups* sobre tablas de potencias de dos. Una vez validada la descomposición, la comparación entera se reduce a

$$x < y \iff (x_{\text{hi}} < y_{\text{hi}}) \vee (x_{\text{hi}} = y_{\text{hi}} \wedge x_{\text{lo}} < y_{\text{lo}}).$$

En consecuencia, el costo algebraico del circuito no queda determinado únicamente por las columnas explícitas: también depende de los recursos auxiliares que Halo2 introduce para verificar estas condiciones de consistencia global.

4.7.8. Uso de crates de MIDNIGHT

La descripción anterior corresponde a Halo2/KZG en abstracto. En MITHRIL, sin embargo, la aritmetización concreta no se escribe directamente contra una API mínima de columnas y *gates*, sino a través de las crates `midnight_proofs` y `midnight_zk_stdlib`. Este detalle importa matemáticamente porque fija una transformación intermedia

$$\mathcal{C}_{\text{Mid}} : \mathcal{R}_{\text{STM-H2}} \rightarrow \mathcal{C}_{\text{Halo2/KZG}},$$

que no es neutra: determina el *backend*, la forma de los componentes especializados del circuito disponibles, la reutilización de columnas y el grado mínimo del dominio necesario para sintetizar el circuito.

En la implementación, la relación STM se expresa como un objeto del tipo `Relation`, y luego se compila mediante `MidnightCircuit::from_relation(&circuit)`. Si denotamos por $\mathcal{C}_{\text{STM}}^{\text{Mid}}$ al circuito resultante, entonces su tamaño mínimo de dominio viene dado por un parámetro $k_{\text{mín}} = \text{min_k}()$, de modo que el dominio $\mathcal{H}$ efectivamente usado tiene cardinalidad $n = 2^{k_{\text{mín}}}$.

Aún si formulamos $\mathcal{R}_{\text{STM-H2}}$ sin hacer referencia a columnas concretas, la complejidad real del sistema queda determinada por la compilación que hace MIDNIGHT de esa relación a un *layout* Halo2.

La particularidad más importante de esa compilación es que los distintos componentes no necesariamente aportan columnas disjuntas. Si denotamos por c_{arith} , c_{ecc} , c_{poseidon} , c_{range} a las necesidades de columnas de cada componente principal, la arquitectura de MIDNIGHT no toma en general la suma

$$c_{\text{arith}} + c_{\text{ecc}} + c_{\text{poseidon}} + c_{\text{range}},$$

sino un *pool* de columnas común cuyo tamaño c_{adv} denota la cantidad de columnas *advice* reservadas por la configuración, y cumple

$$c_{\text{adv}} \approx \text{máx}\{c_{\text{arith}}, c_{\text{ecc}}, c_{\text{poseidon}}, c_{\text{range}}\},$$

más algunos recursos adicionales de soporte. Esto justifica por qué no es correcto estimar el costo del *prover* sumando ingenuamente columnas “por componente”.

4.7.9. Grado algebraico del circuito

Diremos que un circuito de Halo2 tiene *grado algebraico* a lo sumo d si toda restricción/*gate* local $G(X_1, \dots, X_m) \in \mathbb{F}_q[X_1, \dots, X_m]$ puede escribirse mediante un polinomio multivariado de grado total a lo sumo d . $C_{\text{STM}}^{\text{Mid}}$ tiene grado algebraico 5.

Esto es así porque la parte dominante del grado viene de Poseidon*. En efecto, la S-box de Poseidon* viene dada por $S(x) = x^5$. Por lo tanto, toda ronda que aplique esa S-box introduce naturalmente términos de quinto grado en las variables de estado del hash. Si denotamos por $\mathbf{s}^{(r)} = (s_1^{(r)}, \dots, s_t^{(r)})$ al estado de una ronda, la transición de una ronda completa adopta la forma

$$\mathbf{s}^{(r+1)} = \mathbf{M} \cdot \left((s_1^{(r)})^5, \dots, (s_t^{(r)})^5 \right) + \mathbf{c}^{(r)},$$

donde $\mathbf{M}$ es una matriz *MDS* y $\mathbf{c}^{(r)}$ es un vector de constantes de ronda.

En consecuencia, las restricciones que implementan esta transición pueden alcanzar grado local 5. Por eso, cuando afirmamos que “el circuito $C_{\text{STM}}^{\text{Mid}}$ es de grado 5”, nos referimos a la cota algebraica dominante de su sistema de restricciones, determinada en este caso por la S-box de Poseidon*.

La compilación concreta en Halo2 incorpora además restricciones auxiliares, como las asociadas a permutaciones y *lookups*. Estas restricciones pertenecen a la representación del circuito y no forman parte de la definición de la S-box. En la práctica, la representación resultante puede requerir fragmentar Q en varios *chunks* cuando su grado excede el rango soportado por la SRS; esta es una consecuencia del *backend* y de la configuración de la prueba, no una modificación de la S-box.

4.7.10. Costo estructural del *prover*

Denotamos por c_{base} al número de polinomios correspondientes a columnas visibles, y por $c_{\text{aux}} = c_{\text{perm}} + c_{\text{lookup}} + c_{\text{quot}}$ al número de polinomios auxiliares inducidos por permutación, *lookups* y *quotient*. Una cota heurística razonable para el costo del *prover* es

$$T_{\text{prover}} = \mathcal{O}((c_{\text{base}} + c_{\text{aux}}) n \log n) + \mathcal{O}(c_{\text{commit}} \cdot \text{MSM}(n)),$$

donde c_{commit} es el número de compromisos KZG efectivamente producidos. Esta fórmula no es un teorema, pero sí es un resumen razonable del costo: transformaciones tipo FFT sobre dominios de tamaño n , construcción de polinomios auxiliares y MSM grandes sobre la SRS para *batch commitments* en KZG. La operación MSM y su incidencia en el costo se describen en la Subsección B.9.3.

El circuito $C_{\text{STM}}^{\text{Mid}}$ parece estar dominado por aritmética de curva elíptica sobre Jubjub, así como evaluaciones repetidas de Poseidon.

4.8. Puente a la implementación final

Las Secciones 4.3, 4.4 y 4.6 dejan a MITHRIL como la capa que define qué estado remoto puede considerarse autenticado y bajo qué condiciones. A partir de este punto, el diseño del zkBridge puede partir de la siguiente interfaz: certificados de StakeDist, certificados de TxSnapshot, una CertChain, claves de verificación agregadas, mensajes de protocolo y evidencia que puede normalizarse para ser consumida por argumentos ZK.

Esta interfaz separa dos responsabilidades. La primera es mantener en la cadena destino una representación persistente del estado de certificación de MITHRIL. En el diseño que sigue, esa responsabilidad se materializa como un UTxO canónico de StakeDist en $\mathcal{B}$: se inicializa desde un cert_{gen} , se actualiza mediante certificados estándar y conserva el vínculo con la cadena de certificados que autentica los parámetros relevantes del comité. La segunda responsabilidad es justificar reclamos puntuales de aplicación, como que una tx_{lock} pertenece a un TxSnapshot certificado. Para esto, basta con transportar la prueba necesaria en la operación que la consume y verificar que sus entradas públicas coincidan con el certificado autenticado.

Visto así, MITHRIL también explica por qué el bridge no necesita revalidar OUROBOROS PRAOS dentro de un circuito ni rehashar certificados completos dentro de un validador on-chain. Lo que debe preservarse es el *binding* entre tres objetos: el certificado aceptado, el estado persistido por el protocolo y el argumento ZK que será verificado por la transacción correspondiente. Ese *binding* puede expresarse mediante entradas públicas elegidas cuidadosamente, recibos `proof_receipt`, raíces de Merkle y UTxOs canónicos, en lugar de repetir toda la lógica criptográfica en cada operación de aplicación.

El Capítulo 5 toma esta interfaz como punto de partida para fijar las decisiones de diseño del zkBridge: qué estado vive en $\mathcal{B}$, qué datos viajan como evidencia efímera, cómo se separan la capa de actualización y la capa de aplicación, y qué verificaciones deben hacerse on-chain u off-chain. Luego, el Capítulo 6 muestra cómo esas decisiones se concretan en validadores Aiken, circuitos auxiliares y transacciones del flujo final.

5. DECISIONES DE DISEÑO

Este capítulo formaliza *decisiones de diseño* para un zkBridge sobre CARDANO, partiendo de las restricciones del modelo eUTxO y de la capa de ejecución (límites `maxTxSize` y `maxTxExUnits`) discutidas en el Capítulo 3, especialmente en la Sección 3.3. Se propone separar el sistema en dos capas. Por un lado, una *capa de actualización* que mantiene, en $\mathcal{B}$, información verificable sobre $\mathcal{A}$; por otro, una *capa de aplicación* que implementa la lógica de *locking/minting* y, como extensión natural, *burning/unlocking*. Esta estructura guía todas las decisiones posteriores.

La decisión central de diseño es **evitar** un argumento ZK del algoritmo nativo de validación de headers de OUROBOROS PRAOS (VRF, selección de slot leader, certificados operacionales y firmas KES), y **adoptar Mithril como *light client*** para certificar *StakeDist por epoch* e *inclusión de transacciones* mediante certificados de MITHRIL y pruebas de Merkle, con la seguridad anclada en el stake [10]. Una alternativa nativa requeriría una cantidad grande de datos contextuales (por ejemplo, el stake relativo del pool productor y el η de época), complicando el circuito y su actualización por época. El uso de MITHRIL desplaza ese costo a un protocolo dedicado a snapshots/certificación, y reduce la superficie de cambios para la futura evolución del consenso de CARDANO (OUROBOROS PERAS y OUROBOROS LEIOS), ya que el Bridge valida certificados MITHRIL en lugar de reglas de consenso directamente [18, 19].

En la implementación final, las decisiones de estado y atomicidad se concretan con patrones típicos de eUTxO: **UTxOs canónicos de estado** identificados por NFTs, consumidos y recreados en cada transición [5]. En particular:

- Un *registro de transacciones ya procesadas* (anti-replay) representado como la raíz de un SMT en el datum de $o_{\text{txs_set}}$, actualizado *atómicamente* junto con el mint para evitar reutilizar la misma tx_{lock} .
- Un *oráculo de StakeDist* materializado como un UTxO canónico con NFT, inicializado por un certificado génesis autenticado mediante vk_{gen} y luego actualizado por época mediante certificados estándar de MITHRIL [15].
- Un mecanismo de recibos autenticados, **proof_receipt**, que permite separar la verificación pesada de certificados MITHRIL del consumo posterior de esos certificados por tx_{sd} y tx_{mint} .

Finalmente, se justifica una estrategia de verificación sucinta *híbrida*. Para certificados MITHRIL, el flujo implementado verifica on-chain argumentos Halo2 del circuito $C_{\text{STM}}^{\text{Mid}}$ mediante un validador Aiken generado y optimizado. Como esa verificación no entra en una única transacción, se divide en **phase1_setup** y **phase2_verify**, y produce como *output* un **proof_receipt** ligado al **statement_hash** del certificado. Para los circuitos auxiliares del bridge, en cambio, se usa Groth16: la verificación de un SNARK basado en pairings sobre BLS12-381 es viable gracias a primitivas criptográficas incorporadas en PLUTUS V3 y a bibliotecas en Aiken como **ak-381**, integradas con circuitos Circom y **snarkjs** [20, 21, 22]. Como alternativa de evolución, se analiza el uso de *zkVMs* (Risc0/SP1) *envueltas* en Groth16 vía recursión (STARK $\rightarrow$ SNARK), pero se explica por qué esa ruta se descarta como decisión inmediata en favor de circuitos dedicados y verificadores medidos contra los límites reales de CARDANO [23, 24, 25, 26].

5.1. Restricciones de Cardano que gobiernan el diseño

En el contexto de un `zkBridge`, las restricciones de CARDANO en términos de `maxTxSize` y `maxTxExUnits` implican que la verificación de argumentos criptográficos y el manejo de datos asociados (por ejemplo, certificados o entradas públicas) deben ser cuidadosamente diseñados para ajustarse a presupuestos limitados. En particular, resulta necesario minimizar la cantidad de verificaciones costosas realizadas en una misma transacción y controlar el tamaño de los artefactos incluidos.

Adicionalmente, el `Bridge` debe tener en consideración el esquema de validación en dos fases de CARDANO: las transacciones que ejecutan scripts deben incluir colateral, el cual puede ser consumido si la validación falla. En el caso de un `zkBridge`, esto implica que errores en la verificación de argumentos, inconsistencias en los datos o condiciones de carrera sobre el estado on-chain pueden traducirse en pérdidas económicas para el operador. Como consecuencia, el diseño del `Bridge` debe priorizar la robustez de las verificaciones y reducir la probabilidad de fallas en la ejecución on-chain.

Entonces:

- La verificación on-chain debe mantenerse simple y acotada en recursos, favoreciendo argumentos sucintos y de bajo costo de verificación.
- El volumen de datos incluidos en las transacciones debe minimizarse, privilegiando representaciones compactas o referencias a estado previamente publicado.
- Cuando una verificación criptográfica no entra en una única transacción, el diseño debe poder particionarla sin perder el vínculo criptográfico entre prueba, certificado y estado consumido posteriormente.
- El diseño debe contemplar la posibilidad de fallas en la validación y sus costos asociados, especialmente en escenarios de contención o cambios de estado concurrentes.

5.2. Arquitectura

5.2.1. Capas: actualización y aplicación

Como anticipan el esquema general del `zkBridge` de la Subsección 2.3.1 y la interfaz de MITHRIL de la Sección 4.8, un patrón general en el diseño de `Bridges` consiste en separar el sistema en dos capas funcionales: una *capa de actualización*, encargada de mantener en $\mathcal{B}$ una representación verificable del estado relevante de $\mathcal{A}$, y una *capa de aplicación*, que utiliza dicha representación para autorizar las operaciones de transferencia de activos.

Debido a las limitaciones de CARDANO, en lugar de replicar explícitamente el estado de $\mathcal{A}$, la capa de actualización debe basarse en *commitments* o certificados criptográficos. En esta tesis, su instancia concreta es la familia de transacciones `txsd`, que publica y renueva en $\mathcal{B}$ un `UTxO` canónico de `StakeDist` identificado por un NFT. El primer estado se crea con `stake_distribution_genesis_tx`, validando un `certgen` mediante `vkgen`; luego, `stake_distribution_standard_tx` actualiza ese estado consumiendo un `proof_receipt` producido por la verificación STM del dominio `stake_distribution_standard`. El `UTxO` resultante queda disponible para ser consultado por la capa de aplicación como *reference input*, sin necesidad de ser consumido en cada operación de minting.

Por su parte, la capa de aplicación utiliza certificados de `TxSnapshot`, también provistos por MITHRIL, para demostrar la inclusión de transacciones específicas en $\mathcal{A}$. Estos certificados permiten justificar operaciones como el minting de activos en $\mathcal{B}$ sin requerir

la verificación completa del protocolo de consenso. En la implementación final, la tx_{mint} consume un `proof_receipt` del dominio `cardano_transactions`, referencia el o_{sd} vigente, verifica mediante `Groth16` que el `txid` de la tx_{lock} pertenece al `TxSnapshot` autenticado y actualiza atómicamente el SMT local de transacciones ya usadas. Esta formulación evita reconstruir on-chain toda la semántica de la transacción de origen: el circuito de pertenencia fija el identificador canónico dentro del snapshot, mientras que los validadores del Bridge controlan que ese identificador y el estado anti-replay sean consumidos de manera coherente.

5.2.2. Actores, responsabilidades y flujo del sistema

La arquitectura del Bridge involucra distintos actores con responsabilidades diferenciadas:

- **Usuarios:** interactúan con el Bridge realizando transacciones de locking en $\mathcal{A}$ y transacciones de minting en $\mathcal{B}$, donde presentan la evidencia compacta necesaria del estado de $\mathcal{A}$ para poder mintear.
- **Operadores del Bridge:** mantienen actualizada la capa de actualización, inician la cadena de StakeDist desde el certificado génesis y luego publican actualizaciones estándar respaldadas por verificaciones STM.
- **Relayers / provers:** recolectan la evidencia remota asociada a tx_{lock} , construyen los argumentos ZK necesarios y ensamblan la información que luego será presentada por tx_{mint} en la cadena destino.
- **Infraestructura de certificación:** provee los mecanismos para generar y validar los certificados utilizados por el Bridge.

Esta separación permite desacoplar la actualización del estado global de $\mathcal{A}$ de la ejecución de operaciones individuales, reduciendo la carga computacional de las transacciones críticas.

A nivel conceptual, el flujo de interacción del Bridge sigue el siguiente esquema:

1. Los operadores inicializan y luego actualizan periódicamente la representación del estado de $\mathcal{A}$ en $\mathcal{B}$ mediante tx_{sd} , manteniendo vigente el UTxO canónico de StakeDist.
2. Un usuario realiza una transacción tx_{lock} de *locking* de activos en $\mathcal{A}$.
3. El usuario, junto con un *relayer/prover*, construye la evidencia necesaria para demostrar la validez de dicha operación, incluyendo certificados de TxSnapshot, argumentos ZK y recibos `proof_receipt` cuando corresponda.
4. El usuario realiza una transacción tx_{mint} de *minting* de activos en $\mathcal{B}$, donde presenta la evidencia, consulta el UTxO de StakeDist como *reference input* y actualiza atómicamente el estado anti-replay del Bridge.
5. Si la verificación es exitosa, tx_{mint} es satisfactoria y el usuario logra mintear los activos en $\mathcal{B}$.

5.3. Estado en eUTxO: atomicidad y concurrencia

El modelo eUTxO favorece representar el estado de una aplicación como una colección de *outputs* consumibles y recreables, en lugar de un estado global mutable compartido [5]. En este enfoque, cada transición de estado se expresa como una transacción que consume

una representación previa y produce una nueva, garantizando que las invariantes del sistema se mantengan localmente. En el contexto de este **zkBridge**, esto lleva a distinguir con claridad tres piezas persistentes de estado: el **UTxO** de fondos bloqueados en la cadena de origen, el $o_{\text{txs_set}}$ que resume el conjunto de transacciones ya procesadas y el o_{sd} publicado en la cadena destino. A ellas se suman **proof_receipt** intermedios, que no representan estado global del bridge, sino evidencia autenticada producida por la verificación de fase 2.

Un requisito fundamental de cualquier **Bridge** es evitar el reprocesamiento de eventos ya utilizados, lo cual podría derivar en inconsistencias o creación indebida de activos. Dado que el conjunto de eventos procesados crece sin una cota fija, resulta impráctico almacenarlo explícitamente on-chain. Como alternativa, el estado mantiene un *commitment*, concretamente la raíz de un **SMT**, que representa el conjunto de eventos ya procesados. La verificación de una nueva transacción **tx** se basa en demostrar que **tx** no pertenece al conjunto previo y que la transición hacia el nuevo estado en el datum es consistente con marcar **tx** como procesada.

El diseño del sistema requiere que la actualización del estado global y la ejecución de las operaciones de aplicación ocurran de manera atómica. En particular, tx_{mint} no debe entenderse como una simple operación de minting aislada, sino como una transacción compuesta que valida la evidencia remota, mintea los activos envueltos y actualiza simultáneamente el estado anti-replay. En el modelo **eUTxO**, esta propiedad puede lograrse mediante la construcción de transacciones que incorporen simultáneamente la ejecución de múltiples *scripts*, donde se valida que tienen dependencia mutua. De esta forma, se preserva la coherencia del sistema.

5.4. Elección de estrategia ZK

5.4.1. Alternativas consideradas

El diseño de un **zkBridge** involucra dos decisiones fundamentales: qué tipo de información debe ser probada y mediante qué mecanismos criptográficos se construyen dichos argumentos. Verificar directamente el protocolo de consenso de $\mathcal{A}$ es demasiado costoso y fuertemente acoplado a los detalles del protocolo. Por lo tanto, el diseño del **Bridge** consta de varios argumentos y validaciones sucintas con roles separados:

- Una verificación **STM** de certificados de **MITHRIL**, materializada mediante $C_{\text{STM}}^{\text{Mid}}$ y consumida posteriormente como **proof_receipt**.
- Un argumento **Groth16** de pertenencia de un **txid** a un **TxSnapshot** autenticado por **MITHRIL**.
- Un argumento **Groth16** de actualización correcta del **SMT** local que resume las transacciones ya procesadas por el **Bridge**.

En cuanto al mecanismo de argumentos **ZK**, se consideran dos enfoques principales: el uso de circuitos dedicados, donde la lógica se expresa directamente como *constraints*, y el uso de *zkVMs*, donde se demuestra la ejecución de un programa general. La elección entre ambos enfoques implica un *trade-off* entre control sobre la eficiencia de los argumentos y facilidad de desarrollo.

5.4.2. Esquema de argumentos ZK

Por una parte, las *zkVMs* presentan ventajas importantes en términos de mantenibilidad, ya que permiten reutilizar implementaciones existentes del protocolo de la cadena fuente. No obstante, en el contexto de CARDANO, este enfoque introduce desafíos adicionales.

En particular, la mayoría de *zkVMs* generan argumentos del tipo STARK, que son de mayor tamaño y requieren mecanismos de compresión o recursión para ser verificables eficientemente on-chain. Esto implica que, incluso adoptando *zkVMs*, el sistema termina dependiendo de un esquema de argumentos sucintos para la verificación final en la cadena destino (esto se conoce como STARK $\rightarrow$ SNARK). El problema surge porque el programa a demostrar utilizando una *zkVM*, asociado al proceso de votación y generación de certificados de MITHRIL, es demasiado grande para el alcance de esta tesis.

Dado el contexto anterior, se privilegian esquemas de argumentos ZK con verificadores eficientes y tamaño reducido de argumento, pero no se fuerza un único backend criptográfico para todo el sistema. Para la verificación de certificados MITHRIL se reutiliza la relación STM de la Sección 4.6, expresada como C_{STM}^{Mid} y verificada on-chain mediante un verificador Halo2 adaptado a Aiken en la Sección 6.7. Para las dos relaciones auxiliares que pertenecen al Bridge, *snapshot_membership* y *tx_set_update*, se adopta Groth16; ambas se desarrollan en las Secciones 6.9 y 6.8. Esta elección se debe a su sucintez, a su eficiencia de verificación y a la existencia de primitivas BLS12-381 en PLUTUS V3 junto con bibliotecas como *ak-381*. La combinación permite mantener acotadas las transacciones finales sin reimplementar toda la certificación MITHRIL en Circom.

5.4.3. Implicancias de diseño

La estrategia adoptada implica una clara separación entre la generación de argumentos y su verificación:

- La generación de argumentos se realiza off-chain, donde no existen restricciones estrictas de cómputo.
- La verificación on-chain está compuesta por chequeos sucintos y eficientes, adecuados a los límites del protocolo.

En particular, el diseño final evita recomputar on-chain hashes completos de certificados de MITHRIL, y reserva ese costo para la capa de pruebas especializada. En su lugar, el vínculo entre la verificación pesada y las transacciones posteriores queda expresado por el *statement_hash* persistido en *proof_receipt*, por los inputs públicos de los circuitos Groth16 y por el consumo atómico de los UTxOs canónicos correspondientes.

5.5. Transacciones del protocolo

El protocolo del Bridge se estructura en torno a un conjunto de transacciones que reflejan las distintas etapas del flujo de activos entre cadenas. Su materialización completa puede consultarse en la Sección 6.10.

En $\mathcal{A}$, los usuarios inician el proceso mediante una transacción de locking de activos tx_{lock} . Este evento constituye el punto de partida para la generación de evidencia verificable en $\mathcal{B}$.

En $\mathcal{B}$, las transacciones principales consisten en la renovación de certificados de MITHRIL por parte de los operadores para mantener disponible la capa de actualización y en el minting final de activos envueltos. En esta tesis, la instancia concreta de $\text{tx}_{\text{updater}}$ es tx_{sd} , que luego se refina en dos casos: `stake_distribution_genesis_tx` para crear el primer o_{sd} a partir del certificado génesis, y `stake_distribution_standard_tx` para avanzar la cadena de certificados consumiendo un `proof_receipt` del dominio `stake_distribution_standard`. Antes de esas transacciones, el flujo publica *reference scripts* y ejecuta verificaciones `phase1_setup` $\rightarrow$ `phase2_verify` cuando el costo del verificador STM debe repartirse entre dos transacciones.

La otra transacción principal es tx_{mint} , cuya instancia final es `bridge_mint_tx`. Allí los usuarios presentan la evidencia de inclusión de la transacción de locking en $\mathcal{A}$, consumen el `proof_receipt` del dominio `cardano_transactions`, consultan el estado de StakeDist como *reference input*, verifican los argumentos Groth16 auxiliares y actualizan atómicamente el registro anti-replay del Bridge.

En un esquema bidireccional de Bridge, el proceso se completa con una transacción de burning de activos en $\mathcal{B}$ (tx_{burn}) y la correspondiente transacción de unlocking de activos en $\mathcal{A}$ ($\text{tx}_{\text{unlock}}$), completando el ciclo de transferencia. Sin embargo, en esta tesis el foco principal de implementación y validación recae sobre el camino $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$, mientras que el camino inverso se trata como una extensión natural de la misma arquitectura.

6. IMPLEMENTACIÓN DEL zkBridge

Durante este capítulo, mantendremos la notación y la arquitectura fijadas en la Sección 5.2: $\mathcal{A}$ denota la cadena de origen del evento a verificar y $\mathcal{B}$ la cadena destino del zkBridge. La implementación se presenta en dos bloques complementarios. Primero describimos la etapa que materializa la lógica on-chain del camino $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$ sobre el modelo eUTxO usando validadores escritos en Aiken [7, 5]. Luego incorporamos los circuitos ZK concretos, la verificación de certificados de MITHRIL y el flujo transaccional final. El camino inverso $\text{tx}_{\text{burn}} \rightarrow \text{tx}_{\text{unlock}}$ se incluye en la base de código, y en este trabajo se lo trata como una extensión natural para que la arquitectura del zkBridge sea bidireccional, pero no lo consideramos como foco central de la implementación.

6.1. Objetivo y alcance del bloque inicial

La Subsección 5.2.1 estableció la separación entre una capa de actualización y una capa de aplicación. La primera mantiene en $\mathcal{B}$ un estado autenticado y compacto sobre $\mathcal{A}$, mientras que la segunda usa ese estado para autorizar operaciones concretas del Bridge. La implementación de este capítulo baja esa división a componentes precisos: UTxOs de estado identificados por NFTs, *datums* que representan el estado persistente del protocolo, *redeemers* que transportan la evidencia puntual de cada transacción, y validadores que fuerzan la atomicidad entre todos esos componentes.

La elección de Aiken como lenguaje de implementación resulta natural en este contexto. Por un lado, permite programar directamente scripts compatibles con el modelo eUTxO. Por otro, ofrece una interfaz relativamente directa para integrar verificadores de Groth16 sobre BLS12-381 mediante bibliotecas como *ak-381*, lo cual encaja con la decisión arquitectónica de dejar el trabajo pesado off-chain y reservar para la cadena destino una verificación sucinta [20, 21, 22]. En este bloque inicial, sin embargo, el objetivo no es cerrar todavía los circuitos de inclusión y de verificación de certificados, sino fijar correctamente las interfaces entre estado, argumentos y transacciones. Por eso aparecen *placeholder circuits* en las posiciones donde el prototipo ya necesita un argumento ZK; sus implementaciones completas se desarrollan en las Secciones 6.9 y 6.8.

También es importante aclarar un punto metodológico. En una iteración intermedia del prototipo se experimentó con recomputar on-chain el hash completo de certificados de MITHRIL para encadenarlos dentro del script. Esa estrategia no se adopta como decisión final del sistema, ya que hashear certificados completos dentro del validador agrega demasiado *overhead* en bytes procesados y *ExUnits*, mientras que el *binding* entre certificado y estado almacenado on-chain puede lograrse de forma más barata y dentro de un circuito off-chain. En consecuencia, en este capítulo describimos la implementación con la decisión final ya incorporada: el estado on-chain publica los certificados y los encadena con la evidencia criptográfica necesaria, mientras que la validación detallada de esa evidencia se desplaza a circuitos especializados y a entradas públicas cuidadosamente elegidas.

6.2. Componentes principales del prototipo

La implementación se organiza alrededor de un conjunto pequeño de módulos y validadores, cada uno con una responsabilidad muy puntual. El repertorio central puede resumirse en la Tabla 6.1. La distinción más importante es entre validadores que gobiernan estado persistente y validadores que gobiernan una acción de aplicación. En la primera categoría quedan el validador de StakeDist y el actualizador del conjunto de transacciones procesadas. En la segunda categoría queda, para esta etapa, el validador de minting.

Validador	Rol	Responsabilidad principal
<code>stake_distribution.mint</code>	Bootstrap	Mintear una única NFT de StakeDist a partir de un certificado génesis y crear el UTxO inicial de estado.
<code>stake_distribution.spend</code> <code>minting_validator</code>	Actualización Aplicación	Consumir el UTxO que contiene el certificado padre y recrearlo con el nuevo certificado publicado por tx_{sd} . Autorizar el <i>minting</i> de tokens envueltos en $\mathcal{B}$ demostrando que existe una tx_{lock} válida en $\mathcal{A}$ y que su evidencia no fue reutilizada.
<code>txs_updater_minting</code>	Anti-replay	Actualizar (en la misma transacción que el minting) la raíz del SMT de transacciones ya procesadas.
<code>unlocking_validator</code> y <code>txs_updater_unlocking</code>	Extensión	Extender el mismo patrón al camino $\text{tx}_{\text{burn}} \rightarrow \text{tx}_{\text{unlock}}$, reservado aquí como versión bidireccional futura.

Tab. 6.1: Validadores relevantes del bloque inicial de implementación.

Alrededor de esos scripts aparecen tres módulos auxiliares particularmente importantes. El primero define los tipos de dominio compartidos por todo el sistema: el datum del UTxO generado por la tx_{lock} , el datum mantenido por el `txs_updater_unlocking` y los *redeemers* que transportan identificadores de transacción, argumentos ZK Groth16 y referencias a UTxOs. El segundo encapsula la verificación genérica de argumentos ZK Groth16 en Aiken. El tercero concentra funciones auxiliares para encontrar UTxOs candidatos válidos, verificar la presencia de NFTs de estado y coordinar la coherencia entre redeemers de distintos scripts. El objetivo es evitar repetir lógica de eUTxO y dejar aisladas las zonas donde luego se reemplazarán *proof stubs* por verificadores reales.

6.3. Representación del estado del bridge en eUTxO

El bloque inicial del prototipo concreta la decisión de la Sección 5.3: en lugar de modelar el estado del zkBridge como un registro global mutable, lo representa mediante una pequeña familia de UTxOs canónicos, cada uno protegido por un script y marcado por una NFT. Esta elección reaprovecha directamente las fortalezas del modelo eUTxO [5]: cada transición de estado consume un UTxO bien identificado y crea su sucesor, con lo cual la preservación de invariantes queda reducida a un chequeo local sobre la transacción que implementa el paso.

En el camino $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$ aparecen tres piezas persistentes.

1. El UTxO de fondos bloqueados en $\mathcal{A}$. Su dirección de pago corresponde al script del `unlocking_validator`, y su datum contiene dos campos: el identificador del zkBridge y la dirección donde deberán recibirse los *wrapped assets* en $\mathcal{B}$ (es decir, el beneficiario autorizado del *minting*).
2. El UTxO del conjunto de transacciones ya procesadas en $\mathcal{B}$. Su datum no almacena explícitamente una lista creciente de transacciones, sino la raíz de un SMT. El

NFT asociado identifica unívocamente este estado y obliga a que toda actualización consuma exactamente el UTxO correcto.

3. El UTxO de StakeDist en $\mathcal{B}$. Su datum contiene el certificado de MITHRIL más reciente aceptado por el protocolo. Al igual que en el caso anterior, un NFT identifica el estado vigente y fuerza a que cada actualización transfiera el control al mismo script.

Esta descomposición responde a tres necesidades distintas. El UTxO de fondos bloqueados los inmoviliza y deja registrada on-chain la intención de transferencia; el UTxO del conjunto de transacciones procesado resuelve el problema de *anti-replay*; y el UTxO de StakeDist permite verificar que la evidencia presentada por un usuario en $\mathcal{B}$ está conectada con una cadena válida de certificados de MITHRIL. Es relevante observar que ninguna de estas estructuras necesita crecer linealmente con la historia del zkBridge. El costo de mantener el estado se controla usando *commitments* compactos y actualizaciones por sustitución, no por acumulación.

Además, la decisión tiene consecuencias positivas en términos de interfaz de transacción: cada paso del protocolo debe transportar solo la porción de información estrictamente necesaria para reconstruir el invariante local. Por eso los *datums* contienen estado estable y reusable, mientras que los *redeemers* contienen evidencia efímera: argumentos ZK, identificadores de transacción y otros campos puntuales que permiten reconstruir la transacción bloqueante. Esto permite mantener los scripts pequeños y respetar `maxTxSize`.

6.4. Flujo transaccional del bloque inicial

La Figura 6.1 resume el flujo que implementa el prototipo para el caso principal de uso. El orden lógico no es accidental: primero debe haber certificados autenticados de StakeDist; luego puede ocurrir una tx_{lock} en $\mathcal{A}$; y recién entonces una transacción de *minting* puede presentar la evidencia necesaria para emitir los *wrapped assets* en $\mathcal{B}$.

6.4.1. Actualización de StakeDist mediante tx_{sd}

La transacción tx_{sd} pertenece a la capa de actualización. Su objetivo no es mover activos del usuario, sino mantener en $\mathcal{B}$ un resumen autenticado del estado de MITHRIL que luego será utilizado por las transacciones de minting. Conceptualmente, el validador admite dos fases.

En la fase de *bootstrap*, un script de minting crea el NFT que identificará al estado de StakeDist y lo deposita en un UTxO cuyo datum contiene un certificado génesis de MITHRIL. Esto debe poder ocurrir una sola vez; por eso se la ata a un único UTxO capaz de acuñar el NFT, y se exige que el UTxO creado quede bajo el control del mismo validador, para que gobierne las actualizaciones posteriores.

En la fase recurrente de actualización, tx_{sd} consume el UTxO que contiene el certificado padre y produce un nuevo UTxO con el mismo NFT y con el certificado nuevo en el datum. Este bloque inicial ya fija varios invariantes útiles: el nuevo certificado debe ser del tipo correcto, debe estar correctamente encadenado con su padre y el control del estado debe permanecer en el mismo script. La interpretación criptográfica de una actualización correcta se desarrolla en la Sección 4.3; la Sección 6.7 completa cómo esa evidencia se consume mediante recibos de verificación.

Un detalle operativo importante es que el UTxO de StakeDist no se consume en cada tx_{mint} . En lugar de eso, aparece como *reference input*. Esta decisión reduce costos y evita

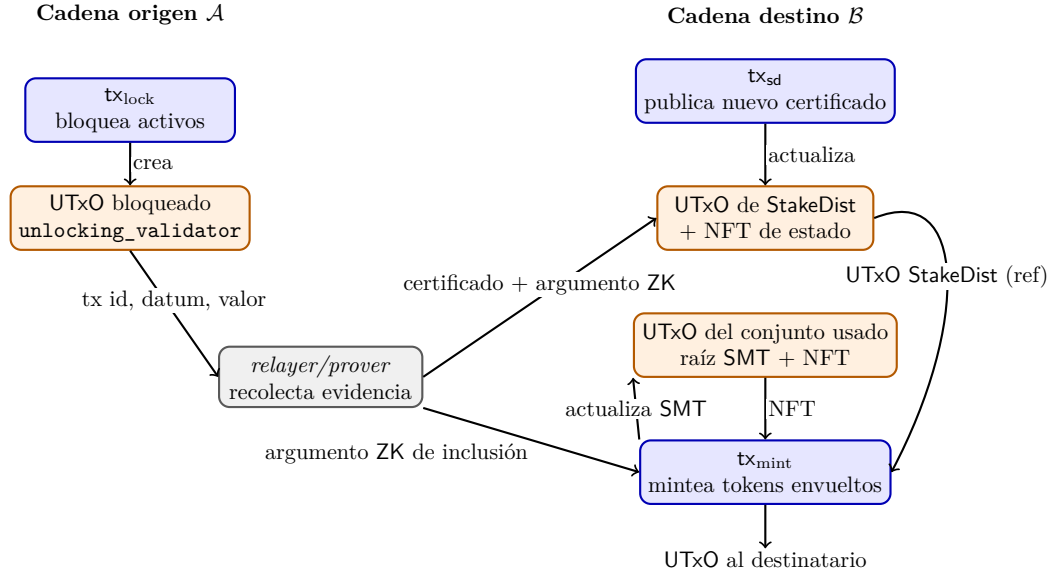


Fig. 6.1: Flujo transaccional del bloque inicial del zkBridge para el camino $tx_{lock} \rightarrow tx_{mint}$. La tx_{lock} crea un UTxO bloqueado en la cadena origen; el *relayer/prover* recolecta el identificador, datum, valor y evidencia criptográfica; en la cadena destino, tx_{sd} actualiza el UTxO de StakeDist y tx_{mint} mientea tokens envueltos mientras actualiza el conjunto anti-replay resumido por una raíz SMT.

que cada usuario compita por modificar un estado que solamente necesita leer.

6.4.2. Transacción de *locking* en la cadena de origen

La tx_{lock} materializa la intención del usuario de iniciar un traslado de valor. En este bloque inicial, la transacción crea un UTxO dirigido al script del `unlocking_validator` de $\mathcal{A}$, el cual contiene la totalidad de los activos bloqueados y un datum con dos componentes: `bridge_id` y `destination_address`. El primero evita que la misma transacción sea reutilizada para reclamar activos en múltiples Bridges incompatibles, mientras que el segundo deja fijado desde $\mathcal{A}$ quién es el destinatario autorizado al hacer tx_{mint} en $\mathcal{B}$.

El campo `destination_address` debe leerse como una dirección de la cadena destino, codificada dentro del datum de la transacción de origen. No asumimos que el espacio de direcciones de $\mathcal{A}$ coincida con el de $\mathcal{B}$, ni que $\mathcal{A}$ pueda interpretar semánticamente esa dirección: para $\mathcal{A}$ es simplemente un dato transportado por el datum, mientras que su significado se recupera al validar tx_{mint} en $\mathcal{B}$.

La implementación adopta en este bloque inicial una convención deliberadamente conservadora: el UTxO bloqueado se toma como la parte semántica relevante de la operación, y la evidencia presentada en tx_{mint} se organiza alrededor de ese UTxO. La información útil para validación queda concentrada en estos campos: referencia del input, dirección de la entrada, datum original, dirección de destino en $\mathcal{B}$ y valor transferido. Esto permite expresar con claridad qué propiedades debe preservar el zkBridge entre la transacción en $\mathcal{A}$ remota y el minting en $\mathcal{B}$, incluso antes de fijar la forma definitiva del circuito de inclusión.

En esta etapa, el prototipo trabaja sobre una formulación isomorfa del problema, donde las cadenas $\mathcal{A}, \mathcal{B}$ son isomorfas. Esto significa que la interfaz de las transacciones en ambas cadenas puede representarse en Aiken con estructuras suficientemente similares

como para chequear localmente la coherencia de la evidencia presentada sobre la tx_{lock} en el script de `minting_validator`. Entonces, la capa de aplicación del `zkBridge` puede validarse antes de introducir las complejidades propias de un circuito completo de inclusión de transacciones de CARDANO. En la Sección 6.9, esta responsabilidad se separa: el circuito fija criptográficamente que el `txid` de la tx_{lock} pertenece a `CardanoTransactions`, pero no prueba por sí solo el destinatario ni el valor transferido. Por eso, la coherencia semántica entre el evento de origen y el minting queda como invariante explícito de la interfaz del Bridge y de los validadores que consumen esa evidencia.

6.4.3. Transacción de minting en la cadena destino

La tx_{mint} es la pieza central de la capa de aplicación. Su trabajo consiste en *mintear* en $\mathcal{B}$ un valor equivalente al bloqueado en $\mathcal{A}$, pero solo si la evidencia aportada por el usuario satisface simultáneamente cuatro condiciones:

1. La tx_{lock} efectivamente pertenece a $\mathcal{A}$ y fue incluida en un `TxSnapshot` autenticado por MITHRIL.
2. El hash o identificador de la tx_{lock} no fue usado antes para mintear.
3. El valor *mintado* del *wrapped asset* coincide con el valor bloqueado en origen.
4. La dirección de destino en $\mathcal{B}$ codificada en tx_{lock} coincide con el receptor efectivo de los *wrapped assets*.

Para verificar esto, el *redeemer* de tx_{mint} asociado a `minting_validator` transporta dos componentes esenciales. La primera es la evidencia ligada a MITHRIL: un certificado de `TxSnapshot`, un `proof_receipt` del dominio `cardano_transactions` que autentica la verificación STM de ese certificado, y un argumento `Groth16` de inclusión del `txid` de tx_{lock} dentro del `TxSnapshot`. La segunda es la información de aplicación necesaria para ligar ese identificador con el reclamo concreto que se está mintiendo.

La implementación del `minting_validator` sigue entonces una estrategia de doble chequeo. Por un lado, verifica la coherencia externa de la evidencia: existe un certificado aceptable, el `TxSnapshot` contiene la transacción indicada y el conjunto anti-replay será actualizado en la misma operación. Por otro lado, verifica la coherencia interna: con los campos provistos en el *redeemer* y con el valor realmente *mintado* por la transacción, reconstruye una versión esperada de la tx_{lock} y comprueba que su identificador coincida con el reclamado. No alcanza con que exista alguna transacción en el `TxSnapshot`, sino que debe tratarse exactamente de tx_{lock} .

Esta última condición se complementa con el mecanismo *anti-replay*. Una vez que existe un camino para probar inclusión, el siguiente problema natural es evitar el reprocesamiento de la misma evidencia. La implementación lo resuelve mediante un SMT cuya raíz se almacena en el datum de un `UTxO` de estado. Cada hoja del árbol representa un *commitment* a una transacción ya utilizada por el `zkBridge`. La decisión de usar un SMT en lugar de una lista explícita responde directamente al crecimiento no acotado del historial: el conjunto puede expandirse indefinidamente sin volver inviable el almacenamiento on-chain. La actualización de ese estado está a cargo del validador `txs_updater_minting`. Su *redeemer* incluye el identificador de la tx_{lock} y del `UTxO` generado por ella, así como un argumento ZK `Groth16` (simulado en este capítulo) que certifica la correcta actualización de la raíz del SMT con la inclusión de la nueva transacción.

Lo importante es que `minting_validator` y `txs_updater_minting` no se validan en transacciones separadas, sino dentro de la misma tx_{mint} . El protocolo no permite que

una transacción agregue un identificador al conjunto usado sin mintear, ni que haga un *minting* sin actualizar ese conjunto. En el modelo eUTxO esto se logra obligando a que ambos scripts participen de la misma operación y comprobando la presencia del otro desde cada uno de ellos. Es precisamente este tipo de decisión la que vuelve defendible al diseño en el marco del modelo eUTxO.

- Desde `minting_validator`, el script exige que entre los inputs exista el UTxO que porta el NFT asociado a `txs_updater_minting`.
- Desde `txs_updater_minting`, el script valida que la transacción está efectivamente minteando tokens del zkBridge.
- Ambos scripts comparan el contenido de sus respectivos *redeemers*, de modo de comprobar que se están refiriendo a la misma `txlock`.

6.5. Decisiones de ingeniería y preparación para los circuitos reales

El bloque inicial no solo implementa validadores; también fija varias decisiones prácticas que condicionan la evolución del sistema. La primera es el uso sistemático de NFTs para marcar estado canónico. Tanto el conjunto de transacciones usadas como el estado de StakeDist dependen de una NFT que identifica de manera unívoca al UTxO válido. Esta técnica evita ambigüedades sobre qué *output* representa el estado vigente y simplifica mucho la lógica de los scripts.

La segunda es la separación entre datos persistentes y evidencia efímera. Los certificados de MITHRIL y las raíces comprometidas viven en datums de UTxOs reutilizables; los argumentos ZK, identificadores de transacción y reconstrucciones parciales de la `txlock` viven en *redeemers*. Esa división es saludable para el costo del protocolo: aquello que muchos usuarios deben leer se publica una sola vez; aquello que cambia en cada operación viaja solo cuando la operación lo necesita.

La tercera, ya mencionada, es la decisión final de no recomputar on-chain hashes completos de certificados de MITHRIL. Desde el punto de vista teórico, tal recomputación podría parecer una forma directa de lograr *binding* entre el dato publicado y el argumento ZK presentado. Sin embargo, en la práctica introduce un costo desproporcionado en serialización, hashing y ExUnits. La versión final del sistema persigue el mismo objetivo de un modo más económico: el UTxO de StakeDist publica el certificado relevante, los argumentos quedan ligados a entradas públicas del circuito que identifican el estado certificado y los validadores solo verifican la consistencia necesaria para que ese enlace sea unívoco. El desarrollo criptográfico de este punto se completa en las Secciones 6.7, 6.9 y 6.8, pero lo mencionamos aquí porque condiciona la forma final de la interfaz on-chain.

Una cuarta decisión es el orden de compilación e instanciación de scripts. Dado que varios validadores necesitan conocer *policy IDs*, *script hashes* o referencias únicas producidas por otros componentes, el prototipo induce un pequeño grafo de dependencias entre validadores. Entonces, este orden es consecuencia directa de cómo los scripts se referencian mutuamente. En la parte del sistema correspondiente a $tx_{lock} \rightarrow tx_{mint}$, dicho grafo puede resumirse como:

1. `stake_distribution.mint` y la política de minting de la NFT del conjunto de transacciones usadas se compilan primero, porque fijan los *policy IDs* de los dos UTxOs de estado sobre los que descansa el resto del protocolo.

2. `stake_distribution.spend` se compila en un segundo paso, ya que depende de la NFT de `StakeDist` definida en el paso anterior para reconocer el UTxO padre y su sucesor.
3. `minting_validator` también se compila en ese segundo paso, porque necesita conocer tanto la política asociada al estado de `StakeDist` como la política de la NFT del actualizador del conjunto usado.
4. `txs_updater_minting` se compila al final, ya que depende de la política del actualizador, pero también del identificador del propio `minting_validator`, al que debe quedar enlazado para forzar la atomicidad de `txmint`.

Por último, la base de código ya incorpora un *test suite* en Aiken para cada uno de los validadores utilizados. Desde el punto de vista de la tesis, estos tests no reemplazan la evaluación de resultados, pero sí aportan una primera validación estructural sobre los casos positivos y negativos donde se rechazan certificados mal encadenados, argumentos ZK inválidos o transacciones que intentan mintear sin actualizar el conjunto *anti-replay*.

6.6. Formalización de los validadores principales

En esta sección fijamos una notación más abstracta para los validadores principales del camino $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$. Seguimos la distinción introducida al presentar el modelo eUTxO entre *spending validators* y *minting validators*. Para los primeros utilizamos la forma $\nu(\text{UtxoRef}, \delta, \rho, \text{tx}) = \text{True}$, mientras que para los segundos utilizamos la forma $\nu_{\text{mint}}(\text{PolicyId}, \rho, \text{tx}) = \text{True}$. En todos los casos, la igualdad a `True` debe leerse como “el validador acepta la transición codificada por la transacción”.

Dos aclaraciones ayudan a leer correctamente esta formalización:

- La notación omite parámetros auxiliares del código Aiken, como referencias locales al input gobernado o constantes de entorno, cuando estos pueden recuperarse del contexto de transacción o forman parte de la instanciación del script.
- El foco está puesto en la semántica de aceptación del validador, no en la organización exacta del código ni en funciones auxiliares de búsqueda sobre ‘inputs’, ‘outputs’, ‘reference inputs’ o ‘redeemers’.

En esta formalización nos concentramos en cuatro validadores: `stake_distribution.mint`, `stake_distribution.spend`, `minting_validator` y `txs_updater_minting`. Los validadores del camino inverso pueden formalizarse de manera análoga.

6.6.1. Validador de bootstrap de StakeDist

$$\text{stake_distribution.mint}(\text{PolicyId}, \rho, \text{tx}) = \text{True}$$

La expresión anterior vale si y sólo si se satisfacen simultáneamente las siguientes condiciones:

1. `tx` consume el único UTxO *o* autorizado para bootstrap del NFT de `StakeDist`.
2. ρ contiene un certificado génesis cert_{gen} de MITHRIL.
3. `tx` crea un UTxO *o** cuyo datum δ^* contiene cert_{gen} .
4. *o** porta el NFT de `StakeDist` bajo el `PolicyId` del `minting_validator`.
5. cert_{gen} es válido según la clave de verificación génesis vk_{gen} fijada por el protocolo.

6.6.2. Validador de actualización de StakeDist

`stake_distribution.spend(UtxoRef,  $\delta$ ,  $\rho$ , tx) = True`

La expresión anterior vale si y sólo si se satisfacen simultáneamente las siguientes condiciones:

1. δ contiene un certificado padre cert_{sd} de MITHRIL.
2. ρ contiene un nuevo certificado $\text{cert}_{\text{sd}}^*$ y un argumento ZK asociado a su validación.
3. tx crea un *output* sucesor o^* cuyo datum δ^* contiene exactamente $\text{cert}_{\text{sd}}^*$.
4. o^* queda bloqueado por el mismo script (el `stake_distribution.spend`).
5. o y o^* portan el mismo NFT de StakeDist.
6. $\text{cert}_{\text{sd}}^*$ es efectivamente un certificado de StakeDist de MITHRIL y está correctamente encadenado con su padre cert_{sd} .
7. El argumento ZK asociado a la validación de $\text{cert}_{\text{sd}}^*$ es aceptado.

6.6.3. Validador de *minting* del zkBridge

`minting_validator(PolicyId,  $\rho$ , tx) = True`

La expresión anterior vale si y sólo si se satisfacen simultáneamente las siguientes condiciones:

1. tx incluye como *reference input* el UTxO canónico de StakeDist o_{sd} , del cual se extrae el certificado padre cert_{sd} necesario para validar el TxSnapshot.
2. ρ contiene un certificado cert_{ts} de TxSnapshot, un `proof_receipt` del dominio `cardano_transactions` y un argumento Groth16 de inclusión del txid de tx_{lock} dentro de cert_{ts} .
3. El `proof_receipt` autentica que cert_{ts} fue aceptado por la verificación STM de fase 2 y que su `statement_hash` coincide con el mensaje firmado del certificado.
4. El argumento Groth16 de inclusión prueba que el identificador reclamado de tx_{lock} pertenece al cert_{ts} autenticado.
5. tx consume el UTxO identificado por el NFT del actualizador del conjunto de transacciones procesadas.
6. El identificador remoto procesado por el actualizador coincide con el identificador reclamado por el `minting_validator`.
7. En la formulación isomorfa de este bloque inicial, a partir de ρ y del valor *mintado* por la transacción puede reconstruirse y verificarse una versión esperada de tx_{lock} . En la versión final, esta condición debe leerse como un invariante de interfaz: la prueba de pertenencia fija el txid, no todos los campos de aplicación.
8. El valor mintado y la dirección de destino en $\mathcal{B}$ coinciden con el valor bloqueado y el destinatario autorizado por tx_{lock} , cuando esos datos forman parte del reclamo de aplicación presentado al validador.

6.6.4. Validador del conjunto de transacciones procesadas

`txs_updater_minting(UtxoRef,  $\delta$ ,  $\rho$ , tx) = True`

La expresión anterior vale si y sólo si se satisfacen simultáneamente las siguientes condiciones:

1. δ contiene la raíz anterior del SMT $r_{\mathcal{T}}$ de transacciones procesadas.
2. tx crea un *output* o^* cuyo datum δ^* contiene la nueva raíz $r_{\mathcal{T}}^*$ del SMT, de modo que queda asociado al conjunto de transacciones procesadas.
3. o^* queda bloqueado por el mismo script (el `txs_updater_minting`).
4. ρ contiene el identificador de tx_{lock} , la referencia al UTxO generado por ella y un argumento ZK de actualización del SMT.
5. El argumento ZK de actualización certifica que la nueva raíz resulta de insertar correctamente la transacción reclamada en el conjunto anterior.
6. tx está efectivamente minteando tokens del zkBridge.
7. Los identificadores de tx_{lock} de ρ y del redeemer ρ^* asociado al `minting_validator` coinciden (porque en tx_{mint} se verifican ambos validadores).

Retomamos la arquitectura fijada en la Sección 5.2 y las interfaces de validadores formalizadas en la Sección 6.6, y nos concentramos ahora en los circuitos ZK concretos, su integración con Aiken y las decisiones necesarias para llegar a una versión viable del zkBridge. En este segundo bloque ya no alcanza con describir el estado persistente del protocolo o la forma abstracta de sus validadores. Hay que mostrar cómo se materializa la verificación criptográfica real dentro de las restricciones de CARDANO, cómo se encadenan los artefactos de MITHRIL con la lógica on-chain y qué compromisos de ingeniería fueron necesarios para que el flujo final entre efectivamente en presupuesto.

La Sección 6.7 se concentra en el caso más exigente de esa traducción: llevar el circuito Halo2 de MITHRIL al entorno transaccional de CARDANO. Allí veremos que la dificultad principal no fue sólo extraer un verificador correcto, sino rediseñar el flujo de verificación y de consumo de certificados para respetar límites de CPU, memoria y tamaño de transacción sin perder *binding* criptográfico entre pruebas, certificados y UTxOs de estado.

6.7. Del circuito Halo2 de Mithril a Cardano

Era necesario cerrar la brecha entre dos entornos que, aun compartiendo la palabra Halo2, imponen restricciones muy distintas. Como se estudió en las Secciones 4.6 y 4.7, el circuito C_{STM}^{Mid} estaba diseñado para producir un argumento sucinto sobre BLS12-381. Del lado de CARDANO no alcanzaba con que el argumento fuera criptográficamente correcto, sino que debía existir un camino realista para transportar el certificado, reconstruir el *transcript*, ejecutar la verificación en Aiken y mantenerse por debajo de los límites simultáneos de `maxTxExUnits` y `maxTxSize`. Esta sección se enfoca en cómo volver la relación $\mathcal{R}_{STM-H2}$ verificable on-chain en CARDANO. Nos apoyamos en un *fork* de `plutus-halo2-verifier-gen`, en una serie de optimizaciones sobre el verificador Aiken emitido por esa herramienta y en una reorganización del flujo de transacciones del zkBridge para lograr tx_{sd} y tx_{mint} .

6.7.1. Necesidad y alcance de la adaptación de `plutus-halo2-verifier-gen`

La herramienta original `plutus-halo2-verifier-gen` fue publicada por IOG para generar verificadores Aiken de circuitos pequeños construidos directamente sobre `halo2_proofs`, con ejemplos como multiplicación simple, *lookups* y circuitos de prueba [27, 28]. A partir de la *verification key* y de una representación de las expresiones del circuito, la herramienta producía el código del validador correspondiente.

Ese flujo no podía aplicarse directamente a C_{STM}^{Mid} . El circuito utilizado por MITHRIL está construido sobre las bibliotecas de MIDNIGHT y emplea tipos, relaciones y pasos de verificación que la herramienta original no podía extraer ni representar. Para salvar esa incompatibilidad desarrollamos un *fork* de `plutus-halo2-verifier-gen`. La adaptación no consistió únicamente en admitir un circuito de mayor tamaño: fue necesario extender el proceso completo de extracción y generación para traducir el verificador de MIDNIGHT a una representación ejecutable en Aiken.

En consecuencia, el *fork* modificó las principales capas del proceso de generación:

- agregamos soporte para el circuito C_{STM}^{Mid} , con sus *gadgets*, asignaciones de *witness*, tipos auxiliares y *runtime* específico.
- incorporamos un extractor nuevo para relaciones de bibliotecas de MIDNIGHT, junto con una capa de conversión para mapear expresiones, compromisos, evaluaciones, rotaciones y pasos del protocolo a la representación intermedia que el generador ya entendía.
- eliminamos el backend Plinth y reorientamos toda la herramienta a Aiken.
- el generador pasó a exportar entidades estables consumibles por el zkBridge. Fija qué bytes representan la prueba, qué valores se usan como *public inputs*, qué parte del acumulador se preserva entre transacciones y cómo se serializa el `ReducedRedeemer` que más tarde se volverá a hashear on-chain.

La necesidad de adaptar esta capa se vuelve concreta al comparar los PCS y protocolos de apertura involucrados. Tanto la herramienta original como el circuito real de MITHRIL usan compromisos KZG [29], pero no la misma estrategia de multi-apertura. Los ejemplos del generador utilizan GWC19, compatible con `halo2_proofs`, mientras que las bibliotecas de MIDNIGHT utilizan Halo2 Multi-Open. La primera estrategia organiza la agregación alrededor de puntos de evaluación; la segunda agrupa los polinomios por *conjuntos de puntos* y reconstruye de otra forma la apertura agregada. Ambas construcciones y su diferencia algebraica se desarrollan en las subsecciones B.7.3 y B.7.4 del Apéndice B. Como el verificador resultante tiene una estructura distinta, no podíamos reutilizar sin cambios el extractor, el código generado ni el análisis de costos de los ejemplos originales.

6.7.2. Optimizaciones del validador Aiken

Una vez obtenido un verificador correcto para C_{STM}^{Mid} , el siguiente obstáculo es económico: pasar a respetar los límites de `maxTxSize` y `maxTxExUnits` de CARDANO. Parte del costo proviene de la frontera entre la representación serializada de la prueba y las operaciones de grupo ejecutadas on-chain. Como se explica en la subsección A.4.1 del Apéndice A, los puntos llegan codificados como bytes y deben descomprimirse para reconstruir elementos de $\mathbb{G}_1$ antes de poder sumarlos o multiplicarlos por escalares. El validador Aiken emitido por el generador realizaba estas descompresiones, además de multiplicaciones escalares, sumas y pairings sobre BLS12-381. Incluso antes de considerar el flujo completo del bridge, el verificador standalone del circuito real de MITHRIL quedaba fuera del presupuesto de CPU.

Las primeras mejoras se hicieron sobre el código Aiken generado, no sobre el protocolo del zkBridge. En el caso del MSM izquierdo se reemplazó la evaluación genérica basada en `foldl` y listas de `MSMElement` por una cadena *inline* de llamadas a `addG1` y `scaleG1`, eliminando *overhead* de la máquina CEK de Aiken, y también omitimos multiplicaciones

escalares redundantes cuando el coeficiente es la identidad. La misma idea se extendió luego al MSM derecho.

Sobre ese mismo MSM derecho eliminamos dos fuentes de costo adicionales. La primera era una conversión redundante del punto `vanishing_g`: una vez computado como elemento de $\mathbb{G}_1$, el verificador lo serializaba nuevamente como `ByteArray` y luego volvía a descomprimirlo antes de utilizarlo. Mantener el punto en su representación de grupo evita ese recorrido sin alterar la semántica de la verificación. La segunda era la descompresión duplicada de los puntos W_i , utilizados tanto en el lado izquierdo como en el derecho. Reutilizar los elementos ya reconstruidos evita repetir llamadas costosas a `bls12_381_G1_uncompress`.

También eliminamos términos algebraicamente muertos del cálculo del acumulador del polinomio *vanishing*. En particular, el patrón `scaleG1(zero, xn)` seguido de una suma con el primer compromiso del bloque es innecesario, porque para todo $s \in \mathbb{F}_r$ valen $s \cdot \mathcal{O} = \mathcal{O}$, y $\mathcal{O} + P = P$. Junto con pequeñas mejoras adicionales, esto produjo ahorros medibles de CPU en todos los circuitos de referencia y permitió que ejemplos pesados del repositorio original, como `atms_with_lookups`, entraran en presupuesto.

Sin embargo, ese conjunto de optimizaciones no resolvió por sí mismo el caso C_{STM}^{Mid} . El verificador de MITHRIL mejoró, pero siguió siendo demasiado costoso para una sola transacción. Por eso la solución final no fue simplemente “emitir mejor código”, sino ver cómo reorganizar el flujo para que el costo pesado de verificar un certificado de MITHRIL se pague una sola vez y luego pueda ser consumido por la cadena de StakeDist y por tx_{mint} sin reejecutar el mismo chequeo.

Aún si el presupuesto de CPU fuera suficiente, cargar en cada transacción los scripts grandes del verificador STM, del `txs_updater_minting` y del `minting_validator` volvía inviable el flujo completo al superar el `maxTxSize` de CARDANO. Para esto, aprovechamos los *reference scripts* introducidos en el CIP-33 [30] para persistir scripts pesados on-chain y referenciarlos desde las transacciones posteriores en lugar de reenviarlos como parte del *witness set*. Podría ser sólo un detalle de implementación, pero fue estrictamente necesario para que el zkBridge final entre efectivamente en el modelo eUTxO de CARDANO. En el flujo concretamente implementado, esa decisión aparece en tres transacciones:

- `publish_phase1_reference_script`,
- `publish_minting_txs_updater_spend_reference_script`,
- `publish_bridge_minting_reference_script`.

Esta decisión de partir el flujo y publicar scripts pesados responde al conjunto de builtins disponible hoy. Si prospera CIP-133, que propone una *builtin* nativa de MSM sobre BLS12-381 [31, 32], una parte importante del bloque lineal que hoy domina el costo del verificador podría colapsar en una sola operación, reduciendo drásticamente tanto el costo algebraico y el *overhead* CEK, y permitiría hacer la verificación de un certificado de MITHRIL sin una transacción de dos fases.

6.7.3. Partición de la verificación en dos transacciones

Como seguíamos fuera del presupuesto de `maxTxExUnits`, debimos partir la validación de un argumento STM de un certificado de MITHRIL en dos transacciones consecutivas. En C_{STM}^{Mid} , el trabajo costoso se concentra en el acumulador derecho (recordemos que estamos en Halo2 Multi-Open), es decir, en la combinación de compromisos por *point sets*, potencias de x_1 y x_4 , reconstrucciones interpoladas y el término final que incorpora $f(x_3)$. Ese acumulador es el bloque lineal dominante de `ExUnits`.

Hicimos el siguiente modelo algebraico usado para diseñar el *split*. Para cada conjunto de puntos S_j , el *verifier* arma una combinación lineal Q_j de compromisos usando potencias de x_1 , evalúa la parte interpolada $R_j(x_3)$, y luego combina los distintos conjuntos con potencias de x_4 para formar el acumulador derecho. Si denotamos por I_j al conjunto de índices de compromisos de S_j , entonces partir el verificador equivale a escribir $I_j = I_j^{(1)} \sqcup I_j^{(2)}$ y descomponer (por linealidad) tanto Q_j como $R_j(x_3)$ y el acumulador final en una suma de una parte de fase 1 y una parte de fase 2. La primera transacción no “verifica media prueba” en un sentido informal, sino que computa un prefijo algebraicamente bien definido del mismo acumulador que la segunda transacción terminará de cerrar.

Para este diseño, debemos decidir cómo transportar un estado intermedio compacto. En lugar de persistir la prueba completa o de recomputar todo en la segunda transacción, la fase 1 produce un *datum* enriquecido que preserva exactamente lo necesario para retomar el cálculo: un acumulador parcial comprimido, tres desafíos del *transcript* y un escalar adicional. En la implementación, esa estructura aparece como **Phase1State**, mientras que la segunda transacción recibe un **ReducedRedeemer** que contiene sólo las contribuciones restantes necesarias para completar el bloque lineal y ejecutar el chequeo final de pairing.

El punto de corte finalmente no se eligió sólo por cálculo estático, sino por medición directa. Aunque el análisis preliminar sugería un corte algo más tardío, al instrumentar por separado **phase1_setup** y **phase2_verify** vimos que la fase 1 absorbía más trabajo fijo del esperado y que el mejor equilibrio real aparecía antes en la entrada 17 del primer conjunto de puntos. Como los cuatro conjuntos de puntos de C_{STM}^{Mid} tienen tamaños 44, 6, 3, 2, la implementación final deja a la fase 1 las contribuciones de los primeros 17 puntos del primer conjunto, y a la fase 2 todo lo restante. Esta partición fue la que produjo el mejor balance medido entre ambas transacciones (en poco más de 7 billones de unidades), y lo hizo viable.

En el flujo integrado esta partición no se ejecuta para todo certificado que aparece en el protocolo. Las dos corridas **phase1_setup** $\rightarrow$ **phase2_verify** que permanecen en el camino on-chain corresponden a los dominios **stake_distribution_standard** y **cardano_transactions**. El certificado génesis de MITHRIL cumple otro rol: inicializa la cadena de StakeDist y se autentica directamente en la *minting policy* del NFT de StakeDist mediante la firma génesis σ_{gen} y la clave vk_{gen} . Por eso no hay un **proof_receipt** asociado al dominio **stake_distribution_genesis**.

6.7.4. Validez del transcript y *binding* entre fases

Partir la verificación en dos transacciones introduce una obligación adicional: la fase 2 no puede aceptar un **ReducedRedeemer** sólo porque completa algebraicamente el acumulador iniciado por la fase 1. Debe comprobar, además, que ese ρ reducido corresponde al mismo *transcript*, a la misma prueba serializada y al mismo *statement* público que la primera transacción empezó a procesar. Si esa continuidad fallara, aparecería una superficie de ataque de empalme: un adversario podría usar una fase 1 válida para fijar un estado intermedio y luego intentar cerrar la fase 2 con contribuciones provenientes de otra prueba, otro certificado o un conjunto incompatible de compromisos.

El diseño cierra esa posibilidad con tres mecanismos complementarios. Primero, **Phase1State** persiste el acumulador parcial y los desafíos necesarios para que la continuación del verificador sea determinista. Segundo, **reduced_hash** ata ese estado intermedio al **ReducedRedeemer** exacto que deberá aparecer en fase 2. Tercero, **statement_hash** conecta

el recibo final con el mensaje firmado por el certificado de MITHRIL que luego consumirán StakeDist y tx_{mint} . En conjunto, estos tres enlaces convierten una verificación partida en dos transacciones en una única verificación lógica del mismo objeto criptográfico.

En la primera transacción, `phase1_setup` recibe la prueba serializada y las dos entradas públicas del circuito, reconstruye el prefijo barato del verificador y deja persistido un datum de tipo `Phase1State` dentro del UTxO que luego consumirá la fase 2. Ese estado contiene `rhs_prefix`, que es un acumulador parcial comprimido en $\mathbb{G}_1$; `reduced_hash`, que fija el hash esperado del `ReducedRedeemer`; los desafíos `x1`, `x3` y `x4`; y el escalar `v`, ya pre-computado. Además, el datum extendido del UTxO de fase 2 conserva un `statement_hash`, que en la familia de circuitos **Lagrange** coincide directamente con la segunda entrada pública del circuito, es decir, con el mensaje firmado por el certificado de MITHRIL.

Es importante notar qué no se persiste. El desafío x_2 no reaparece en el datum porque su información ya quedó absorbida en los cálculos que produce la fase 1: una vez reconstruidos $f(x_3)$ y v , la segunda transacción ya no necesita rederivar x_2 para cerrar el chequeo. Esta decisión reduce el tamaño del estado intermedio y, al mismo tiempo, hace explícita una propiedad conceptual relevante: el datum no guarda “todo el transcript”, sino solamente la porción de estado necesaria para volver determinista la continuación de la verificación.

El segundo mecanismo de *binding* es el hash del `ReducedRedeemer`. La fase 1 serializa canónicamente ese ρ reducido, computa BLAKE2S-256 y persiste el resultado como `reduced_hash`. Ese mismo valor se usa además como nombre del NFT emitido en fase 1 y también del NFT que la fase 2 mintea cuando la verificación final tuvo éxito. La consecuencia es fuerte: el estado intermedio, el activo que identifica el UTxO de fase 2 y el recibo final de prueba verificada quedan todos ligados al mismo objeto serializado. Si la fase 2 intentara presentar un `ReducedRedeemer` distinto, el rehash fallaría antes de ejecutar la parte cara del verificador.

El *binding* final se da entre el `statement_hash` y el certificado de MITHRIL. La fase 2, cuando termina con éxito, no sólo consume el UTxO intermedio sino que crea un `proof_receipt` con un datum `ProofReceiptDatum { statement_hash }`. Ese valor es el mismo que más tarde leerán los validadores de StakeDist y de tx_{mint} para comprobar que el certificado presentado lleva exactamente el *signed_message* que ya fue autenticado por $C_{\text{STM}}^{\text{Mid}}$. Además, ese `statement_hash` no queda respaldado meramente por estar escrito en un datum: queda respaldado por el hecho de que el UTxO fue creado junto con el NFT mintado bajo la `PolicyId` de la fase 2, y los validadores downstream buscan precisamente `proof_receipt` marcados por esa política. Así, el `proof_receipt` no es sólo un portador de datos, sino una evidencia autenticada por los scripts que ejecutaron la segunda mitad del verificador.

6.7.5. Conexión de UTxOs, tx_{sd} y tx_{mint}

El `proof_receipt` anteriormente generado conecta el verificador STM con el resto del bridge sólo para los certificados que pasan por la ruta de dos fases. En el subflujo de StakeDist existe primero un caso de arranque: `stake_distribution_genesis_tx` no consume un `proof_receipt`, sino que recibe el `cert_gen` y verifica su firma génesis σ_{gen} contra vk_{gen} mediante la primitiva Ed25519 disponible en Aiken. Si esa autenticación y las condiciones del certificado son válidas, la transacción mintea el NFT de distribución y crea el primer o_{sd} .

Recién después aparece la ruta estándar de StakeDist. `stake_distribution_standard_tx`

consume el recibo del dominio `stake_distribution_standard` junto con el o_{sd} padre, verifica la continuidad entre el certificado padre y el certificado nuevo, y emite el nuevo o_{sd} canónico que el resto del protocolo considerará vigente. De esta manera, el estado persistente del bridge en $\mathcal{B}$ queda formado por una cadena de UTxOs que parte de un certificado génesis autenticado por vk_{gen} y luego sólo puede avanzar si cada certificado estándar nuevo está ligado a un `proof_receipt` de verificación STM ya aceptado on-chain.

En el camino $tx_{lock} \rightarrow tx_{mint}$, la transacción `bridge_mint_tx` consume el `proof_receipt` del dominio `cardano_transactions`, consume también el o_{txs_set} y referencia el o_{sd} vigente (como *reference input*, ver CIP-31 [33]). La separación de roles es aquí muy precisa. El `proof_receipt` de `cardano_transactions` prueba que un $cert_{ts}$ ha pasado la verificación STM y fija el `statement_hash` que debe coincidir con el *signed_message* del certificado. El o_{sd} aporta el certificado padre confiable contra el cual debe encadenarse ese $cert_{ts}$. Y el `txs_updater_minting` garantiza que la transacción bloqueante de $\mathcal{A}$ no pueda reutilizarse para mintear dos veces, reemplazando la nueva raíz de SMT en el $o_{txs_set}^*$ actualizado.

Recién sobre esa base, el argumento Groth16 de pertenencia de la tx_{lock} dentro del TxSnapshot certificado adquiere el significado que el zkBridge necesita: argumenta inclusión en un snapshot autenticado por un certificado de MITHRIL ya enlazado a la CertChain mantenida on-chain.

6.8. El circuito `tx_set_update`

En la Subsección 6.4.3 introdujimos el problema *anti-replay* del camino $tx_{lock} \rightarrow tx_{mint}$: aun si el zkBridge dispone de una prueba correcta de que cierta tx_{lock} ocurrió en CARDANO y pertenece a un TxSnapshot autenticado por MITHRIL, todavía falta evitar que esa misma evidencia se reutilice para mintear dos veces. La función del circuito `tx_set_update` es precisamente cerrar esa brecha. No prueba pertenencia de una transacción en un snapshot certificado por MITHRIL; esa es la responsabilidad del circuito desarrollado en la Sección 6.9. Lo que prueba aquí es que el SMT local del bridge, que resume el conjunto de transacciones ya consumidas por el protocolo, fue actualizado correctamente al pasar de un estado donde la transacción reclamada estaba ausente a otro donde esa misma transacción figura como presente.

Esta distinción es importante porque el circuito se verifica on-chain en un lugar muy preciso del flujo. El argumento Groth16 asociado a `tx_set_update` es consumido por `txs_updater_minting`, mientras que el argumento de pertenencia en el snapshot certificado es consumido por `minting_validator`. Ambos scripts deben validarse dentro de la misma tx_{mint} . Por eso conviene separar con cuidado tres niveles que en el código aparecen superpuestos: el *statement* semántico del circuito, su interfaz aritmética compilada para Groth16 y la manera en que el validador Aiken recupera de la transacción los datos necesarios para invocar al verificador.

6.8.1. Modelo matemático del SMT

Fijamos un SMT $\mathcal{T}$ de altura $H = 256$, cuyas hojas están indexadas por elementos de $\mathcal{B}_{32}$, o equivalentemente de $\{0, 1\}^{256}$. El circuito trabaja sobre el cuerpo escalar $\mathbb{F}_r$ de BLS12-381, de modo que cada nodo interno del árbol es un elemento de $\mathbb{F}_r$. Las hojas usan como codificación $\perp = 0 \in \mathbb{F}_r$ para ausencia de la transacción y $1 \in \mathbb{F}_r$ si está presente.

El hash interno del árbol es `Poseidon` sobre $\mathbb{F}_r$. En la implementación concreta del repositorio, esta instancia corresponde a la variante conocida como `Poseidon255` sobre BLS12-381, pero para la formalización de la relación basta verla como una función determinista $\text{Poseidon}_2 : \mathbb{F}_r \times \mathbb{F}_r \longrightarrow \mathbb{F}_r$, de modo que cada nodo interno se obtiene aplicando `Poseidon` a su hijo izquierdo y su hijo derecho, en ese orden.

La posición de una transacción dentro del SMT viene determinada por el identificador canónico de la transacción en CARDANO. Si notamos $\text{txid} \in \mathcal{B}_{32}$ y escribimos sus bits en orden *big-endian* como $\text{txid} = (b_0, \dots, b_{255})$ con $b_d \in \{0, 1\}$, entonces el bit b_d fija la dirección tomada en la profundidad d . En la implementación `circom`, esto está codificado en el arreglo `mt_path_indexes`.

Sea ahora $\text{sib} = (h_0, \dots, h_{255}) \in \mathbb{F}_r^{256}$ el vector de hermanos del camino asociado a txid , escrito también en orden de raíz hacia hoja. El mismo vector `sib` sirve simultáneamente para probar ausencia antes de la actualización y presencia después de ella, porque la única diferencia entre ambos estados es el valor de la hoja terminal. Ningún nodo hermano fuera de ese camino se modifica.

6.8.2. *Public inputs y witness* del circuito

Los *public inputs* del circuito están dados por la tripla $\mathbf{x} = (\text{txid}, r_{\text{in}}, r_{\text{out}})$, donde $\text{txid} \in \mathcal{B}_{32}$ es el identificador de la transacción, $r_{\text{in}} \in \mathbb{F}_r$ es la raíz del conjunto antes de procesarla y $r_{\text{out}} \in \mathbb{F}_r$ es la raíz después de marcarla como usada. El *witness* es simplemente $\mathbf{w} = \text{sib} = (h_0, \dots, h_{255}) \in \mathbb{F}_r^{256}$.

La instancia `circom` es más detallada porque debe hacer explícita la codificación binaria del *statement*. Recibe:

1. `tx_id_b[32]`, que representa `txid` como 32 bytes, el cual sirve para imponer chequeos de rango byte a byte, obtener la descomposición binaria de `txid` y empaquetar luego el digest en el formato esperado por el verificador `Groth16`;
2. `mt_root_in` y `mt_root_out`, que copian las raíces como elementos de $\mathbb{F}_r$;
3. `mt_path_indexes[256]`, que explicita el vector de bits del camino, de modo que el circuito fuerza cada una de sus entradas a coincidir con el bit *big-endian* correspondiente de `tx_id_b`;
4. `mt_path_values[256]`, que contiene el vector `sib`.

Los *outputs* públicos efectivos del circuito, que luego pasan a ser los *public inputs* del verificador `Groth16` on-chain, son: $(\text{tx_id_hi}, \text{tx_id_lo}, \text{mt_root_in}, \text{mt_root_out}) \in \mathbb{F}_r^4$. Las dos raíces se exponen directamente como un único elemento del cuerpo cada una, mientras que `txid` se divide en dos mitades de 16 bytes (porque $2^{256} > |\mathbb{F}_r|$) y cada mitad se interpreta como un entero *big-endian* de 128 bits:

$$\text{txid}_{\text{hi}} = \sum_{i=0}^{15} \text{txid}[i] \cdot 256^{15-i}, \quad \text{txid}_{\text{lo}} = \sum_{i=16}^{31} \text{txid}[i] \cdot 256^{31-i}.$$

En el validador de Aiken, usamos esta convención para reconstruir `tx_id_hi` y `tx_id_lo` a partir del *redeemer*.

6.8.3. La relación NP de `tx_set_update`

Tenemos $\mathcal{X} = \mathcal{B}_{32} \times \mathbb{F}_r \times \mathbb{F}_r$, $\mathcal{W} = \mathbb{F}_r^{256}$, y consideramos la relación $\mathcal{R}_{\text{tx-set-update}} \subseteq \mathcal{X} \times \mathcal{W}$. Consideramos $\mathbf{x} = (\text{txid}, r_{\text{in}}, r_{\text{out}}) \in \mathcal{X}$ y $\mathbf{w} = (h_0, \dots, h_{255}) \in \mathcal{W}$. Sea además

$(b_0, \dots, b_{255})$ la expansión *big-endian* de txid.

Definimos dos secuencias de acumuladores, una para la reconstrucción con hoja vacía y otra para la reconstrucción con hoja presente. Para $u \in \{0, 1\}$, definimos recursivamente $a_{256}^{(u)} = u$, y para cada $d = 255, 254, \dots, 0$:

$$a_d^{(u)} = \begin{cases} \text{Poseidon}_2(a_{d+1}^{(u)}, h_d), & \text{si } b_d = 0, \\ \text{Poseidon}_2(h_d, a_{d+1}^{(u)}), & \text{si } b_d = 1. \end{cases}$$

La recurrencia recorre el camino en sentido desde la hoja hacia la raíz. El valor $a_0^{(u)}$ es entonces la raíz reconstruida cuando se fija la hoja terminal igual a u . Por lo tanto, decimos que $\mathcal{R}_{\text{tx-set-update}}(\mathbf{x}, \mathbf{w}) = 1$ si y sólo si se satisfacen simultáneamente las condiciones $a_0^{(0)} = r_{\text{in}}$ y $a_0^{(1)} = r_{\text{out}}$. Es decir, la relación $\mathcal{R}_{\text{tx-set-update}}$ afirma:

El camino del SMT determinado por los bits de txid cumple que, usando los mismos 256 hashes hermanos sobre ese camino, la hoja vacía 0 reconstruye la raíz anterior r_{in} y la hoja presente 1 reconstruye la nueva raíz r_{out} .

Como ambos chequeos utilizan los mismos vectores, la nueva raíz r_{out} no puede corresponder a una mutación arbitraria del árbol: debe ser el resultado de insertar exactamente la transacción txid en el conjunto representado por r_{in} , sin modificar ninguna otra hoja. El lenguaje asociado a $\mathcal{R}_{\text{tx-set-update}}$ es

$$\mathcal{L}_{\text{tx-set-update}} = \{(\text{txid}, r_{\text{in}}, r_{\text{out}}) \in \mathcal{X} \mid \exists \text{sib} \in \mathcal{W} : \mathcal{R}_{\text{tx-set-update}}(\mathbf{x}, \mathbf{w}) = 1\}.$$

6.8.4. Familias de restricciones del circuito

La relación anterior es implementada en `circom` para ser convertida a un circuito aritmético y luego a un QAP.

1. Un chequeo de rango de cada byte en `tx_id_b`: para cada $0 \leq j < 32$, el circuito instancia un componente `AssertByte` y exige $0 \leq \text{tx_id_b}[j] < 256$.
2. La descomposición de cada byte `tx_id_b[j]` en bits mediante un componente `ToBits(8)`. Luego, forzamos cada entrada de `mt_path_indexes` a ser booleana usando `AssertBit`, e imponemos la igualdad `mt_path_indexes[d] = b_d`, para cada $0 \leq d < 256$. Esto impide sustituir el camino de otra clave por uno compatible con el mismo sib.
3. El componente `SparseMerkleRoot(256)` llamado `inPath` recibe la hoja 0, el vector `mt_path_indexes[256]` y el vector `mt_path_values[256]`, y restringe `inPath.root = mt_root_in`, lo que equivale a $a_0^{(0)} = r_{\text{in}}$.
4. El componente `SparseMerkleRoot(256)` llamado `inPath` recibe la hoja 0, el vector `mt_path_indexes[256]` y el vector `mt_path_values[256]`, y restringe `inPath.root = mt_root_out`, lo que equivale a $a_0^{(1)} = r_{\text{out}}$.
5. Dos componentes `Pack16Bytes` empaquetan la mitad alta y la mitad baja de `tx_id_b[32]` en los enteros públicos `tx_id_hi` y `tx_id_lo`. Las dos raíces se copian además a `mt_root_in_out` y `mt_root_out_out`. Son exactamente los cuatro *public inputs* contra los cuales verifica el validador on-chain.

6.8.5. Vínculo con txs_updater_minting

La utilidad del circuito en el protocolo aparece al mirar `txs_updater_minting`. Sus condiciones de aceptación fueron formalizadas en la Subsección 6.6.4 y forman parte de las condiciones necesarias de tx_{mint} . El mapeo entre la matemática del argumento Groth16 y la validación on-chain es el siguiente:

1. El UTxO $o_{\text{txs_set}}$ de entrada contiene la raíz anterior r_{in} en su *datum*;
2. El *output* $o_{\text{txs_set}}^*$ contiene la nueva raíz r_{out} en su *datum*;
3. el campo `redeemer.tx_id` trae los *public inputs* `tx_id_hi` y `tx_id_lo`;
4. los tres campos del argumento Groth16 $\pi = (\pi_A, \pi_B, \pi_C)$ vienen como campos del *redeemer*: `piA`, `piB`, `piC`.

6.9. El circuito de pertenencia a una TxSnapshot

Para orientar la lectura, esta sección separa tres niveles de la misma verificación. Primero fija la relación matemática que se quiere probar: que un txid pertenece al conjunto de transacciones comprometido por una raíz r_{tx} certificada por MITHRIL. Luego distingue qué datos deben quedar como *public inputs* del argumento Groth16, concretamente txid, r_{sub} y r_{tx} , y qué datos permanecen como *witness*, principalmente los caminos Merkle y la cadena ASCII que identifica el rango de bloques. Finalmente, muestra cómo esos valores públicos se empaquetan para el verificador on-chain y cómo el `minting_validator` los enlaza con el `certts` ya validado.

Este circuito resuelve cómo convencer a la cadena destino $\mathcal{B}$ de que la tx_{lock} de $\mathcal{A}$ efectivamente pertenece al TxSnapshot de CARDANO autenticado por un certificado de MITHRIL. Es decir, argumenta la pertenencia respecto de la Merkle Root de `CardanoTransactions` que un `certts` expone en su `msg`. Esto importa porque el `minting_validator` no solamente valida un `certts` contra el `proof_receipt` que dejó la fase 2 del verificador STM, sino que además consume un argumento Groth16 generado por `snapshot_membership` para verificar que el txid de la tx_{lock} pertenece al snapshot autenticado por `certts`.

6.9.1. Modelo matemático del TxSnapshot Pythagoras de Mithril

En la implementación actualmente compatible con el *aggregator*, la entidad `CardanoTransactions` se autentica mediante el esquema `legacy` de la era Pythagoras. Como acabamos de describir, la prueba tiene estructura de dos niveles: un *sub-tree* que agrupa un rango contiguo de transacciones de CARDANO, y un *master tree* cuyas hojas son un compromiso de cada *sub-tree* junto con el rango que representa.

Fijamos la función de hash BLAKE2S-256 : $\{0, 1\}^* \rightarrow \{0, 1\}^{256}$, y para cada longitud de bytes $\ell \in \mathbb{N}$ escribimos $\mathcal{B}_\ell = \{0, 1\}^{8\ell}$. Si $\text{txid} \in \mathcal{B}_{32}$ es el hash canónico de una transacción de CARDANO, la hoja en el *sub-tree* es su codificación hexadecimal ASCII en minúsculas $\text{hex}_{\text{lc}}(\text{txid}) \in \mathcal{B}_{64}$, que luego se hashea con BLAKE2S-256 a $\mathcal{B}_{32}$.

Sea ahora $r_{\text{sub}} \in \mathcal{B}_{32}$ la raíz del *sub-tree* que contiene a txid. La hoja correspondiente del *master tree* se construye como:

$$\text{leaf}_{\text{master}} = \text{BLAKE2S-256}(\alpha \parallel r_{\text{sub}}), \quad \alpha \in \mathcal{B}_L, \quad 0 \leq L \leq 32,$$

donde α es la cadena `range_ascii` de longitud variable L que identifica el rango de bloques del *sub-tree*.

El argumento completo de pertenencia queda descompuesto en:

1. **sub_prefix**: pasos inferiores del sub-tree.
2. **sub_upper**: continuación del camino hasta r_{sub} , ya con hermanos en $\mathcal{B}_{32}$;
3. **master_prefix**: pasos iniciales por encima de $\text{leaf}_{\text{master}}$;
4. **master_upper**: continuación final hasta la raíz certificada del snapshot.

6.9.2. *Public inputs y witness del circuito*

Para este circuito, tenemos $\mathbf{x} = (\text{txid}, r_{\text{sub}}, r_{\text{tx}}) \in \mathcal{X}$, donde $\text{txid} \in \mathcal{B}_{32}$ es el identificador de transacción de CARDANO cuya pertenencia queremos demostrar, $r_{\text{sub}} \in \mathcal{B}_{32}$ es la Merkle Root del *sub-tree* que contiene a txid , y $r_{\text{tx}} \in \mathcal{B}_{32}$ es la Merkle Root certificada por el cert_{ts} de MITHRIL.

Por otra parte, el *witness* está dado por $\mathbf{w} = (\Psi_{\text{sub}}^{\text{pre}}, \Psi_{\text{sub}}^{\text{up}}, \alpha, \Psi_{\text{mst}}^{\text{pre}}, \Psi_{\text{mst}}^{\text{up}}) \in \mathcal{W}$, donde:

- α es la cadena ASCII de longitud variable `range_ascii`;
- $\Psi_{\text{sub}}^{\text{pre}}$ es la secuencia activa de pasos del **sub_prefix**;
- $\Psi_{\text{sub}}^{\text{up}}$ es la secuencia activa de siblings de 32 bytes del **sub_upper**;
- $\Psi_{\text{mst}}^{\text{pre}}$ es la secuencia activa del **master_prefix**;
- $\Psi_{\text{mst}}^{\text{up}}$ es la secuencia activa del **master_upper**.

En la instancia `mithril_legacy_tx_membership.circom`, compilada hoy como `MithrilLegacyTxMembership(10, 32, 32, 1, 32)`, se expone a través de estos *inputs*:

1. `cardano_tx_hash_b[32]`,
2. `range_ascii_b[32]` y `range_ascii_len` (porque L es variable),
3. todas las familias `sub_prefix_*`, `sub_upper_*`, `master_prefix_*`, y `master_upper_*`,
4. `expected_root_b[32]`, que usamos para imponer la igualdad byte a byte con r_{tx} (porque tenemos que separarlo en dos partes al no entrar en un elemento de $\mathbb{F}_r$). No aporta información nueva, pero es conveniente.

txid , r_{sub} y r_{tx} pasan al verificador Groth16 on-chain como seis enteros, porque necesitamos dos elementos de $\mathbb{F}_r$ por cada ítem de $\mathbf{x}$. Partimos cada elemento de $\mathcal{B}_{32}$ en dos mitades de $\mathcal{B}_{16}$, y cada mitad se interpreta como un entero *big-endian* de 128 bits.

$$\begin{pmatrix} \text{cardano_tx_hash_hi}, \text{cardano_tx_hash_lo}, \\ \text{sub_root_hi}, \text{sub_root_lo}, \\ \text{master_root_hi}, \text{master_root_lo} \end{pmatrix}.$$

6.9.3. La relación NP de `snapshot_membership`

Tenemos $\mathcal{X} = \mathcal{B}_{32}^3$ y $\mathcal{W} = \{(\Psi_{\text{sub}}^{\text{pre}}, \Psi_{\text{sub}}^{\text{up}}, \alpha, \Psi_{\text{mst}}^{\text{pre}}, \Psi_{\text{mst}}^{\text{up}})\}$ compatibles con los formatos ya descritos. Formalmente, cada paso activo del **sub_prefix** o del **master_prefix** es una tripla $\psi = (k, s, b)$ con $k, b \in \{0, 1\}$, donde k indica si el hermano s se interpreta como bytes o como hash, y b indica si ese hermano se ubica a la izquierda del nodo actual. Para compactar la notación, escribimos

$$\text{Merge}(x, s, b) = \begin{cases} \text{BLAKE2S-256}(x \parallel s), & \text{si } b = 0, \\ \text{BLAKE2S-256}(s \parallel x), & \text{si } b = 1. \end{cases}$$

Si $\psi = (k, s, b)$ es un paso tipado activo, definimos además $\text{Step}(x, \psi) = \text{Merge}(x, s_\phi, b_\phi)$.

Tomemos $\mathbf{x} = (\text{txid}, r_{\text{sub}}, r_{\text{tx}}) \in \mathcal{X}$, $\mathbf{w} = (\Psi_{\text{sub}}^{\text{pre}}, \Psi_{\text{sub}}^{\text{up}}, \alpha, \Psi_{\text{mst}}^{\text{pre}}, \Psi_{\text{mst}}^{\text{up}}) \in \mathcal{W}$. Escribimos:

$$\begin{aligned}\Psi_{\text{sub}}^{\text{pre}} &= (\psi_0^{\text{sub}}, \dots, \psi_{a-1}^{\text{sub}}), & \Psi_{\text{sub}}^{\text{up}} &= ((h_0^{\text{sub}}, b_0^{\text{sub}}), \dots, (h_{u-1}^{\text{sub}}, b_{u-1}^{\text{sub}})), \\ \Psi_{\text{mst}}^{\text{pre}} &= (\psi_0^{\text{mst}}, \dots, \psi_{c-1}^{\text{mst}}), & \Psi_{\text{mst}}^{\text{up}} &= ((h_0^{\text{mst}}, b_0^{\text{mst}}), \dots, (h_{v-1}^{\text{mst}}, b_{v-1}^{\text{mst}})).\end{aligned}$$

La relación $\mathcal{R}_{\text{tx-snapshot-membership}}$ se define recursivamente en cuatro etapas.

1. Derivación de la hoja del *sub-tree*: fijamos $x_0 = \text{hex}_{\text{lc}}(\text{txid}) \in \mathcal{B}_{64}$, y luego $x_1 = \text{Step}(x_0, \psi_0^{\text{sub}})$, $x_{i+1} = \text{Step}(x_i, \psi_i^{\text{sub}})$ para $1 \leq i < a$. Al terminar esta primera fase, $x_a \in \mathcal{B}_{32}$ es el digest obtenido al consumir todos los pasos activos del *sub_prefix*.
2. Reconstrucción de r_{sub} : definimos $y_0 = x_a$, $y_{j+1} = \text{Merge}(y_j, h_j^{\text{sub}}, b_j^{\text{sub}})$ para $0 \leq j < u$. La primera condición es $y_u = r_{\text{sub}}$.
3. Construcción de $\text{leaf}_{\text{master}}$: con la cadena ASCII del rango y la r_{sub} recién reconstruida, definimos $z_0 = \text{BLAKE2S-256}(\alpha \parallel r_{\text{sub}})$.
4. Reconstrucción de r_{tx} : hacemos los pasos del *master_prefix* y del *master_upper*: $z_1 = \text{Step}(z_0, \psi_0^{\text{mst}})$, $z_{i+1} = \text{Step}(z_i, \psi_i^{\text{mst}})$ para $1 \leq i < c$, y luego $t_0 = z_c$, $t_{j+1} = \text{Merge}(t_j, h_j^{\text{mst}}, b_j^{\text{mst}})$ para $0 \leq j < v$. La segunda condición es $t_v = r_{\text{tx}}$.

Decimos que $\mathcal{R}_{\text{tx-snapshot-membership}}(\mathbf{x}, \mathbf{w}) = 1$ si y sólo si $y_u = r_{\text{sub}}$ y $t_v = r_{\text{tx}}$. El lenguaje asociado es: $\mathcal{L}_{\text{tx-snapshot-membership}} = \{\mathbf{x} \in \mathcal{X} \mid \exists \mathbf{w} \in \mathcal{W} : \mathcal{R}_{\text{tx-snapshot-membership}}(\mathbf{x}, \mathbf{w}) = 1\}$.

Existen una descomposición válida del Merkle Path de la transacción de $\mathcal{A}$ identificada como *txid*, una cadena ASCII de rango y un Merkle Path en el *master tree* tales que la transacción pertenece al *sub-tree* de raíz r_{sub} y ese mismo *sub-tree*, comprometido como $\text{leaf}_{\text{master}}$, pertenece al *master tree* de raíz r_{tx} autenticado por MTHRIL en un cert_{ts} .

6.9.4. Familias de restricciones del circuito

La relación anterior es implementada en *circom* para ser convertida a un circuito aritmético y luego a un QAP. En conjunto, estas restricciones prueban pertenencia, y además descartan *witness* inválidos, incoherencias de longitud y usos incompatibles.

1. Un chequeo de rango $0 \leq b < 256$ para todos los arreglos de bytes mediante componentes `AssertByte/AssertBytes`.
2. La derivación determinista de la hoja en ASCII, para que el *prover* no pueda sustituir la transacción real por una alternativa.
3. Garantizar que los *flags enabled* sean contiguos. Con esto, no se puede “saltar” un nivel intermedio y reactivar luego otro más alto.
4. En cada paso tipado, un bit *kind* determina si el hermano relevante es una cadena de bytes o un hash. Se garantiza además que el slot inactivo quede anulado.
5. Cuando un paso está deshabilitado, el circuito impone que tanto sus selectores como sus datos sean cero y que el *output* propague exactamente el valor de entrada.
6. `TypedPrefixFirstStep`, `TypedPrefixHashedStep` y `MerkleUpperPathBlake2s` son componentes que implementan la reconstrucción desde $\text{hex}_{\text{lc}}(\text{txid})$ hasta r_{sub} .
7. El componente `MasterLeafHash` selecciona la longitud efectiva `range_ascii_len`, obliga a cero todos los bytes fuera de esa longitud y luego computa $\text{BLAKE2S-256}(\alpha \parallel r_{\text{sub}})$. De esta forma, dos *witnesses* con el mismo prefijo activo no pueden diferir sólo en bytes “de relleno” fuera de la longitud declarada.

8. Se recompone la r_{tx} a partir de la hoja $\text{BLAKE2S-256}(\alpha \parallel r_{sub})$.
9. Se impone byte a byte $\text{master_root_b} = \text{expected_root_b}$.
10. Los tres hashes $(txid, r_{sub}, r_{tx}) \in \mathcal{B}_{32}^3$ se empaquetan en seis enteros públicos de $\mathcal{B}_{16}$.

6.9.5. Vínculo exacto con minting_validator

La utilidad del circuito en el protocolo aparece al mirar la interfaz del `minting_validator`, cuyas condiciones de aceptación fueron formalizadas en la Subsección 6.6.3 y forman parte de las condiciones necesarias de tx_{mint} . El mapeo entre la matemática del argumento Groth16 y la validación on-chain es el siguiente:

1. El validador construye un `ReducedMithrilCertificate` a partir de los campos planos del *redeemer*.
2. Inspecciona los inputs de la transacción para recuperar el `statement_hash` persistido en el `proof_receipt` de fase 2 y, con eso, ejecuta la verificación del certificado contra el certificado padre autenticado por el o_{sd} de referencia.
3. Recién después de ese chequeo corre la verificación de `snapshot_membership`, que en el código del repositorio aparece como `snapshot_membership.proof_verifies(proof, locking_tx_hash, sub_root, snapshot_root)`.
4. Los *public inputs* on-chain del argumento Groth16 vienen, en ese mismo orden, dados por el hash canónico de la transacción de CARDANO, la raíz pública del sub-tree r_{sub} y el `CardanoTransactionsMerkleRoot` del certificado reducido, o sea r_{tx} .

6.9.6. Transición experimental a Lagrange

Junto con el circuito *legacy* integrado hoy al operador, el repositorio mantiene un experimento separado: `mithril_lagrange_tx_membership_experimental.circom`. Ese archivo adelanta la geometría esperada para la era Lagrange: un único Merkle Tree plano de transacciones, un `leaf_index`, un vector de siblings y una función de hash Poseidon* de MIDNIGHT. En su forma actual, el circuito experimental recibe:

1. `cardano_tx_hash[32];`
2. `leaf_index;`
3. `siblings[MAX_DEPTH][2];`
4. `expected_root[2].`

Y expone exactamente los mismos seis *outputs* públicos que el circuito *legacy*.

Si notamos $\tilde{H}_{Mid}$ a la función hash *mockeada* que hoy reemplaza provisionalmente a Poseidon*, la relación NP tentativa de ese circuito puede enunciarse así: existen un índice j y un camino de hermanos tales que el digest hoja $\tilde{\ell} = \tilde{H}_{Mid}(txid)$ reconstruye, mediante el Merkle Path determinado por los bits de j , una Merkle Root igual a r_{tx}^{Lag} .

Este carril debe leerse hoy como trabajo futuro y no como una alternativa lista para usar, ya que la función hash todavía no es la variante definitiva de Poseidon* que MITHRIL planea usar en producción, y el contrato de *witness* del circuito experimental todavía no coincide con la forma de prueba que hoy entrega la API real del *aggregator*, que se encuentra en la era Pythagoras.

Componente	Versión	Rol en el flujo final
Aiken	v1.1.22	compilación de validadores y ejecución de <code>aiken check</code>
<code>trix</code>	0.26.2	validación, build e inspección de <code>main.tx3</code>
<code>cshell</code>	0.15.0	resolución, firma y envío de transacciones
<code>dolos</code>	1.3.2	nodo local de datos y entorno runtime del bridge

Tab. 6.2: Versiones de la toolchain utilizada por el flujo final del zkBridge.

6.10. Flujo final del zkBridge

Esta sección describe el flujo operacional completo que une los validadores de la Sección 6.6, la adaptación de Halo2 de la Sección 6.7 y los circuitos auxiliares de las Secciones 6.8 y 6.9 en un protocolo transaccional ejecutable. En particular, explica cómo se ensamblan las transacciones, qué ajustes de *tooling* fueron necesarios para volver reproducible el flujo y cómo se organizó la validación mediante tests.

6.10.1. El ecosistema Tx3/`trix`/`cshell`/`dolos`

La implementación final del zkBridge se apoya en un pequeño ecosistema coordinado de herramientas, además de Aiken. Es el ecosistema Tx3, `trix`, `cshell` y `dolos`. Esta capa fija el protocolo transaccional concreto del zkBridge, materializa plantillas de transacciones, resuelve UTxOs, firma y envía transacciones, y mantiene un entorno local donde todo el protocolo se pueda ejecutar de forma determinista.

Tx3 define una interfaz declarativa para protocolos de blockchains UTxO, buscando volver explícita la información off-chain que normalmente quedaría dispersa entre *scripts*, *SDKs* y conocimiento implícito del autor del protocolo [34]. El archivo `main.tx3` cumple exactamente ese rol: es la fuente de verdad del protocolo transaccional del zkBridge.

Sobre esa interfaz opera `trix`, la CLI principal del ecosistema Tx3 [35]. Para el zkBridge, `trix` cumple tres funciones distintas. Primero, valida estáticamente la semántica y sintaxis con `trix check`. Segundo, compila `main.tx3` a representaciones intermedias consumibles por otras herramientas, en particular la *Transaction Invocation Interface* (TII) y la *Transaction Intermediate Representation* (TIR), que son un puente entre la descripción del protocolo y su materialización [34, 35]. Tercero, sirve como punto de entrada para inspección y depuración, por ejemplo con `trix build` y `trix inspect tir`, lo cual fue útil en la etapa de ajustes finales del zkBridge.

`cshell` toma la interfaz ya compilada por `trix` y la usa para resolver, firmar y someter transacciones reales [36]. `cshell` es el componente que transforma una plantilla simbólica como `bridge_mint_tx` o `phase2_verify` en una transacción concreta, ya balanceada y lista para ser firmada o enviada [36].

`dolos` provee el entorno local de datos y de *devnet* sobre el que se apoyan `trix` y `cshell` [37]. `dolos` sincroniza estado, expone servicios y permite que el resto de herramientas resuelvan UTxOs y observen confirmaciones en un nodo local [37]. En nuestro prototipo, `dolos` fue la pieza necesaria para tener una reproducción completa del zkBridge sin depender de un nodo remoto de CARDANO cada vez que se reejecutaba el flujo.

La Tabla 6.2 resume las versiones de las herramientas usadas durante el desarrollo del zkBridge y que volvieron operable su lógica criptográfica. Tx3 no tiene una versión, mientras que las herramientas que lo interpretan y ejecutan sí.

6.10.2. Estrategia de tests con aiken check

La estrategia de tests del repositorio está organizada alrededor de `aiken check`. En CARDANO, los tests verdaderamente útiles para validadores no solamente reproducen una propiedad lógica, sino que recorren el camino de traducción a UPLC. Por eso, `aiken check` es central: valida semántica, produce trazas y reporta unidades de ejecución suficientemente cercanas a las reales como para guiar el diseño y detectar problemas antes de correr el flujo integrado [7]. En el repositorio, esta estrategia adopta una forma cercana a una pirámide de validación. La ventaja de esta organización es que separa bien los tipos de falla.

1. En la base aparecen los tests de aceptación y rechazo de validadores. Su función principal es fijar las condiciones de aceptación del protocolo y también sus fallas esperadas. La convención adoptada es bastante clara: casos positivos escritos como `accepts_*` o equivalentes, y casos negativos marcados con `fail`, de modo que un cambio semántico se vuelva visible enseguida.
2. Un segundo nivel lo forman los tests donde se comprueba que los *wrappers* Aiken de Groth16 acepten una prueba dada, así que el orden y el *packing* de *public inputs* coincidan exactamente con lo que esperan los circuitos Groth16.
3. Un tercer nivel lo ocupan los tests de *fixture* E2E. Estos fijan valores exportados desde los JSON canónicos del flujo, y verifican que los *redeemer* reales sigan alineados con las pruebas y con los hashes de helpers compartidos. Busca evitar *drift* entre el lado off-chain del bridge (con los *scripts* para coordinar los flujos) y los tests de Aiken.
4. Finalmente, la capa más costosa la forman los *runtime probes* con pruebas reales. Estos casos intentan recrear de la forma más fiel posible los argumentos criptográficos o estados intermedios que el flujo integrado realmente consumirá.

Además, integramos `aiken check` al mismo flujo principal del zkBridge. En particular, `./scripts/bridge.sh run -strict` ejecuta primero un *preflight* que valida la coherencia del estado actual del repositorio para poder llevar a cabo las mediciones y ejecutar validadores, y sólo después corre `aiken check`. Del mismo modo, si `aiken check` falla, tampoco vale la pena arrancar `dolos` ni construir la secuencia completa de transacciones.

6.10.3. El flujo principal del zkBridge

El punto de entrada principal del zkBridge es `./scripts/bridge.sh run -strict`. Consideramos a este comando como la implementación final de la prueba de concepto del zkBridge en CARDANO. Podemos resumirlo en:

1. Validación de las versiones de herramientas en el repositorio: Aiken, `trix`, `cshell`, `dolos`, `uv` y dependencias de Python.
2. Construcción los argumentos STM del flujo, que se concentran en un JSON final.
3. Ejecución obligatoria del *preflight*; muy útil para tener errores rápidos.
4. Ejecución de `aiken check`.
5. Ejecución del flujo completo de transacciones del zkBridge con herramientas Tx3.

El *preflight* es especialmente importante porque fija el contrato entre el material criptográfico off-chain y el estado que luego verán los validadores on-chain. Verifica, entre otras cosas, que el bundle STM siga cumpliendo sus invariantes, que los snapshots

de referencia no hayan quedado obsoletos y que fixtures como `bridge_fixture.ak` y `cardano_transactions.ak` continúen alineados con la r_{tx} y con los *statement hashes* que usará el flujo real. Recién cuando todo eso pasa, el runner avanza hacia la parte transaccional propiamente dicha.

Una vez dentro del runtime, el bridge se organiza en etapas bien diferenciadas. La primera es la publicación compartida de `publish_phase1_reference_script`, necesaria para reutilizar el script pesado de la fase 1 del verificador STM. La segunda es la ejecución secuencial de las dos corridas `phase1_setup` $\rightarrow$ `phase2_verify` que siguen usando verificación Halo2 on-chain: `stake_distribution_standard` y `cardano_transactions`. Cada una de esas corridas termina produciendo un `proof_receipt` distinto.

La tercera etapa es preparar el o_{sd} . La transacción `stake_distribution_genesis_tx` no consume un resultado de fase 2: recibe el $cert_{gen}$, valida la firma génesis σ_{gen} contra vk_{gen} con Ed25519 y emite el primer o_{sd} . Luego `stake_distribution_standard_tx` consume ese estado junto con el `proof_receipt` del dominio `stake_distribution_standard`, verifica continuidad de certificado padre e hijo, y produce el o_{sd} vigente que el zkBridge usará más tarde como *reference input* para la tx_{mint} .

En paralelo conceptual con esa cadena corre la cadena del actualizador de transacciones usadas. Primero `locking_txs_updater_seed_tx` crea el ancla de unicidad para la política de mint del actualizador. Después aparecen dos publicaciones adicionales de *reference scripts*: `publish_minting_txs_updater_spend_reference_script` y `publish_bridge_minting_reference_script`. Ambas son necesarias para mantener el flujo por debajo de los límites de tamaño de transacción, ya que el mint final necesita reutilizar scripts pesados sin reenviarlos inline. Recién entonces `minting_txs_updater_seed_tx` inicializa el o_{txs_set} del bridge con la Merkle Root vacía del conjunto de locking transactions ya consumidas.

La última etapa es `bridge_mint_tx`. Esta transacción consume el `proof_receipt` del dominio `cardano_transactions`, consume también el o_{txs_set} y toma como *reference input* el o_{sd} vigente. En su *redeemer* viajan el certificado de TxSnapshot, el argumento Groth16 de pertenencia al snapshot y el argumento Groth16 de actualización del conjunto de transacciones usadas por el zkBridge.

La Figura 6.2 resume la dependencia lógica del flujo final. El bundle STM alimenta las dos corridas de fase 1/fase 2 que permanecen en el flujo, mientras que el certificado génesis inicializa la cadena de StakeDist sin `proof_receipt`. Luego, las corridas `stake_distribution_standard` y `cardano_transactions` producen dos `proof_receipt` distintos: el primero actualiza la cadena de StakeDist y el segundo habilita el mint final.

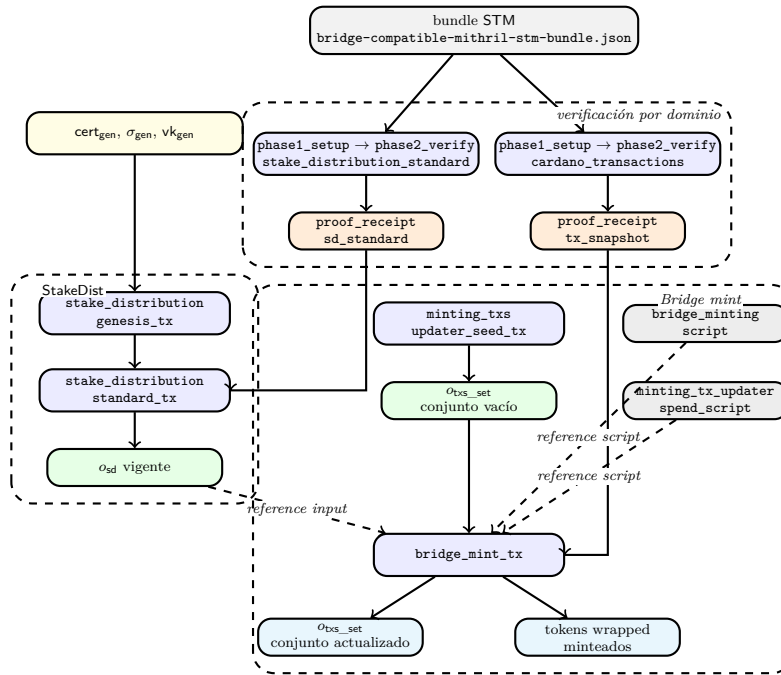


Fig. 6.2: Flujo final implementado del zkBridge. El bundle STM alimenta las dos verificaciones por dominio que producen `proof_receipt` para `stake_distribution_standard` y `cardano_transactions`; el `cert_gen` inicializa la cadena de StakeDist sin receipt; la transacción `bridge_mint_tx` consume el receipt de transacciones y el `o_txs_set`, usa el `o_sd` como *reference input* y reutiliza scripts pesados como *reference scripts* para emitir tokens envueltos y actualizar el conjunto anti-replay.

7. RESULTADOS

Este capítulo resume el resultado principal de la implementación final: el flujo transaccional de la Sección 6.10 entra dentro de los límites de recursos de CARDANO que motivaron el diseño. En particular, retomamos `maxTxSize` y `maxTxExUnits`, introducidos en la Sección 3.3. Para evaluarlo, tomamos las transacciones concretas del flujo final y comparamos su tamaño serializado, su consumo de CPU y su consumo de memoria contra los parámetros de protocolo de la Tabla 3.1.

La Tabla 7.1 concentra ese contraste. El resultado más ajustado en tamaño es `publish_phase1_reference_script`, que ocupa el 96.4% de `maxTxSize`. Del lado de `ExUnits`, las transacciones más costosas son las dos fases del verificador STM, con picos de 72.0% del presupuesto de CPU y 96.5% del presupuesto de memoria. La `bridge_mint_tx` final también queda por debajo del límite, usando 58.5% del presupuesto de CPU y 18.7% del de memoria.

Transacción	% maxTxSize	% CPU maxTxExUnits	% memory maxTxExUnits
<code>publish_phase1_reference_script</code>	96.4	0.0	0.0
<code>phase1_setup_stake_dist_std</code>	28.5	71.5	96.5
<code>phase2_verify_stake_dist_std</code>	40.2	72.0	5.7
<code>phase1_setup_cardano_txs</code>	28.5	70.5	94.0
<code>phase2_verify_cardano_txs</code>	40.2	72.0	5.7
<code>stake_distribution_genesis_tx</code>	66.3	4.5	8.4
<code>stake_distribution_standard_tx</code>	37.7	4.7	11.1
<code>locking_txs_updater_seed_tx</code>	1.7	0.0	0.0
<code>publish_minting_updater_ref_script</code>	41.6	0.0	0.0
<code>publish_bridge_minting_ref_script</code>	52.0	0.0	0.0
<code>minting_txs_updater_seed_tx</code>	7.7	0.2	0.4
<code>bridge_mint_tx</code>	14.7	58.5	18.7

Tabla 7.1. Comparación entre las transacciones del flujo final del zkBridge y los límites de recursos de CARDANO. El porcentaje de CPU se calcula respecto del segundo componente de `maxTxExUnits` (1×10^{10}), mientras que el de memoria se calcula respecto del primero ($1,4 \times 10^7$), que coincide con el límite práctico considerado en este trabajo.

La implementación se encuentra en el repositorio de GitHub [eryxcoop/zk-bridge](https://github.com/eryxcoop/zk-bridge) [38]. Allí se encuentran los componentes de implementación que sostienen las mediciones anteriores: los validadores Aiken, los circuitos, el operador off-chain, los scripts de ejecución y demás objetos auxiliares usados para reproducir el flujo final.

Además, ese repositorio aporta tres resultados complementarios que ayudan a delimitar el alcance del proyecto:

1. El flujo integrado del prototipo, cuya secuencia se detalla en la Subsección 6.10.3, se ejecuta de punta a punta mediante `./scripts/bridge.sh run -strict`, encadenando construcción del bundle de MITHRIL, *preflight*, `aiken check` y corrida completa sobre un nodo local `dolos`.
2. La suite de `aiken check`, descrita en la Subsección 6.10.2, cubre los validadores principales del sistema: *minting*, *StakeDist*, actualizador de transacciones usadas, pertenencia al *snapshot*, *phase1*, *phase2* y *fixtures* de integración.
3. La validación es reproducible porque el flujo local se apoya en *fixtures* canónicos versionados, en un bundle derivado de manera determinística y en chequeos previos que detectan *drift* entre pruebas, certificados y datos auxiliares antes de enviar transacciones.

8. CONCLUSIONES Y TRABAJO FUTURO

8.1. Conclusiones

La conclusión principal de este trabajo es que sí es posible materializar un `zkBridge` sobre `CARDANO` respetando los límites de unidades de ejecución impuestos por el protocolo. En particular, las mediciones del Capítulo 7 muestran que, mediante la partición de la verificación `STM` en dos fases de la Subsección 6.7.3, el uso de *reference scripts* y la separación entre capa de actualización y capa de aplicación de la Subsección 5.2.1, el flujo final puede mantenerse dentro del presupuesto on-chain disponible sin abandonar el modelo `eUTxO`.

8.1.1. Modelo de seguridad y alcance del prototipo

El modelo de seguridad obtenido se apoya en `MITHRIL` como capa intermedia de autenticación de estado, según la cadena de certificados formalizada en la Sección 4.3. El validador exige una cadena de certificados `MITHRIL` anclada en un `certgen`, actualizada mediante certificados de `StakeDist` y conectada con `proof_receipt` producidos por la verificación `STM` en dos fases. En la implementación Aiken, esos recibos quedan ligados al *transcript* del verificador `Halo2`: `phase1_setup` reconstruye el *transcript* a partir de la prueba serializada y de las entradas públicas, persiste los desafíos y el hash del ρ reducido necesarios para continuar, y `phase2_verify` sólo puede cerrar la verificación si ese ρ coincide con el estado intermedio y satisface el chequeo final de pairing. Bajo el supuesto de que el umbral de `MITHRIL` representa una fracción honesta suficiente del stake, y bajo la solidez de los argumentos `Halo2/Groth16` usados, esos recibos actúan como evidencia on-chain de que el mensaje certificado fue aceptado por la capa de actualización y no por una ejecución desconectada del *transcript* original.

Dentro de ese modelo, el prototipo fuerza tres invariantes principales para el camino $\text{tx}_{\text{lock}} \rightarrow \text{tx}_{\text{mint}}$, cuyo ensamblado completo se presenta en la Subsección 6.10.3. Primero, el `txid` reclamado debe pertenecer a un `TxSnapshot` certificado por `MITHRIL`. Segundo, la transacción de minting queda ligada al cuerpo canónico de la `txlock`: el monto bloqueado, el identificador del bridge y el destinatario codificado en el datum participan en el hash que luego se prueba contra el snapshot. Tercero, la misma operación actualiza atómicamente el estado anti-replay del `Bridge` mediante una raíz `SMT`, de modo que una misma transacción de origen no pueda reutilizarse para mintear dos veces.

El alcance del prototipo, por lo tanto, debe leerse como una prueba de viabilidad para una instancia de `CARDANO`, no como un bridge productivo y heterogéneo listo para cualquier par de cadenas. La implementación trabaja con una formulación isomorfa de las transacciones relevantes: las cadenas $\mathcal{A}$ y $\mathcal{B}$ pueden modelarse con estructuras suficientemente compatibles para reconstruir y validar localmente el evento de origen. En un escenario plenamente heterogéneo, esa capa debería reemplazarse por circuitos y codificaciones específicos de la cadena origen, junto con reglas explícitas para normalizar direcciones, valores, fees y formatos de transacción.

Quedan también fuera del modelo cerrado de esta tesis algunos aspectos operativos: disponibilidad del operador off-chain para construir *witnesses* generales a partir de un `SMT` mantenido a lo largo de ejecuciones sucesivas, más allá de la raíz canónica que el

protocolo sí persiste on-chain; gestión de claves de verificación ante cambios de versión de MITHRIL, experiencia de usuario, manejo productivo de wallets y operación sostenida sobre una red pública. Esos puntos, sin invalidar la arquitectura criptográfica presentada, delimitan con precisión qué parte fue demostrada: la composición on-chain de certificados MITHRIL, recibos de verificación y argumentos sucintos dentro de los límites reales de CARDANO.

8.2. Trabajo futuro

8.2.1. Sobre CIP-133

Una línea de trabajo futuro particularmente relevante para este proyecto es la evolución del soporte de MSM en la capa de lenguajes y herramientas de CARDANO. En la implementación actual, el principal cuello de botella del verificador STM no estuvo dado por la corrección criptográfica del argumento, sino por el costo de materializar on-chain una gran MSM sobre puntos de BLS12-381. Esa operación domina buena parte del costo lineal del verificador emitido en Aiken y fue una de las razones centrales por las que el flujo final debió partir la verificación en dos transacciones consecutivas.

CIP-133 propone precisamente incorporar una *builtin* nativa para MSM sobre BLS12-381 [31, 32]. Este punto merece una precisión importante: desde la perspectiva del protocolo, esa dirección ya no es meramente especulativa. El soporte para estas primitivas ya forma parte de la evolución reciente de Plutus en CARDANO, pero al momento de escribir esta tesis todavía no se encuentra expuesto de manera estable en Aiken. En particular, el trabajo para incorporar estas *builtins* en el compilador y en la tipificación del lenguaje aparece reflejado en el *pull request* `aiken-lang/aiken#1349`, que al 1 de julio de 2026 seguía abierto y proponía justamente agregar soporte para las operaciones de MSM en $\mathbb{G}_1$ y $\mathbb{G}_2$.

Para este zkBridge, la consecuencia potencial no sería solamente una optimización incremental. Si Aiken pasa a exponer de manera usable las primitivas asociadas a CIP-133, una fracción importante del trabajo que hoy se expresa como cadenas largas de `scaleG1`, `addG1`, descompresiones y *overhead* de la máquina CEK podría colapsar en una operación primitiva mucho más compacta. Eso volvería plausible verificar un certificado de MITHRIL dentro de una sola transacción, sin necesidad de introducir el *split phase1/phase2* ni de transportar un estado intermedio entre ambas fases.

La consecuencia arquitectónica sería importante. Desaparecería una parte de la complejidad hoy dedicada a los `proof_receipt`, al *binding* entre fases y a la coordinación secuencial de dos transiciones para autenticar un mismo certificado. Incluso si siguieran siendo necesarios algunos *reference scripts* por motivos de tamaño, el diseño global del bridge podría simplificarse de manera apreciable. Por eso, una continuación natural de este trabajo sería reevaluar toda la arquitectura a la luz de CIP-133 una vez que ese soporte quede disponible de extremo a extremo en la superficie de programación efectivamente usada por el prototipo.

8.2.2. Sobre ‘zk-bridge-operator’

El comando `zk-bridge-operator tx prove <transaction-hash>` genera hoy un *bundle* con dos pruebas distintas para el mismo `tx_id` canónico de CARDANO. La primera, `snapshot_membership`, sigue la ruta *legacy* modelada en la Subsección 6.9.1 y prueba que

la transacción pertenece al snapshot certificado. La segunda, `tx_set_update`, produce la prueba de actualización del SMT local del bridge según la relación de la Subsección 6.8.3.

En el estado actual, el operador no construye aún un *witness* completamente general para la segunda prueba. La función `single_insert_empty_tree_witness` arma un caso canónico de inserción sobre árbol vacío: parte de la raíz del SMT completamente vacío, usa el camino inducido por `tx_id` y calcula la raíz resultante al cambiar la hoja final de 0 a 1. Esto es suficiente para la prueba de concepto actual y no altera la semántica matemática del circuito, pero claramente estrecha el conjunto de instancias que el operador sabe producir hoy.

La consecuencia conceptual es nítida. `tx_set_update` ya prueba la relación general $\mathcal{R}_{\text{tx-set-update}}$: cualquier par $(r_{\text{in}}, r_{\text{out}})$ compatible con un mismo camino y un mismo vector de hermanos entra en su lenguaje. Lo que permanece como trabajo futuro no es generalizar el circuito, sino generalizar la capa off-chain del operador para que construya *witnesses* de actualización sobre un SMT arbitrario mantenido a lo largo de ejecuciones sucesivas, y no sólo sobre el caso canónico de primer inserto.

8.2.3. Sobre ‘preview testnet’ de Cardano

Otra continuación natural de este trabajo sería agregar soporte on-chain real sobre la *preview testnet* de CARDANO. El prototipo actual ya recorre de punta a punta el flujo transaccional del zkBridge en un entorno local reproducible, apoyado en `dolos`, `Tx3`, `trix` y `cshell`. Ese recorrido, hoy orquestado por el flujo integrado del repositorio, fue suficiente para validar la lógica del protocolo, sus presupuestos de ejecución y la coherencia entre pruebas, certificados y UTxOs. Sin embargo, todavía queda el paso de ejecutar el mismo flujo sobre una red pública de pruebas, con interacción real con la infraestructura y con las convenciones operativas de CARDANO.

En términos prácticos, esto implica integrar el bridge con una *wallet* de usuario, por ejemplo `Lace`, para firmar y enviar las transacciones del flujo desde una interfaz más cercana al uso real. Ese paso permitiría validar aspectos que en el entorno local quedan parcialmente abstraídos: selección y manejo de UTxOs desde una billetera estándar, provisión de colateral, publicación y reutilización de *reference scripts* en una red compartida, latencias y confirmaciones reales, y la fricción concreta de llevar el flujo desde una herramienta de desarrollo hacia un uso más cercano al de un operador o usuario final.

Desde el punto de vista de la arquitectura, esta migración no debería entenderse sólo como una etapa de despliegue, sino como una validación adicional del diseño. Si el flujo actual logra sostenerse sobre *preview testnet* con firmas reales de una wallet como `Lace`, entonces la prueba de concepto dejaría de estar limitada a una demostración controlada en *devnet* y pasaría a aproximarse mucho más al escenario operativo que tendría un zkBridge usable sobre CARDANO. También permitiría empezar a estudiar con mayor seriedad la experiencia de uso del bridge, cuestión que en esta tesis quedó deliberadamente subordinada a la viabilidad criptográfica y al encaje on-chain del protocolo.

8.2.4. Sobre σ_{gen} en la era Lagrange

Un último frente de trabajo futuro queda asociado al caso particular del certificado génesis de MITHRIL en la transición hacia la era **Lagrange**. En el flujo desarrollado en esta tesis, el `certgen` se usa para inicializar el `osd` y se autentica directamente dentro del validador de mint mediante una σ_{gen} verificada contra la vk_{gen} . Esa decisión fue deliberada:

el certificado génesis no representa una actualización recurrente de STM, por lo que no conviene forzarlo a pasar por el mecanismo `phase1/phase2` ni generar un `proof_receipt` sólo para el arranque del estado.

Sin embargo, la ruta **Lagrange** introduce una dificultad criptográfica adicional. Mientras el caso más simple se reduce a una firma Ed25519 verificable con primitivas disponibles en Aiken, el formato *SNARK-friendly* de MITHRIL incorpora una **StandardSchnorrSignature** sobre Jubjub. Verificar esa firma directamente on-chain no es una opción práctica en el corto plazo: Jubjub no es una curva nativa de CARDANO y una implementación pura en Aiken de aritmética de curva, multiplicación escalar y recomputación del desafío consumiría demasiadas unidades de ejecución. Por lo tanto, una extensión natural del sistema es mover esa verificación a un circuito auxiliar y dejar que el validador de StakeDist verifique sólo una prueba sucinta.

Concretamente, ese trabajo consiste en definir un circuito Groth16 sobre BLS12-381 que capture la relación de verificación Schnorr usada por el génesis de **Lagrange**: reconstruir el punto $R' = rG + cvk^{\text{sch}}$, recomputar el desafío con la variante de Poseidon/Poseidon* usada por MITHRIL sobre el *domain separation tag*, la clave, el punto reconstruido y el mensaje base, y exigir que ese desafío coincida con el de la firma. Además, el circuito debe fijar las condiciones de pertenencia al subgrupo correcto y conectar sus entradas públicas con la codificación real del certificado, no sólo con valores algebraicos ya parseados.

Desde el punto de vista del bridge, la integración esperada mantendría la misma filosofía del flujo final: `stake_distribution_genesis_tx` seguiría siendo la transacción de bootstrap y no reaparecería un dominio `stake_distribution_genesis` dentro de las dos fases de verificación STM. La diferencia sería que el redeemer del caso génesis podría transportar, además de la firma Ed25519, una prueba Groth16 de la firma Schnorr sobre Jubjub. El validador combinaría entonces dos chequeos: la autenticación nativa con vk_{gen} y la verificación del argumento sucinto que ata la componente **Lagrange** del certificado al mismo mensaje génesis.

Aunque una versión posterior del prototipo ya avanzó en esta dirección, este trabajo queda correctamente clasificado como fuera del alcance original de la tesis.

Bibliografía

- [1] R. Belchior, A. Vasconcelos, S. Guerreiro, and M. Correia, “A survey on blockchain interoperability: Past, present, and future trends,” *ACM Computing Surveys*, vol. 54, p. 1–41, Oct. 2021.
- [2] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system.” <http://www.bitcoin.org/bitcoin.pdf>, May 2009.
- [3] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, “zkbridge: Trustless cross-chain bridges made practical,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS ’22*, p. 3003–3017, ACM, Nov. 2022.
- [4] N. Belenkov, V. Callens, A. Murashkin, K. Bak, M. Derka, J. Gorzny, and S.-S. Lee, “Sok: A review of cross-chain bridge hacks in 2023,” 2025.
- [5] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler, *The Extended UTXO Model*, p. 525–539. Springer International Publishing, 2020.
- [6] I. O. G. (IOG), “The EUTXO Handbook.” <https://www.essentialcardano.io/article/the-eutxo-handbook>, 2022. Accessed: 2026-03-22.
- [7] Aiken Team, “Aiken language documentation.” <https://aiken-lang.org/>, 2026.
- [8] B. David, P. Gaži, A. Kiayias, and A. Russell, “Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake protocol.” *Cryptology ePrint Archive*, Paper 2017/573, 2017.
- [9] P. Chaidos and A. Kiayias, “Mithril: Stake-based threshold multisignatures.” *Cryptology ePrint Archive*, Paper 2021/916, 2021.
- [10] I. O. G. . M. Team, “Mithril documentation (overview, protocol, certificates, client),” 2024–2026. Documentación oficial del protocolo y la red Mithril.
- [11] M. Team, “Cardano transactions certification (mithril documentation),” 2024–2026.
- [12] M. Team, “Mithril aggregator node and rest api documentation,” 2024–2026.
- [13] M. Team, “Protocol phases (mithril documentation),” 2024–2026.
- [14] I. O. G. . M. Team, “mithril-stm crate readme and implementation notes,” 2024–2026.
- [15] M. Team, “Certificate chain design (mithril documentation),” 2024–2026.
- [16] M. Team, “Threat model analysis (mithril documentation),” 2024–2026.
- [17] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge.” *Cryptology ePrint Archive*, Paper 2019/953, 2019.

-
- [18] C. S. . P. team, “Ouroboros peras faq,” 2024–2026.
 - [19] C. S. . L. team, “Ouroboros leios (specification and design materials),” 2024–2026.
 - [20] C. Foundation, “Cip-0381: Plutus support for pairings over bls12-381,” 2022.
 - [21] I. O. Global, “Unlocking more opportunities with plutus v3,” 2024.
 - [22] M.-P. contributors, “ak-381: A snark verification library using bls12-381 (aiken),” 2024–2026.
 - [23] R. Zero, “Recursive proving and stark-to-snark wrapping documentation,” 2025–2026.
 - [24] S. Labs, “Sp1 testnet launch (recursion + stark-to-snark wrapper),” 2024.
 - [25] E. contributors, “Technical survey / research notes on groth16 verification cost on cardano,” 2024–2026. Repositorio público con mediciones/argumentos de costo; usar como heurística, validar empíricamente en este proyecto.
 - [26] C. C. (community), “Protocol parameters (cardano-cli query protocol-parameters –mainnet example),” 2024.
 - [27] I. O. Global, “plutus-halo2-verifier-gen,” 2025–2026. Repositorio GitHub del generador de verificadores Halo2 para PLUTUS y Aiken.
 - [28] I. O. Global, “Unlocking zero-knowledge proofs for cardano: the halo2 plutus verifier,” 2025.
 - [29] A. Kate, G. M. Zaverucha, and I. Goldberg, *Constant-Size Commitments to Polynomials and Their Applications*, p. 177–194. Springer Berlin Heidelberg, 2010.
 - [30] C. Foundation, “Cip-0033: Reference scripts,” 2021.
 - [31] C. Foundation, “Cip-0133: Multi-scalar multiplication over bls12-381,” 2024.
 - [32] I. O. Global, “plutus-msm-bench,” 2024–2026. Repositorio GitHub de benchmarks preliminares de MSM asociado al esfuerzo de implementación de CIP-133.
 - [33] C. Foundation, “Cip-0031: Reference inputs,” 2021.
 - [34] TxPipe, “Tx3 introduction.” <https://docs.txpipe.io/tx3>, 2026.
 - [35] TxPipe, “Trix documentation.” <https://docs.txpipe.io/tx3/tooling/trix>, 2026.
 - [36] TxPipe, “CShell usage guide.” <https://docs.txpipe.io/cshell/usage>, 2026.
 - [37] TxPipe, “Dolos quickstart and operation modes.” <https://docs.txpipe.io/dolos>, 2026.
 - [38] E. Coop, “zk-bridge.” <https://github.com/eryxcoop/zk-bridge>, 2026. Repositorio público con los artefactos de implementación del zkBridge para CARDANO, consultado el 1 de julio de 2026.
 - [39] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996.

-
- [40] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*. CRC Press, 2020.
- [41] V. Shoup, “Lower bounds for discrete logarithms and related problems,” *Journal of Cryptology*, vol. 10, no. 2, pp. 97–123, 1997.
- [42] N. P. Smart, “The discrete logarithm problem on elliptic curves of trace one,” *Journal of Cryptology*, vol. 12, p. 193–196, June 1999.
- [43] A. Menezes, T. Okamoto, and S. Vanstone, “Reducing elliptic curve logarithms to logarithms in a finite field,” *IEEE Transactions on Information Theory*, vol. 39, no. 5, p. 1639–1646, 1993.
- [44] F. Hess, *Weil Descent Attacks*, p. 151–180. Cambridge University Press, Apr. 2005.
- [45] V. S. Miller, “Short programs for functions on curves,” *Journal of Cryptology*, 2004. Originally presented in 1986.
- [46] C. Costello, “Pairings for beginners.” Available online: <https://static1.squarespace.com/static/5fdbb09f31d71c1227082339/t/5ff394720493bd28278889c6/1609798774687/PairingsForBeginners.pdf>, 2010.
- [47] M. Bellare and P. Rogaway, “Random oracles are practical: a paradigm for designing efficient protocols,” in *Proceedings of the 1st ACM conference on Computer and communications security - CCS '93*, CCS '93, p. 62–73, ACM Press, 1993.
- [48] P.-A. Fouque and M. Tibouchi, *Indifferentiable Hashing to Barreto–Naehrig Curves*, p. 1–17. Springer Berlin Heidelberg, 2012.
- [49] R. S. Wahby and D. Boneh, “Fast and simple constant-time hashing to the bls12-381 elliptic curve,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, p. 154–179, Aug. 2019.
- [50] A. Faz-Hernandez, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood, “Hashing to Elliptic Curves.” RFC 9380, Aug. 2023.
- [51] D. J. Bernstein, M. Hamburg, A. Krasnova, and T. Lange, “Elligator: Elliptic-curve points indistinguishable from uniform random strings.” Cryptology ePrint Archive, Paper 2013/325, 2013.
- [52] R. C. Merkle, *A Digital Signature Based on a Conventional Encryption Function*, p. 369–378. Springer Berlin Heidelberg, 1988.
- [53] D. Catalano and D. Fiore, *Vector Commitments and Their Applications*, p. 55–72. Springer Berlin Heidelberg, 2013.
- [54] M. Naor and K. Nissim, “Certificate revocation and certificate update,” *IEEE Journal on Selected Areas in Communications*, vol. 18, p. 561–570, Apr. 2000.
- [55] A. Miller, M. Hicks, J. Katz, and E. Shi, “Authenticated data structures, generically,” in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '14, p. 411–423, ACM, Jan. 2014.
- [56] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM J. Comput.*, vol. 18, no. 1, pp. 186–208, 1989.

-
- [57] A. Shamir, “Ip = pspace,” *Journal of the ACM*, vol. 39, p. 869–877, Oct. 1992.
 - [58] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*. Cambridge University Press, 2006.
 - [59] O. Goldreich, *Foundations of Cryptography*. Cambridge University Press, Aug. 2001.
 - [60] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, “From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again,” in *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS ’12, p. 326–349, ACM, Jan. 2012.
 - [61] J. Thaler, “Proofs, arguments, and zero-knowledge,” *Found. Trends Priv. Secur.*, vol. 4, no. 2-4, pp. 117–660, 2022.
 - [62] B. Parno, J. Howell, C. Gentry, and M. Raykova, “Pinocchio: nearly practical verifiable computation,” *Communications of the ACM*, vol. 59, p. 103–112, Jan. 2016.
 - [63] J. von zur Gathen and G. Seroussi, “Boolean circuits versus arithmetic circuits,” *Information and Computation*, vol. 91, p. 142–154, Mar. 1991.
 - [64] R. Gennaro, C. Gentry, B. Parno, and M. Raykova, *Quadratic Span Programs and Succinct NIZKs without PCPs*, p. 626–645. Springer Berlin Heidelberg, 2013.
 - [65] T. P. Pedersen, *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*, p. 129–140. Springer Berlin Heidelberg, 1991.
 - [66] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast reed-solomon interactive oracle proofs of proximity,” in *Electron. Colloquium Comput. Complex.*, 2017.
 - [67] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity.” Cryptology ePrint Archive, Paper 2018/046, 2018.
 - [68] A. Fiat and A. Shamir, *How To Prove Yourself: Practical Solutions to Identification and Signature Problems*, p. 186–194. Springer Berlin Heidelberg, 1986.
 - [69] J. Groth, “On the size of pairing-based non-interactive arguments.” Cryptology ePrint Archive, Paper 2016/260, 2016.
 - [70] S. Goldwasser and Y. Kalai, “On the (in)security of the fiat-shamir paradigm,” in *44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings.*, SFCS-03, p. 102–113, IEEE Computer. Soc, 2003.
 - [71] Q. Dao, J. Miller, O. Wright, and P. Grubbs, “Weak fiat-shamir attacks on modern proof systems,” in *2023 IEEE Symposium on Security and Privacy (SP)*, p. 199–216, IEEE, May 2023.
 - [72] H. Nguyen, U. Ho, and A. Biryukov, “Fiat-shamir in the wild.” Cryptology ePrint Archive, Paper 2024/1565, 2024.
 - [73] D. Khovratovich, R. D. Rothblum, and L. Soukhanov, *How to Prove False Statements: Practical Attacks on Fiat-Shamir*, p. 3–26. Springer Nature Switzerland, 2025.

-
- [74] Q. Yuan, C. Sun, and T. Takagi, “Revisiting the security of fiat-shamir signature schemes under superposition attacks.” Cryptology ePrint Archive, Paper 2024/590, 2024.
 - [75] G. Fuchsbauer, E. Kiltz, and J. Loss, *The Algebraic Group Model and its Applications*, p. 33–62. Springer International Publishing, 2018.
 - [76] N. Pippenger, “On the evaluation of powers and related problems,” in *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*, p. 258–263, IEEE, Oct. 1976.
 - [77] G. Botrel, T. Piellard, Y. E. Housni, I. Kubjas, A. Tabaie, and contributors, “gnark: A go library for zk-snark circuits and proof systems.” <https://github.com/Consensys/gnark>, 2025. GitHub repository, accessed 2026-03-06.
 - [78] arkworks contributors, “arkworks zksnark ecosystem.” <https://arkworks.rs>, 2022.
 - [79] zkcrypto contributors, “bellman: A rust library for building zk-snark circuits.” <https://github.com/zkcrypto/bellman>, 2023. GitHub repository, accessed 2026-03-06.
 - [80] Modulo-P, “groth-validator: Groth16 verifier implementation for cardano.” <https://github.com/Modulo-P/groth-validator>, 2024. GitHub repository, accessed 2026-03-06.

Apéndice

A. PRIMITIVAS CRIPTOGRÁFICAS

Este apéndice reúne herramientas algebraicas y criptográficas que el cuerpo principal de la tesis da por sabidas o utiliza de manera recurrente. Su función no es desarrollar una narrativa independiente, sino fijar el vocabulario mínimo y las construcciones de base necesarias para sostener la discusión sobre certificados de la Sección 4.3, pruebas de inclusión de la Sección 6.9, actualización autenticada de la Sección 6.8 y verificación sucinta en el zkBridge propuesto.

A.1. Cuerpos Finitos

A.1.1. Cuerpos

Antes de introducir formalmente la noción de cuerpo, resulta útil pensar qué tipo de estructura algebraica buscamos modelar. Intuitivamente, un cuerpo es un sistema numérico donde las operaciones aritméticas habituales (suma, resta, multiplicación y división por un elemento no nulo) se comportan de manera esperada.

Los conjuntos numéricos más familiares, como los números racionales, reales o complejos, comparten esta propiedad: es posible hacer estas operaciones sin ambigüedades ni inconsistencias. La abstracción de cuerpo captura precisamente estas reglas algebraicas esenciales, independientemente de la naturaleza concreta de sus elementos.

Definición A.1 (Cuerpo). Sea $\mathbb{K}$ un conjunto, y sean $+$: $\mathbb{K} \times \mathbb{K} \rightarrow \mathbb{K}$ (suma) y $\cdot$: $\mathbb{K} \times \mathbb{K} \rightarrow \mathbb{K}$ (producto) operaciones binarias en $\mathbb{K}$. Diremos que $(\mathbb{K}, +, \cdot)$ es un cuerpo (notado simplemente $\mathbb{K}$) si valen:

- $+$ y $\cdot$ son operaciones asociativas y conmutativas.
- Existe un elemento neutro para la suma, denotado $0_{\mathbb{K}}$. Es decir, tal que $x + 0_{\mathbb{K}} = x$ para todo $x \in \mathbb{K}$.
- Existe un elemento neutro para el producto, denotado $1_{\mathbb{K}}$. Es decir, tal que $x \cdot 1_{\mathbb{K}} = x$ para todo $x \in \mathbb{K}$.
- Todo $x \in \mathbb{K}$ tiene un inverso aditivo, notado $-x$. Es decir, tal que $x + (-x) = 0_{\mathbb{K}}$.
- Todo $x \in \mathbb{K} \setminus \{0_{\mathbb{K}}\}$ tiene un inverso multiplicativo, notado x^{-1} . Es decir, tal que $x \cdot x^{-1} = 1_{\mathbb{K}}$.
- La operación $\cdot$ es distributiva sobre la operación $+$.
- $1_{\mathbb{K}} \neq 0_{\mathbb{K}}$.

La definición de cuerpo implica que si $\mathbb{K}$ es un cuerpo, entonces $0_{\mathbb{K}} \cdot x = 0_{\mathbb{K}}$ para todo $x \in \mathbb{K}$, y que el producto de cualesquiera elementos no nulos es no nulo. Además, $(\mathbb{K}, +)$ y $(\mathbb{K} \setminus \{0_{\mathbb{K}}\}, \cdot)$ son grupos abelianos. Por coloquialidad, utilizaremos el símbolo 0 en vez de $0_{\mathbb{K}}$ y el símbolo 1 en lugar de $1_{\mathbb{K}}$.

Por ejemplo, $(\mathbb{R}, +, \cdot)$ es un cuerpo, así como lo son $(\mathbb{Q}, +, \cdot)$, $(\mathbb{C}, +, \cdot)$ y $(\mathbb{K}, +, \cdot)$ para $\mathbb{K} = \{a + b\sqrt{2} : a, b \in \mathbb{Q}\}$.

A.1.2. Cuerpos Finitos

Definición A.2 (Cuerpo finito). Un cuerpo finito es un cuerpo $(\mathbb{K}, +, \cdot)$ donde $|\mathbb{K}| < \infty$.

El conjunto de enteros módulo n siempre forma un anillo. Este anillo es un cuerpo si y sólo si n es primo. Para un primo p , notamos $\mathbb{F}_p$ a su cuerpo finito asociado, con las operaciones binarias de suma y producto definidas módulo p . Más generalmente, todo cuerpo finito tiene cardinalidad $q = p^k$ para algún primo p y algún entero $k \geq 1$, y es habitual notarlo $\mathbb{F}_q$. Intuitivamente, un cuerpo finito puede pensarse como un sistema numérico donde todas las operaciones usuales funcionan, pero con una cantidad finita de elementos.

Desde el punto de vista criptográfico, los cuerpos finitos permiten definir operaciones eficientes, controlar la complejidad computacional (al tener una cantidad finita de elementos) y formular problemas matemáticos difíciles de resolver, sobre los que sostener la seguridad de sistemas criptográficos.

Definición A.3 (Característica). *Sea $\mathbb{K}$ un cuerpo, y sean $0_{\mathbb{K}}, 1_{\mathbb{K}}$ sus elementos neutros aditivos y multiplicativos, respectivamente. La característica de $\mathbb{K}$, notada $\text{char}(\mathbb{K})$ se define como el menor número natural n tal que $\underbrace{1_{\mathbb{K}} + 1_{\mathbb{K}} + \dots + 1_{\mathbb{K}}}_{n \text{ veces}} = 0_{\mathbb{K}}$, y en caso de que no exista ningún natural que cumpla esto, la característica se define como 0.*

Es notable que $\mathbb{Q}, \mathbb{R}, \mathbb{C}$ son cuerpos de característica 0, mientras que para todo primo $p \in \mathbb{N}$, $\text{char}(\mathbb{F}_p) = p$. Es conocido que todo cuerpo tiene característica 0 o característica prima.

A.1.3. Estructura multiplicativa del grupo no nulo de $\mathbb{F}_q$

El conjunto de elementos no nulos de un cuerpo finito forma un grupo bajo la multiplicación. En particular, $(\mathbb{F}_q^\times, \cdot)$ es un grupo finito de $q - 1$ elementos.

Recordamos algunas nociones básicas de teoría de grupos.

Definición A.4 (Subgrupo, grupo cíclico, generador). *Dado un grupo $(\mathbb{G}, \cdot)$ y $g \in \mathbb{G}$, consideramos el conjunto $\langle g \rangle = \{g^k : k \in \mathbb{Z}\}$. El par $(\langle g \rangle, \cdot)$ es un subgrupo de $\mathbb{G}$, y nos referimos a $\langle g \rangle$ como un subgrupo cíclico de $\mathbb{G}$. Decimos que g genera $\langle g \rangle$. Si $\langle g \rangle = \mathbb{G}$, entonces decimos que $\mathbb{G}$ es cíclico y que g es un generador de $\mathbb{G}$.*

Observar que siempre g es un generador del grupo $\langle g \rangle$. Para un grupo finito, vale que $\langle g \rangle = \{1, g, g^2, \dots, g^{n-1}\}$, y decimos que n es el orden de $\langle g \rangle$. También nos podemos referir a n como el orden de g . Veremos una propiedad relevante de los subgrupos, que es corolario del Teorema de Lagrange sobre subgrupos:

Teorema A.5 (Corolario del Teorema de Lagrange). *Sea $\mathbb{G}$ un grupo y sea $\mathbb{G}$ un subgrupo de $\mathbb{G}$. Entonces, la cantidad de elementos de $\mathbb{G}$ divide a la cantidad de elementos de $\mathbb{G}$.*

Teorema A.6 ($(\mathbb{F}_q^\times, \cdot)$ es cíclico). *Sea $\mathbb{F}_q$ un cuerpo finito. Entonces, existe $a \in \mathbb{F}_q^\times$ (no necesariamente único) tal que $\mathbb{F}_q^\times = \{1, a, a^2, \dots, a^{q-2}\}$.*

A.2. Problema del logaritmo discreto (DLP)

En numerosos contextos algebraicos, ciertas operaciones resultan computacionalmente sencillas, mientras que sus inversas pueden ser significativamente más complejas. Un ejemplo es el problema del logaritmo discreto (DLP). Intuitivamente, elevar un elemento de un grupo a una potencia es una operación eficiente. Sin embargo, determinar qué exponente

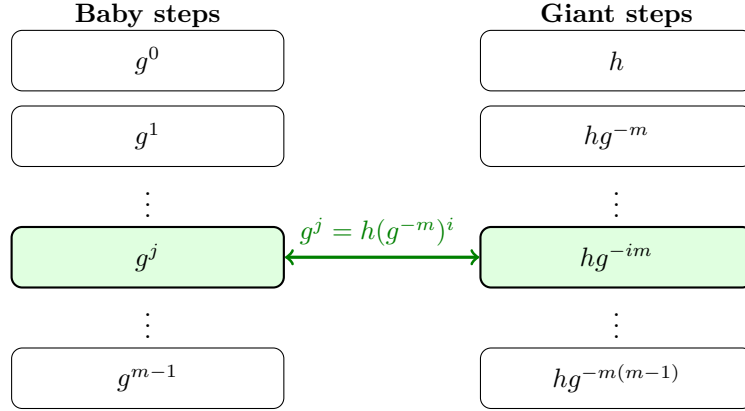


Fig. A.1: Idea del algoritmo *Baby-Step Giant-Step* para resolver un DLP en un grupo cíclico $\mathbb{G} = \langle g \rangle$. La tabla de la izquierda enumera los *baby steps* g^j , la de la derecha enumera los *giant steps* $h(g^{-m})^i$, y una coincidencia determina un exponente $x = im + j$ tal que $h = g^x$.

fue utilizado (aun conociendo el resultado) puede ser computacionalmente difícil. Esta asimetría constituye la base de múltiples construcciones criptográficas.

El problema del logaritmo discreto ha sido ampliamente estudiado en la literatura criptográfica [39, 40].

Definición A.7 (DLP). Sea $(\mathbb{G}, \cdot)$ un grupo cíclico finito, y sea g un generador de $\mathbb{G}$. Dado $h \in \mathbb{G}$, el problema del logaritmo discreto consiste en hallar un $x \in \mathbb{Z}$ que satisfaga $h = g^x$. Al valor x se lo denomina logaritmo discreto de h en base g , denotado $\log_g(h)$ cuando existe y es único módulo $|\mathbb{G}|$.

A diferencia del logaritmo en los números reales, donde existen algoritmos eficientes para su cálculo, en grupos finitos no se conocen métodos generales eficientes para resolver el DLP. La dificultad del problema depende fuertemente de la estructura del grupo subyacente.

En grupos cíclicos genéricos, DLP se considera computacionalmente difícil en el sentido de la complejidad algorítmica. Se sabe que cualquier algoritmo probabilístico que resuelva DLP con alta probabilidad requiere, en el peor caso, un número de operaciones del orden de $\Omega(\sqrt{n})$, donde n es el orden del grupo cíclico [41]. No se sabe si el DLP pertenece a la clase de complejidad P, ni si es NP-completo.

Un algoritmo clásico que muestra que esta barrera es alcanzable en el modelo genérico de grupo es el algoritmo *Baby-Step Giant-Step* (BSGS). Sea $\mathbb{G} = \langle g \rangle$ un grupo cíclico de orden n , y sea $h \in \mathbb{G}$. Queremos hallar $x \in \mathbb{Z}$ tal que $h = g^x$. Si $m = \lceil \sqrt{n} \rceil$, entonces todo $0 \leq x < n$ puede escribirse como $x = im + j$ para algunos $0 \leq i, j < m$. Por lo tanto:

$$h = g^x = g^{im+j} = (g^m)^i g^j,$$

y el DLP equivale a buscar índices i, j tales que $h(g^{-m})^i = g^j$.

Cuando el grupo considerado es el grupo multiplicativo de un cuerpo finito, por ejemplo $\mathbb{F}_q^\times$, la estructura aritmética adicional permite algoritmos más eficientes que en un grupo genérico. En el caso particular de cuerpos primos grandes, el mejor algoritmo clásico conocido es el Number Field Sieve para logaritmos discretos (NFS-DL), cuya complejidad es subexponencial.

Algoritmo 4 Algoritmo Baby-Step Giant-Step para DLP**Requiere:** Grupo cíclico finito $\mathbb{G} = \langle g \rangle$ de orden n , elemento $h \in \mathbb{G}$ **Asegura:** Un exponente $x \in \mathbb{Z}$ tal que $h = g^x$, si existe

```

1:  $(m, \gamma, y) \leftarrow (\lceil \sqrt{n} \rceil, g^{-m}, h)$ 
2:  $T \leftarrow \{(g^j, j) : 0 \leq j < m\}$ 
3: for  $i = 0$  to  $m - 1$  do
4:   if  $y \in T$  then
5:     Sea  $j$  tal que  $T[y] = j$ 
6:     return  $x = im + j$ 
7:   end if
8:    $y \leftarrow y \cdot \gamma$ 
9: end for
10: return  $\perp$ 

```

El DLP ocupa un lugar central en la criptografía moderna, ya que la seguridad de numerosos esquemas depende de su dificultad computacional. Entre ellos se encuentran el intercambio de claves de Diffie-Hellman, diversos esquemas de clave pública y varios esquemas de firma digital, como Schnorr y DSA. En este marco, una clave privada suele modelarse como un exponente $x \in \mathbb{N}$, mientras que la clave pública asociada es el elemento g^x .

En síntesis, calcular potencias en un grupo cíclico es eficiente, mientras que invertir esa operación resulta computacionalmente prohibitivo con recursos realistas. Esta asimetría entre cómputo directo e inversión es precisamente lo que vuelve útil al DLP como fuente de seguridad criptográfica.

A.3. Polinomios**A.3.1. Anillo de polinomios**

Los polinomios constituyen una de las herramientas algebraicas más versátiles en matemática y criptografía. Permiten representar funciones, modelar relaciones algebraicas y describir cálculos completos. En esta sección introducimos el anillo de polinomios y algunas de sus propiedades fundamentales.

Definición A.8 (Anillo de polinomios). *Dado un cuerpo $\mathbb{K}$, se define el anillo de polinomios en una indeterminada x sobre $\mathbb{K}$, denotado $\mathbb{K}[x]$, como el conjunto de todas las expresiones formales de la forma*

$$P(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m = \sum_{i=0}^m a_ix^i,$$

donde $m \in \mathbb{N}$ y $a_0, a_1, a_2, \dots, a_m \in \mathbb{K}$. Llamamos grado de P , notado $\deg P$, al mayor $k \in \mathbb{N}$ tal que $a_k \neq 0$. Las operaciones de este anillo se definen como:

■ *Suma (+):*

$$\left(\sum_{i=0}^m a_ix^i \right) + \left(\sum_{i=0}^n b_ix^i \right) = \sum_{i=0}^{\max\{m,n\}} (a_i + b_i)x^i,$$

donde se toma $a_i = 0$ o $b_i = 0$ si el índice es mayor que el grado correspondiente.

- *Producto* $(\cdot)$:

$$\left(\sum_{i=0}^m a_i x^i \right) \cdot \left(\sum_{i=0}^n b_i x^i \right) = \sum_{k=0}^{m+n} \left(\sum_{i+j=k} a_i b_j \right) x^k.$$

Con estas operaciones, $\mathbb{K}[x]$ es un anillo conmutativo cuyo neutro aditivo es $C(x) = 0$ (el polinomio nulo), y cuyo neutro multiplicativo es $I(x) = 1$.

Se observan las siguientes propiedades para el grado de un polinomio:

- El polinomio nulo no tiene un grado definido.
- $\deg(P + Q) \leq \max\{\deg P, \deg Q\}$.
- Si $P, Q \neq 0$, entonces $\deg(P \cdot Q) = \deg P + \deg Q$.

Definición A.9 (Raíz de un polinomio). *Dados un polinomio $P \in \mathbb{K}[x]$ y un $x^* \in \mathbb{K}$, decimos que x^* es una raíz de P si $P(x^*) = 0$.*

Las siguientes propiedades sobre polinomios serán útiles para trabajar con polinomios en las secciones posteriores.

Teorema A.10 (Algoritmo de división de polinomios). *Sea $\mathbb{K}$ un cuerpo y sean $P, Q \in \mathbb{K}[x]$ tales que $Q \neq 0$. Entonces, existen únicos polinomios $C, R \in \mathbb{K}[x]$ (cociente, resto) tales que $P = Q \cdot C + R$, donde o bien $R = 0$, o bien $\deg R < \deg Q$.*

Como corolario de este teorema, se observa que si x^* es una raíz de P , entonces al dividir P por el polinomio $x - x^*$, se obtiene como resto el polinomio nulo, y vale $\deg Q = \deg P - 1$. Puede ocurrir que x^* sea raíz de Q , así como no. Lo que es seguro es que el multiconjunto de raíces de Q (conjunto de raíces contando multiplicidad) se obtiene al quitarle un x^* al multiconjunto de raíces de P .

Aunque los polinomios se definen como expresiones formales, resulta natural asociarles una función mediante evaluación, es decir $P : \mathbb{K} \rightarrow \mathbb{K}$. En cuerpos finitos, esta distinción es particularmente relevante: polinomios distintos en $\mathbb{F}_q[x]$ pueden inducir la misma función.

A.3.2. Lema de Schwartz-Zippel

El lema de Schwartz–Zippel es una herramienta fundamental en el estudio de las Pruebas de Identidad de Polinomios (Polynomial Identity Testing, PIT). Las PIT son centrales en los proving systems modernos y aparecen de manera recurrentemente en diversos protocolos criptográficos, donde permiten obtener garantías de corrección probabilísticas.

Teorema A.11. *Sea $\mathbb{F}_q$ un cuerpo finito, y sean $P, Q \in \mathbb{F}_q[x]$ dos polinomios de grado a lo sumo n . Se toma un $k \in \mathbb{F}_q$ de forma uniformemente aleatoria. Si $P \neq Q$, entonces*

$$\mathbb{P}[P(k) = Q(k)] \leq \frac{n}{q}.$$

Si $P \neq Q$, entonces el polinomio $R = P - Q$ es no nulo y tiene grado a lo sumo n . Por lo tanto, posee a lo sumo n raíces. En consecuencia, la igualdad $P(k) = Q(k)$ sólo puede ocurrir en un conjunto reducido de puntos.

En contextos donde el tamaño del cuerpo es significativamente mayor al grado de los polinomios, esta probabilidad resulta despreciable. Por ejemplo, si $q \approx 2^{255}$ y $n = 2^{64}$,

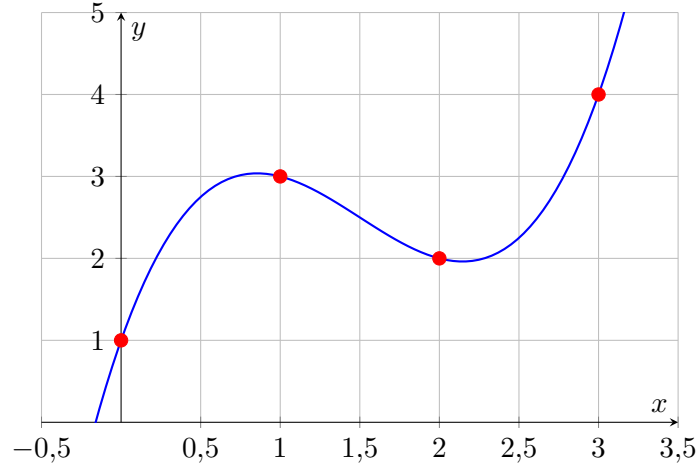


Fig. A.2: Ejemplo de interpolación polinómica sobre $\mathbb{R}$. Los cuatro puntos rojos $D = \{(0,1), (1,3), (2,2), (3,4)\}$ determinan un único polinomio de grado a lo sumo 3, mostrado en azul: $P(x) = x^3 - \frac{9}{2}x^2 + \frac{11}{2}x + 1$.

entonces $\frac{n}{q} \approx 2^{-191}$. Esto quiere decir que si $P(k) = Q(k)$, es extremadamente probable que $P = Q$. Esto se logra evaluando los polinomios únicamente en un punto, lo que es eficiente.

La hipótesis de elegir k de forma aleatoria es extremadamente importante para la seguridad de los protocolos criptográficos que usan este lema como una de sus herramientas. Si el punto de evaluación pudiera ser elegido de forma maliciosa por un agente que conoce P, Q , el lema dejaría de ser aplicable. Ya que si existe un $k^* \in \mathbb{F}_q$ tal que $P(k^*) = Q(k^*)$, entonces el agente malicioso puede usarlo para darnos una garantía (falsa) de que $P = Q$. Esto indica que se requieren mecanismos seguros para la generación de aleatoriedad y su correcta incorporación en los procesos criptográficos que usen el lema de Schwartz-Zippel.

A.3.3. Interpolación de polinomios

La interpolación de polinomios permite reconstruir un polinomio a partir de sus evaluaciones. La interpolación sobre cuerpos finitos desempeña un rol central en construcciones criptográficas modernas, en particular en la aritmetización de cómputos en sistemas de pruebas sucintas.

Definición A.12 (Polinomio interpolador). Sean $\mathbb{K}$ un cuerpo, $P \in \mathbb{K}[x]$ un polinomio, y $D = \{(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)\}$ un conjunto de puntos de $\mathbb{K} \times \mathbb{K}$ donde los x_i son distintos dos a dos. Si se cumple $P(x_i) = y_i$ para cada $0 \leq i \leq n$, entonces decimos que P es un polinomio interpolador de D .

Teorema A.13 (Existencia y unicidad del polinomio interpolador). Existe un único polinomio interpolador P^* con $\deg P^* \leq n$ para un conjunto D de $n+1$ puntos, y está dado por:

$$P^*(x) = \sum_{j=0}^n y_j L_j(x), \quad L_j(x) = \prod_{i=0, i \neq j}^n \frac{x - x_i}{x_j - x_i}.$$

Algoritmo 5 Interpolación de Lagrange**Requiere:** Puntos $D = \{(x_0, y_0), \dots, (x_n, y_n)\}$ con $x_i \neq x_j$ para $i \neq j$ **Asegura:** Polinomio $P^*(x)$ tal que $P^*(x_i) = y_i$ para todo $0 \leq i \leq n$

```

1:  $P^*(x) \leftarrow 0$ 
2: for  $j = 0$  to  $n$  do
3:    $L_j(x) \leftarrow 1$ 
4:   for  $i = 0$  to  $n$  do
5:     if  $i \neq j$  then
6:        $L_j(x) \leftarrow L_j(x) \cdot \frac{x-x_i}{x_j-x_i}$ 
7:     end if
8:   end for
9:    $P^*(x) \leftarrow P^*(x) + y_j L_j(x)$ 
10: end for
11: return  $P^*(x)$ 

```

Intuitivamente, podemos ver que $L_j(x) = 0$ para cada $i \neq j$, y $L_j(x) = 1$. Así mismo, la unicidad se puede deducir: si hubiera dos polinomios P, Q de grado n que interpolan a un conjunto de $n + 1$ puntos, entonces el polinomio $P - Q$ tendría raíces en $x_0, \dots, x_n$, lo que son al menos $n + 1$ raíces, contadas con multiplicidad. Esto quiere decir que $P - Q$ es el polinomio nulo, i.e., $P = Q$.

A.4. Curvas Elípticas

En esta sección introducimos las curvas elípticas sobre un cuerpo, y las dotamos de una estructura de grupo abeliano. Las curvas elípticas presentan propiedades de seguridad criptográfica favorables, en particular, para niveles de seguridad comparables, los grupos de curvas elípticas permiten tamaños de parámetros significativamente menores que los grupos multiplicativos clásicos.

Definición A.14 (Curva elíptica). Sea $\mathbb{K}$ un cuerpo con $\text{char}(\mathbb{K}) \neq 2, 3$. Una curva elíptica $\mathcal{E}$ sobre $\mathbb{K}$ es una curva, junto con un punto distinguido $\mathcal{O}$, que puede describirse mediante una ecuación de la forma

$$\mathcal{E} : y^2 = x^3 + ax + b, \quad a, b \in \mathbb{K},$$

y sujeta a la condición de no singularidad $4a^3 + 27b^2 \neq 0$.

El conjunto de puntos de la curva elíptica sobre $\mathbb{K}$ se define como

$$\mathcal{E}(\mathbb{K}) = \{(x, y) \in \mathbb{K} \times \mathbb{K} : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

donde $\mathcal{O}$ denota el punto del infinito o punto identidad.

La condición de no singularidad garantiza que la curva no posee puntos singulares, y por lo tanto que la curva es lisa. Es decir, si consideramos la función en dos variables $F(x, y) = y^2 - x^3 - ax - b$, sus derivadas parciales son $\frac{\partial F}{\partial x}(x, y) = -3x^2 - a$, y $\frac{\partial F}{\partial y}(x, y) = 2y$. Un punto de la curva es singular si ambas derivadas parciales de F se anulan en él.

Definición A.15 (Definición geométrica de suma en curvas elípticas). Sea $\mathbb{K}$ un cuerpo, y $\mathcal{E}(\mathbb{K})$ una curva elíptica. Dados $P, Q \in \mathcal{E}(\mathbb{K})$, se define la suma $+$: $\mathcal{E}(\mathbb{K}) \times \mathcal{E}(\mathbb{K}) \rightarrow \mathcal{E}(\mathbb{K})$ desde una perspectiva geométrica, como:

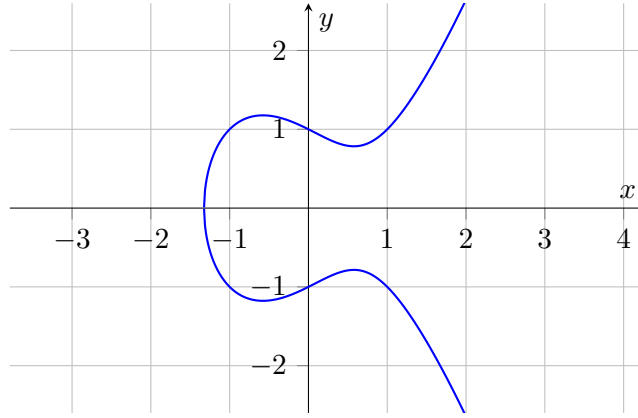


Fig. A.3: Curva elíptica real $\mathcal{E}(\mathbb{R})$ dada por $y^2 = x^3 - x + 1$. La figura muestra las dos ramas $y = \pm\sqrt{x^3 - x + 1}$ en el plano real; la condición de no singularidad garantiza que esta curva pueda equiparse con la operación geométrica de suma de puntos junto con el punto en el infinito $\mathcal{O}$.

1. Se considera la recta que pasa por P y Q . En el caso particular $P = Q$, se toma la recta tangente a la curva en P . En el caso particular $Q = -P$, la recta es vertical.
2. La recta interseca a la curva elíptica en un tercer punto R (contando multiplicidades).
3. Si R' es la reflexión de R por el eje x , se define $P + Q := R'$. En el caso particular de la recta vertical, tenemos $Q = -P$ y se define $P + Q := \mathcal{O}$.

La introducción del punto $\mathcal{O}$ es necesaria para que la suma de puntos esté definida en todos los casos, en particular cuando se suman puntos opuestos cuya recta secante es vertical. Sin este punto adicional, la operación no sería cerrada en $\mathcal{E}(\mathbb{K})$. Esta operación cumple el resto de propiedades para poder considerar a $(\mathcal{E}(\mathbb{K}), +)$ como un grupo abeliano:

- $P + \mathcal{O} = \mathcal{O} + P = P$ para todo $P \in \mathcal{E}(\mathbb{K})$ (elemento neutro).
- $P + (-P) = \mathcal{O}$ para todo $P \in \mathcal{E}(\mathbb{K})$ (elemento inverso). Un detalle geométrico relevante es que $-P$ se obtiene al reflejar P por el eje x .
- $P + (Q + R) = (P + Q) + R$ para todos $P, Q, R \in \mathcal{E}(\mathbb{K})$ (asociatividad).
- $P + Q = Q + P$ para todos $P, Q \in \mathcal{E}(\mathbb{K})$ (conmutatividad).

Por convención, utilizaremos notación aditiva en vez de multiplicativa al trabajar con grupos de curva elíptica. Dados $n \in \mathbb{N}$ y $P \in \mathcal{E}(\mathbb{K})$, podemos definir $nP := P + P + \dots + P$, donde hay n sumandos.

Al pasar a considerar curvas elípticas sobre cuerpos finitos, es decir curvas de la forma $\mathcal{E}(\mathbb{F}_q)$ con $q = p^k$, p primo y $\text{char}(\mathbb{F}_q) \neq 2, 3$, la interpretación geométrica deja de ser literal, aunque sigue siendo una guía conceptual valiosa. La estructura de grupo abeliano se preserva, y en particular la operación de suma continúa estando bien definida. Sin embargo, en este contexto ya no resulta conveniente describir la suma en términos geométricos, sino mediante fórmulas algebraicas explícitas.

Definición A.16 (Definición algebraica de suma en curvas elípticas). Sea $\mathbb{K}$ un cuerpo finito, sea $\mathcal{E}(\mathbb{K})$ una curva elíptica y sean $P, Q \in \mathcal{E}(\mathbb{K})$. Se define la suma $+$: $\mathcal{E}(\mathbb{K}) \times$

$\mathcal{E}(\mathbb{K}) \rightarrow \mathcal{E}(\mathbb{K})$ desde una perspectiva algebraica mediante fórmulas explícitas y el siguiente algoritmo:

1. Si $P = \mathcal{O}$, entonces $P + Q := Q$. Similarmente, si $Q = \mathcal{O}$, entonces $P + Q := P$.
2. Si $P, Q \neq \mathcal{O}$, entonces son puntos de la forma $P = (x_P, y_P)$, $Q = (x_Q, y_Q)$ con coordenadas en $\mathbb{F}_q$. Si $Q = -P$, lo que es equivalente a $x_P = x_Q$ y $y_P = -y_Q$, entonces $P + Q := \mathcal{O}$.
3. Se calcula la pendiente λ de la recta que pasa por P y Q .
 - En el caso $P = Q$, es una recta tangente a P y $\lambda := \frac{3x_P^2 + a}{2y_P}$.
 - En el caso $P \neq Q$, se calcula de forma usual: $\lambda := \frac{y_Q - y_P}{x_Q - x_P}$.
4. Para $x^* := \lambda^2 - x_P - x_Q$ y $y^* = \lambda(x_P - x^*) - y_P$, se define $P + Q := (x^*, y^*)$.

Podemos ver un ejemplo concreto sobre la curva $\mathcal{E}(\mathbb{F}_{19})$ dada por la ecuación $y^2 = x^3 - x + 1$. Tomemos $P = (2, 8), Q = (3, 5) \in \mathcal{E}(\mathbb{F}_{19})$. Como $P \neq Q$ y $Q \neq -P$, hacemos los siguientes cálculos:

$$\begin{aligned}\lambda &= \frac{y_Q - y_P}{x_Q - x_P} = \frac{5 - 8}{3 - 2} = -3 \equiv 16 \pmod{19}, \\ x^* &= \lambda^2 - x_P - x_Q = 16^2 - 2 - 3 = 251 \equiv 4 \pmod{19}, \\ y^* &= \lambda(x_P - x^*) - y_P = 16(2 - 4) - 8 = -40 \equiv 17 \pmod{19}.\end{aligned}$$

Por lo tanto, $P + Q = (4, 17)$.

Una curva elíptica sobre cuerpos finitos tiene finitos puntos. Pero es notable que, en general, $|\mathcal{E}(\mathbb{F}_q)| \neq |\mathbb{F}_q| = q$. La cota de Hasse establece un límite preciso para el número de puntos racionales de una curva elíptica definida sobre un cuerpo finito.

Teorema A.17 (Cota de Hasse). *Sea $\mathbb{F}_q$ un cuerpo finito, y sea $\mathcal{E}(\mathbb{F}_q)$ una curva elíptica. Entonces, $|\mathcal{E}(\mathbb{F}_q)| - (q + 1) \leq 2\sqrt{q}$.*

En particular, es relevante en numerosos protocolos criptográficos que $|\mathcal{E}(\mathbb{F}_q)|$ tenga un divisor primo grande. Usualmente se eligen curvas elípticas donde $|\mathcal{E}(\mathbb{F}_q)| = hr$, donde r es un primo grande y h (llamado el *cofactor* de la curva) es un número pequeño, como 1, 2, o 4. Ahora sí estamos preparados para adentrarnos en los subgrupos cíclicos de una curva elíptica sobre un cuerpo finito.

Definición A.18 (Orden de un punto en una curva elíptica). *Sea $\mathbb{F}_q$ un cuerpo finito, y sea $\mathcal{E}(\mathbb{F}_q)$ una curva elíptica. Para cada punto $P \in \mathcal{E}(\mathbb{F}_q)$, definimos su orden como el menor $r \in \mathbb{N}$ tal que $rP = \mathcal{O}$. Es decir, $|\langle P \rangle|$ para $\langle P \rangle \subset \mathcal{E}(\mathbb{F}_q)$.*

Observamos que el orden de P siempre existe y es un divisor del orden de $\mathcal{E}(\mathbb{F}_q)$ por el corolario del Teorema de Lagrange. Dentro de este grupo cíclico podemos volver a considerar el DLP, pero sobre un grupo de curva elíptica.

Definición A.19 (ECDLP). *Sea $\mathbb{F}_q$ un cuerpo finito, y sea $\mathcal{E}(\mathbb{F}_q)$ una curva elíptica. Dados un punto generador $G \in \mathcal{E}(\mathbb{F}_q)$ y un punto $P \in \langle G \rangle$, el problema del logaritmo discreto sobre curvas elípticas (ECDLP) es encontrar un $k \in \mathbb{N}$ tal que $P = kG$.*

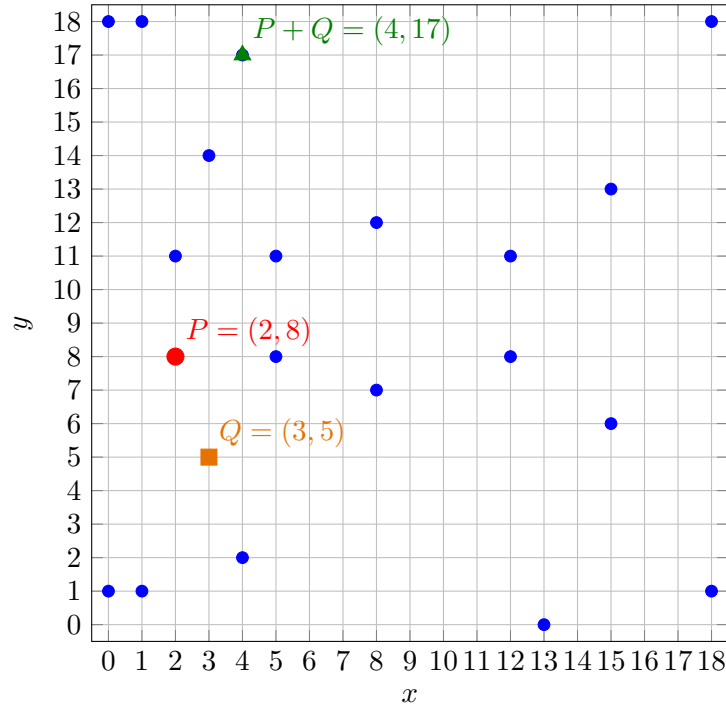


Fig. A.4: Puntos de $\mathcal{E}(\mathbb{F}_{19})$ para la curva $y^2 = x^3 - x + 1$. Los puntos azules son todas las soluciones sobre $\mathbb{F}_{19}$; se destacan $P = (2, 8)$ en rojo, $Q = (3, 5)$ en naranja y el resultado de la suma algebraica $P + Q = (4, 17)$ en verde, calculado con las fórmulas de grupo sobre cuerpos finitos.

En este contexto, se entiende por qué es útil buscar que el orden r de $\langle G \rangle$ sea un número primo grande. A diferencia del DLP en el grupo multiplicativo de un cuerpo finito, los algoritmos conocidos más eficientes para resolver el ECDLP para cualquier curva elíptica son genéricos en la estructura de grupo, por lo que su complejidad está en $\Omega(\sqrt{r})$. Esta diferencia algorítmica se traduce en una ventaja práctica en términos de bits de seguridad para algoritmos criptográficos, y por lo tanto en operaciones más rápidas, menos uso de memoria, claves de menor tamaño, y pruebas más pequeñas. En este contexto, las claves privadas serán de la forma $k \in \mathbb{F}_r$, mientras que sus claves públicas derivadas serán de la forma $K = kG \in \mathcal{E}(\mathbb{F}_q)$.

Bits de seguridad	$\mathbb{F}_q^\times$ (DLP clásico)	$\mathcal{E}(\mathbb{F}_q)$ (ECDLP)
128	~ 3072 bits	~ 256 bits
192	~ 7680 bits	~ 384 bits
256	~ 15360 bits	~ 512 bits

Tab. A.1: Comparación de tamaños de parámetros para niveles de seguridad equivalentes.

Sin embargo, no todas las curvas elípticas son igualmente adecuadas para fines criptográficos. Existen familias especiales para las cuales se conocen algoritmos más eficientes, e incluso polinomiales, para resolver el ECDLP. Por ello, la selección de una curva no depende solo de la eficiencia de sus operaciones, sino también de sus propiedades aritméticas y de su resistencia frente a ataques conocidos. Estas consideraciones motivan el estudio detallado

de subgrupos cíclicos de orden primo, que constituyen el entorno algebraico fundamental para numerosas construcciones criptográficas modernas.

Cuando se descubre un algoritmo significativamente más eficiente para resolver el ECDLP sobre una familia de curvas, dicha familia deja de ser considerada segura para uso criptográfico. En ese contexto, se habla de un **ataque** criptográfico. El lector interesado puede consultar, por ejemplo, los ataques a curvas anómalas [42], el *MOV attack* sobre curvas supersingulares [43] o los ataques basados en *Weil descent* [44].

A lo largo del proyecto aparecen varias curvas elípticas con roles diferentes. En general, al describir una curva elíptica sobre un cuerpo finito, conviene distinguir los siguientes parámetros:

- El cuerpo base $\mathbb{F}_q$ sobre el que está construida la curva $\mathcal{E}(\mathbb{F}_q)$, donde $q = p^k$ con p primo.
- Un generador $G \in \mathcal{E}(\mathbb{F}_q)$ de un subgrupo $\langle G \rangle \subset \mathcal{E}(\mathbb{F}_q)$.
- El cuerpo escalar $\mathbb{F}_r$, donde $r \in \mathbb{N}$ es el orden del subgrupo generado por G . Idealmente, r será un primo grande.

A.4.1. Representación y compresión de puntos

Para transmitir o almacenar un punto de una curva elíptica es necesario convertirlo en una secuencia de bytes. Una representación directa de un punto afín $P = (x, y)$ incluye ambas coordenadas. Sin embargo, en una curva dada por $y^2 = x^3 + ax + b$, la coordenada x determina como máximo dos valores posibles para y , que corresponden a puntos opuestos. Una representación comprimida puede entonces almacenar x junto con un bit que indique cuál de esos dos valores debe elegirse, mientras que el punto en el infinito utiliza una codificación distinguida.

En este contexto, “compresión” significa utilizar una serialización reversible de menor tamaño. La descompresión interpreta los bytes, reconstruye la coordenada faltante y rechaza representaciones inválidas o no canónicas. Dependiendo de la interfaz criptográfica, también puede comprobar que el punto pertenezca al subgrupo esperado. Sólo después de esta conversión el punto puede utilizarse en operaciones de grupo como sumas o multiplicaciones escalares. Esta frontera entre bytes serializados y puntos de curva aparece de forma explícita en las interfaces criptográficas utilizadas posteriormente en el proyecto.

A.4.2. Curvas elípticas útiles para este trabajo

- **BLS12-381.** Es la curva emparejable más importante para esta tesis. Aparece en la discusión de Groth16 y de KZG, en la motivación de las decisiones de diseño y en la implementación on-chain del verificador, ya que CARDANO expone primitivas criptográficas sobre BLS12-381 [20] y bibliotecas como **ak-381** se apoyan precisamente en esa curva. Además, en el bloque de circuitos del capítulo de implementación, el cuerpo escalar asociado a BLS12-381 se reutiliza para modelar nodos internos del SMT y para formalizar circuitos basados en **Poseidon**.
- **Jubjub.** Esta curva aparece asociada al circuito STM de MTHRIL en la era Lagrange. Allí las firmas Schnorr de los participantes, así como parte de la aritmetización de Halo2, se expresan sobre Jubjub. En consecuencia, aunque Jubjub no sea la curva usada para pairings on-chain en CARDANO, sí es central para entender la evidencia criptográfica que el zkBridge recibe y posteriormente comprime o verifica.

- **BN254.** Históricamente fue muy utilizada en otros entornos blockchain, en particular en ETHEREUM, y constituye una de las curvas emparejables más conocidas en la literatura de pruebas zk-SNARK. Su presencia en el texto sirve para contrastar alternativas posibles y para explicar por qué, en el ecosistema concreto de CARDANO, la curva emparejable relevante termina siendo BLS12-381.

A.5. Pairings bilineales

Los pairings bilineales son una primitiva criptográfica fundamental en las construcciones modernas. Informalmente, un pairing es una aplicación que relaciona puntos de curvas elípticas con elementos de un grupo multiplicativo, preservando una estructura algebraica específica. Gracias a sus propiedades, los pairings permiten reducir ciertos problemas difíciles en curvas elípticas [43] (como el ECDLP) a problemas equivalentes en otros grupos, lo que habilita nuevas construcciones criptográficas que no son posibles en el modelo clásico sin pairings.

Definición A.20 (Pairing bilineal). Sea $\mathcal{E}(\mathbb{F}_q)$ una curva elíptica definida sobre un cuerpo finito $\mathbb{F}_q$, y sea $r \in \mathbb{N}$ primo tal que r divide a $|\mathcal{E}(\mathbb{F}_q)|$. Un pairing bilineal es una función

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T,$$

donde $\mathbb{G}_1, \mathbb{G}_2$ son subgrupos cíclicos de orden r de $\mathcal{E}(\mathbb{F}_q)$ (o de extensiones de $\mathcal{E}(\mathbb{F}_q)$), y $\mathbb{G}_T$ es un subgrupo multiplicativo de un cuerpo finito (o de una extensión), también de orden r . Además, debe cumplir las siguientes propiedades:

- **Bilinealidad:** Para todos $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, a, b \in \mathbb{F}_r$, valen $e(aP, Q) = e(P, Q)^a$ y $e(P, bQ) = e(P, Q)^b$. Observar que en el lado derecho de la ecuación, se está utilizando la notación de grupo multiplicativo.
- **No degeneración:** Para cualesquiera generadores $G_1 \in \mathbb{G}_1, G_2 \in \mathbb{G}_2$ de los subgrupos de orden r , $e(G_1, G_2) \neq 1_{\mathbb{G}_T}$.
- **Computabilidad eficiente:** Existe un algoritmo eficiente en tiempo polinomial para computar $e(P, Q)$, para todos $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$.

Una forma útil de interpretar la bilinealidad es observar que el pairing bilineal convierte operaciones aditivas en operaciones multiplicativas:

$$e(P + P', Q) = e(P, Q) \cdot e(P', Q), \quad e(P, Q + Q') = e(P, Q) \cdot e(P, Q').$$

Esto permite trasladar relaciones lineales entre puntos de curvas elípticas a relaciones multiplicativas en $\mathbb{G}_T$, lo cual resulta extremadamente poderoso en construcciones criptográficas. Más aún, observamos que el pairing *mezcla* los escalares a, b en su producto, lo que induce otra operación difícil de revertir. Sin pairings, muchas construcciones modernas de pruebas sucintas y de autenticación algebraica no serían posibles.

La construcción explícita de un pairing bilineal involucra una cantidad considerable de teoría adicional, incluyendo el uso de cuerpos de extensión, divisores en curvas algebraicas, funciones racionales y algoritmos específicos como el algoritmo del loop de Miller [45]. El desarrollo completo de estos conceptos excede el objetivo de esta tesis y requeriría una extensión significativa que no sería reutilizada en las secciones posteriores. Ejemplos clásicos

de pairings bilineales utilizados en la práctica son el pairing de Weil y el pairing de Tate. Por lo tanto, se asumirá la existencia de los pairings bilineales junto con las propiedades mencionadas anteriormente, sin entrar en los detalles de su construcción matemática. Se remite al lector interesado a [46], que proporciona una introducción accesible a los pairings bilineales.

Los pairings bilineales que vamos a necesitar se dicen asimétricos, es decir, son tales que no se conoce ningún isomorfismo eficiente entre los subgrupos $\mathbb{G}_1, \mathbb{G}_2$. Además, el ECDLP debe ser difícil de resolver en ellos.

A.6. Hashes

A.6.1. Funciones de hash criptográficas

Las funciones de hash son una primitiva criptográfica sumamente popular. Ejemplos conocidos son MD5, SHA-1, SHA-256, BLAKE2, BLAKE3, RIPEMD-160, Poseidon, entre muchas otras. Estas serán utilizadas frecuentemente en el desarrollo del proyecto, y permiten construir primitivas criptográficas más avanzadas.

Definición A.21 (Función de hash criptográfica). *Sea $n \in \mathbb{N}$. Se dice que $\mathcal{H} : \{0,1\}^* \rightarrow \{0,1\}^n$ es una función de hash criptográfica si satisface las siguientes propiedades:*

- *Pueden ser implementadas determinísticamente en forma eficiente.*
- *Resistencia a preimagen: Dado $y \in \{0,1\}^n$, es difícil encontrar $x \in \{0,1\}^*$ tal que $\mathcal{H}(x) = y$.*
- *Resistencia a segunda preimagen: Dados $x \in \{0,1\}^*$, $\mathcal{H}(x) \in \{0,1\}^n$, es difícil encontrar $x' \neq x$ tal que $\mathcal{H}(x) = \mathcal{H}(x')$.*
- *Resistencia a colisiones: Es difícil encontrar $x \neq x'$ tales que $\mathcal{H}(x) = \mathcal{H}(x')$.*

La definición anterior es deliberadamente informal, ya que todavía no precisa qué significa *difícil*. En este apéndice alcanza con la intuición de *computacionalmente prohibitivo*; las formulaciones rigurosas de estas propiedades pueden encontrarse en [39, 40].

Existe una relación jerárquica entre estas propiedades. En particular, la resistencia a colisiones implica resistencia a segunda preimagen, y esta última implica resistencia a preimagen.

La compresión de un dominio infinitamente grande $\{0,1\}^*$ en un codominio finito $\{0,1\}^n$ implica inevitablemente la existencia de colisiones. La seguridad no radica en su inexistencia, sino en la imposibilidad computacional de encontrarlas.

En criptografía, la seguridad de muchas construcciones se analiza en modelos idealizados. En particular, las funciones de hash suelen tratarse no como objetos matemáticos explícitos, sino como primitivas ideales. Por ello, numerosas demostraciones se formulan en modelos como el Random Oracle Model (ROM) [47]. En este marco, se supone que $\mathcal{H} : \{0,1\}^* \rightarrow \{0,1\}^n$ se comporta como una función completamente aleatoria. Bajo esa idealización, cualquier algoritmo genérico para hallar una preimagen tiene complejidad $\Theta(2^n)$, mientras que cualquier algoritmo genérico para encontrar colisiones tiene complejidad $\Theta(2^{n/2})$ (ver [39]).

Podemos transformar cualquier estructura matemática o información finita a una tira de bytes, es decir al conjunto $\{0,1\}^*$, y elegir un n suficientemente grande como para que valga la propiedad de resistencia a colisiones. Esto es lo que se hace en la práctica. Entonces, dada una función de hash $\mathcal{H}$ y un conjunto arbitrario X donde sus elementos

tienen tamaño finito, podemos hablar de la función de hash asociada $\mathcal{H}_X : X \rightarrow \{0, 1\}^n$, y por comodidad de notación usar simplemente $\mathcal{H}$.

A.6.2. Domain separation

En construcciones criptográficas modernas es habitual reutilizar una misma función de hash en múltiples contextos dentro de un protocolo. Sin embargo, esta reutilización puede introducir interacciones indeseadas entre componentes que, idealmente, deberían comportarse de manera independiente. La técnica de *domain separation* consiste en forzar dicha independencia lógica modificando la entrada de la función de hash mediante etiquetas, prefijos o parámetros públicos.

Es decir, permite que una misma función $\mathcal{H}$ actúe como si fueran múltiples funciones independientes. Por ejemplo, en lugar de computar simplemente $\mathcal{H}(x)$, se evalúan expresiones del tipo $\mathcal{H}(\text{commit} \parallel x)$ o $\mathcal{H}(\text{challenge} \parallel x)$. De esta forma, aunque los valores de entrada coincidan, los hashes pertenecen a dominios criptográficamente aislados. Esta técnica resulta fundamental a lo largo de esta tesis, por ejemplo, en la heurística de Fiat–Shamir.

A.6.3. Hash hacia curvas elípticas

En numerosos protocolos criptográficos modernos resulta necesario mapear datos arbitrarios (es decir, tiras de bits de $\{0, 1\}^*$) a puntos de una curva elíptica. Este problema no es trivial: una función de hash produce cadenas de bits, mientras que los puntos de una curva elíptica deben satisfacer una ecuación algebraica específica. Formalmente, buscamos una función determinística y eficiente

$$\text{HashToCurve} : \{0, 1\}^* \rightarrow \mathcal{E}(\mathbb{F}_q)$$

que modele, idealmente, una distribución uniforme sobre un subgrupo criptográficamente relevante de $\mathcal{E}(\mathbb{F}_q)$ [48, 49]. Es decir, que ambas distribuciones (la del hash y la uniforme) sean computacionalmente indistinguibles.

En esta tesis asumiremos la existencia de funciones HashToCurve seguras y eficientes, sin profundizar en sus construcciones explícitas. El estándar moderno que formaliza estas construcciones es RFC 9380 [50], que especifica algoritmos concretos. Conceptualmente, dichos algoritmos implementan funciones del tipo

$$\{0, 1\}^* \rightarrow \mathbb{F}_q \rightarrow \mathcal{E}(\mathbb{F}_q)$$

minimizando sesgos estadísticos [48] y evitando ataques estructurales.

En la práctica, estas funciones no se construyen directamente, sino mediante transformaciones intermedias. Típicamente, el proceso involucra:

1. Expandir la entrada mediante una función de hash convencional.
2. Interpretar el *output* como uno o más elementos del cuerpo base $\mathbb{F}_q$ de la curva.
3. Aplicar un mapeo algebraico hacia la curva [49, 51], es decir una función $f : \mathbb{F}_q \rightarrow \mathcal{E}(\mathbb{F}_q)$.
4. (Opcionalmente) Forzar la pertenencia al subgrupo de orden primo deseado mediante multiplicación por el cofactor de la curva.

Más precisamente, estos mapeos algebraicos consisten en funciones racionales definidas sobre $\mathbb{F}_q$, diseñadas de modo que su imagen pertenezca a $\mathcal{E}(\mathbb{F}_q)$ con alta probabilidad o de forma determinística. Dichas funciones se construyen utilizando identidades algebraicas que garantizan la satisfacción de la ecuación de la curva.

A.6.4. Firmas Schnorr sobre curvas elípticas

Si fijamos un subgrupo cíclico de una curva elíptica $\langle G \rangle \subseteq \mathcal{E}(\mathbb{F}_q)$ de orden primo r , es natural construir sobre él primitivas criptográficas concretas. Un ejemplo es el esquema de firmas de Schnorr, que será utilizado en la tesis.

- El firmante tiene una clave privada, que es un escalar $sk \in \mathbb{F}_r$, y una clave pública, que es un punto $pk = skG \in \langle G \rangle$.
- Para firmar un mensaje $m \in \{0, 1\}^*$, el firmante elige un $k \in \mathbb{F}_r$ aleatorio, calcula $R = kG$, deriva el *challenge* $c = \mathcal{H}(m \| R) \in \mathbb{F}_r$, define la respuesta $s = k + c \cdot sk \in \mathbb{F}_r$, y publica como firma el par $\sigma = (R, s)$.
- La verificación consiste en recomputar $c = \mathcal{H}(m \| R)$ y comprobar si se verifica la igualdad $sG \stackrel{?}{=} R + c \cdot pk$. Esta igualdad se satisface si el firmante conocía la clave privada sk , ya que $sG = (k + c \cdot sk)G = kG + c \cdot skG = R + c \cdot pk$.

Informalmente, producir firmas válidas sin conocer sk requiere quebrar la dificultad esperada del ECDLP o manipular indebidamente el *challenge* derivado con la función de hash. Por esa razón, Schnorr constituye una de las construcciones más limpias para entender cómo se pasa de la teoría de grupos de curvas elípticas a primitivas criptográficas concretas.

A.7. Merkle Trees

A.7.1. Definición e intuición

Los Merkle Trees son estructuras de datos basadas en árboles binarios y funciones de hash criptográficas que se utilizan extensivamente como parte de protocolos criptográficos en el ROM. Por ejemplo, son utilizadas en la red de BITCOIN desde sus inicios [2]. Fueron introducidas por Merkle en [52].

Definición A.22 (Merkle Tree). *Sea $L = \{\ell_1, \dots, \ell_n\} \subset \{0, 1\}^*$ una secuencia ordenada de datos, y sea $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^n$ una función de hash criptográfica. Un Merkle Tree es un árbol binario $\mathcal{T}_{L, \mathcal{H}}$ (notado $\mathcal{T}$ por comodidad) completo cuyas hojas corresponden a los elementos de L , definido recursivamente de la siguiente manera:*

1. *En el nivel inferior, están las hojas del árbol, que son los hashes de los datos ordenados: $h_i = \mathcal{H}(\ell_i)$, para $1 \leq i \leq n$.*
2. *Cada nodo interno N con hijos N_i (nodo hijo izquierdo) y N_d (nodo hijo derecho) se define como $N = \mathcal{H}(N_i \| N_d)$, donde $\|$ denota la operación de concatenación en $\{0, 1\}^*$.*
3. *La raíz del árbol se nota $r_{\mathcal{T}}$. Es el nodo que sintetiza criptográficamente toda la información de L .*

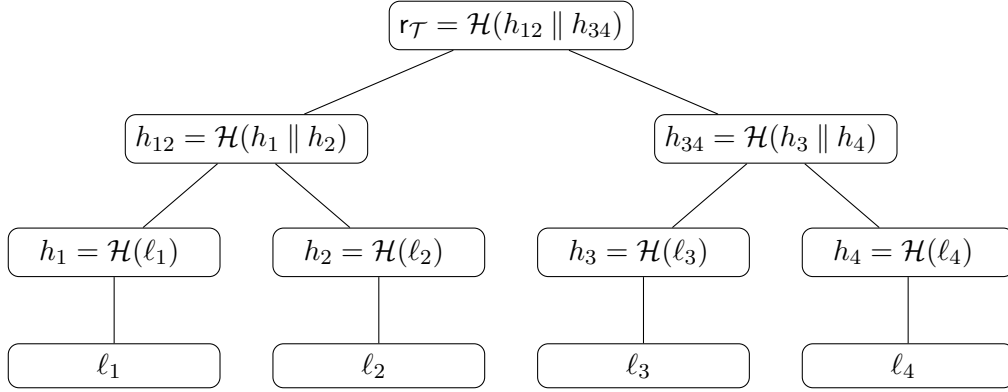


Fig. A.5: Merkle Tree binario construido sobre la secuencia ordenada $L = (\ell_1, \ell_2, \ell_3, \ell_4)$. Las hojas del árbol son los hashes $\mathcal{H}(\ell_i)$, los nodos internos se obtienen hasheando la concatenación ordenada de sus hijos, y la raíz $r_{\mathcal{T}}$ compromete criptográficamente a toda la secuencia.

Si n no es una potencia de 2, se completa la lista de hojas mediante una función de padding determinística.

La propiedad clave de un Merkle Tree $r_{\mathcal{T}}$ es que cualquier modificación en un dato ℓ_i provoca un cambio en su hoja asociada $\mathcal{H}(\ell_i)$, y se propaga hasta $r_{\mathcal{T}}$ de una forma impredecible. Esto permite verificar la integridad de un elemento específico sin necesidad de recorrer o almacenar toda la secuencia de datos. Cualquier modificación en un dato (incluso en un solo bit) permite detectar alteraciones de forma inmediata solamente verificando si cambió el valor de $r_{\mathcal{T}}$.

Por supuesto, los elementos de la secuencia de datos pueden pertenecer a cualquier conjunto arbitrario X mientras todos sus datos sean finitos, y transformarse a tiras de bits, replicando la codificación discutida en la Sección A.6.

Observar que en las imágenes se indica la secuencia de datos por debajo de $\mathcal{T}$, pero se recalca que estos no pertenecen directamente a $\mathcal{T}$, sino que el nivel inferior de $\mathcal{T}$ está compuesto por los $\mathcal{H}(\ell_i)$.

A.7.2. Merkle Proofs

Una de las propiedades más importantes de esta estructura es que permite construir pruebas de pertenencia, también conocidas como Merkle Proofs. Una Merkle Proof permite demostrar que un elemento pertenece a un Merkle Tree $\mathcal{T}$ sin necesidad de revelar todos los elementos de la secuencia de datos asociada. En particular, esto permite verificar un solo dato conociendo únicamente la raíz $r_{\mathcal{T}}$ y el camino autenticado correspondiente, sin necesidad de replicar el árbol completo en cada lugar donde se quiera verificar pertenencia.

Definición A.23 (Merkle Proof). Sea $\mathcal{T}$ un Merkle Tree con raíz $r_{\mathcal{T}}$, construido sobre una secuencia de datos $L = \{\ell_1, \dots, \ell_n\}$ y usando una función de hash criptográfica $\mathcal{H}$. Una Merkle Proof para un dato $\ell = (b_1, \dots, b_k) \in L$ es una lista ordenada $\pi = [h_1, \dots, h_k]$ que representa un camino (Merkle Path) desde la hoja correspondiente a ℓ hasta $r_{\mathcal{T}}$, donde:

- Cada $h_i \in \{0,1\}^n$ es el hash del hermano del nodo del nivel i en el Merkle Path desde la hoja correspondiente a ℓ hasta $r_{\mathcal{T}}$.

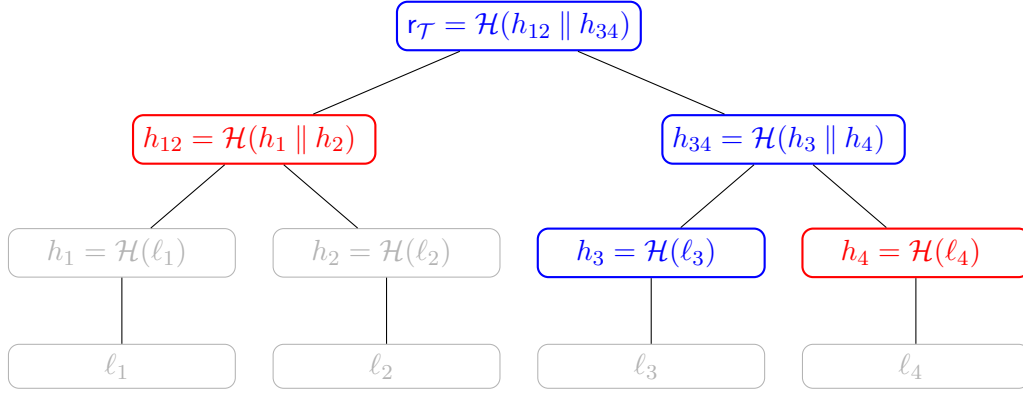


Fig. A.6: Merkle Proof de pertenencia de ℓ_3 respecto de la raíz $r_{\mathcal{T}}$. Los nodos azules forman el camino que se recomputa desde $\mathcal{H}(\ell_3)$ hasta la raíz, mientras que los nodos rojos son los hashes hermanos que debe aportar la prueba; en este ejemplo, $\pi = [h_4, h_{12}]$.

- Cada $b_i \in \{0, 1\}$ es el i -ésimo bit de ℓ , e indica si el hash h_i está a la izquierda ($b_i = 0$) o a la derecha ($b_i = 1$) del nodo en el Merkle Path.
- $k = \lceil \log_2 n \rceil$ es la altura de $\mathcal{T}$, por lo tanto la cantidad de pasos en el Merkle Path hasta $r_{\mathcal{T}}$.

Intuitivamente, una Merkle Proof consiste en los nodos hermanos a lo largo del camino que conecta una hoja con la raíz del Merkle Tree.

En la imagen de una Merkle Proof, se observa que es una prueba de pertenencia de ℓ_3 , y que $\pi = [h_4, h_{12}]$. Se puede definir una función (y su algoritmo correspondiente) para verificar que una Merkle Proof es correcta. La idea es reconstruir determinísticamente los nodos del Merkle Path del dato en $\mathcal{T}$ (en azul en la figura A.6) usando π (en rojo en la figura A.6) para llegar hasta $r_{\mathcal{T}}$, y verificar que es igual al valor esperado. Los bits b_i son esenciales para preservar el orden de concatenación dentro del input de $\mathcal{H}$.

La seguridad criptográfica de un Merkle Tree se reduce a la resistencia a colisiones de su función de hash. En efecto, una colisión permitiría construir dos subárboles distintos con la misma raíz, comprometiendo la solidez de las pruebas de pertenencia (ver [39, 40]).

Definición A.24 (Función de verificación de Merkle Proof). Sea $\mathcal{T}$ un Merkle Tree con raíz $r_{\mathcal{T}}$, construido usando una función de hash criptográfica $\mathcal{H}$, y sea π una Merkle Proof para un dato $\ell \in L$. Definimos la función de verificación de π como:

$$\text{Verify}(\ell, \pi, r_{\mathcal{T}}) = \llbracket x_k = r_{\mathcal{T}} \rrbracket,$$

donde

$$x_0 := \mathcal{H}(\ell), \quad x_i := \begin{cases} \mathcal{H}(h_i \parallel x_{i-1}) & \text{si } b_i = 0, \\ \mathcal{H}(x_{i-1} \parallel h_i) & \text{si } b_i = 1, \end{cases} \quad 1 \leq i \leq k.$$

Notar que el algoritmo 6 tiene una complejidad temporal de $\mathcal{O}(k) = \mathcal{O}(\log n)$, así como el tamaño de la Merkle Proof. Para árboles con muchos niveles, esto resulta clave. Además, no resulta necesario almacenar todo el vector camino x durante el mismo.

Desde una perspectiva criptográfica, los Merkle Trees pueden interpretarse como esquemas de compromiso vectorial [53] con pruebas de tamaño logarítmico. Esta interpretación resulta especialmente útil para entender por qué la tesis utiliza Merkle Trees dentro de circuitos con mayor estructura.

Algoritmo 6 Verificación de una Merkle Proof**Requiere:** Dato $\ell \in \{0, 1\}^*$, Merkle Proof $\pi = [h_1, \dots, h_k]$, raíz $r_{\mathcal{T}} \in \{0, 1\}^n$ **Asegura:** **true** si π prueba que $\ell \in \mathcal{T}$, **false** en caso contrario

```

1:  $(b_1, \dots, b_k) \leftarrow \ell$  ▷ representación binaria de  $\ell$ 
2:  $x \leftarrow \mathcal{H}(\ell)$ 
3: for  $i = 1$  to  $k$  do
4:   if  $b_i = 0$  then ▷  $h_i$  es hijo izquierdo
5:      $x \leftarrow \mathcal{H}(h_i \parallel x)$ 
6:   else ▷  $h_i$  es hijo derecho
7:      $x \leftarrow \mathcal{H}(x \parallel h_i)$ 
8:   end if
9: end for
10: return  $\llbracket x = r_{\mathcal{T}} \rrbracket$ 

```

A.8. Sparse Merkle Trees (SMT)**A.8.1. Definición e Intuición**

Los Merkle Trees tradicionales permiten realizar pruebas eficientes de pertenencia a un conjunto de datos finito. Pero hay una limitación fundamental a la hora de realizar pruebas de exclusión, es decir de la **no pertenencia** de un dato ℓ a un conjunto L . En un Merkle Tree clásico, las hojas corresponden exclusivamente a los elementos de L . Una posible solución sería definir un orden total sobre L (si fuera posible), construyendo una prueba de exclusión mostrando que el elemento consultado se encuentra entre dos hojas consecutivas del árbol. Observamos que en protocolos criptográficos esto no es ideal, pues se debe revelar información adicional sobre elementos vecinos en L .

Para superar estas limitaciones se introducen los SMT. La idea de una estructura de datos que permite tanto actualizar como revocar certificados fue introducida por primera vez en [54].

Definición A.25 (Default hashes, Sparse Merkle Tree). Sean $k, n \in \mathbb{N}$, y $\mathcal{H}$ una función de hash criptográfica, y sea $D = \{0, 1\}^k$ el dominio completo de posibles elementos (cadenas de k bits). Definimos primero los **default hashes** por nivel de forma recursiva:

$$z_0 := \mathcal{H}(\perp), \quad z_i := \mathcal{H}(z_{i-1} \parallel z_{i-1}) \quad \text{para } 0 < i \leq k.$$

Dado $L \subseteq D$, un **SMT** $\mathcal{T}_{L, \mathcal{H}} = \mathcal{T}$ de altura k y datos almacenados L es un árbol binario completo de altura k definido recursivamente de la siguiente manera:

1. *Hojas:* Cada elemento $x \in D$ se asocia en forma ordenada a una hoja h_x de la siguiente forma:

$$h_x = \begin{cases} \mathcal{H}(x), & \text{si } x \in L \\ z_0 & \text{si } x \notin L \end{cases}$$

2. Cada nodo interno N con hijos N_i (nodo hijo izquierdo) y N_d (nodo hijo derecho) se define como $N = \mathcal{H}(N_i \parallel N_d)$.
3. La raíz $r_{\mathcal{T}}$ es el nodo que sintetiza criptográficamente toda la información de L sobre el dominio D .

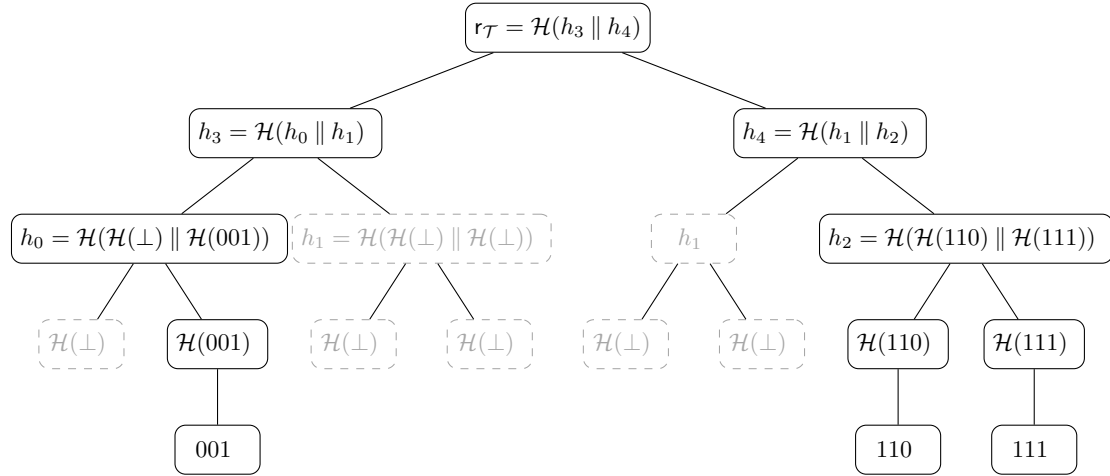


Fig. A.7: Sparse Merkle Tree de altura 3 con hojas ocupadas 001, 110 y 111. Los nodos sólidos son los que deben almacenarse explícitamente, mientras que los nodos punteados representan subárboles vacíos reemplazables por *default hashes*; por eso el árbol completo de 2^3 hojas puede representarse sin materializar todos sus nodos.

4. Aunque conceptualmente $\mathcal{T}$ tiene 2^k hojas, en la práctica solo se almacenan y calculan los nodos necesarios para representar los elementos de L y los hashes por defecto de las ramas no ocupadas. Esto permite mantener las pruebas de pertenencia y exclusión de tamaño $\mathcal{O}(k)$.

Cada default hash z_i es un valor que representa un subárbol completamente vacío de altura i , cuyos nodos hoja contienen únicamente el valor por defecto. Entonces, la raíz de un SMT vacío $\mathcal{T}_\emptyset$ de altura k es exactamente z_k . Esto provee a los SMT de una estructura algebraica bien definida.

La ventaja de representar implícitamente la mayor parte de los nodos no se aprecia del todo en la figura A.7, ya que allí la altura del árbol es muy pequeña. Sin embargo, para valores de k usados en la práctica, como 224 o 256, almacenar explícitamente todos los nodos del árbol completo resulta completamente inviable. Esa es precisamente la motivación operativa detrás de los SMT.

A.8.2. Sparse Merkle Proofs

Los SMT permiten definir pruebas de exclusión eficientes: para demostrar que $\ell \notin L$, basta con exhibir el camino **Path** desde la hoja asociada a $\ell \in D \setminus L$ (que contiene el default hash z_0) hasta la raíz del árbol. La verificación de dicha prueba depende únicamente del valor de $r_{\mathcal{T}}$ y de la resistencia a colisiones de $\mathcal{H}$.

Definición A.26 (Sparse Merkle Proof). Sea $\mathcal{T}$ un Sparse Merkle Tree de altura k con raíz $r_{\mathcal{T}}$, construido sobre un conjunto finito $L \subseteq \{0, 1\}^k$ y de función criptográfica asociada $\mathcal{H}$. Sea $\ell = (b_1, \dots, b_k) \in \{0, 1\}^k$. Una Sparse Merkle Proof de ℓ en L es una lista ordenada $\pi = [h_1, \dots, h_k]$, donde:

- Cada $h_i \in \{0, 1\}^n$ es el hash del hermano del nodo del nivel i en el camino desde la hoja correspondiente a ℓ hasta $r_{\mathcal{T}}$.

- Cada $b_i \in \{0, 1\}$ es un bit que indica si el hash h_i está a la izquierda ($b_i = 0$) o a la derecha ($b_i = 1$) del nodo en el camino. Los bits b_i no forman parte de la prueba sino de la representación binaria de ℓ .

Nos referimos a π como una prueba de inclusión si $\ell \in L$, y como una prueba de exclusión si $\ell \notin L$.

Se puede dar una función de verificación (y su algoritmo correspondiente) para una Sparse Merkle Proof, así como hicimos con la Merkle Proof tradicional. Nuevamente, vamos reconstruyendo el camino **Path** de ℓ en un vector x , y la prueba es válida si y solo si el valor obtenido al final del camino es consistente con la raíz: $x_k = r_{\mathcal{T}}$. Si los hermanos no estuvieran almacenados explícitamente en $\mathcal{T}$, hay que usar los default hashes de los niveles correspondientes.

Definición A.27 (Función de verificación de una Sparse Merkle Proof). Sea $\mathcal{T}$ un Sparse Merkle Tree de altura k con raíz $r_{\mathcal{T}}$ y función de hash criptográfica asociada $\mathcal{H}$ y valor por defecto $\perp$, y sea π una Sparse Merkle Proof para un elemento $\ell = (b_1, \dots, b_k) \in \{0, 1\}^k$. La función de verificación de π se define como:

$$\text{Verify}(\ell, \pi, r_{\mathcal{T}}, b) = \llbracket x_k = r_{\mathcal{T}} \rrbracket,$$

donde

$$x_0 := \begin{cases} \mathcal{H}(\ell) & (\text{prueba de inclusión}), \\ z_0 & (\text{prueba de exclusión}), \end{cases}$$

y para cada $1 \leq i \leq k$:

$$x_i = \begin{cases} \mathcal{H}(h_i \parallel x_{i-1}) & \text{si } b_i = 0, \\ \mathcal{H}(x_{i-1} \parallel h_i) & \text{si } b_i = 1, \end{cases} \quad 1 \leq i \leq k.$$

Para el algoritmo de verificación, podemos simplemente indicar con un bit adicional b' si π es una prueba de inclusión o exclusión: $b = 1$ indica que es una prueba de inclusión, mientras que $b = 0$ implica una prueba de exclusión.

Algoritmo 7 Verificación de una Sparse Merkle Proof

Requiere: Elemento $\ell \in \{0, 1\}^*$, Sparse Merkle Proof $\pi = [h_1, \dots, h_k]$, raíz $r_{\mathcal{T}} \in \{0, 1\}^n$, bit $b \in \{0, 1\}$

Asegura: **true** si π prueba que $\ell \in L$ o $\ell \notin L$, **false** en caso contrario

```

1:  $(b_1, \dots, b_k) \leftarrow \ell$  ▷ representación binaria de  $\ell$ 
2:  $x \leftarrow$  if  $b = 1$  then  $\mathcal{H}(\ell)$  else  $z_0$ 
3: for  $i = 1$  to  $k$  do
4:   if  $b_i = 0$  then ▷  $h_i$  es hijo izquierdo
5:      $x \leftarrow \mathcal{H}(h_i \parallel x)$ 
6:   else ▷  $h_i$  es hijo derecho
7:      $x \leftarrow \mathcal{H}(x \parallel h_i)$ 
8:   end if
9: end for
10: return  $\llbracket x = r_{\mathcal{T}} \rrbracket$ 
```

Nada impide combinar una Sparse Merkle Proof de exclusión con una de inclusión para argumentar la corrección de una actualización sobre un SMT. En efecto, se trata de una estructura de datos **autenticada**: toda actualización de $\mathcal{T}$ induce una nueva raíz que puede ser verificada por terceros a partir de la información adecuada [55]. Esta propiedad se utiliza concretamente en el circuito `tx_set_update` de la Sección 6.8.

A.8.3. Actualización de un Sparse Merkle Tree

En comparación con los Merkle Trees de la Sección A.7, la diferencia central es la función de actualización de los SMT, ya que idealmente se usan como una estructura de datos dinámica: al agregar un nuevo elemento $\ell \in \{0, 1\}^k$, se deben añadir nuevos nodos al árbol, correspondientes al Merkle Path de ℓ en $\mathcal{T}$ (la parte que no haya sido creada antes). El Algoritmo 8 explica cómo hacer esto en tiempo $O(k)$.

En una implementación concreta no se materializa el árbol disperso completo: se almacenan únicamente los nodos no triviales y los caminos de autenticación necesarios para recomputar la raíz luego de cada inserción o eliminación.

Algoritmo 8 Actualización de una hoja en un Sparse Merkle Tree

Requiere: $\ell \in \{0, 1\}^k$, Sparse Merkle Proof $\pi = [h_1, \dots, h_k]$

Asegura: $\mathcal{T}$ es actualizado correctamente

```

1:  $(b_1, \dots, b_k) \leftarrow \ell$  ▷ representación binaria de  $\ell$ 
2:  $x \leftarrow \mathcal{H}(\ell)$ 
3: for  $i = 1$  to  $k$  do
4:   if  $b_i = 0$  then
5:      $x \leftarrow \mathcal{H}(h_i \parallel x)$ 
6:   else
7:      $x \leftarrow \mathcal{H}(x \parallel h_i)$ 
8:   end if
9:   Guardar  $x$  como valor del nodo correspondiente al prefijo  $(b_1, \dots, b_i)$ 
10: end for
11: return  $x$ 

```

En muchos protocolos no solo interesa actualizar la estructura abstracta del árbol, sino recomputar de manera autenticada la nueva raíz a partir de una prueba válida respecto de la raíz anterior. Esto puede expresarse con el siguiente algoritmo, que combina verificación y actualización local del camino.

Si la prueba original era de exclusión ($b = 0$), el algoritmo anterior primero verifica que la hoja correspondiente a ℓ estaba vacía respecto de la raíz antigua r , y luego sustituye ese valor por $\mathcal{H}(\ell)$ para producir la raíz nueva r^* . En consecuencia, formaliza exactamente el tipo de transición autenticada que más tarde reutilizaremos al discutir conjuntos *anti-replay* resumidos por SMT.

En la figura A.8, podemos ver la actualización de un Sparse Merkle Tree de altura 3 luego de agregarle el elemento $100 \in \{0, 1\}^3$. Observar que se encuentra coloreado su Merkle Path asociado, en donde los nodos marcados como $\mathcal{H}(0)$ y h_5 no existían antes, mientras que los nodos marcados con h_6 y $r^*(\mathcal{T})$ vieron actualizado su hash.

Las implementaciones reales representan sólo los caminos relevantes mediante tries binarios, pues la mayoría de los nodos son implícitos (*default hashes*). Cada clave ℓ determina una ruta y sólo existen nodos para los prefijos compartidos por claves insertadas.

Algoritmo 9 Actualización autenticada de la raíz de un Sparse Merkle Tree

Requiere: Elemento $\ell \in \{0,1\}^k$, Sparse Merkle Proof $\pi = [h_1, \dots, h_k]$, raíz anterior $r \in \{0,1\}^n$, bit $b \in \{0,1\}$

Asegura: Nueva raíz $r^* \in \{0,1\}^n$ si la prueba es consistente con r

```

1:  $(b_1, \dots, b_k) \leftarrow \ell$  ▷ representación binaria de  $\ell$ 
2:  $x \leftarrow$  if  $b = 1$  then  $\mathcal{H}(\ell)$  else  $z_0$ 
3: for  $i = 1$  to  $k$  do
4:   if  $b_i = 0$  then
5:      $x \leftarrow \mathcal{H}(h_i \parallel x)$ 
6:   else
7:      $x \leftarrow \mathcal{H}(x \parallel h_i)$ 
8:   end if
9: end for
10: if  $x \neq r$  then
11:   return fail
12: end if
13:  $x^* \leftarrow \mathcal{H}(\ell)$ 
14: for  $i = 1$  to  $k$  do
15:   if  $b_i = 0$  then
16:      $x^* \leftarrow \mathcal{H}(h_i \parallel x^*)$ 
17:   else
18:      $x^* \leftarrow \mathcal{H}(x^* \parallel h_i)$ 
19:   end if
20: end for
21: return  $x^*$ 

```

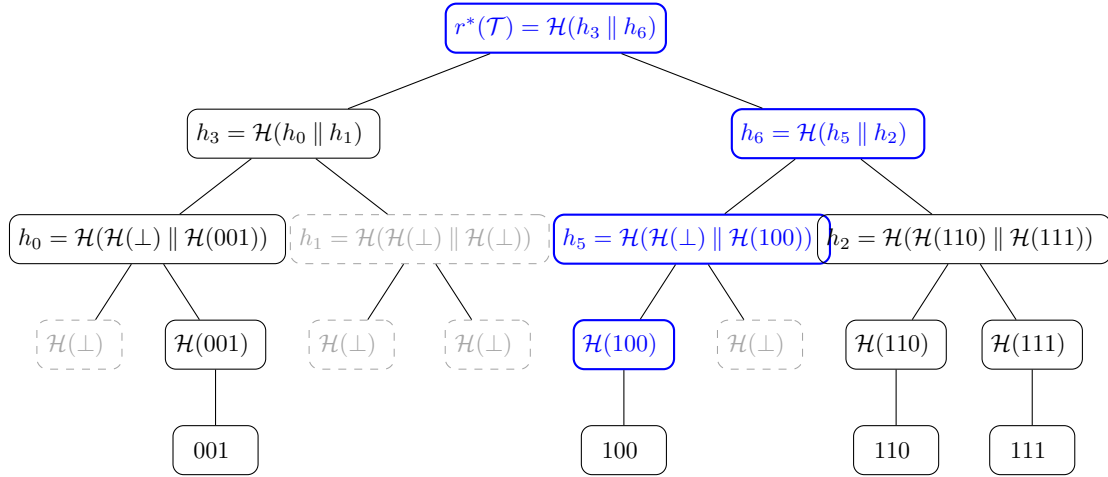


Fig. A.8: Actualización de un SMT de altura 3 al insertar la clave 100. El camino azul muestra los nodos que se crean o recomputan desde la nueva hoja $\mathcal{H}(100)$ hasta la nueva raíz $r^*(\mathcal{T})$; las ramas no afectadas, como el subárbol de 001, 110 y 111, se reutilizan sin cambios.

Además, los SMT suelen apoyarse en almacenamiento direccionado por hash: cada nodo se identifica por su hash y el árbol se reconstruye desde una base de datos *key-value*.

B. ARITMETIZACIÓN Y CONSTRUCCIÓN DE UN SNARK

Este apéndice reúne la teoría criptográfica subyacente a los argumentos sucintos utilizados en la tesis. Su propósito es concentrar en un único lugar las definiciones, transformaciones y construcciones que permiten pasar de un cómputo expresado como relación en NP a un argumento **zk-SNARK** verificable de manera eficiente. En particular, estas nociones sostienen la estrategia elegida en la Sección 5.4, la adaptación on-chain de la Sección 6.7 y los circuitos auxiliares de las Secciones 6.9 y 6.8.

Puede leerse de dos maneras. Una lectura lineal, de principio a fin, reconstruye la cadena completa que va desde la representación algebraica del cómputo hasta el esquema de prueba finalmente adoptado en el proyecto. En cambio, un lector que sólo necesite contexto para seguir el argumento principal puede usarlo como material de consulta selectiva: las secciones sobre QAP, KZG y Groth16 son las más directamente conectadas con los capítulos de diseño e implementación, mientras que las secciones iniciales fijan la intuición necesaria para que esas construcciones no aparezcan como cajas negras.

B.1. Introducción a pruebas ZK

Una prueba de conocimiento cero (*zero-knowledge proof*) o prueba **ZK** es un protocolo entre dos partes, en donde una quiere convencer a otra de que posee un valor, sin revelar nada sobre el conocimiento de este valor más allá de la veracidad de la información. Estas partes se conocen como **Prover** $\mathcal{P}$ y **Verifier** $\mathcal{V}$. Podemos pensar a estas partes como personas ejecutando un algoritmo, o como algoritmos en sí mismos (usaremos estos conceptos de forma indistinta). Esta idea fue introducida inicialmente por Goldwasser, Micali y Rackoff en [56].

Intuitivamente, el *witness* es la información privada que posee el prover y que hace verdadera la afirmación pública que quiere demostrar. La instancia pública, en cambio, contiene los datos que el verifier sí conoce. Así, una prueba **ZK** no busca revelar el *witness*, sino convencer al verifier de que existe un *witness* válido para la instancia pública considerada. Cuando el protocolo es interactivo, llamamos *transcript* a la secuencia ordenada de mensajes intercambiados entre prover y verifier durante una ejecución concreta del protocolo.

Definición B.1 (Protocolo de pruebas ZK). *Sea $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ una relación, donde $\mathcal{X}$ es el conjunto de public inputs y $\mathcal{W}$ el conjunto de witnesses. Asociado a $\mathcal{R}$, definimos el lenguaje*

$$\mathcal{L}_{\mathcal{R}} := \{x \in \mathcal{X} : \exists w \in \mathcal{W}, (x, w) \in \mathcal{R}\}.$$

Un protocolo de pruebas ZK para $\mathcal{R}$ está dado por un prover $\mathcal{P}$ y un verifier $\mathcal{V}$ tales que, sobre una instancia pública $x \in \mathcal{X}$ y un witness $w \in \mathcal{W}$, generan una interacción verificable por el verifier. Si el protocolo es interactivo, el objeto matemático relevante es el transcript completo de esa interacción; si es no interactivo, dicho objeto se reduce a una prueba $\pi \in \Pi$.

Informalmente, el protocolo debe satisfacer las siguientes propiedades:

1. *Compleitud: si $(x, w) \in \mathcal{R}$ y ambas partes siguen el protocolo, entonces $\mathcal{V}$ acepta con probabilidad alta.*

2. *Solidez: si la afirmación pública es falsa, es decir, si $x \notin \mathcal{L}_{\mathcal{R}}$, entonces un prover deshonesto no puede hacer que $\mathcal{V}$ acepte salvo con probabilidad negligible.*
3. *Conocimiento cero: el verifier no aprende nada esencial sobre w más allá de la validez de la afirmación demostrada.*

Diremos que el protocolo es no interactivo si toda la comunicación de $\mathcal{P}$ a $\mathcal{V}$ consiste en un único mensaje $\pi \in \Pi$; en ese caso llamaremos prueba a dicho mensaje. En caso contrario, diremos que es interactivo y hablaremos del transcript completo de la ejecución.

Vamos a ganar intuición sobre esta definición, apoyados también en el protocolo de ejemplo de la imagen B.1. Supongamos que tenemos un valor secreto $w \in \{0,1\}^*$ y calculamos $x = \mathcal{H}(w) \in \{0,1\}^n$ para una función de hash criptográfica $\mathcal{H}$, de modo que x es interesante (por ejemplo, empieze con muchos ceros) y por lo cual nos darían un premio al haber hallado w . Si alguien nos pide ver w para verificar la igualdad $\mathcal{H}(w) = x$, podemos no compartirlo, pero demostrar que lo conocemos (por esto se habla de prueba de conocimiento). En este contexto:

- La relación $\mathcal{R}$ está dada por los pares (x, w) tales que $\mathcal{H}(w) = x$. Notar que pueden existir múltiples witnesses válidos para el mismo x . Se puede describir de forma concisa como $\mathcal{R} = \{(\mathcal{H}(w), w) : w \in \mathcal{W}\}$.
- El conjunto $\mathcal{X} = \{0,1\}^n$ sería el de *public inputs*, ya que ambos conocemos x .
- El conjunto $\mathcal{W} = \{0,1\}^*$ sería el *witness*, ya que solamente nosotros conocemos w .
- Hay un conjunto de pruebas Π que describe todas las posibles combinaciones de datos que le enviamos al verifier. Veremos cómo son estos conjuntos cuando vayamos a protocolos de pruebas ZK concretos.
- Pensemos que si un w tal que $(x, w) \in \mathcal{R}$ fuera fácil de calcular, el protocolo de pruebas ZK no tendría sentido.

En la imagen B.1, el protocolo de ejemplo muestra una interacción de tres pasos entre $\mathcal{P}$ y $\mathcal{V}$. En este caso, el objeto relevante no es una prueba aislada sino el *transcript* completo de la interacción, formado por todos los mensajes intercambiados.

Por otra parte, podemos desglosar las tres propiedades que debe cumplir un protocolo de pruebas ZK.

- Si $\mathcal{P}$ ejecuta su algoritmo con un w tal que $(x, w) \in \mathcal{R}$, porque efectivamente **conoce** un w que cumple esto, entonces seguramente podrá convencer a $\mathcal{V}$ de que lo conoce.
- Si la afirmación pública es falsa, es decir, si $x \notin \mathcal{L}_{\mathcal{R}}$, entonces un prover deshonesto $\mathcal{P}^*$ probablemente no podrá convencer a $\mathcal{V}$. La probabilidad de engaño no es nula, pero es negligible. Qué tan baja queremos que sea esta probabilidad depende del nivel de bits de seguridad que exigimos a nuestro sistema criptográfico. Observamos que la propiedad de solidez, a diferencia de las otras, vale para **cualquier** prover.
- En esta introducción estamos trabajando con la noción de *honest-verifier zero knowledge*. Esto significa que $\mathcal{V}$ no aprende ninguna información esencial sobre w a partir del *transcript* del protocolo, más allá de la validez de la afirmación, siempre que siga honestamente el protocolo. La propiedad se enuncia mediante un simulador: un algoritmo que, sin conocer un *witness* válido, puede generar una vista del protocolo computacionalmente indistinguible de una ejecución real para ese verifier honesto. Intuitivamente, si existe tal simulador, entonces todo lo que ve el verifier durante el protocolo podría haberse generado sin conocer el *witness*; por lo tanto, la interacción

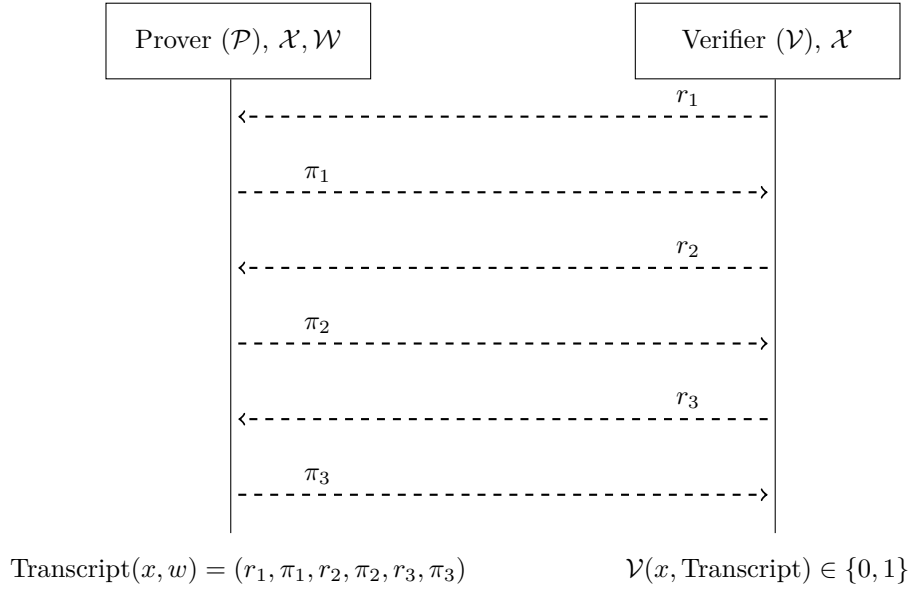


Fig. B.1: Prototipo de un protocolo ZK interactivo entre $\mathcal{P}$ y $\mathcal{V}$ sobre una instancia pública x y un witness privado w . Las flechas r_i representan desafíos del verifier, las flechas π_i representan respuestas del prover, y el objeto verificable final es el *transcript* completo $(r_1, \pi_1, r_2, \pi_2, r_3, \pi_3)$.

no revela información adicional sobre w . Además, el simulador debe ser eficiente: si generar la vista simulada requiriera mucho más tiempo que una ejecución real, ese costo también podría servir para distinguir ambas distribuciones.

- Un detalle adicional es que, si bien la propiedad de solidez trata el caso donde un prover puede ser deshonesto, acá no estamos considerando verifiers arbitrarios o maliciosos. La noción usada en esta sección es, precisamente, la de *honest-verifier zero knowledge*.

Desde el punto de vista de la teoría de la complejidad, los protocolos de pruebas ZK interactivos se estudian dentro del marco más general de las pruebas interactivas, correspondiente a la clase *Interactive Proofs* (IP). No todo protocolo en IP es de conocimiento cero, pero todo protocolo ZK interactivo es, en particular, una prueba interactiva. Informalmente, una clase de lenguajes L pertenece a IP si existe un protocolo interactivo $(\mathcal{P}, \mathcal{V})$ tal que para toda instancia $x \in \mathcal{X}$, el verifier $\mathcal{V}$, que se ejecuta en tiempo polinomial, acepta con probabilidad 1 cuando $x \in L$ y rechaza con probabilidad al menos $1 - \epsilon$ cuando $x \notin L$. El prover $\mathcal{P}$ no está restringido computacionalmente, y la interacción puede contar con un número polinomial de rondas. Podemos pensar al lenguaje L como inducido por $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$, donde $x \in L$ si y sólo si existe $w \in \mathcal{W}$ tal que $(x, w) \in \mathcal{R}$.

Un resultado central de la teoría de la complejidad es que la potencia expresiva de los protocolos interactivos coincide exactamente con la de la clase PSPACE. Es decir, todo lenguaje que puede decidirse utilizando una cantidad polinomial de espacio admite un protocolo de prueba interactivo con un verifier probabilístico en tiempo polinomial. En esta tesis, solamente nos interesa hablar del mismo como indicador de la enorme expresividad de los protocolos interactivos.

Teorema B.2. $\text{IP} = \text{PSPACE}$.

Este teorema fue demostrado inicialmente por Shamir en [57], lo cual implica que problemas como la equivalencia de circuitos booleanos o la validez de fórmulas cuantificadas (QBF) admiten protocolos interactivos con verificación eficiente. Para lectores experimentados en complejidad computacional, en [58] hay una exposición moderna y rigurosa del resultado.

La igualdad tiene implicancias relevantes incluso al restringirse a clases de complejidad más bajas, como NP. En efecto, todo lenguaje en NP admite trivialmente un protocolo interactivo eficiente: dado un lenguaje inducido por una relación $R \subseteq \mathcal{X} \times \mathcal{W}$, el prover puede convencer al verifier de que $x \in L$ demostrando el conocimiento de un witness w tal que $(x, w) \in R$. Esto permite existencia de protocolos interactivos para problemas clásicos de NP como la satisfacibilidad booleana, el isomorfismo de grafos, la 3-colorabilidad de un grafo, la existencia de un ciclo hamiltoniano, la existencia de k -cliques, entre muchos otros.

Por ahora, consideramos protocolos de pruebas ZK en su forma más general, y vimos que estos protocolos capturan una clase extremadamente amplia de problemas. Sin embargo, el hecho de que sean interactivos y puedan tener un costo comunicacional elevado vuelve poco prácticos a estos protocolos en muchos escenarios. En particular, los protocolos de pruebas ZK son usados en sistemas distribuidos y blockchains, donde requieren pruebas que puedan ser verificadas de manera no interactiva, con complejidad de verificación muy baja (no solamente polinomial), y tamaño de prueba reducido.

Más aún, podemos pensar a los protocolos ZK como una herramienta general para la verificación de cómputo. Dado un algoritmo eficiente y una entrada secreta, el prover puede convencer al verifier de que el algoritmo fue calculado correctamente, sin revelar información adicional sobre w ni sobre el proceso interno de cómputo. Si el tiempo de verificación es bajo, y el tamaño de prueba es reducido, podemos incluso obviar el hecho de que no estamos revelando información sobre w y hablar de protocolos ZK que permiten verificar cálculos de gran complejidad en sistemas con muy escasos recursos (como una blockchain). También se puede hablar de *cómputo delegado*.

Definición B.3 (Esquema de cómputo verificable (ZK)). *Sea $f : \mathcal{X} \times \mathcal{W} \rightarrow \mathcal{Y}$ una función computable en tiempo polinomial. Un esquema de cómputo verificable para f es un protocolo $(\mathcal{P}, \mathcal{V})$ asociado a la relación $\mathcal{R}_f \subseteq \mathcal{X}^* \times \mathcal{W}$, donde el conjunto de public inputs es $\mathcal{X}^* = \mathcal{X} \times \mathcal{Y}$ y el conjunto de witness es $\mathcal{W}$, y que está definida por:*

$$(x, y, w) \in \mathcal{R}_f \iff f(x, w) = y.$$

El protocolo $(\mathcal{P}, \mathcal{V})$ debe satisfacer:

1. *Compleitud:* Para todo $x \in \mathcal{X}$ y todo $w \in \mathcal{W}$, si definimos $y := f(x, w)$ y $\pi = \mathcal{P}(x, w)$, entonces vale $\mathbb{P}[\mathcal{V}(x, y, \pi) = 1] = 1$.
2. *Solidez:* Para todo prover probabilístico en tiempo polinomial posiblemente deshonesto $\mathcal{P}^*$ que, dada una entrada pública (x, y) , produce una prueba $\pi = \mathcal{P}^*(x, y)$, vale $\mathbb{P}[\mathcal{V}(x, y, \pi) = 1 \mid y \neq f(x, w) \text{ para todo } w \in \mathcal{W}] \leq \epsilon$, donde ϵ es un valor negligible.
3. *Eficiencia del verifier:* El tiempo de ejecución de $\mathcal{V}$ es polinomial en el tamaño de la entrada (x, y) , y sublineal respecto al costo requerido para computar directamente $f(x, w)$.

Si además el protocolo satisface la propiedad de conocimiento cero, diremos que el esquema de cómputo verificable es ZK.

En la definición anterior, asumimos que la función f es fija, pública y conocida tanto por $\mathcal{P}$ como por $\mathcal{V}$. Si la función no fuera fija, tendríamos que agregarla al conjunto de *inputs* públicos, es decir que serían de la forma (x, y, f) .

También observamos que en la definición anterior nos referimos al esquema de cómputo verificable era ZK, pero no hablamos de que $(\mathcal{P}, \mathcal{V})$ era un protocolo de pruebas ZK. Esto es porque debemos hacer una distinción fundamental antes de pasar a la siguiente sección de la tesis, entre *prueba ZK* y *argumento ZK*. Es notable cómo coloquialmente suele haber confusión entre estos dos términos y pensar que se refieren a lo mismo, usándose indistintamente. Daremos una distinción informal, basada en la distinción formal que se puede encontrar en el séptimo capítulo de [59].

1. En una prueba, la propiedad de solidez vale contra provers arbitrarios, incluso computacionalmente no acotados. Es decir, aun concediendo poder de cómputo ilimitado al prover deshonesto, la probabilidad de convencer al verifíer de una afirmación falsa permanece acotada por un valor negligible.
2. En un argumento, la propiedad de solidez sólo se exige contra provers eficientes. En particular, se cuantifica sobre provers probabilísticos en tiempo polinomial. Por eso también se habla de *computationally sound proof systems*. Si además vale la propiedad de conocimiento cero, diremos que se trata de un argumento ZK.

Ahora bien, ¿por qué usaríamos algo más débil que una prueba ZK, que puede ser teóricamente roto? Porque los argumentos ZK pueden ser mucho más eficientes y escalables. En general, los argumentos ZK asumen la imposibilidad de ciertos problemas criptográficos, como ECDLP, BDH, o la resistencia a colisiones de funciones de hash criptográficas.

B.2. SNARKs

Entre los distintos tipos de argumentos de conocimiento introducidos en la Sección B.1, los *SNARK* (*Succinct Non-Interactive ARgument of Knowledge*) ocupan un lugar central por su eficiencia y aplicabilidad práctica. Fueron introducidos por primera vez en [60]. Estos sistemas permiten demostrar la correctitud de cálculos arbitrariamente complejos mediante pruebas extremadamente cortas y verificables en tiempo reducido. La combinación de no interactividad, sucintez y solidez convierte a los SNARK en una herramienta fundamental en criptografía moderna, particularmente en contextos donde la verificación eficiente resulta crítica, como sistemas distribuidos, protocolos de privacidad y blockchain.

Definición B.4 (SNARK, zk-SNARK). Sea $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ una relación en la clase de complejidad NP. Un SNARK para $\mathcal{R}$ es una tripla de algoritmos en tiempo polinomial

$$\mathcal{S} = (\text{Setup}, \mathcal{P}, \mathcal{V}),$$

definidos como:

- $\text{Setup}(1^\lambda)$ es un algoritmo aleatorio que genera un conjunto de parámetros públicos *crs*, conocido como *Common Reference String*, que es usado tanto por el prover como por el verifíer. Es decir, $\text{Setup}(1^\lambda) \rightarrow \text{crs}$. Aquí λ es un parámetro de seguridad.
- $\mathcal{P}_{\text{crs}} : \mathcal{X} \times \mathcal{W} \rightarrow \Pi$ es un algoritmo aleatorio que, a partir de una instancia pública x y un witness w , produce una prueba π .

- $\mathcal{V}_{\text{crs}} : \mathcal{X} \times \Pi \rightarrow \{0, 1\}$ es un algoritmo determinístico que, dada una instancia pública x y una prueba π , acepta si π certifica que existe un witness w tal que $(x, w) \in \mathcal{R}$.

La tripla de algoritmos $\mathcal{S}$ debe, además, cumplir con las siguientes propiedades:

- *Compleitud:* Para todo $(x, w) \in \mathcal{R}$, se cumple

$$\mathbb{P}[\mathcal{V}_{\text{crs}}(x, \mathcal{P}_{\text{crs}}(x, w)) = 1] = 1.$$

- *Solidez por extracción:* Sea

$$\mathcal{L}_{\mathcal{R}} := \{x \in \mathcal{X} : \exists w \in \mathcal{W}, (x, w) \in \mathcal{R}\}.$$

Para todo prover probabilístico en tiempo polinomial $\mathcal{P}^*$, existe un extractor probabilístico en tiempo polinomial $\mathcal{E}^{\mathcal{P}^*}$, entendido como un algoritmo teórico con acceso a $\mathcal{P}^*$ que intenta recuperar un witness válido a partir de su capacidad de producir pruebas aceptadas, tal que, para toda instancia $x \in \mathcal{X}$, si

$$\mathbb{P}[\mathcal{V}_{\text{crs}}(x, \mathcal{P}^*(x)) = 1]$$

es no negligible, entonces

$$\mathbb{P}[(x, \mathcal{E}^{\mathcal{P}^*}(x)) \in \mathcal{R}]$$

también es no negligible. Es por esto que hablamos de que π es un argumento y no una prueba. En adelante, cuando hablamos de SNARK nos referimos a argumentos de conocimiento.

- *Sucintez:* Esta propiedad se divide, a su vez, en otras subpropiedades:
 - Existen polinomios p y q tales que, para todo $(x, w) \in \mathcal{R}$, si $\pi = \mathcal{P}_{\text{crs}}(x, w)$, entonces $|\pi| \leq p(\lambda, |x|)$. En particular, el tamaño de la prueba es independiente de $|w|$.
 - El tiempo de ejecución de $\mathcal{V}_{\text{crs}}$ está acotado por $q(\lambda, |x|)$. También es independiente de $|w|$.
 - Intuitivamente, el costo de verificación de $\pi = \mathcal{P}_{\text{crs}}(x, w)$ debe ser asintóticamente menor que el de recomputar por completo la relación a partir del witness.
- *No interactividad:* El prover $\mathcal{P}_{\text{crs}}$ genera un único mensaje π , que luego es verificado por $\mathcal{V}_{\text{crs}}$ sin interacción adicional.

Si además $\mathcal{S}$ satisface la siguiente propiedad, decimos que es un zk-SNARK.

- *Conocimiento cero:* Para todo $x \in \mathcal{X}$, existe un simulador probabilístico en tiempo polinomial $S(\text{crs}, x)$, en el mismo sentido de simulación indistinguible discutido para protocolos ZK interactivos, tal que la distribución de las pruebas simuladas $S(\text{crs}, x)$ es computacionalmente indistinguible de la distribución de pruebas reales $\mathcal{P}_{\text{crs}}(x, w)$, para cualquier $w \in \mathcal{W}$ tal que $(x, w) \in \mathcal{R}$. Intuitivamente, esto implica que el verifier podría haber generado por sí mismo una prueba indistinguible de la real, sin conocer el witness.

La definición de sucintez que dimos es informal. Para acercarnos a la formalidad, consideramos familias de instancias $(x_n, w_n) \in \mathcal{R}$ para las cuales el tamaño mínimo de w_n necesario para verificar $\mathcal{R}$ con x_n crece de forma no acotada. En ese régimen, el costo de verificación debe ser asintóticamente menor que el costo de verificar explícitamente la pertenencia de (x_n, w_n) a $\mathcal{R}$. Es decir, el verifier no debe volver a ejecutar el cómputo implícito en el witness.

Recordar que, como $\mathcal{R}$ está en NP, entonces existe una máquina de Turing determinística $M_{\mathcal{R}}$ que decide si $(x, w) \in \mathcal{R}$ en tiempo polinomial respecto de $|x| + |w|$. Lo que no necesariamente admite una decisión eficiente a partir sólo de la instancia pública es el lenguaje inducido

$$\mathcal{L}_{\mathcal{R}} = \{x \in \mathcal{X} : \exists w \in \mathcal{W}, (x, w) \in \mathcal{R}\},$$

es decir, el problema de decidir la pertenencia a la relación sin conocer el *witness* w . Tener presente que una relación en NP representa un cómputo siempre resulta útil a la hora de aprender o reforzar estos conceptos.

Ahora, vamos a hablar más informalmente de SNARK y zk-SNARK para ganar mayor intuición respecto a estos conceptos. En primer lugar, un SNARK puede entenderse como un mecanismo que permite a una parte demostrar que conoce un *witness* w consistente con un *input* público x , sin interacción adicional y con una prueba de tamaño pequeño. La no interactividad surge del hecho de que primero se ejecuta $\mathcal{P}_{\text{crs}}$ y luego $\mathcal{V}_{\text{crs}}$, de modo que toda la comunicación entre el *prover* y el *verifier* es un único mensaje π .

La propiedad de solidez computacional asegura que ningún *prover* eficiente puede convencer al *verifier* de una afirmación falsa, salvo con probabilidad negligible. La noción de solidez por extracción formaliza rigurosamente la intuición presentada en la sección anterior: una prueba válida no sólo convence al *verifier* de la veracidad de una afirmación, sino que además certifica que el *prover* posee efectivamente el conocimiento subyacente. La existencia de un extractor eficiente implica que cualquier *prover* capaz de generar pruebas aceptadas con probabilidad no negligible debe, implícitamente, contener suficiente información como para reconstruir un *witness* válido. En este sentido, la prueba actúa como una evidencia criptográfica de conocimiento.

La sucintez es la característica distintiva de los SNARK. Intuitivamente, esta propiedad establece que el costo de verificación depende únicamente del tamaño del *input* público x , y no de la complejidad del cómputo de $\mathcal{R}$. En particular, el *verifier* no vuelve a ejecutar el algoritmo implícito en el *witness* w , sino que se limita a verificar una prueba corta de la correctitud del cómputo. Este desacople entre el costo de probar y el costo de verificar es lo que permite utilizar los SNARK para la verificación de cálculos arbitrariamente grandes.

Finalmente, los zk-SNARK tienen la propiedad adicional de conocimiento cero, que garantiza que la prueba π no revela información adicional sobre el *witness* w más allá de su existencia. Conceptualmente, podemos separar completamente la correctitud del cómputo de la confidencialidad de los datos involucrados.

Para concluir con esta sección, hablaremos sobre el parámetro de seguridad λ , que es una abstracción en sistemas criptográficos. Intuitivamente, λ mide el nivel de protección del sistema criptográfico frente a ataques. A mayor λ , menor probabilidad de que un algoritmo adversario logre romper las garantías de seguridad del sistema. En la práctica, λ se relaciona en forma directa con el número de bits de seguridad del sistema: si decimos que ofrece 128 bits de seguridad, significa que el mejor ataque conocido requiere del orden de 2^{128} operaciones para tener éxito, lo cual es computacionalmente inviable con la tecnología actual (y por eso 128 suele ser el nivel deseado). Incrementar el valor de λ aumenta la

dificultad de los ataques, pero también incrementa el tiempo de ejecución de los algoritmos, así como el tamaño de los argumentos y los parámetros públicos.

Como nota al pie, la notación 1^λ utilizada como entrada del algoritmo **Setup** parece insensata, pero es una convención que indica que el parámetro se proporciona en forma unaria. En particular y como es indicado en [59], esto evita que algoritmos que dependen exponencialmente de λ aparezcan artificialmente como eficientes debido a una codificación binaria del parámetro, y que los algoritmos considerados eficientes sean precisamente aquellos cuyo tiempo de ejecución es polinomial en la longitud de sus entradas, preservando así la interpretación estándar.

En la Sección B.3 veremos cómo representar cálculos generales mediante circuitos aritméticos, lo cual permitirá instanciar relaciones $\mathcal{R}$ adecuadas para la construcción de SNARK concretos. Se remite al lector a [61], donde podrá observar que hay múltiples formas de construir una SNARK, muy diferentes a la que desarrollaremos en este capítulo.

B.3. Circuitos Aritméticos

B.3.1. Introducción a los circuitos aritméticos

En la Sección B.2 tratamos a las relaciones $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ de forma abstracta, asumiendo únicamente su pertenencia a la clase NP. Sin embargo, para construir SNARK concretos es necesario describir explícitamente el algoritmo/cálculo (polinomial) que define a $\mathcal{R}$, de modo que su correctitud pueda ser verificada eficientemente sin tener que volver a ejecutar dicho cálculo. En este contexto, resulta fundamental contar con una representación estructurada y algebraica de los algoritmos que se desean verificar.

Una forma natural de modelar un cálculo es mediante un circuito, en el cual cada operación elemental se representa como una puerta y el flujo de datos se describe mediante conexiones entre ellas. En general conocemos a los circuitos booleanos, que son adecuados para describir computación a nivel lógico, y estamos acostumbrados a trabajar con ellos.

Pero también sabemos que en el contexto de la construcción de SNARK, resulta más conveniente trabajar sobre cuerpos finitos y operaciones algebraicas. Esto motiva la introducción del modelo computacional de los circuitos aritméticos [61]. Dichos circuitos permiten representar cálculos como composiciones de sumas y multiplicaciones sobre un cuerpo finito.

Definición B.5 (Circuito aritmético). *Sea $\mathbb{K}$ un cuerpo finito. Un **circuito aritmético** C sobre $\mathbb{K}$ es un grafo dirigido acíclico (DAG) cuyas aristas representan el flujo de valores y cuyos nodos se clasifican en los siguientes tipos:*

- **Nodos de entrada**, que reciben valores en $\mathbb{K}$. Estos nodos representan tanto los public inputs como el witness privado del cálculo.
- **Nodos de constante**, etiquetados con un elemento fijo de $\mathbb{K}$.
- **Puertas aritméticas**, que computan operaciones de suma o multiplicación en $\mathbb{K}$. Cada puerta aritmética tiene grado de entrada 2, grado de output 1, y está etiquetada por una operación en $\{+, \times\}$.

*El valor de un nodo se obtiene evaluando el circuito en orden topológico, comenzando por los nodos de entrada. Uno o más nodos del circuito se designan como **nodos de output**, y su valor define el resultado del cálculo. Entonces, si hay $n \in \mathbb{N}$ nodos de entrada y $m \in \mathbb{N}$ nodos de output, el circuito induce una función $f_C : \mathbb{K}^n \rightarrow \mathbb{K}^m$.*

El **tamaño** $|C|$ de un circuito aritmético se define como el número total de nodos que contiene, mientras que su **profundidad** se define como la longitud máxima de un camino dirigido desde un nodo de entrada hasta un nodo de output.

Como primera aclaración, si bien los *public inputs* y el *witness* forman parte de los nodos de entrada, la diferencia no es ahora en el circuito, sino en la información que $\mathcal{P}$ va a revelar a partir del cómputo. Es seguro que revelará los *public inputs*, e idealmente revelará lo menos posible del *witness*.

Observamos que la corrección de la evaluación de un circuito aritmético C puede expresarse como la verificación de igualdades algebraicas locales: una por cada puerta del circuito. Esta propiedad será central en las dos siguientes secciones, donde veremos cómo transformar circuitos aritméticos en representaciones algebraicas aún más estructuradas, adecuadas para la construcción de argumentos **SNARK** eficientes.

Este modelo de cómputo permite representar funciones polinómicas de varias variables. Para representar funciones booleanas en un circuito aritmético, se puede usar codificación polinómica: AND se traduce en multiplicación $a \wedge b = a \cdot b$, OR en $a \vee b = a + b - a \cdot b$, y NOT en $\neg a = 1 - a$. Estas identidades son válidas únicamente bajo la restricción adicional $a, b \in \{0, 1\} \subseteq \mathbb{K}$, identificando 0 y 1 con los elementos correspondientes del cuerpo. Esto se impone mediante restricciones algebraicas del tipo $a(a - 1) = 0$. Esto garantiza que se pueden construir circuitos aritméticos equivalentes a cualquier función lógica.

Dado que cualquier verificador polinómico puede representarse mediante combinaciones de sumas y productos (por ejemplo, representando condiciones booleanas como polinomios), es posible construir un circuito aritmético que verifique la pertenencia de un *witness* w a una relación $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$, y concluir que los circuitos aritméticos son suficientemente expresivos [58] para capturar relaciones en NP. Formalmente, dada cualquier relación $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ en NP con verificador polinómico

$$f_{\mathcal{R}} : \mathcal{X} \times \mathcal{W} \rightarrow \{0, 1\},$$

existe un circuito aritmético $C_{\mathcal{R}}$ sobre un cuerpo finito $\mathbb{K}$ tal que la evaluación de $C_{\mathcal{R}}$ sobre (x, w) devuelve el valor esperado si y solo si $f_{\mathcal{R}}(x, w)$ acepta el par (x, w) .

B.3.2. Construcción de un ejemplo

Consideramos el cuerpo finito $\mathbb{F}_p$. Supongamos que queremos verificar la siguiente relación (trivial) $\mathcal{R} \subseteq \mathbb{F}_p^4 \times \mathbb{F}_p^3$:

$$\mathcal{R} = \{((x_1, x_2, x_3, y), (w_1, w_2, w_3)) \mid y = (w_1 + x_1) \cdot (w_2 + x_2 + 1) + w_3 \cdot x_3\}.$$

En la relación anterior, interpretamos a x_1, x_2, x_3, y como *inputs* públicos y a w_1, w_2, w_3 como el *witness*. El valor y representa el *output* público esperado del cómputo, mientras que el circuito calcula explícitamente un valor y' a partir de (x, w) . La relación exige entonces que ambos coincidan. Podemos observar un circuito aritmético C para la relación $\mathcal{R}$ en la figura B.2.

Introducimos explícitamente todas las variables del circuito, incluyendo las variables intermedias:

- *Public inputs*: $x_1, x_2, x_3, y \in \mathbb{F}_p$.
- *Witness*: $w_1, w_2, w_3 \in \mathbb{F}_p$.

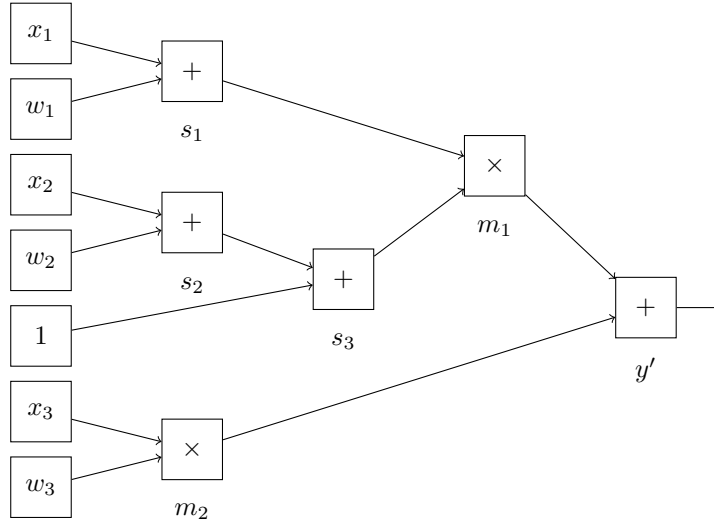


Fig. B.2: Circuito aritmético sobre $\mathbb{F}_p$ para la relación $y = (w_1 + x_1)(w_2 + x_2 + 1) + w_3 x_3$. Los nodos de entrada mostrados son x_1, x_2, x_3 como entradas públicas, w_1, w_2, w_3 como witness y la constante 1; las puertas $+$ y $\times$ calculan valores intermedios s_i, m_i , y el *output* y' debe coincidir con el valor público y .

- Constantes: $1 \in \mathbb{F}_p$.
- Variables intermedias:
 - $s_1 = x_1 + w_1$.
 - $s_2 = x_2 + w_2$.
 - $s_3 = s_2 + 1$.
 - $m_1 = s_1 \cdot s_3$.
 - $m_2 = x_3 \cdot w_3$.
 - $y' = m_1 + m_2$.

Al final, debe chequearse que el *output* calculado coincida con el valor público declarado, es decir, que $y' = y$. En esta primera presentación lo mantenemos separado del dibujo para distinguir pedagógicamente entre el cómputo que produce y' y la condición de aceptación de la relación. Esta condición no debe pensarse necesariamente como una puerta booleana adicional: al pasar a un sistema de restricciones, se incorporará como una igualdad algebraica sobre las variables del sistema. Aquí y' es la evaluación del circuito, mientras que y es el *input* público esperado.

En el ejemplo anterior, el circuito aritmético C que representa la relación $\mathcal{R}$ permite separar de manera clara el cómputo de su verificación. El *prover* evalúa el circuito completo, calculando los valores intermedios asociados a cada puerta, mientras que conceptualmente la corrección del circuito puede descomponerse en verificaciones locales y la consistencia final con el *witness* declarado. Este esquema es consistente con la noción de cómputo verificable introducida previamente: el costo de verificación no depende de ejecutar nuevamente el algoritmo subyacente, sino de validar una prueba estructurada de su correcta ejecución.

Desde la perspectiva de los SNARK discutidos en la sección introductoria sobre argumentos sucintos, el circuito aritmético actúa como la representación explícita de la relación $\mathcal{R}$ de NP que se desea probar. El *witness* (w_1, w_2, w_3) , junto con los valores intermedios

del circuito, conforman la información que $\mathcal{P}$ tiene pero $\mathcal{V}$ no. Por otra parte, los *public inputs* (x_1, x_2, x_3, y) así como la estructura particular de C son conocidos por $\mathcal{V}$.

En particular, la verificación de un circuito aritmético puede expresarse como un conjunto de restricciones algebraicas (constraints), una por cada puerta del circuito. Esta observación es más profunda de lo que podría parecer inicialmente. En lugar de pensar al circuito como un objeto gráfico compuesto por puertas, podemos reinterpretar toda la computación como un sistema algebraico de ecuaciones, donde cada puerta induce una restricción polinómica de grado a lo sumo 2. Cada puerta impone una restricción polinómica de bajo grado sobre las variables del cómputo, y la corrección global del circuito equivale entonces a la satisfacibilidad simultánea de todas dichas restricciones.

Esta reformulación puramente algebraica del cómputo es precisamente la perspectiva adoptada por los sistemas de prueba modernos. En la Sección B.4 introduciremos Rank-1 Constraint Systems (R1CS), que son una representación canónica y particularmente conveniente de circuitos aritméticos para llegar a los SNARK. En este caso, un SNARK va a permitir demostrar, mediante un argumento sucinto (asintóticamente menos que la cantidad de compuertas) y no interactivo, que existe una asignación de valores que satisface todas las igualdades inducidas por el circuito C , sin revelar dicha asignación.

B.3.3. Lo que tenemos hasta ahora

Por ahora, $\mathcal{P}$ podría confeccionar un argumento π a partir del circuito C . La forma inmediata de hacerlo consistiría en enviar todos los valores involucrados en la evaluación del circuito:

$$\pi = (w_1, w_2, w_3, s_1, s_2, s_3, m_1, m_2, y') \in \mathbb{F}_p^9.$$

Dado este argumento, el *verifier* puede comprobar localmente la corrección de cada puerta del circuito y verificar que el valor final calculado coincide con el *input* público declarado. Desde este punto de vista, π actúa como un certificado explícito de que el *prover* conoce una asignación válida que satisface todas las igualdades inducidas por C . Esta idea refleja fielmente el principio de los argumentos de conocimiento: convencer al *verifier* de la existencia de un *witness* consistente con el *input* público.

Desde un punto de vista puramente matemático, este enfoque ya constituye un argumento de correctitud completamente válido. Aunque esta construcción es simple, tiene problemas fundamentales que la hacen inapropiada para un SNARK. En primer lugar, el tamaño del argumento π crece linealmente con el tamaño de C , por lo que no cumple la propiedad de sucintez, y el costo de verificación se vuelve comparable al del cómputo original. En segundo lugar, el esquema no tiene una estructura algebraica que permita verificar en forma global la corrección del circuito en forma eficiente.

Por último, tampoco sería un argumento ZK, ya que no cumple la propiedad de conocimiento cero, pues $\mathcal{P}$ está exponiendo información sobre el *witness*. En un SNARK real, π no incluye los valores intermedios en claro, sino compromisos o pruebas polinómicas [62].

Estos problemas del argumento π motivan la necesidad de transformar la descripción del circuito en una representación algebraica más estructurada. En lugar de manipular puertas individualmente, buscaremos un formalismo que capture simultáneamente todas las restricciones del cómputo en una forma uniforme y algebraicamente conveniente, como se desarrolla en las secciones sobre R1CS y QAP. Así mismo, se observa necesario introducir mecanismos criptográficos que permitan comprometer valores sin revelarlos, como se discute en las secciones sobre esquemas de compromiso y KZG.

B.3.4. Limitaciones de los circuitos aritméticos

Nos estamos comprometiendo a trabajar con un modelo de cómputo específico, que tiene propiedades favorables para implementar construcciones criptográficas basadas en SNARK. Por lo tanto, debemos ser conscientes lo antes posible de qué limitaciones tenemos a la hora de trabajar con él. Cabe notar que la representatividad de los circuitos aritméticos para relaciones en NP no implica eficiencia automática; optimizar el tamaño y la profundidad de los circuitos es un problema no trivial al que los desarrolladores se enfrentan con frecuencia.

Los circuitos aritméticos representan funciones polinómicas sobre un cuerpo $\mathbb{K}$. Pero en general, la mayoría de funciones requieren una codificación adicional para ser expresadas como polinomios en varias variables. Esto puede incrementar el tamaño de los circuitos en forma drástica. Por ejemplo, la mayoría de funciones de hash criptográficas son sumamente costosas de representar, evaluar y verificar en este modelo computacional, ya que están compuestas mayoritariamente de operaciones booleanas costosas de construir a partir de sumas y productos en $\mathbb{K}$. Otro ejemplo son las funciones recursivas.

Operaciones que son baratas en circuitos booleanos (por ejemplo XOR o rotaciones) pueden resultar costosas en circuitos aritméticos, mientras que multiplicaciones (caras en hardware tradicional) son operaciones nativas en este modelo [63].

Además, algunas propiedades de la función original pueden perderse o volverse más complicadas de representar si el campo elegido es demasiado pequeño o no admite ciertas operaciones de manera directa. En general, $\mathbb{F}_p$ es el cuerpo de nuestro circuito aritmético. Así que, por ejemplo, tenemos que considerar todo el tiempo la aritmética modular a la hora de hacer operaciones de suma y producto en $\mathbb{F}_p$.

B.4. Sistemas de Restricciones R1CS (Rank-1 Constraint Systems)

B.4.1. Introducción a R1CS

En la Sección B.3, el argumento π asociado a un circuito aritmético C consiste en una asignación explícita de valores a todas las variables del cómputo: el witness y cada valor intermedio producido por las puertas del circuito. En el ejemplo de la Subsección B.3.2, esto implicaba transmitir todos los valores. Si bien este enfoque permite al verificador comprobar localmente la corrección de cada operación, su tamaño crece linealmente con el número de puertas del circuito, por lo que no cumple la propiedad de sucintez. Más aún, como también se transmite el witness, no cumple la propiedad de conocimiento cero.

Para evitar esto, resulta necesario abstraer el cómputo de una manera estructurada. Observamos que la correctitud del circuito no depende de los valores individuales en sí mismos, sino del hecho de que estos satisfacen un conjunto de igualdades algebraicas bien definidas. En el ejemplo considerado, la sección anterior separó pedagógicamente el cálculo de y' del chequeo $y' = y$. Al pasar a R1CS, esa condición de aceptación se absorbe dentro del propio sistema de restricciones, de modo que ya no trabajaremos con y' como variable separada, sino directamente con la igualdad $y = m_1 + m_2$. En consecuencia, la ejecución correcta queda expresada como la existencia de valores $w_1, w_2, w_3, s_1, s_2, s_3, m_1, m_2 \in \mathbb{F}_p$ tales que se cumplen simultáneamente las ecuaciones

- $s_1 = x_1 + w_1$.
- $s_2 = x_2 + w_2$.
- $s_3 = s_2 + 1$.

- $m_1 = s_1 \cdot s_3$.
- $m_2 = x_3 \cdot w_3$.
- $y = m_1 + m_2$.

Más generalmente, cada puerta del circuito induce una relación algebraica entre un número *reducido de variables*, y la validez del cómputo completo se reduce a satisfacer todas estas ecuaciones a la vez. Esta observación permite reformular el problema de verificación de un circuito como un problema puramente algebraico: demostrar la existencia de una asignación a un conjunto de variables que satisface un sistema de ecuaciones estructuradas sobre un cuerpo finito. Desde esta perspectiva, el prover ya no necesita revelar la asignación completa, sino únicamente convencer al verifier de que dicha asignación existe. Durante todo este capítulo siempre debemos recordar el paso conceptual fundamental en la construcción de SNARK: transformar cómputo en álgebra [64, 62]. Esta sección y la siguiente se dedican a introducir representaciones algebraicas. Ahora definiremos una de ellas.

Definición B.6 (R1CS). *Sea $\mathbb{K}$ un cuerpo finito. Un sistema de restricciones de rango uno (Rank-1 Constraint System, R1CS) sobre $\mathbb{K}$ es una colección de $m \in \mathbb{N}$ restricciones de la forma*

$$\langle \mathbf{a}_i, \mathbf{z} \rangle \cdot \langle \mathbf{b}_i, \mathbf{z} \rangle = \langle \mathbf{c}_i, \mathbf{z} \rangle, \quad 1 \leq i \leq m,$$

donde los $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \mathbb{K}^n$ son vectores fijos, y $\mathbf{z} \in \mathbb{K}^n$ es un vector de variables.

En este contexto, $\langle \cdot, \cdot \rangle : \mathbb{K}^n \times \mathbb{K}^n \rightarrow \mathbb{K}$ denota el producto interno estándar en $\mathbb{K}^n$. El vector $\mathbf{z}$ se denomina vector de asignación del sistema, e incluye una coordenada constante igual a 1, los public inputs, las variables correspondientes al witness del problema original y las variables auxiliares que representan valores intermedios del cómputo.

Intuitivamente, un R1CS no describe cómo computar una función, sino qué igualdades deben satisfacerse para que una computación sea válida.

El término rango uno proviene del hecho de que cada restricción involucra el producto de dos formas lineales, en lugar de polinomios arbitrarios.

Los R1CS poseen un alto poder expresivo, ya que permiten capturar la semántica de cualquier cómputo expresable mediante algún circuito aritmético sobre un cuerpo finito $\mathbb{K}$. Aunque cada restricción individual impone únicamente una igualdad bilineal entre combinaciones lineales de variables, la conjunción de muchas de estas restricciones permite describir de manera completa y estructurada la correcta ejecución de un algoritmo. En este marco, el R1CS no especifica cómo computar una función, sino cómo verificar que una ejecución dada es consistente con el cómputo esperado: cada restricción actúa como un chequeo local de una operación elemental, y satisfacer todas a la vez garantiza que el conjunto de valores proporcionado por el prover corresponde a una evaluación válida del circuito considerado. Esto puede ser expresado como un teorema.

Teorema B.7. *Existe una transformación eficiente (en tiempo polinomial) que, dado un circuito aritmético C sobre un cuerpo finito $\mathbb{K}$ junto con una codificación explícita de sus variables de entrada, constantes y valores intermedios, produce un R1CS sobre $\mathbb{K}$ que preserva la semántica del cómputo. En particular, las evaluaciones correctas de C inducen vectores de asignación que satisfacen el R1CS, y recíprocamente, todo vector de asignación que satisface el R1CS corresponde a una ejecución consistente con esa codificación del circuito.*

Un R1CS puede representarse de forma compacta mediante tres matrices $A, B, C \in \mathbb{K}^{m \times n}$, donde cada fila codifica una restricción del sistema. Dado un vector de asignación $\mathbf{z} \in \mathbb{K}^n$, el R1CS queda expresado como

$$(A\mathbf{z}) \circ (B\mathbf{z}) = C\mathbf{z},$$

donde $\circ$ denota el producto de Hadamard (producto componente a componente). En esta representación, la i -ésima fila de las matrices A, B, C corresponde a los vectores $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i$ de la restricción original, y la ecuación resultante impone que la i -ésima restricción de rango uno se satisfaga. Esta notación matricial no introduce nueva semántica, sino que organiza el conjunto de restricciones de forma algebraicamente conveniente.

B.4.2. Construcción de un ejemplo

Vamos a continuar el ejemplo iniciado en la Subsección B.3.2: un circuito aritmético para la relación $\mathcal{R} \subseteq \mathbb{F}_p^4 \times \mathbb{F}_p^3$ dada por:

$$\mathcal{R} = \{((x_1, x_2, x_3, y), (w_1, w_2, w_3)) \mid y = (w_1 + x_1) \cdot (w_2 + x_2 + 1) + w_3 \cdot x_3\}$$

Ya tenemos la descomposición del cómputo de la relación en operaciones elementales con la introducción de variables intermedias. En este caso, el vector de asignación $\mathbf{z}$ estará dado por:

$$\mathbf{z} = (1, x_1, x_2, x_3, w_1, w_2, w_3, s_1, s_2, s_3, m_1, m_2, y) \in \mathbb{F}_p^{13}.$$

Observamos que el vector $\mathbf{z}$ contiene una coordenada constante, los public inputs, el witness, y las variables intermedias del circuito. En comparación con la sección anterior, la simplificación importante es que ya no incluimos una variable separada y' para el *output* computado: el chequeo externo $y' = y$ fue absorbido como la última restricción del sistema, escrita directamente como $y = m_1 + m_2$. Hemos dicho que no queremos revelar tampoco las variables intermedias. Por lo tanto, el R1CS terminará siendo para la relación (determinísticamente derivable desde $\mathcal{R}$) $\mathcal{R}^* \subseteq \mathbb{F}_p^4 \times \mathbb{F}_p^8$, donde la relación extendida incorpora en el vector de asignación las variables auxiliares s_1, s_2, s_3, m_1, m_2 . En este ejemplo continuaremos así por ser más intuitivo pedagógicamente, pero se suele estandarizar que las variables intermedias se llamen luego w_4, w_5, w_6 , etc.

Vamos a ver cómo representar cada una de las ecuaciones del circuito anterior utilizando restricciones de rango uno (para familiarizarnos con estos conceptos, resulta útil verificar esto a mano). Obtendremos un R1CS. La transformación es completamente sistemática: expresar ambos lados como combinaciones lineales, y forzar la igualdad mediante un producto con el vector canónico adecuado para $\mathbf{b}_i$.

- La restricción $s_1 = x_1 + w_1$ se reescribe como $\langle \mathbf{a}_1, \mathbf{z} \rangle \cdot \langle \mathbf{b}_1, \mathbf{z} \rangle = \langle \mathbf{c}_1, \mathbf{z} \rangle$ para:

$$\begin{aligned} \mathbf{a}_1 &= (0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{b}_1 &= (1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{c}_1 &= (0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0) \end{aligned}$$

- La restricción $s_2 = x_2 + w_2$ se reescribe como $\langle \mathbf{a}_2, \mathbf{z} \rangle \cdot \langle \mathbf{b}_2, \mathbf{z} \rangle = \langle \mathbf{c}_2, \mathbf{z} \rangle$ para:

$$\begin{aligned} \mathbf{a}_2 &= (0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{b}_2 &= (1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{c}_2 &= (0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0) \end{aligned}$$

- La restricción $s_3 = s_2 + 1$ se reescribe como $\langle \mathbf{a}_3, \mathbf{z} \rangle \cdot \langle \mathbf{b}_3, \mathbf{z} \rangle = \langle \mathbf{c}_3, \mathbf{z} \rangle$ para:

$$\begin{aligned}\mathbf{a}_3 &= (1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0), \\ \mathbf{b}_3 &= (1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{c}_3 &= (0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0)\end{aligned}$$

- La restricción $m_1 = s_1 \cdot s_3$ se reescribe como $\langle \mathbf{a}_4, \mathbf{z} \rangle \cdot \langle \mathbf{b}_4, \mathbf{z} \rangle = \langle \mathbf{c}_4, \mathbf{z} \rangle$ para:

$$\begin{aligned}\mathbf{a}_4 &= (0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0), \\ \mathbf{b}_4 &= (0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0), \\ \mathbf{c}_4 &= (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0)\end{aligned}$$

- La restricción $m_2 = x_3 \cdot w_3$ se reescribe como $\langle \mathbf{a}_5, \mathbf{z} \rangle \cdot \langle \mathbf{b}_5, \mathbf{z} \rangle = \langle \mathbf{c}_5, \mathbf{z} \rangle$ para:

$$\begin{aligned}\mathbf{a}_5 &= (0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{b}_5 &= (0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0), \\ \mathbf{c}_5 &= (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0)\end{aligned}$$

- La restricción $y = m_1 + m_2$ se reescribe como $\langle \mathbf{a}_6, \mathbf{z} \rangle \cdot \langle \mathbf{b}_6, \mathbf{z} \rangle = \langle \mathbf{c}_6, \mathbf{z} \rangle$ para:

$$\begin{aligned}\mathbf{a}_6 &= (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0), \\ \mathbf{b}_6 &= (1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), \\ \mathbf{c}_6 &= (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)\end{aligned}$$

Para pasar el R1CS anterior a forma matricial, agrupamos los vectores $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \mathbb{F}_p^{13}$ como filas de tres matrices $A, B, C \in \mathbb{F}_p^{6 \times 13}$, de modo que el sistema completo se escriba como $(A\mathbf{z}) \circ (B\mathbf{z}) = C\mathbf{z}$. Para compactar la notación, utilizaremos los vectores $e_1, e_2, \dots, e_{13}$ de la base canónica de $\mathbb{F}_p^{13}$; cuando aparezcan como filas de las matrices, escribiremos sus transpuestas $e_i^\top$.

$$A = \begin{pmatrix} e_2^\top + e_5^\top \\ e_3^\top + e_6^\top \\ e_1^\top + e_9^\top \\ e_8^\top \\ e_4^\top \\ e_{11}^\top + e_{12}^\top \end{pmatrix}, \quad B = \begin{pmatrix} e_1^\top \\ e_1^\top \\ e_1^\top \\ e_{10}^\top \\ e_7^\top \\ e_1^\top \end{pmatrix}, \quad C = \begin{pmatrix} e_8^\top \\ e_9^\top \\ e_{10}^\top \\ e_{11}^\top \\ e_{12}^\top \\ e_{13}^\top \end{pmatrix}.$$

B.4.3. Lo que tenemos hasta ahora

A pesar de haber transformado el cómputo original en un sistema de restricciones algebraicas compacto y estructurado, el problema fundamental de qué argumento π publicar persiste. En efecto, una prueba ingenua de que existe un vector de asignación $\mathbf{z}$ que satisface el R1CS consistiría nuevamente en revelar dicho vector completo, lo cual expone el witness y todos los valores intermedios del cómputo. Si bien la forma matricial organiza las restricciones de manera conveniente, verificar explícitamente su validez sigue requiriendo acceso directo a $\mathbf{z}$, violando tanto la propiedad de sucintez como la de conocimiento cero que se esperan de un SNARK (o zk-SNARK).

La dificultad central puede formularse de la siguiente manera: necesitamos convencer al verificador de que existe un vector $\mathbf{z}$ que satisface $(A\mathbf{z}) \circ (B\mathbf{z}) = C\mathbf{z}$ sin revelar dicho vector, y sin que el costo de verificación dependa del tamaño del R1CS. Resolver este problema requiere herramientas adicionales que permitan comprimir globalmente las restricciones de forma eficiente. En particular, buscamos una representación en la cual todas las restricciones puedan verificarse simultáneamente mediante una única condición algebraica.

Este objetivo se alcanza reformulando el R1CS como una identidad polinómica. En lugar de verificar ecuaciones individuales, podemos construir polinomios cuidadosamente diseñados cuya correcta relación encapsule simultáneamente todas las restricciones del R1CS. De esta manera, el problema combinatorio de verificar muchas ecuaciones se transforma en un problema algebraico de verificar una única igualdad entre polinomios.

Las estructuras que emergen de esta transformación se denominan Quadratic Arithmetic Programs (QAP). Los QAP preservan exactamente la semántica del R1CS, pero la expresan en un lenguaje polinómico particularmente adecuado para la construcción de SNARK basados en compromisos algebraicos y pairings bilineales. La Sección B.5 introduce formalmente esta representación.

B.5. QAP: Quadratic Arithmetic Programs

B.5.1. Equivalencia entre R1CS y QAP

Un QAP puede entenderse como un mecanismo algebraico que codifica simultáneamente todas las restricciones de un R1CS de la Definición B.6 en una única relación polinómica global. La idea fundamental es que los polinomios recuerden cada restricción. Usaremos aquí la interpolación polinómica de la Subsección A.3.3: dados valores sobre un conjunto de puntos distintos, existe un único polinomio de grado acotado que interpola esos valores. En un QAP, esta herramienta permite reemplazar las columnas de las matrices de un R1CS por polinomios. Daremos la definición formal ahora mismo.

Definición B.8 (QAP, Polinomio objetivo). *Sea $\mathbb{K}$ un cuerpo finito, y sean $m, n \in \mathbb{N}$. Un Quadratic Arithmetic Program (QAP) sobre $\mathbb{K}$ de tamaño m y dimensión n está dado por:*

- Tres familias de n polinomios en $\mathbb{K}[x]$:

$$\mathcal{A} = \{A_i(x)\}_{i=1}^n, \quad \mathcal{B} = \{B_i(x)\}_{i=1}^n, \quad \mathcal{C} = \{C_i(x)\}_{i=1}^n,$$

cada uno de grado a lo sumo $m - 1$.

- Un polinomio objetivo $Z \in \mathbb{K}[x]$ (también llamado vanishing polynomial) dado por

$$Z(x) = \prod_{j=1}^m (x - r_j),$$

donde $r_1, \dots, r_m \in \mathbb{K}$ son distintos dos a dos.

Notaremos en forma sintética a un QAP como una 4-upla $(\mathcal{A}, \mathcal{B}, \mathcal{C}, Z)$.

Definición B.9 (Satisfacción de un QAP). *Sean $\mathbb{K}$ un cuerpo finito, $(\mathcal{A}, \mathcal{B}, \mathcal{C}, Z)$ un QAP sobre $\mathbb{K}$ de tamaño m y dimensión n , sea $\mathbf{z} \in \mathbb{K}^n$ un vector de asignación. Decimos que $\mathbf{z}$ satisface el QAP si existe un polinomio $H \in \mathbb{K}[x]$ tal que*

$$\left(\sum_{i=1}^n \mathbf{z}_i A_i \right) \cdot \left(\sum_{i=1}^n \mathbf{z}_i B_i \right) - \left(\sum_{i=1}^n \mathbf{z}_i C_i \right) = HZ.$$

Vamos a demostrar que hay una correspondencia entre los R1CS y los QAP. Es decir, son dos vistas algebraicas diferentes del mismo objeto. Para ello, primero generaremos intuición a partir de la comparación de los parámetros que definen en cada uno.

- Ambos se definen sobre un cuerpo finito $\mathbb{K}$.
- n es el número de variables del sistema que hay en un R1CS (longitud de los vectores de asignación incluyendo la constante 1), mientras que es el tamaño en un QAP.
- m es el número de restricciones que hay en un R1CS, mientras que en un QAP controla el grado máximo de los polinomios $A_1, \dots, A_n, B_1, \dots, B_n, C_1, \dots, C_n$.
- En un R1CS, la información está condensada en las matrices A, B, C . En un QAP, se busca que los polinomios $A_1, \dots, A_n, B_1, \dots, B_n, C_1, \dots, C_n$ interpolen las columnas de A, B, C . Nos referiremos a estos $3n$ polinomios como los polinomios interpoladores del QAP.
- En un QAP, tomamos m elementos diferentes de $\mathbb{K}$, uno por cada restricción. En particular, construimos el polinomio objetivo Z de modo que se anule en exactamente estos m puntos.

Dados m puntos distintos $r_1, \dots, r_m \in \mathbb{K}$ y valores $y_1, \dots, y_m \in \mathbb{K}$, existe exactamente un polinomio de grado a lo sumo $m-1$ que los interpola. En particular, existe exactamente un polinomio interpolador de grado a lo sumo $m-1$ para el conjunto de elementos de $\mathbb{K}$ tomados de cada una de las columnas de las matrices $A, B, C \in \mathbb{K}^{m \times n}$ de un R1CS asignados uno a uno con $\{r_1, \dots, r_m\}$, y más aún, tenemos su construcción explícita. Por ejemplo, para la primera columna de A , podemos definir exactamente un polinomio A_1 de grado a lo sumo $m-1$ que cumpla:

$$A_1(r_1) = (A)_{1,1}, A_1(r_2) = (A)_{2,1}, \dots, A_1(r_m) = (A)_{m,1}.$$

Intuitivamente, observar que cada columna de las matrices puede verse como una función definida en los m puntos $r_1, \dots, r_m$. El QAP simplemente reemplaza esa tabla de valores por su polinomio interpolador.

El polinomio resultante A_1 pasará por el conjunto de puntos $\{(r_1, (A)_{1,1}), \dots, (r_m, (A)_{m,1})\} \subset \mathbb{K} \times \mathbb{K}$. Por lo tanto, podemos definir $3n$ polinomios en $\mathbb{K}[x]$ mediante interpolación, de modo que los primeros n correspondan a las columnas de A , los siguientes n correspondan a las columnas de B , y los últimos n correspondan a las columnas de C :

$$A_i(r_j) = (A)_{j,i}, \quad B_i(r_j) = (B)_{j,i}, \quad C_i(r_j) = (C)_{j,i}, \quad \forall 1 \leq i \leq n, \forall 1 \leq j \leq m.$$

Luego, observamos que evaluando la identidad polinómica que define al QAP en cada uno de $r_1, \dots, r_m$, el lado derecho se hace cero (pues son las m raíces del polinomio objetivo Z). En cada caso, la identidad se reduce a una igualdad puramente algebraica determinada por la j -ésima restricción del R1CS. Este es el paso crucial.

$$\begin{aligned} \left(\sum_{i=1}^n \mathbf{z}_i A_i(r_j) \right) \cdot \left(\sum_{i=1}^n \mathbf{z}_i B_i(r_j) \right) &= \sum_{i=1}^n \mathbf{z}_i C_i(r_j) \\ \left(\sum_{i=1}^n \mathbf{z}_i (A)_{j,i} \right) \cdot \left(\sum_{i=1}^n \mathbf{z}_i (B)_{j,i} \right) &= \sum_{i=1}^n \mathbf{z}_i (C)_{j,i} \\ \langle \mathbf{a}_j, \mathbf{z} \rangle \cdot \langle \mathbf{b}_j, \mathbf{z} \rangle &= \langle \mathbf{c}_j, \mathbf{z} \rangle. \end{aligned}$$

Es decir que la identidad polinómica que define al QAP se anula en r_j si $\mathbf{z}$ satisface la j -ésima restricción del R1CS. Por lo tanto, que se anule en $r_1, \dots, r_m$ indica que $\mathbf{z}$ satisface todas las restricciones del R1CS. Concluimos entonces que si $\mathbf{z}$ satisface el R1CS, entonces satisface el QAP asociado. Tenemos demostrada entonces la primera dirección del siguiente teorema (la segunda queda como ejercicio para el lector, y no es una transformación que nos sirva hacer en este trabajo):

Teorema B.10 (Equivalencia entre R1CS-QAP). *Sea $\mathbb{K}$ un cuerpo finito. Para todo R1CS sobre $\mathbb{K}$, definido por matrices $A, B, C \in \mathbb{K}^{m \times n}$, existe un QAP canónicamente asociado (A, B, C, Z) sobre $\mathbb{K}$ de tamaño m y dimensión n tal que, para todo vector de asignación $\mathbf{z}$,*

$$(A\mathbf{z}) \circ (B\mathbf{z}) = C\mathbf{z} \iff \exists H \in \mathbb{K}[x] \text{ tal que}$$

$$\left(\sum_{i=1}^n \mathbf{z}_i A_i \right) \cdot \left(\sum_{i=1}^n \mathbf{z}_i B_i \right) - \left(\sum_{i=1}^n \mathbf{z}_i C_i \right) = HZ,$$

donde la igualdad del lado derecho debe entenderse como una identidad entre polinomios de $\mathbb{K}[x]$.

Recíprocamente, todo QAP induce un R1CS equivalente, en el sentido de que ambos definen la misma relación algebraica sobre los vectores de asignación.

B.5.2. Construcción de un ejemplo

Para continuar con nuestro ejemplo, con $m = 6$ y $n = 13$, debemos interpolar los 39 polinomios de grado menor o igual a 5 dados por cada columna de $A, B, C \in \mathbb{F}_p^{6 \times 13}$. Para ello, primero debemos elegir un conjunto de 6 puntos diferentes de $\mathbb{F}_p$, que van a definir el polinomio objetivo. Sin perder generalidad, pueden ser $r_i = i$ para $1 \leq i \leq 6$ (suponiendo, desde luego, que $|\mathbb{F}_p| \geq 6$). Entonces:

$$Z(x) = (x-1)(x-2)(x-3)(x-4)(x-5)(x-6) = \prod_{i=1}^6 (x-i).$$

Hacer todas las interpolaciones paso por paso a mano sería sumamente engorroso. Por lo tanto, nos limitaremos a mostrar cómo se haría para construir el polinomio A_1 . Observamos que la primera columna de A es el vector $(0, 0, 1, 0, 0, 0)$. Es decir, A_1 es el único polinomio de grado menor o igual a 5 tal que $A_1(1) = A_1(2) = A_1(4) = A_1(5) = A_1(6) = 0$ y $A_1(3) = 1$, usando la base de Lagrange.

$$A_1(x) = \frac{(x-1)(x-2)(x-4)(x-5)(x-6)}{(3-1)(3-2)(3-4)(3-5)(3-6)}.$$

B.5.3. Lo que tenemos hasta ahora

Reformulamos un circuito aritmético como un R1CS, y luego como un QAP. El problema se transformó en una única condición de divisibilidad polinómica. Basta verificar que cierta identidad polinómica se anula en cierto conjunto de puntos, o equivalentemente, que es múltiplo del polinomio objetivo $Z \in \mathbb{K}[x]$.

Esta transformación es precisamente lo que habilita la construcción de pruebas sucintas. Recordemos el Lema de Schwartz-Zippel ya introducido. Esa herramienta permitía verificar la igualdad de dos polinomios en forma probabilística utilizando solamente un

punto elegido aleatoriamente para evaluarlos, sin necesidad de manipular y comparar sus coeficientes.

Del mismo modo, verificar la divisibilidad de polinomios puede realizarse de manera probabilística mediante evaluaciones en puntos aleatorios. Las Secciones B.6 y B.7 muestran cómo hacerlo, abriendo el camino hacia mecanismos de verificación más compactos que la comprobación explícita de cada restricción por separado. En el camino, también se introduce la propiedad de conocimiento cero, ya que la representación polinómica permite agregar criptografía para comprometer evaluaciones sin revelar los valores.

B.6. Esquemas de Compromiso (Commitment Schemes)

B.6.1. Motivación y definición formal

Muchos protocolos criptográficos tienen el mismo patrón estructural: una parte desea fijar un valor sin revelarlo inmediatamente, conservando la posibilidad de demostrar posteriormente que dicho valor ya estaba determinado desde el inicio [39, 40].

Los esquemas de compromiso formalizan precisamente este mecanismo criptográfico. Intuitivamente, un compromiso actúa como un “sobre sellado”: permite ocultar información en una primera fase, garantizando simultáneamente que dicha información no puede modificarse más adelante. Las dos palabras fundamentales son *ocultamiento* e *inmutabilidad*. En la segunda fase, el emisor revela dicho valor junto con información auxiliar para poder verificar la consistencia del compromiso.

Más formalmente, un esquema de compromiso define un protocolo de dos fases. En la fase de *commit*, el emisor fija un valor secreto. En la fase de *open* (o *reveal*), el emisor revela información auxiliar que permite verificar la consistencia de la apertura con el compromiso emitido. Un ejemplo fundamental de esquema de compromiso es el de Pedersen [65].

Definición B.11 (Esquema de Compromiso). *Sean $\mathcal{M}$ un espacio de mensajes, $\mathcal{R}$ un espacio de aleatoriedad, y $\mathcal{C}$ un espacio de compromisos. Adoptaremos la convención simplificada de describir un esquema de compromiso mediante un algoritmo de compromiso y un procedimiento de verificación de apertura. Un esquema de compromiso (o commitment scheme) es entonces un par de algoritmos probabilísticos en tiempo polinomial:*

$$\begin{aligned} \text{Commit} &: \mathcal{M} \times \mathcal{R} \rightarrow \mathcal{C}, \\ \text{VerifyOpen} &: \mathcal{C} \times \mathcal{M} \times \mathcal{R} \rightarrow \{0, 1\}, \end{aligned}$$

tales que para todo mensaje $m \in \mathcal{M}$ y valores aleatorios $r \in \mathcal{R}$, si $c = \text{Commit}(m, r)$, entonces $\text{VerifyOpen}(c, m, r) = 1$.

Además, un esquema de compromiso debe satisfacer las siguientes propiedades:

- *Hiding*: el compromiso no revela información computacionalmente distinguible sobre el mensaje comprometido.
- *Binding*: es computacionalmente inviable encontrar un compromiso $c \in \mathcal{C}$ y dos aperturas válidas (m, r) y (m', r') con $m \neq m'$ tales que $\text{VerifyOpen}(c, m, r) = \text{VerifyOpen}(c, m', r') = 1$.

Intuitivamente, abrir un compromiso consiste en revelar el mensaje junto con la aleatoriedad utilizada durante la fase de compromiso, permitiendo su verificación pública. Notamos que los espacios $\mathcal{M}$, $\mathcal{C}$ y $\mathcal{R}$ pueden ser conjuntos arbitrarios, de modo que este esquema es instanciable para diferentes estructuras algebraicas.

Un detalle sumamente importante es que la aleatoriedad $r \in \mathcal{R}$ usada en ambos algoritmos sea la misma. El procedimiento **VerifyOpen** no introduce nueva aleatoriedad $r' \neq r$, ya que si permitiera hacerlo, se podría abrir el mismo c como infinitas cosas distintas.

Notar que en esta definición general **VerifyOpen** cumple el rol de procedimiento de verificación de apertura. En muchas referencias, se separa además un algoritmo de apertura que produce evidencia explícita y otro de verificación, que suele escribirse como **Open** y **Verify** respectivamente. Dependiendo del caso, los vamos a separar para hacer que la idea fundamental sea más explícita: en las próximas definiciones, **Open** generará una prueba o argumento π en un espacio de pruebas/argumentos Π , que será recibida por **Verify**.

Las propiedades de *hiding* y *binding* capturan exactamente la intuición del sobre sellado: *hiding* indica que nadie puede ver el contenido del sobre, mientras que *binding* indica que el contenido del sobre no puede ser modificado luego de sellarse.

Es importante remarcar que un compromiso no es un cifrado criptográfico: un cifrado protege confidencialidad frente a terceros, pero el emisor puede modificar el mensaje antes de enviarlo. Un compromiso, en cambio, protege consistencia temporal: fija criptográficamente un valor [40].

Aunque la definición formal anterior contempla un procedimiento de verificación de apertura **VerifyOpen**, en muchas aplicaciones criptográficas modernas el mensaje completo no se revela, sino únicamente información derivada verificable. En la práctica (en particular, en la construcción de los SNARK) no se revelará el mensaje completo, ya que queremos tener propiedades de sucintez y conocimiento cero.

Los esquemas de compromiso son un mecanismo criptográfico que permite transformar objetos matemáticos en artefactos verificables públicamente. En las siguientes subsecciones veremos instancias concretas de esta abstracción. Primero, observaremos que los Merkle Trees pueden interpretarse naturalmente como compromisos vectoriales. Luego, extendemos esta idea a los polinomios.

B.6.2. Compromisos Vectoriales

Los esquemas de compromiso de la Subsección B.6.1 permiten fijar un único mensaje. Sin embargo, numerosos protocolos criptográficos requieren comprometer colecciones estructuradas de datos. En particular, resulta frecuente la necesidad de comprometer un vector $\mathbf{v} \in \mathbb{K}^n$ (donde $\mathbb{K}$ es un cuerpo finito), de modo tal que posteriormente pueda demostrarse la consistencia de componentes individuales sin revelar el vector completo.

Esta generalización conduce naturalmente a la noción de compromisos vectoriales [53]. Intuitivamente, un compromiso vectorial permite:

- Fijar criptográficamente un conjunto de valores.
- Abrir posiciones individuales del vector.
- Verificar dichas aperturas de manera eficiente.

La idea de esta subsección es observar que un esquema de compromiso vectorial es exactamente lo que podíamos hacer con los Merkle Trees, y que por lo tanto no son algo muy diferente a lo que venimos trabajando. Ahora vamos con la definición formal.

Definición B.12 (VCS). Sean $\mathcal{M}$ un espacio de mensajes, $\mathcal{C}$ un espacio de compromisos, Π un espacio de pruebas, y $n \in \mathbb{N}$. Un esquema de compromiso vectorial (vector commit-

ment scheme, VCS) es una tripla de algoritmos probabilísticos en tiempo polinomial:

$$\begin{aligned}\text{Commit} &: \mathcal{M}^n \rightarrow \mathcal{C}, \\ \text{Open} &: \mathcal{M}^n \times [n] \rightarrow \mathcal{M} \times \Pi, \\ \text{Verify} &: \mathcal{C} \times [n] \times \mathcal{M} \times \Pi \rightarrow \{0, 1\},\end{aligned}$$

tales que para todo vector $\mathbf{v} \in \mathcal{M}^n$, si $c := \text{Commit}(\mathbf{v})$ y $(\mathbf{v}_i, \pi) := \text{Open}(\mathbf{v}, i)$, entonces $\text{Verify}(c, i, \mathbf{v}_i, \pi) = 1$.

Para simplificar la exposición, omitimos explícitamente el espacio de aleatoriedad $\mathcal{R}$. Esta omisión no afecta la construcción que utilizaremos (Merkle Trees), la cual es determinística, y la seguridad no se basa en la aleatoriedad, sino en la resistencia a colisiones de una función de hash criptográfica $\mathcal{H}$ (en el ROM) [40].

Por otra parte, decidimos incluir el espacio de pruebas Π , así como separar los algoritmos Open y Verify , para poder ver con más detalle la relación entre ambos y las primitivas ya introducidas.

Podemos definir un VCS a partir de los Merkle Trees de la siguiente forma: dado un vector $\mathbf{v} \in \mathbb{K}^n$, el compromiso se define como $c := r_{\mathcal{T}}$, donde $\mathcal{T}$ es el Merkle Tree cuyas hojas son $h_i = \mathcal{H}(v_i)$, y la apertura de la i -ésima posición se corresponde exactamente con una Merkle Proof. Bajo esta interpretación:

- $\text{Commit}(\mathbf{v}) := r_{\mathcal{T}}$.
- $\text{Open}(\mathbf{v}, i)$ devuelve el par $(\mathbf{v}_i, \pi)$, donde π es la Merkle Proof asociada a la hoja ℓ_i .
- $\text{Verify}(c, i, \mathbf{v}_i, \pi)$ se corresponde con verificar la Merkle Proof (con el algoritmo Verify del Merkle Tree ya introducido).

La propiedad de binding se deduce de la resistencia a colisiones de $\mathcal{H}$: no es computacionalmente posible abrir una posición del vector de dos maneras distintas y llegar a la misma raíz. Por otra parte, los Merkle Trees no garantizan *hiding* en el sentido tradicional de los esquemas de compromiso, ya que el compromiso es determinístico. En esta tesis los utilizaremos por sus propiedades de autenticidad e inclusión, no como mecanismos de ocultamiento criptográfico general.

Además, el compromiso $r_{\mathcal{T}}$ tiene tamaño constante $\mathcal{O}(1)$, las Merkle Proofs π_i tienen tamaño logarítmico: $\mathcal{O}(\log n)$, y la verificación requiere de $\mathcal{O}(\log n)$ evaluaciones de $\mathcal{H}$. Esto refleja la propiedad de sucintez que tanto deseamos en el esquema.

B.6.3. Compromisos Polinomiales

En numerosos protocolos criptográficos (y particularmente en sistemas SNARK) los objetos de interés no son vectores arbitrarios, sino polinomios sobre cuerpos finitos de la forma $P \in \mathbb{K}[x]$. Si bien un polinomio puede verse como un vector de coeficientes en $\mathbb{K}$, y por lo tanto un compromiso sobre un polinomio puede ser reducido a un compromiso vectorial, los polinomios tienen estructura algebraica adicional, que permite construir protocolos criptográficos más eficientes.

Intuitivamente, un compromiso polinomial permite fijar criptográficamente un polinomio $P \in \mathbb{K}[x]$ sin revelarlo, conservando la posibilidad de demostrar posteriormente evaluaciones del mismo. Es decir, permite demostrar afirmaciones del tipo

$$P(z) = y, \quad y, z \in \mathbb{K}$$

sin necesidad de revelar completamente P . Esta noción es el *argumento de evaluación*. Una prueba de evaluación es un argumento criptográfico que permite convencer al verificador de que una evaluación reclamada $y = P(z)$ es consistente con un compromiso c , sin revelar P . Daremos ahora la definición formal de los compromisos polinomiales.

Definición B.13 (PCS). Sean $\mathbb{K}$ un cuerpo finito, $d \in \mathbb{N}$, $\mathcal{P} \subseteq \mathbb{K}[x]_{\leq d}$ un espacio de polinomios sobre $\mathbb{K}$, $\mathcal{C}$ un espacio de compromisos, y Π un espacio de argumentos. Un esquema de compromiso polinomial (*polynomial commitment scheme, PCS*) es una tripla de algoritmos probabilísticos en tiempo polinomial:

$$\begin{aligned} \text{Commit} &: \mathcal{P} \rightarrow \mathcal{C}, \\ \text{Open} &: \mathcal{P} \times \mathbb{K} \rightarrow \mathbb{K} \times \Pi, \\ \text{Verify} &: \mathcal{C} \times \mathbb{K} \times \mathbb{K} \times \Pi \rightarrow \{0, 1\}, \end{aligned}$$

tales que para todo polinomio $P \in \mathcal{P}$, si $c := \text{Commit}(P)$, entonces $\text{Open}(P, z)$ produce un par (y, π) con $y = P(z)$, y $\text{Verify}(c, z, y, \pi) = 1$.

B.6.4. De compromisos polinomiales a identidades de QAP

Formalmente, el concepto de un argumento de evaluación es el objeto $\pi \in \Pi$ generado por el algoritmo **Open**. En esta subsección explicamos por qué esta noción, suficiente para afirmar evaluaciones individuales, todavía necesita estructura adicional cuando se la aplica a las identidades polinómicas que aparecen en un QAP.

Ahora, vamos a conectar estas ideas con las primitivas criptográficas ya introducidas. Desde una perspectiva puramente algebraica, $P(z) = y$ equivale a que el polinomio $Q(x) := P(x) - y$ tiene a z como raíz, es decir que existe un polinomio $H \in \mathbb{K}[x]$ tal que $Q(x) = (x - z)H(x)$. El problema se reduce entonces a argumentar esta afirmación.

El Lema de Schwartz–Zippel establece que dos polinomios distintos de grado acotado d coinciden en un punto aleatorio con probabilidad despreciable, a lo sumo $\frac{d}{|\mathbb{K}|}$. Por lo tanto, si el verificador pudiera evaluar ambos lados de la identidad en un punto aleatorio r , podría chequear la igualdad

$$P(r) - y \stackrel{?}{=} (r - z)H(r).$$

Sin embargo, en un esquema de compromiso polinomial no basta con verificar evaluaciones explícitas de P , ya que el verificador no tiene acceso a P , sino al compromiso $c \in \mathcal{C}$. Surge entonces el problema fundamental: ¿cómo verificar identidades algebraicas sin acceso directo a los objetos algebraicos? Necesitamos un mecanismo que permita “transportar” esta verificación algebraica al espacio de los compromisos. Como problema aún más fundamental, no hicimos ninguna especificación de la estructura del espacio de los compromisos $\mathcal{C}$.

Un requisito mínimo para cualquier esquema de este tipo es que las evaluaciones puedan manipularse algebraicamente en el espacio de los compromisos. Sea $\mathbb{G}_1$ un grupo cíclico aditivo con generador G_1 . Entonces, una evaluación $a \in \mathbb{K}$ puede codificarse como aG_1 . En este contexto, las operaciones lineales se preservan naturalmente: $aG_1 + bG_1 = (a + b)G_1$. Esto permite transportar combinaciones lineales de evaluaciones al espacio de los compromisos.

No obstante, las relaciones algebraicas relevantes en sistemas de prueba no son meramente lineales. En particular, si queremos resolver el problema que expresamos mediante un QAP utilizando un argumento de evaluación, vemos que se involucran productos:

$$A(r) \cdot B(r) - C(r) = H(r)Z(r).$$

Esto es una dificultad central. La estructura de grupo no ofrece un mecanismo directo para preservar productos. Si $a = A(r)$ y $b = B(r)$, vamos a necesitar codificar el elemento $(ab)G_1$ de alguna forma, y no podemos hacerlo usando sólo la estructura aditiva de $\mathbb{G}_1$.

El problema es que aquí vamos a querer comprometer polinomios como A, B, C provenientes de la aritmetización del QAP. Solamente usando una estructura de grupo aditiva, el esquema de compromiso polinomial no preservaría estructura multiplicativa. Es decir que no podríamos transportar ese mecanismo al mundo de los compromisos.

En particular, necesitamos una estructura criptográfica que preserve relaciones multiplicativas entre exponentes. Los pairings bilineales proporcionan precisamente esta herramienta, pero utilizando más de un grupo. Sean $\mathbb{G}_1$ y $\mathbb{G}_2$ grupos cíclicos aditivos con generadores G_1 y G_2 respectivamente, sea $\mathbb{G}_T$ un grupo multiplicativo, y sea

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T,$$

se cumple la propiedad fundamental

$$e(aG_1, bG_2) = e(G_1, G_2)^{ab},$$

donde usamos notación aditiva en los grupos fuente y notación multiplicativa en el grupo objetivo $\mathbb{G}_T$. En particular, el producto ab aparece automáticamente en el exponente del elemento del grupo objetivo. Esta propiedad permite transportar identidades multiplicativas al espacio de los compromisos, haciendo viable la verificación eficiente de relaciones polinómicas complejas.

Por ahora, esto puede sonar muy abstracto. La Sección B.7 introduce una instancia concreta de estas ideas: el esquema de compromisos polinomiales de Kate–Zaverucha–Goldberg (KZG) [29], donde la divisibilidad polinomial se traduce directamente en una única ecuación de pairing.

B.7. KZG

B.7.1. Definición formal e intuición

Definición B.14 (KZG PCS). *Consideremos el cuerpo finito $\mathbb{F}_p$, y sea $\mathcal{P} = \mathbb{F}_p[x]_{\leq d}$ el espacio de polinomios de grado a lo sumo d . Sean además $\mathbb{G}_1$ y $\mathbb{G}_2$ grupos cíclicos aditivos con generadores $G_1 \in \mathbb{G}_1$ y $G_2 \in \mathbb{G}_2$, sea $\mathbb{G}_T$ un grupo multiplicativo, y sea un pairing bilineal no degenerado*

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T.$$

El esquema de compromisos polinomiales KZG consta de los siguientes algoritmos:

- **Setup:** *Se elige $\tau \xleftarrow{\$} \mathbb{F}_p$ y se publica la Structured Reference String (SRS):*

$$\text{SRS} = (G_1, \tau G_1, \tau^2 G_1, \dots, \tau^d G_1, G_2, \tau G_2).$$

- **Commit:** *Dado un polinomio $P(x) = \sum_{i=0}^d a_i x^i \in \mathcal{P}$, se define el compromiso*

$$c := \text{Commit}(P) = \sum_{i=0}^d a_i \tau^i G_1 = P(\tau) G_1 \in \mathbb{G}_1.$$

- **Open:** Dado el polinomio comprometido $P \in \mathcal{P}$ y un punto $z \in \mathbb{F}_p$ enviado por el verificador, se computan $y := P(z)$ y el polinomio cociente $Q(x) := \frac{P(x)-y}{x-z}$. El argumento de evaluación se define como $\pi := \text{Commit}(Q) = Q(\tau)G_1 \in \mathbb{G}_1$.
- **Verify:** Dados (c, z, y, π) , se acepta si

$$e(c - yG_1, G_2) = e(\pi, (\tau - z)G_2).$$

Para relacionarlo con la Definición B.13, observamos la instancia específica utilizada para cada uno de los conjuntos:

- El espacio de polinomios $\mathcal{P}$ es $\mathbb{F}_p[x]_{\leq d}$.
- El espacio de compromisos $\mathcal{C}$ es $\mathbb{G}_1$.
- El espacio de argumentos Π también es $\mathbb{G}_1$. Efectivamente, el argumento contiene un solo elemento de grupo, de modo que la prueba es de tamaño constante y por lo tanto sucinta.

Ahora, vamos a verificar efectivamente que el esquema de compromisos polinomiales KZG satisface la propiedad de completitud. Recordemos que esto quiere decir que un prover honesto siempre va a poder convencer a un verificador honesto. Como $y = P(z)$ y $Q(x) := \frac{P(x)-y}{x-z}$, se tiene $P(\tau) - y = (\tau - z)Q(\tau)$. Entonces:

$$\begin{aligned} e(c - yG_1, G_2) &= e((P(\tau) - y)G_1, G_2) \\ &= e(G_1, G_2)^{P(\tau)-y} \quad (\text{bilinealidad del pairing}) \\ &= e(G_1, G_2)^{(\tau-z)Q(\tau)} \\ &= e(Q(\tau)G_1, (\tau - z)G_2) \quad (\text{bilinealidad del pairing}) \\ &= e(\pi, (\tau - z)G_2) \quad (\text{definición de } \pi) \end{aligned}$$

Por otra parte, la solidez de KZG se sostiene en que el compromiso codifica algebraicamente la evaluación oculta $P(\tau)$ sin revelar τ . Si un prover intenta convencer al verificador de una evaluación falsa $y \neq P(z)$, entonces el polinomio $P(x) - y$ no será divisible por el polinomio $x - z$. En consecuencia, no existe ningún polinomio $Q(x)$ tal que $P(x) - y = (x - z)Q(x)$. La verificación exige precisamente que esa identidad se cumpla “en el exponente” vía la ecuación de pairing bilineal. Fabricar un argumento válido $\pi = Q(\tau)G_1$ sin que la divisibilidad sea cierta implicaría resolver una relación algebraica que, bajo el supuesto t -Bilinear Strong Diffie-Hellman (t -BSDH) es computacionalmente inviable [29]. Intuitivamente: sin conocer τ , no se puede falsificar coherentemente la aritmética polinomial codificada en la SRS.

En la práctica, los grupos $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ no son grupos arbitrarios, sino grupos derivados de curvas elípticas emparejables (*pairing-friendly elliptic curves*). Estas curvas están construidas específicamente para admitir pairings bilineales eficientes y seguros. En construcciones tipo KZG, los compromisos y pruebas viven en $\mathbb{G}_1$, los elementos de verificación utilizan $\mathbb{G}_2$, y las ecuaciones se evalúan en $\mathbb{G}_T$. En la práctica, estos grupos suelen instanciarse mediante curvas emparejables como BLS12-381 o BN254, ampliamente utilizadas en sistemas de argumentos zk-SNARK.

B.7.2. Trusted Setup

El *trusted setup* en KZG es, esencialmente, el mecanismo mediante el cual se genera la Structured Reference String (SRS). Intuitivamente, la SRS contiene “potencias secretas” de un valor desconocido $\tau \in \mathbb{F}_p$. Es decir, de la forma

$$\text{SRS} = (G_1, \tau G_1, \tau^2 G_1, \dots, \tau^d G_1, G_2, \tau G_2),$$

que son las necesarias para poder calcular $P(\tau)G_1$ para cualquier $P \in \mathcal{P}$, así como $(\tau - z)G_2$ para cualquier $z \in \mathbb{F}_p$.

Toda la seguridad del esquema depende de que *nadie* conozca el valor de τ . Conocer τ permite computar directamente $Q(\tau)$ para cualquier $Q \in \mathbb{F}_p[x]_{\leq d}$, lo que posibilita construir aperturas válidas para valores $y \in \mathbb{F}_p$ arbitrarios, rompiendo la propiedad de binding.

Para eliminar el supuesto de confiar en una sola entidad, se utilizan ceremonias *multi-party computation* (MPC). Supongamos n participantes. Cada participante i elige un secreto $\tau_i \in \mathbb{F}_p$, y actualiza la SRS multiplicando todos los valores por τ_i . Es decir que cuando terminen los n participantes, el valor final será

$$\tau := \tau_1 \cdot \tau_2 \cdot \dots \cdot \tau_n = \prod_{i=1}^n \tau_i.$$

Notamos que si al menos uno de los participantes destruye su τ_i , entonces es computacionalmente imposible que alguien pueda conocer τ . No hace falta confiar en todos los participantes, sino en que hay alguno que es honesto. Es por esto que este modelo se llama *at least one honest participant*. Por este motivo, las ceremonias modernas pueden involucrar cientos o miles de contribuyentes.

Observamos que una SRS para polinomios en el espacio $\mathbb{F}_p[x]_{\leq d}$ requiere $d+3$ elementos de grupo ($d+1$ elementos de $\mathbb{G}_1$ y 2 elementos de $\mathbb{G}_2$), es decir, un tamaño lineal en d . Como hacer circuitos aritméticos más grandes implica que el grado máximo necesario va a incrementar, que esto escale de forma lineal genera dificultades prácticas. Una SRS KZG es universal para todos los polinomios de grado menor o igual a d , lo cual limita el tamaño máximo que pueden tener los circuitos aritméticos (medido en el número de restricciones de su R1CS asociado) para los que queremos hacer SNARK.

Para SNARK pequeños, suelen ser miles de elementos de la curva. Pero para sistemas grandes, pueden ser millones de elementos, lo que puede ocupar numerosos GB de almacenamiento, copiado en dispositivos de muchos desarrolladores. Como el proceso no es rápido y no pueden realizarlo dos desarrolladores al mismo tiempo, hay una complejidad social adicional. Por eso, es que hay alternativas para sistemas de pruebas que no requieren una fase de *trusted setup*, o que el mismo es universal para cualquier circuito [17, 66, 67]. Por otra parte, estos sistemas de pruebas alternativos tienen como contrapartida un mayor tamaño de prueba, al mismo tiempo que más tiempo de ejecución en sus algoritmos.

B.7.3. Aperturas múltiples y GWC19

El esquema KZG presentado hasta ahora resuelve el problema de abrir *un* polinomio en *un* punto. Sin embargo, los sistemas de pruebas reales rara vez necesitan una sola evaluación aislada. En protocolos *PLONKish*, como Halo2, el verifier debe chequear simultáneamente evaluaciones de muchos polinomios. Verificar cada *Open* por separado sería

prohibitivo, porque implicaría una ecuación de pairing independiente por cada evaluación. El objetivo de GWC19, introducido en el paper de PLONK [17], es precisamente colapsar estos Open en una verificación sucinta conservando la estructura algebraica de KZG. El número total de polinomios consultados puede ser grande, pero si los puntos de evaluación distintos son pocos, el esquema logra mantener pruebas cortas y verificación sucinta. Esta es precisamente la situación que aparece en protocolos *PLONKish* como Halo2.

La idea central es separar dos fuentes de redundancia: varios polinomios se abren en el mismo punto, y aún habiendo más de un punto de evaluación, el número de puntos distintos sigue siendo pequeño. GWC19 explota estas observaciones mediante combinaciones lineales aleatorias. Para cada punto, combina todos los polinomios abiertos en él. Luego, agrega los distintos puntos en una sola ecuación de pairing. El costo del *verifier* pasa a depender del número de puntos distintos consultados.

Modelo algebraico de multi-apertura. Sea $s \in \mathbb{N}$ el número de puntos de evaluación distintos, y sean $z_1, \dots, z_s \in \mathbb{F}_p$ los puntos de evaluación. Para $1 \leq j \leq s$, en z_j queremos abrir t_j polinomios $P_{j,0}, P_{j,1}, \dots, P_{j,t_j-1} \in \mathbb{F}_p[x]_{\leq d}$ con compromisos

$$c_{j,i} := \text{Commit}(P_{j,i}) = P_{j,i}(\tau)G_1 \in \mathbb{G}_1, \quad 0 \leq i < t_j.$$

El prover afirma conocer las evaluaciones $y_{j,i} := P_{j,i}(z_j) \in \mathbb{F}_p$.

Si aplicáramos KZG de manera independiente a cada par $(P_{j,i}, z_j)$, obtendríamos una prueba distinta por cada par de índices (j, i) . GWC19 reemplaza ese esquema. Para ello, el *verifier* toma un *challenge* aleatorio $v \in \mathbb{F}_p$ y define para cada j las combinaciones lineales agregadas

$$F_j(X) := \sum_{i=0}^{t_j-1} v^i P_{j,i}(X) \in \mathbb{F}_p[x]_{\leq d}, \quad \alpha_j := \sum_{i=0}^{t_j-1} v^i y_{j,i} \in \mathbb{F}_p.$$

Por construcción, si todas las aperturas son correctas, entonces $F_j(z_j) = \alpha_j$. Como este compromiso es lineal, el *verifier* puede reconstruir el compromiso agregado a F_j usando la misma combinación lineal, sin necesidad de conocerlo:

$$C_j := \sum_{i=0}^{t_j-1} v^i c_{j,i} = \sum_{i=0}^{t_j-1} v^i P_{j,i}(\tau)G_1 = F_j(\tau)G_1 \in \mathbb{G}_1.$$

Por lo tanto, todas las aperturas en z_j quedan reducidas a la afirmación $F_j(z_j) = \alpha_j$, que es una apertura KZG, y el *prover* puede definir el polinomio cociente $W_j(X) := \frac{F_j(X) - \alpha_j}{X - z_j}$ y enviar como argumento de apertura el compromiso $\pi_j := \text{Commit}(W_j) = W_j(\tau)G_1 \in \mathbb{G}_1$, reduciendo el problema a s aperturas KZG ordinarias.

$$e(C_j - \alpha_j G_1, G_2) = e(\pi_j, (\tau - z_j)G_2), \quad 1 \leq j \leq s.$$

Usando la bilinealidad del pairing, esta ecuación puede reescribirse como

$$e(\pi_j, \tau G_2) = e(z_j \pi_j + C_j - \alpha_j G_1, G_2).$$

La segunda etapa de GWC19 agrega estas s ecuaciones de pairing en una sola. Para ello, el *verifier* toma un segundo *challenge* aleatorio $u \in \mathbb{F}_p$ y combina linealmente todas las instancias con potencias de u . Definimos

$$\Pi := \sum_{j=1}^s u^{j-1} \pi_j \in \mathbb{G}_1, \quad E := \sum_{j=1}^s u^{j-1} (z_j \pi_j + C_j - \alpha_j G_1) \in \mathbb{G}_1.$$

Entonces, por bilinealidad,

$$e(\Pi, \tau G_2) = \prod_{j=1}^s e(\pi_j, \tau G_2)^{u^{j-1}} = \prod_{j=1}^s e(z_j \pi_j + C_j - \alpha_j G_1, G_2)^{u^{j-1}} = e(E, G_2).$$

En consecuencia, el *verifier* sólo necesita chequear la ecuación final $e(\Pi, \tau G_2) = e(E, G_2)$.

Complejidad e intuición de solidez. La completitud de GWC19 es una consecuencia directa de la completitud de KZG y de la linealidad de las combinaciones introducidas. Si todas las afirmaciones $P_{j,i}(z_j) = y_{j,i}$ son correctas, entonces para cada j el polinomio $F_j(X) - \alpha_j$ es divisible por $X - z_j$, de modo que $W_j(X)$ está bien definido y satisface la ecuación KZG ordinaria. Multiplicar esas ecuaciones por potencias de u y reagruparlas no altera su validez, porque el pairing es bilineal en ambos argumentos.

La intuición de solidez también sigue la estructura de KZG. En primer lugar, para un punto fijo z_j , si el *prover* mintiera alguna de las evaluaciones $y_{j,i}$, entonces la igualdad $F_j(z_j) = \alpha_j$ dejaría de ser cierta salvo cancelaciones fortuitas en la combinación lineal inducida por v , pero esto tiene probabilidad negligible porque v se elige uniformemente al azar en $\mathbb{F}_p$. En segundo lugar, aún si el *prover* intentara mezclar comportamientos distintos entre varios puntos, la agregación posterior con u obliga a que todas las ecuaciones parciales sean simultáneamente consistentes, de modo que falsificar la ecuación final implicaría fabricar una identidad de divisibilidad agregada en el exponente sin conocer τ , lo cual sigue estando fuera del alcance computacional bajo los supuestos de seguridad de KZG.

B.7.4. Aperturas por conjuntos de puntos en Halo2 Multi-Open

La construcción anterior organiza las aperturas a partir de cada punto de evaluación: primero combina los polinomios consultados en un mismo punto y luego agrega las ecuaciones resultantes. Otra estrategia, utilizada por las bibliotecas de MIDNIGHT que sostienen el circuito $C_{\text{STM}}^{\text{Mid}}$ [14], agrupa los polinomios que comparten un mismo *conjunto* de puntos de evaluación. Esta diferencia no cambia el compromiso KZG subyacente, pero sí modifica la forma algebraica del argumento de apertura y el trabajo que debe reconstruir el *verifier*.

Consideremos r conjuntos de puntos. Para $0 \leq j < r$, escribimos

$$S_j = \{z_{j,0}, \dots, z_{j,m_j-1}\}$$

y denotamos por $p_{j,i}(X) \in \mathbb{F}_p[x]_{\leq d}$ a los polinomios asociados al conjunto S_j . Sean además $x_1, x_2, x_3, x_4 \in \mathbb{F}_p$ desafíos escalares del protocolo. En una ejecución no interactiva, estos valores se derivan del *transcript* mediante la transformación de Fiat–Shamir de la Sección B.8; por ahora sólo importa su función como coeficientes aleatorios de agregación.

Para cada conjunto, el *verifier* forma la combinación lineal

$$Q_j(X) = \sum_i x_1^i p_{j,i}(X) \in \mathbb{F}_p[x]_{\leq d}.$$

Luego debe reconstruir el valor de Q_j en el punto adicional x_3 a partir de las evaluaciones conocidas sobre todo S_j . Para ello define

$$R_j(x_3) = \sum_{\ell=0}^{m_j-1} \lambda_{j,\ell}(x_3) Q_j(z_{j,\ell}), \quad Z_j(X) = \prod_{\ell=0}^{m_j-1} (X - z_{j,\ell}),$$

donde $\lambda_{j,\ell}(x_3)$ son los coeficientes de interpolación de Lagrange y Z_j es el polinomio que se anula en los puntos de S_j . A diferencia de verificar una apertura KZG independiente por cada evaluación, el argumento contiene un único elemento de grupo π y valores auxiliares $q_j(x_3)$. Con esos datos, el *verifier* reconstruye

$$f(x_3) = \sum_{j=0}^{r-1} x_2^j \cdot \frac{q_j(x_3) - R_j(x_3)}{Z_j(x_3)}.$$

Finalmente, agrega las contribuciones de los distintos conjuntos con potencias de x_4 y forma

$$\text{final_com} = \sum_{j=0}^{r-1} x_4^j Q_j(\tau) G_1 + x_4^r f(\tau) G_1, \quad v = \sum_{j=0}^{r-1} x_4^j q_j(x_3) + x_4^r f(x_3).$$

La verificación termina en una única ecuación de pairing:

$$e(\pi, \tau G_2) \stackrel{?}{=} e(\text{final_com} - v G_1 + x_3 \pi, G_2).$$

La diferencia esencial con GWC19 está en el lugar donde se realiza la agregación. GWC19 combina aperturas KZG organizadas por punto de evaluación; Halo2 Multi-Open mueve parte del trabajo al proceso de reconstruir una apertura común para cada conjunto de puntos, mediante interpolaciones y polinomios de anulación. Ambos caminos conservan una verificación final sucinta, pero inducen estructuras de datos, acumuladores y costos de verificación diferentes. Esta distinción es la que impide trasladar mecánicamente un verificador generado para el primer protocolo al circuito de MTHRIL basado en el segundo.

B.7.5. Lo que tenemos hasta ahora

Las dos estrategias de apertura múltiple anteriores muestran cómo la linealidad de KZG permite condensar muchas evaluaciones en una ecuación final de pairing, aunque organicen esa agregación de manera diferente. Este refinamiento no altera el papel que los compromisos polinomiales cumplen dentro de la construcción general. En la sección de QAP mostramos que la verificación de un circuito aritmético puede reformularse como una identidad entre polinomios de la forma

$$AB - C = HZ,$$

entendida como igualdad en el anillo $\mathbb{F}_p[x]$, donde A, B, C, H y Z son polinomios. Conceptualmente, esto transforma la verificación de muchas restricciones en una única condición algebraica global. Sin embargo, ese enfoque presentaba un obstáculo fundamental: el *verifier* debía conocer explícitamente los polinomios (o al menos sus evaluaciones). Esto es incompatible con los objetivos criptográficos del sistema. Queremos que el witness permanezca oculto, la prueba sea sucinta y no interactiva, y que el *verifier* no necesite reconstruir objetos grandes.

Los compromisos polinomiales, y en particular KZG, nos permiten reemplazar el acceso explícito a los polinomios involucrados por compromisos criptográficos junto con argumentos de evaluación de tamaño constante (ya que son elementos de $\mathbb{G}_1$), y la verificación se hace en el grupo objetivo $\mathbb{G}_T$ del pairing bilineal e . Esta reducción es extremadamente poderosa: la complejidad del *verifier* deja de depender del tamaño del circuito y pasa a

depender únicamente del número de ecuaciones de pairing, que es constante. Del mismo modo, se pueden verificar identidades polinómicas sin necesidad de revelar los polinomios mismos.

No obstante, todavía faltan componentes esenciales para obtener un zk-SNARK completo:

- No interactividad: KZG asume desafíos enviados por el verifier (en concreto, el punto $z \in \mathbb{F}_p$). Para eliminar esta interacción, necesitaremos la transformación de Fiat–Shamir de la Sección B.8.
- Conocimiento cero: KZG no garantiza conocimiento cero por sí mismo. El ocultamiento va a aparecer al combinar compromisos, aleatoriedad y demás estructura del protocolo. En particular, vamos a ver cómo los compromisos a A, B, C, H se ensamblan en una prueba coherente en la última sección del capítulo.

En las siguientes secciones formalizaremos estos pasos finales. Primero, eliminaremos la interacción mediante la transformación de Fiat–Shamir [68]. Luego, presentaremos Groth16 [69] como una instanciación concreta donde todas las piezas desarrolladas en el capítulo se combinan en un sistema de zk-SNARK funcional, y que utilizamos en nuestro proyecto.

B.8. Protocolo Fiat–Shamir y No-Interactividad

Esta sección presenta la transformación que elimina la interacción en protocolos de desafío público y muestra su aplicación al esquema KZG.

B.8.1. De protocolos interactivos a no interactivos

Los protocolos de argumentos de conocimiento suelen ser interactivos entre el prover y el verifier. Sin ir más lejos, el argumento sucinto KZG de la Sección B.7 requiere que el verifier envíe un valor aleatorio como challenge al prover.

Sin embargo, en entornos como blockchains, donde la verificación debe ocurrir en un único mensaje publicado onchain, la interactividad es un obstáculo. En 1986, Amos Fiat y Adi Shamir propusieron una transformación que convierte muchos de estos protocolos interactivos en protocolos no interactivos (NIZK) sin pérdida significativa del valor de seguridad, cuando se modela la función de desafío como un oráculo aleatorio [68, 47].

La idea central es la siguiente: en lugar de que el *verifier* genere un desafío aleatorio y se lo envíe al *prover*, el *prover* calcula el desafío como el resultado de aplicar una función de hash criptográfica a la transcripción parcial del protocolo y utiliza ese resultado como si fuera el desafío.

Para que esta transformación sea segura, se necesita trabajar en el ROM, de modo que la función de hash $\mathcal{H}$ se comporte como una función aleatoria. Este enfoque es central para la mayoría de construcciones de zk-SNARK modernas. Aunque no existen funciones hash verdaderamente aleatorias en la práctica, modelar una función criptográfica como oráculo aleatorio permite demostrar seguridad de manera formal y luego instanciar la construcción con hashes como SHA-256, BLAKE2 o Poseidon bajo heurísticas de seguridad razonables. Para formalizar esta transformación, primero planteamos formalmente un protocolo interactivo público genérico de tres mensajes (lo cual puede trivialmente generalizarse a un número arbitrario de mensajes), conocido como Σ -protocol.

Definición B.15 (Σ -protocol). Decimos que $\Pi = (\text{Commit}, \text{Open}, \text{Verify})$ es un Σ -protocol para una relación $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ con public inputs x y witness w tales que $(x, w) \in \mathcal{R}$, entre un prover $\mathcal{P}$ y un verifier $\mathcal{V}$, si consiste de los siguientes tres mensajes:

1. El prover le envía al verifier un compromiso $c = \text{Commit}(x, w; r)$ (r es la aleatoriedad interna del prover, fijada al inicio de la ejecución).
2. El verifier responde con un desafío uniformemente aleatorio z .
3. El prover le envía una respuesta $\pi = \text{Open}(c, z, w, x; r)$.

Una ejecución completa del Σ -protocol produce la tripla (c, z, π) . El verifier acepta la respuesta π si $\text{Verify}(x, c, z, \pi) = 1$.

Definición B.16 (Transformación Fiat–Shamir). Sea $\Pi = (\text{Commit}, \text{Open}, \text{Verify})$ un Σ -protocol para una relación $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$, y supongamos que su espacio de desafíos es compatible con el output de una función hash criptográfica $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^n$. La transformación Fiat–Shamir es un protocolo no interactivo $\Pi_{FS} = (\text{Prove}_{FS}, \text{Verify}_{FS})$ definido como:

- $\text{Prove}_{FS}(x, w)$, ejecutado por el prover:
 - Se computa $c = \text{Commit}(x, w; r)$.
 - Se computa un challenge $z \in \{0, 1\}^n$ como $z = \mathcal{H}(x \parallel c)$, donde la concatenación $x \parallel c$ representa una codificación no ambigua del transcript parcial.
 - Se computa la respuesta $\pi = \text{Open}(c, z, w, x; r)$.
- $\text{Verify}_{FS}(x, c, \pi)$, ejecutado por el verifier:
 - Se recomputa el challenge $z = \mathcal{H}(x \parallel c)$.
 - Se acepta si y solo si $\text{Verify}(x, c, z, \pi) = 1$.

B.8.2. Transformación Fiat–Shamir en KZG

Vamos a ver un ejemplo de la transformación de Fiat–Shamir. En particular, la vamos a aplicar al esquema de compromisos polinomiales KZG. Esto nos permitirá ganar intuición sobre la transformación, lo que será necesario para la sección final del capítulo. Observemos que esto es para las etapas posteriores al trusted setup.

Sea una función hash criptográfica $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{F}_p$, modelada como oráculo aleatorio en el ROM. La transformación consiste en reemplazar el desafío aleatorio $z \in \mathbb{F}_p$ por $z := \mathcal{H}(\text{encode}(c))$, donde c es el compromiso polinomial generado por el prover. En implementaciones reales, la entrada al hash incluye una codificación determinística y no ambigua del compromiso, así como cualquier otro contexto público que deba formar parte del transcript.

El protocolo consiste de los algoritmos Prove_{FS} 10, a ser ejecutado por el prover, y Verify_{FS} 11, a ser ejecutado por el verifier.

La transformación de Fiat–Shamir aplicada al protocolo de apertura de KZG muestra un principio fundamental en el diseño de pruebas criptográficas modernas: la eliminación de la interacción mediante la derivación determinística de desafíos aleatorios a partir de información pública. Esta técnica permite eliminar la interactividad en protocolos sin sacrificar las demás propiedades deseables en un SNARK.

En el caso del esquema KZG, el desafío z pasa de ser generado por el verificador a determinarse como el resultado de una función hash aplicada al compromiso. En contraste,

Algoritmo 10 Prove_{FS} para el esquema KZG**Requiere:** Polinomio $P \in \mathbb{F}_p[x]_{\leq d}$, SRS $\{G_1, \tau G_1, \dots, \tau^d G_1, G_2, \tau G_2\}$.**Asegura:** (c, y, π) es una prueba no interactiva que será aceptada por el algoritmo Verify_{FS} .

- 1: $c \leftarrow P(\tau)G_1$ ▷ cómputo del compromiso c
- 2: $z \leftarrow \mathcal{H}(\text{encode}(c))$ ▷ cómputo del *challenge* z
- 3: $y \leftarrow P(z)$ ▷ evaluación del polinomio P en el *challenge*
- 4: $Q(x) \leftarrow \frac{P(x)-y}{x-z}$ ▷ cómputo del polinomio cociente Q
- 5: $\pi \leftarrow Q(\tau)G_1$ ▷ cómputo del argumento de evaluación π
- 6: **return** (c, y, π)

Algoritmo 11 Verify_{FS} para el esquema KZG**Requiere:** Compromiso polinomial $c \in \mathbb{G}_1$, evaluación $y \in \mathbb{F}_p$, argumento de evaluación $\pi \in \mathbb{G}_1$, y los elementos $G_2, \tau G_2 \in \mathbb{G}_2$ de la SRS.**Asegura:** Si el algoritmo devuelve 1, entonces el prover conoce un polinomio P consistente con c tal que $y = P(z)$, excepto por probabilidad negligible.

- 1: $z \leftarrow \mathcal{H}(\text{encode}(c))$ ▷ recómputo del *challenge* z
- 2: **return** $\llbracket e(c - yG_1, G_2) = e(\pi, (\tau - z)G_2) \rrbracket$

el esquema Groth16 de la Sección B.9 ya se presenta directamente como un argumento no interactivo, y la aleatoriedad que en un Σ -protocol sería provista por el verificador aparece absorbida en la estructura del *trusted setup*. La transformación de Fiat–Shamir reaparece además en el *transcript* de la verificación Halo2 discutido en la Subsección 6.7.4.

B.8.3. Limitaciones

Aunque el esquema Fiat–Shamir es extremadamente útil y ampliamente utilizado para obtener pruebas no interactivas, no es una panacea sin riesgos. La seguridad de la transformación se demuestra únicamente en el ROM, y no existe una garantía absoluta en el mundo real de que una instancia concreta del hash cumpla exactamente ese ideal. De hecho, múltiples trabajos han documentado varios problemas asociados a implementaciones mal hechas o a condiciones de seguridad insuficientes. La bibliografía es sumamente extensa al respecto; acá dejamos un pequeño resumen.

En particular, está demostrado que existen protocolos interactivos seguros para los cuales la transformación Fiat–Shamir falla para cualquier función hash real, lo que subraya que la seguridad depende críticamente de la idealización de la función de hash [70].

En diversos trabajos recientes [71, 72, 73], se analizan distintos tipos de errores en la aplicación de la transformación en sistemas reales, ilustrando ejemplos en los que la construcción incorrecta de los inputs al hash puede comprometer la seguridad de pruebas basadas en esta heurística. Los autores de estos trabajos demuestran que un uso débil o parcial de la transformación puede permitir ataques efectivos contra protocolos de SNARK concretos, incluso pudiendo falsificar pruebas.

Finalmente, observamos que en el contexto de la computación cuántica, las transformaciones Fiat–Shamir con funciones de hash reales tienen limitaciones de seguridad bajo ataques que explotan superposiciones de consultas [74].

Como lecciones principales para el desarrollo de nuestro trabajo, es fundamental espe-

cificar con precisión qué partes del transcript se incluyen en el hash, utilizar codificaciones no ambiguas y evitar reutilización de desafíos entre instancias distintas del protocolo.

B.9. Groth16

B.9.1. El protocolo Groth16

En esta sección presentamos una descripción simplificada del argumento de conocimiento subyacente al esquema Groth16, pensada para fijar la intuición algebraica principal sin reproducir todos los detalles técnicos de una formalización estándar. Sea $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ una relación en NP. Como en la Sección B.5, suponemos que la relación ha sido aritmetizada como un QAP de tamaño m y dimensión n sobre un cuerpo finito $\mathbb{K} = \mathbb{F}_p$, y todo se reduce a argumentar sucintamente una identidad polinomial. Vamos a ligar todos estos nombres de objetos antes de entrar a describir el protocolo propiamente dicho.

En particular, existen $3n$ polinomios $\{A_i(x)\}_{i=1}^n$, $\{B_i(x)\}_{i=1}^n$, $\{C_i(x)\}_{i=1}^n$ en $\mathbb{F}_p[x]_{<m}$ tales que, definiendo el vector extendido de asignación

$$\mathbf{z} = (z_1, \dots, z_n) \in \mathbb{F}_p^n$$

donde $\mathbf{z}$ representa el vector completo de asignación del QAP: incluye la coordenada constante, las coordenadas correspondientes a los *public inputs*, las correspondientes al witness original del problema, y las variables auxiliares introducidas por la aritmetización. En particular, adoptamos la convención usual de escribir $x = (x_1, \dots, x_\ell)$ para la parte pública con $x_1 = 1$, de modo que la constante queda incluida dentro de los *public inputs*; si $w = (w_1, \dots, w_t)$ denota el witness original, entonces tanto x como w aparecen embebidos dentro de $\mathbf{z}$. Definiendo los polinomios agregados

$$A(x) = \sum_{i=1}^n \mathbf{z}_i A_i(x), B(x) = \sum_{i=1}^n \mathbf{z}_i B_i(x), C(x) = \sum_{i=1}^n \mathbf{z}_i C_i(x),$$

y para un polinomio objetivo Z construido sobre $r_1, \dots, r_m \in \mathbb{F}_p$ se cumple que $(x, w) \in \mathcal{R}$ si y solo si existe un polinomio H tal que

$$AB - C = HZ.$$

Además, sean $\mathbb{G}_1$ y $\mathbb{G}_2$ grupos cíclicos aditivos de orden primo p , sean $G_1 \in \mathbb{G}_1$ y $G_2 \in \mathbb{G}_2$ generadores, y sea $\mathbb{G}_T$ un grupo multiplicativo, con un pairing bilineal no degenerado $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.

Definición B.17 (Protocolo interactivo Groth16). *El protocolo interactivo Groth16 busca argumentar la existencia de un polinomio H y consta de los siguientes algoritmos:*

- **Setup:** Se eligen de forma uniforme $\tau, \alpha, \beta, \gamma, \delta \in \mathbb{F}_p^*$, que se usan para construir la SRS, y luego son destruidos. La SRS contiene:
 - Las potencias de τ necesarias para evaluar polinomios de hasta grado $m-1$ en los grupos $\mathbb{G}_1, \mathbb{G}_2$: $\{\tau^k G_1, \tau^k G_2\}_{k=0}^{m-1}$.
 - Los elementos $\alpha G_1, \beta G_2, \gamma G_2, \delta G_1, \delta G_2$.

- Para $\ell + 1 \leq i \leq n$ (i.e., los índices correspondientes a la parte privada del vector de asignación), los elementos

$$\frac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\delta} G_1.$$

- Para los inputs públicos $1 \leq i \leq \ell$, la clave de verificación contiene los elementos

$$\frac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\gamma} G_1,$$

que permiten al verifier reconstruir la combinación lineal asociada a la instancia pública.

- Los elementos $\left\{ \frac{\tau^k Z(\tau)}{\delta} G_1 \right\}_{k=0}^{m-2}$.
- **Prove:** El prover $\mathcal{P}$, que conoce x y w , calcula $A(\tau), B(\tau), C(\tau), H(\tau)$. Luego elige aleatoriamente $r, s \in \mathbb{F}_p^*$, y define:
 1. $\pi_A = (\alpha + A(\tau) + r\delta) G_1$.
 2. $\pi_B = (\beta + B(\tau) + s\delta) G_2$.
 3. $\pi_C = \left(\sum_{i=\ell+1}^n \mathbf{z}_i \frac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\delta} \right) G_1 + \left(\frac{H(\tau)Z(\tau)}{\delta} + rB(\tau) + sA(\tau) - rs\delta \right) G_1$.

El output del algoritmo **Prove** es la tripla $\pi = (\pi_A, \pi_B, \pi_C) \in (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_1)$.

- **Verify:** El verifier $\mathcal{V}$, que solamente conoce los inputs públicos $x_1, \dots, x_\ell$, reconstruye la combinación

$$V_{\text{pub}}(\tau) := \sum_{i=1}^{\ell} x_i \frac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\gamma}.$$

Luego, acepta si:

$$e(\pi_A, \pi_B) = e(\alpha G_1, \beta G_2) \cdot e(V_{\text{pub}}(\tau) G_1, \gamma G_2) \cdot e(\pi_C, \delta G_2).$$

No está demás recordar que **Groth16** es un argumento de conocimiento. Intuitivamente, esto significa que para todo prover eficiente que logra producir pruebas aceptadas con probabilidad no despreciable, existe un extractor probabilístico en tiempo polinomial, con acceso a dicho prover, que recupera un witness w tal que $(x, w) \in \mathcal{R}$.

El paper original de Groth [69] describe el protocolo en términos de vectores y combinaciones lineales, no en términos de exponenciaciones de grupo. La formulación moderna que presentamos (y sobre la cual están basadas las implementaciones principales) interpreta cada elemento del SRS $a \in \mathbb{F}_p$ como elementos de $\mathbb{G}_1$ mediante la aplicación $a \mapsto aG_1$, de modo que las combinaciones lineales se vuelven MSM (multi-scalar multiplications) y el verifier pueda usar pairings bilineales.

Observamos que si $(x, w) \in \mathcal{R}$, entonces por la definición del QAP existe un polinomio H tal que $AB - C = HZ$, y en particular se cumple la identidad evaluada en el trapdoor del setup:

$$A(\tau)B(\tau) - C(\tau) = H(\tau)Z(\tau).$$

Si expandimos la ecuación de verificación y utilizamos la bilinealidad de e , se observa que todos los términos dependientes de r y s se cancelan, y la igualdad se satisface completamente. Por lo tanto, el protocolo cumple la propiedad de completitud. La solidez se

fundamenta en supuestos criptográficos estándar sobre grupos bilineales y en la estructura algebraica de la construcción.

A diferencia de los Σ -protocols discutidos en la Sección B.8, Groth16 no expone un *challenge* explícito del *verifier*. En cambio, la randomización está embebida en la fase del *trusted setup* mediante la tupla $(\tau, \alpha, \beta, \gamma, \delta)$. Groth16 ya es NIZK por diseño. En particular, no se enmarca en el ROM, sino en el AGM (Algebraic Group Model) [75].

Para finalizar esta subsección, vamos a observar que Groth16 realmente está diseñado para verificar la identidad polinomial en el exponente (en $\mathbb{G}_T$), pero separando la parte pública $1 \leq i \leq \ell$, de la parte privada $\ell + 1 \leq i \leq n$, y de la parte de aleatorización. En efecto:

$$\begin{aligned} \pi_C = & \left(\sum_{i=\ell+1}^n \mathbf{z}_i \frac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\delta} \right) G_1 \\ & + \frac{H(\tau)Z(\tau)}{\delta} G_1 + (rB(\tau) + sA(\tau) - rs\delta)G_1. \end{aligned}$$

Así, vemos que todo lo que está dividido por δ corresponde a la parte algebraica del QAP, mientras que los términos $rB(\tau) + sA(\tau) - rs\delta$ aparecen como una perturbación cuadrática simétrica en r, s .

B.9.2. Groth16 como extensión natural de KZG

Es instructivo interpretar Groth16 como una evolución del esquema KZG aplicado a la aritmetización mediante QAP. Recordemos que en KZG, el prover demuestra que conoce un polinomio $P(x)$ tal que $y = P(z)$. La verificación consiste en chequear, mediante un pairing bilineal, que $P(\tau) - y$ es divisible por $\tau - z$. En esencia, KZG permite verificar una identidad polinomial en un punto oculto τ .

En el caso del QAP, lo que queremos verificar es: $A(x) \cdot B(x) - C(x) = H(x)Z(x)$. En vez de argumentar divisibilidad por el polinomio $x - z$, lo hacemos por el polinomio $Z(x)$. Pero KZG no sería suficiente para esto, por los siguientes tres motivos que Groth16 soluciona.

- En KZG, el compromiso revela implícitamente la evaluación $P(\tau)$, sin soportar separación entre parte pública y privada. En cambio, en Groth16 el *input* puede ocultarse, y los *public inputs* deben poder verificarse en forma separada. Es por esto que aparecen los parámetros $\alpha, \beta, \gamma, \delta$, que mezclan algebraicamente los polinomios de forma que $\mathcal{V}$ pueda aislar la parte pública, y el *witness* quede enmascarado.
- En KZG, necesitaríamos un compromiso para cada uno de $A(x), B(x), C(x), H(x)$, lo que conlleva varias pruebas de divisibilidad con correspondientes ecuaciones de pairing bilineal. Groth16 comprime toda la verificación del QAP en una única ecuación de emparejamiento.
- Como se señaló en la Subsección B.7.5, KZG por sí solo no es ZK. La aleatoriedad r, s introducida por Groth16, así como los términos cruzados $(rB(\tau) + sA(\tau) - rs\delta)G_1$ garantizan que la distribución de la prueba sea independiente del witness, y por lo tanto es ZK.

B.9.3. Complejidad de Groth16

En la fase de **Setup** (que recordemos, en la práctica es una ceremonia en la que varias personas deben contribuir con aleatoriedad y al menos una debe comportarse de forma honesta), para computar la SRS en su totalidad hay que computar:

- $\Theta(m)$ multiplicaciones escalares en $\mathbb{G}_1$.
- $\Theta(m)$ multiplicaciones escalares en $\mathbb{G}_2$ (en la práctica, $\mathbb{G}_2$ suele tener el doble de bits que $\mathbb{G}_1$).
- La SRS tiene una cantidad lineal de elementos en m y ℓ , con $\ell \leq n$. En particular, a medida que aumenta el tamaño del circuito, aumenta también el tamaño del *trusted setup* necesario. El *trusted setup* de Groth16 es una fase costosa, y no es universal, sino específica para cada circuito.

En la fase de generación de la prueba, el costo principal proviene de las MSM necesarias para computar π_A, π_B, π_C . Recordemos que una MSM es una operación del estilo $\sum s_i P_i$, donde los s_i son escalares y los P_i son puntos de curva, de modo que en Groth16 las MSM son aquellas operaciones donde multiplicamos un polinomio evaluado en τ por el generador correspondiente. Esto permite el uso de algoritmos especializados, principalmente el de Pippenger [76], usado en las bibliotecas estándar de sistemas zk-SNARK [77, 78, 79]. Cada una de ellas cuesta $\Theta(m)$ operaciones escalares en $\mathbb{G}_1$ o $\mathbb{G}_2$ respectivamente. Por ejemplo:

$$A(\tau)G_1 = \sum_{i=0}^{m-1} a_i \tau^i G_1.$$

- Una MSM en $\mathbb{G}_1$ para calcular $A(\tau)G_1$ durante el cómputo de π_A .
- Una MSM en $\mathbb{G}_2$ para calcular $B(\tau)G_2$ durante el cómputo de π_B .
- Una MSM en $\mathbb{G}_1$ durante el cómputo de π_C . Aquí es donde se observa que los términos de la SRS correspondientes al witness permiten aliviar el cómputo:

$$\begin{aligned} \pi_C = & \sum_{i=\ell+1}^n \mathbf{z}_i \left(\frac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\delta} G_1 \right) + \sum_{j=0}^{m-2} h_j \left(\frac{\tau^j Z(\tau)}{\delta} G_1 \right) \\ & + \sum_{i=1}^n (r \mathbf{z}_i) B_i(\tau) G_1 + \sum_{i=1}^n (s \mathbf{z}_i) A_i(\tau) G_1 - r s (\delta G_1). \end{aligned}$$

y más aún, en la práctica se fusionan algebraicamente todas las contribuciones en un único vector de escalares de modo que se hace una MSM.

- En el cálculo de $H(x)$, se requiere una multiplicación polinomial y una división por $Z(x)$. Con una FFT, esto cuesta $\Theta(n \log n)$ operaciones en $\mathbb{F}_p$.
- La prueba generada $\pi = (\pi_A, \pi_B, \pi_C)$ tiene tamaño constante $\Theta(1)$ (tres elementos de grupo).

En la fase de verificación de la prueba, la complejidad temporal del algoritmo no depende del tamaño total del circuito, sino linealmente en el número de public inputs ℓ :

- Reconstruir la combinación lineal correspondiente a los inputs públicos, lo que es una MSM en $\mathbb{G}_1$ de tamaño ℓ , y típicamente $\ell \ll m$.
- Evaluar la ecuación de pairing bilineal, para lo cual se requieren evaluar tres pairings (pues $e(\alpha G_1, \beta G_2)$ es constante desde el **Setup**) y algunas multiplicaciones en $\mathbb{G}_T$.

B.9.4. Aplicación de **Groth16** al proyecto

En contextos de blockchain, la métrica relevante no es únicamente la complejidad asintótica, sino el costo concreto en términos de operaciones criptográficas soportadas por la máquina virtual subyacente. En particular, las operaciones dominantes en la verificación de **Groth16** son los pairings bilineales. Si la plataforma expone precompilados o primitivas nativas eficientes para emparejamientos (como ocurre en ciertos entornos basados en curvas elípticas tipo BN254 o BLS12-381), el costo de verificación es esencialmente constante y pequeño, lo que hace a **Groth16** especialmente adecuado para validación *onchain*.

Esta observación es particularmente relevante en el contexto de CARDANO, donde las capacidades criptográficas disponibles on-chain determinan directamente qué construcciones zk-SNARK usar. La decisión correspondiente se justifica en la Subsección 5.4.2, y su integración concreta con los dos circuitos auxiliares se desarrolla en las Secciones 6.9 y 6.8. En particular, las primitivas de pairings bilineales sobre BLS12-381 disponibles en PLUTUS, junto con la extrema sucintez del argumento **Groth16**, determinan que sea el zk-SNARK utilizado en el proyecto [80].